

UNIVERSITY
OF SKÖVDE

DOCTORAL DISSERTATION

CONFORMAL ANOMALY DETECTION

Detecting abnormal trajectories in surveillance applications

RIKARD LAXHAMMAR
Informatics

CONFORMAL ANOMALY DETECTION

Detecting abnormal trajectories in surveillance applications

DOCTORAL DISSERTATION

CONFORMAL ANOMALY DETECTION

Detecting abnormal trajectories in surveillance applications

RIKARD LAXHAMMAR
Informatics



UNIVERSITY
OF SKÖVDE

This research has been supported by:



SAAB

KK-stiftelsen

Rikard Laxhammar, 2014

Title: Conformal Anomaly Detection
Detecting abnormal trajectories in surveillance applications

University of Skövde 2014, Sweden
www.his.se

Printer: Runit AB, Skövde

ISBN 978-91-981474-2-1
Dissertation Series, No. 3 (2014)

Abstract

Human operators of modern surveillance systems are confronted with an increasing amount of *trajectory data* from moving objects, such as people, vehicles, vessels, and aircraft. A large majority of these trajectories reflect routine traffic and are uninteresting. Nevertheless, some objects are engaged in dangerous, illegal or otherwise interesting activities, which may manifest themselves as unusual and abnormal trajectories. These *anomalous* trajectories can be difficult to detect by human operators due to cognitive limitations.

In this thesis, we have studied algorithms for the automated detection of anomalous trajectories in surveillance applications. The main results and contributions of the thesis are two-fold. Firstly, we propose and discuss a novel approach for anomaly detection, called *conformal anomaly detection*, which is based on *conformal prediction* (Vovk et al.). In particular, we propose two general algorithms for anomaly detection: the *conformal anomaly detector* (CAD) and the computationally more efficient *inductive conformal anomaly detector* (ICAD). A key property of conformal anomaly detection, in contrast to previous methods, is that it provides a well-founded approach for the tuning of the anomaly threshold that can be directly related to the expected or desired alarm rate. Secondly, we propose and analyse two parameter-light algorithms for unsupervised online learning and sequential detection of anomalous trajectories based on CAD and ICAD: the *sequential Hausdorff nearest neighbours conformal anomaly detector* (SHNN-CAD) and the *sequential sub-trajectory local outlier inductive conformal anomaly detector* (SSTLO-ICAD), which is more sensitive to local anomalous sub-trajectories.

We have implemented the proposed algorithms and investigated their classification performance on a number of real and synthetic datasets from the video and maritime surveillance domains. The results show that SHNN-CAD achieves competitive classification performance with minimum parameter tuning on video trajectories. Moreover, we demonstrate that SSTLO-ICAD is able to accurately discriminate realistic anomalous vessel trajectories from normal background traffic.

Keywords: Anomaly detection, conformal prediction, trajectory analysis, video surveillance, maritime surveillance.

Sammanfattning

Operatörer av moderna övervakningssystem ställs inför en allt större mängd data i form av rörelsebanor från olika typer av objekt, t.ex. människor, fordon, fartyg och flygplan. En stor majoritet av dessa rörelsebanor motsvarar rutinmässig trafik och är ointressant. Några av objekten är dock involverade i farlig, olovlig eller på annat sätt intressant aktivitet, vilket kan yttra sig i form av ovanliga rörelsebanor. Sådana avvikande rörelsebanor kan vara svåra att upptäcka för en mänsklig operatör p.g.a. kognitiva begränsningar, och det finns därför ett behov av ett automatiserat stöd.

I denna avhandling studeras algoritmer for automatiserad upptäckt av avvikande rörelsebanor inom övervakningstillämpningar. De huvudsakliga resultaten och bidragen hos avhandlingen är tvåfaldiga. För det första, föreslås, analyseras och diskuteras en ny ansats till avvikelsedetektion kallad *conformal anomaly detection,* som är baserad på *conformal prediction* (Vovk et al.). Framförallt föreslås två generella algoritmer för avvikelsedetektion: *conformal anomaly detector* (CAD) och *inductive conformal anomaly detector* (ICAD), där den senare är beräkningsmässigt mer effektiv. En nyckelegenskap hos conformal anomaly detection är att denna metod, till skillnad från tidigare metoder, erbjuder en teoretiskt välgrundad ansats till tröskelsättning, som direkt kan relateras till önskad eller förväntad larmfrekvens. För det andra, förelås och analyseras två parameterlätta algoritmer, baserade på CAD och ICAD, för oövervakad och inkrementell inlärning samt sekventiell detektion av avvikande banor: *Sequential Hausdorff nearest neighbours conformal anomaly detector* (SHNN-CAD) och *sequential sub-trajectory local outlier inductive conformal anomaly detector* (SSTLO-ICAD). Den senare är känsligare för lokala avvikelser hos rörelsebanorna.

Vi implementerar och undersöker klassificeringsprestanda hos de två föreslagna algoritmerna på ett antal verkliga och syntetiska datamängder inom videoövervakning och sjöövervakning. Resultaten visar att SHNN-CAD uppnår konkurrenskraftig prestanda med minimal parametertrimning på videodata. Vidare demonstrerar vi att SSTLO-ICAD med hög noggrannhet kan särskilja realistiskt simulerade fartygsavvikelser från verkliga fartygsrörelser som motsvarar normal bakgrundstrafik.

Acknowledgments

First and foremost, I would like to thank my main supervisor, Göran Falkman. You have shown great commitment to my research project at all times, and your advice and support have been invaluable to me. There were many times when I felt discouraged and resigned before our supervision meetings. Yet, on each such occasion, I left the meeting feeling relieved and encouraged. I would also like to express my sincerest gratitude to Klas Wallenius, who has been my mentor at Saab. Without your support and commitment, this research project would never have been realised in the first place. Your advice and feedback have been highly important to my research and the writing of this thesis.

This research has been supported by my employer Saab AB, and I am very grateful and proud of the unique opportunity they have offered me. I would like to extend special thanks to Egils Sviestins at Saab, the co-author of one of my papers, who has given me valuable feedback on my research, including a draft of this thesis and all my published papers. Other Saab employees, who have given me feedback, and with whom I have had many interesting discussions, include Mattias Björkman, Thomas Kronhamn, Martin Smedberg, and Håkan Warston.

I am also very thankful to my current and former colleagues of the GSA research group at the University of Skövde: Christoffer Brax, who kindly provided the maritime dataset with labelled anomalies, and with whom I have co-authored two papers and had many interesting discussions; Lars Niklasson, who has been my co-advisor; Fredrik Johansson, Anders Dahlbom, Maria Riveiro, Tina Erlandsson, and Tove Helldin. I appreciate not only your feedback and our scientific discussions, but also our social intercourse during lunches, coffee breaks and other social activities. I would also like to acknowledge Volodya Vovk, at the Royal Holloway University of London, for giving me valuable feedback regarding my work on conformal anomaly detection, and Stefan Arnborg, at the Royal Institute of Technology in Stockholm, who first introduced me to the exciting research area of conformal prediction.

Last, but not least, I would like to thank my beloved Kajsa, for always being there and for putting up with, at times, an absent-minded researcher at home.

List of Publications

I. **Laxhammar, R.** (2014) Anomaly Detection. To appear in *Conformal Prediction for Reliable Machine Learning: Theory, Adaptations, and Applications* (eds. Balasubramanian, V., Ho, S. and Vovk, V.), Elsevier Insight Series, Morgan-Kaufmann Publishers, 2014.

II. **Laxhammar, R.** and Falkman, G. (2013) Inductive Conformal Anomaly Detection for Sequential Detection of Anomalous Sub-Trajectories. In *Annals of Mathematics and Artificial Intelligence: Special Issue on Conformal Prediction and its Applications*, September 2013.

III. **Laxhammar, R.** and Falkman, G. (2013) Online Learning and Sequential Anomaly Detection in Trajectories. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 99, PrePrints, September 2013.

IV. **Laxhammar, R.** and Falkman, G. (2012) Online Detection of Anomalous Sub-Trajectories - A Sliding Window Approach Based on Conformal Anomaly Detection and Local Outlier Factor. In *Proceedings of the AIAI 2012 Workshop on Conformal Prediction and its Applications*, volume 382 of *IFIP Advances in Information and Communication Technology*, pages 192–202, Berlin, 2012.

V. Holst, A., Bjurling, B., Ekman, J., Rudström, Å., Wallenius, K., **Laxhammar, R.**, Björkman, M., Fooladvandi, F. and Trönninger, J. (2012) A Joint Statistical and Symbolic Anomaly Detection System Increasing Performance in Maritime Surveillance. In *Proceedings of the 15th International Conference on Information Fusion*, Singapore, July 2012.

VI. **Laxhammar, R.** (2011) *Anomaly Detection in Trajectory Data for Surveillance Applications*. Licentiate Thesis, Örebro University, 2011.

VII. **Laxhammar, R.** and Falkman, G. (2011) Sequential Conformal Anomaly Detection in Trajectories based on Hausdorff Distance. In *Proceedings of the 14th International Conference on Information Fusion*, Chicago, USA, July 2011.

VIII. **Laxhammar, R.** and Falkman, G. (2010) Conformal Prediction for Distribution-Independent Anomaly Detection in Streaming Vessel Data. In *Proceedings of the First International Workshop on Novel Data Stream Pattern Mining Techniques* (ACM), Washington D.C., USA, July 2010.

IX. **Laxhammar, R.**, Falkman, G. and Sviestins, E. (2009) Anomaly Detection in Sea Traffic - a Comparison of the Gaussian Mixture Model and the Kernel Density Estimator. In *Proceedings of the 12th International Conference on Information Fusion*, Seattle, USA, July 2009.

X. Brax, C., Niklasson, L. and **Laxhammar, R.** (2009) An ensemble approach for increased anomaly detection performance in video surveillance data. In *Proceedings of the 12th International Conference on Information Fusion, Seattle*, USA, July 2009.

XI. Brax, C., **Laxhammar, R.** and Niklasson, L. (2008) Approaches for detecting behavioural anomalies in public areas using video surveillance data. In *Proceedings of SPIE Electro-Optical and Infrared Systems: Technology and Applications V*, Cardiff, Wales, September 2008.

XII. **Laxhammar, R.** (2008) Anomaly detection for sea surveillance. In *Proceedings of the 11th International Conference on Information Fusion*, Cologne, Germany, July 2008.

Abbreviations and Acronyms

AIS	Automatic identification system
AUC	Area under the receiver operating characteristics curve
CAD	Conformal anomaly detector
CP	Conformal prediction
DH-kNN-NCM	Directed Hausdorff k-nearest neighbours nonconformity measure
DTW	Dynamic time warping
ED	Euclidean distance
EM	Expectation-maximization
GMM	Gaussian mixture model
HD	Hausdorff distance
HMM	Hidden Markov model
ICAD	Inductive conformal anomaly detector
ICP	Inductive conformal prediction
IID	Independent and identically distributed
KDE	Kernel density estimation
LCSS	Longest common sub-sequence
LOF	Local outlier factor
MAP	Maximum a posterior
ML	Maximum likelihood
NCM	Nonconformity measure

pAUC	Partial area under the receiver operating characteristics curve
PDF	Probability density function
ROC	Receiver operating characteristics
SBAD	State-based anomaly detection
SHNN-CAD	Sequential Hausdorff nearest neighbours conformal anomaly detector
SOM	Self-organising map
SSTLO-ICAD	Sequential sub-trajectory local outlier inductive conformal anomaly detector
SSTLO-NCM	Sequential sub-trajectory local outlier nonconformity measure
SVM	Support Vector Machine

List of Symbols

δ_H	Undirected Hausdorff distance
$\overrightarrow{\delta_H}$	Directed Hausdorff distance
A	Nonconformity measure (NCM)
B	Multi-set
ϵ	Significance level
p	p-value
α	Nonconformity score
k	Number of nearest neighbours
w	Window (sub-trajectory) length

Contents

1 Introduction **1**
1.1 Research Question and Objectives 4
1.2 Research Methodology 4
1.3 Scientific Contributions 5
1.3.1 Anomaly Detection 5
1.3.2 Anomalous Trajectory Detection 6
1.3.3 Conformal Prediction 7
1.4 Publications 8
1.5 Thesis Outline 13

2 Background **15**
2.1 Anomaly Detection 15
2.1.1 General Aspects of Anomaly Detection 16
2.1.2 Statistical Anomaly Detection 20
2.1.3 Other Anomaly Detection Algorithms 25
2.2 Anomalous Trajectory Detection 28
2.2.1 Video Surveillance 29
2.2.2 Maritime Surveillance 33

3 The Research Gap **37**
3.1 Anomaly Detection 37
3.1.1 Inaccurate Statistical Models 37
3.1.2 Parameter-laden Algorithms 39
3.1.3 Ad-hoc Anomaly Thresholds and High False Alarm Rates . 39
3.2 Anomalous Trajectory Detection 41
3.2.1 Offline Learning 41
3.2.2 Offline Anomaly Detection 41
3.2.3 Insensitivity to Local Sub-Trajectory Anomalies 42
3.3 Identification of Key Properties 43

4 Conformal Anomaly Detection **45**
4.1 Preliminaries: Conformal Prediction 45
4.2 Conformal Prediction for Multi-Class Anomaly Detection 48
4.2.1 A Nonconformity Measure for Multi-class Anomaly Detection 48
4.3 Conformal Anomaly Detection 50
4.3.1 The Conformal Anomaly Detector 50
4.3.2 Offline and Online Conformal Anomaly Detection 52
4.3.3 Unsupervised and Semi-Supervised Conformal Anomaly Detection 52
4.3.4 Importance of the Nonconformity Measure 53
4.3.5 Tuning of the Anomaly Threshold 54
4.4 Inductive Conformal Anomaly Detection 54
4.4.1 Offline and Semi-Offline Inductive Conformal Anomaly Detection 55
4.4.2 Online Inductive Conformal Anomaly Detection 56
4.5 Summary 58

5 Sequential Detection of Anomalous Trajectories **59**
5.1 Sequential Conformal Anomaly Detection 59
5.1.1 Preliminaries: Hausdorff Distance 60
5.1.2 A Nonconformity Measure for Examples Represented as Sets of Points 60
5.1.3 The Sequential Hausdorff Nearest Neighbours Conformal Anomaly Detector 62
5.1.4 Discussion 65
5.2 Empirical Investigations 66
5.2.1 Accuracy of Trajectory Outlier Measures 67
5.2.2 Unsupervised Online Learning and Sequential Anomaly Detection Performance 70
5.2.3 Discussion 74
5.3 Summary 78

6 Sequential Detection of Anomalous Sub-Trajectories **81**
6.1 A Nonconformity Measure for the Sequential Detection of Local Anomalous Sub-Trajectories 81
6.1.1 Preliminaries: Local Outlier Factor 82
6.1.2 The Sequential Sub-Trajectory Local Outlier Nonconformity Measure 84
6.2 The Sequential Sub-Trajectory Local Outlier Inductive Conformal Anomaly Detector 90
6.3 Empirical Investigations in the Maritime Domain 90
6.3.1 Preliminary Investigations 91
6.3.2 Empirical Alarm Rate 97
6.3.3 Detection Delay: Simulated Anomalies 98
6.3.4 Classification Performance: Realistic Anomalies 105

6.4 Summary 129

7 Conclusion **133**
7.1 Contributions 133
7.1.1 Research Question 138
7.2 Future Work 141
7.3 Generalisation 142
7.4 Final Remarks 143

A Performance Results: Realistic Maritime Anomalies **145**
A.1 Circle and Land 146
A.2 Missed Turn 148
A.3 Unexpected Stop 150
A.4 Unusual Speed 152
A.5 Strange Manoeuvres 154
A.6 Large Vessel in Unusual Location 156
A.7 Unexpected Vessel Type/Behaviour 158
A.8 Overall Performance 160

References **162**

List of Figures

3.1 Estimated PDF for vessel position using GMM and EM 38
3.2 Illustration of global vs. local trajectory anomaly 43

4.1 Problem with previous k-nearest neighbours nonconformity measure for anomaly detection 49

5.1 Illustration of directed Hausdorff distance between two curves . . . 61
5.2 Plot of trajectories from the first synthetic dataset 68
5.3 Plot of trajectories from the first dataset of recorded video trajectories 69
5.4 Plot of trajectories from the second dataset of recorded video trajectories 70
5.5 Plot of trajectories from the second synthetic dataset 72
5.6 Plots of sensitivity and precision dependent on size of the training set during online learning and sequential anomaly detection 75
5.7 Plot of the mean detection delay dependent on the size of the training set during online learning and sequential anomaly detection . . 76
5.8 Histogram of the empirical alarm rate for the unsupervised online SHNN-CAD 77

6.1 Illustration of the local density problem 83
6.2 Illustration of the smoothening effect of the reachability distance . 85
6.3 Illustration of top three anomalous vessel trajectories according to SSTLO-NCM with $w = 6$ and $k = 5$ 93
6.4 Illustration of top three anomalous vessel trajectories according to SSTLO-NCM with $w = 6$ and $k = 2$ 94
6.5 Illustration of top three anomalous vessel trajectories according to SSTLO-NCM with $w = 61$ and $k = 5$ 95
6.6 Illustration of top three anomalous vessel trajectories according to SSTLO-NCM with $w = 61$ and $k = 2$ 96
6.7 Histograms of the empirical alarm rates for different parameter values of the semi-offline SSTLO-ICAD 99
6.8 Histograms of the empirical alarm rates for different parameter values of the alternative online SSTLO-ICAD 100

6.9 Plot of the accumulated nr of anomalous trajectories detected as a function of the nr of trajectories 101
6.10 Plot of extracted vessel trajectories from port area of Gothenburg . 102
6.11 Plot of simulated anomalous vessel trajectories in port area of Gothenburg 103
6.12 Geographical coverage of the dataset of recorded vessel trajectories and realistic anomalies. 109
6.13 Plot of vessel trajectories from training set and the normal test set 110
6.14 Plot of maritime anomalies: vessels that circle and land. 114
6.15 Plot of maritime anomalies: vessels that miss a turn. 115
6.16 Plot of maritime anomalies: unexpected stops. 116
6.17 Plot of a maritime anomalies: unusual speed 117
6.18 Plot of a maritime anomaly: strange manoeuvres. 118
6.19 Plot of maritime anomalies: large vessels in unusual locations. . . . 119
6.20 Plot of maritime anomalies: passenger vessels behaving strangely. . 120
6.21 Plot of maritime anomalies: tankers behaving strangely. 121
6.22 Example of ROC curves. 124

List of Tables

1.1 Overview: research objectives, publications, thesis chapters 14

5.1 Accuracy on the first dataset of synthetic trajectories 68
5.2 Accuracy on the second dataset of recorded video trajectories . . . 71
5.3 F_1-score and detection delay on the second dataset of synthetic trajectories 74

6.1 Mean detection delay on simulated anomalous vessel trajectories . 106
6.2 Median detection delay on simulated anomalous vessel trajectories 107
6.3 Overview of the different maritime anomaly classes. 112
6.4 Average classification performance of SSTLO-NCM on set of maritime anomalies for different values of k. 126
6.5 Average classification performance of SSTLO-NCM on set of maritime anomalies for different sub-trajectory lengths. 126
6.6 pAUC for SSTLO-NCM and discords based on spatial trajectory representation. 127
6.7 pAUC for SSTLO-NCM and discords based on velocity vector trajectory representation. 127
6.8 Sensitivity and mean detection delay on each maritime anomaly class for SSTLO-NCM, discords and SBAD. 129

7.1 Overview of the identified key properties and the proposed and evaluated algorithms in the thesis. 139

List of Algorithms

4.1 Conformal predictor 46
4.2 The Conformal Anomaly Detector (CAD) 51
4.3 The Inductive Conformal Anomaly Detector (ICAD) 55
5.1 The Sequential Hausdorff Nearest Neighbours Conformal Anomaly Detector (SHNN-CAD) 64
6.1 K-d tree based sequential sub-trajectory local outlier nonconformity measure (SSTLO-NCM) 87
6.2 Sub-trajectory neighbourhood algorithm 88

Chapter 1

Introduction

Anomalous behaviour, also referred to as strange, suspicious, unusual, novel or unexpected behaviour, may indicate interesting objects or events in a wide variety of domains. An important example is surveillance in the military and civil security domains, where there is a clear trend towards more and more advanced sensor systems producing huge amounts of geo-spatial and temporal *trajectory data* from moving objects, including people, vehicles, vessels and aircraft. In *maritime surveillance,* anomalous vessel trajectories corresponding to unexpected stops, deviations from standard routes, speeding, traffic direction violations etc., may indicate threats and dangers related to smuggling, command of a vessel while drunk[1], collisions (Danish Maritime Authority, 2003), grounding (Swedish Maritime Safety Inspectorate, 2004), terrorism[2], hijacking[3], piracy[4] etc. In *video surveillance* of on-land activities, anomalous trajectories may indicate illegal and adverse activity related to personal assault, robbery, burglary, infrastructural sabotage etc. The timely detection of these relatively infrequent events is critical for enabling pro-active measures to be taken and requires constant analysis of all trajectories. This is typically a significant challenge for human analysts due to information overload, fatigue and inattention (Riveiro, 2011). At any one time in the Baltic sea, for example, there are typically 3000–4000 commercial vessels being monitored by only a few human analysts[5]. Hence, there is need of a system that supports the automated analysis of all collected trajectory data and alerts human analysts of potentially interesting targets. In this thesis, we are concerned with the problem of the automated detection of anomalous trajectories in surveillance applications. The main contribution of our work is the proposal, analysis and empirical investigations of algorithms appropriate for the online detection of

[1] http://www.swedishwire.com/economy/5738-drunk-captain-runs-aground-in-sweden
[2] http://www.globalsecurity.org/security/profiles/uss_cole_bombing.htm
[3] http://news.bbc.co.uk/2/hi/uk_news/8196640.stm
[4] http://en.wikipedia.org/wiki/Piracy_in_Somalia
[5] http://www.idg.se/2.1085/1.376546/sjobasis-battre-an-stalmannens-rontgensyn?utm_source=tip-friend&utm_medium=email

anomalous trajectories during surveillance operations.

In the research fields of *data fusion* (Hall and McMullen, 2004) and *data mining* (Tan et al., 2006), various computational methods that support surveillance analysts in detecting abnormal and interesting behaviour have been proposed (Riveiro, 2011; Brax, 2011). According to Rhodes (2009), "timely identification and assessment of anomalous activity within an area of interest is an increasingly important capability — one that falls under the enhanced situation awareness objective of higher-level [data] fusion". *Signature-based methods* (Patcha and Park, 2007) assume that specific models for interesting behaviour, such as rules and templates, can be defined a priori and used for automated pattern recognition in new data. Such models are constructed according to two main knowledge extraction strategies: incorporation of human *expert knowledge* regarding suspicious and interesting behaviours (Edlund et al., 2006; Dahlbom et al., 2009) and *supervised learning* from *labelled data* of interesting and non-interesting behaviour (Fooladvandi et al., 2009) using methods from *machine learning* (Mitchell, 1997). Of particular interest is a general machine learning method called *conformal prediction* (Vovk et al., 2005), which provides reliable and well-calibrated measures of *confidence* in the predictions made by traditional machine learning algorithms. The system's confidence in predicting that a particular behaviour is interesting, or non-interesting, may be important information for determining whether the situation needs to be further examined by a human analyst. However, it has been argued that signature-based methods are not sufficient, since accurate models of all possible behaviour of interest cannot be acquired in practice (Patcha and Park, 2007). This is mainly due to the lack of expert knowledge and annotated data that covers the full spectrum of interesting behaviour, as well as practical engineering difficulties in encoding expert knowledge. According to Kraiman et al. (2002), "there is a need for robust, non-template-based processing techniques that monitor large tracking and surveillance datasets". It has also been argued that the analysis should be focused towards detecting "strange" and abnormal patterns that deviate from the expected or "normal" patterns. Such approaches, typically referred to as *anomaly*, *novelty* or *outlier detection* methods (Chandola et al., 2009), benefit from the fact that there is usually large amounts of historical data, which can be exploited to learning normal behaviour. In contrast to signature-based approaches, these methods require less prior knowledge regarding the abnormal or interesting behaviour. Nevertheless, signature-based methods and anomaly detection methods complement each other and should be combined in order to achieve improved overall performance (Holst et al., 2012). The remainder of the thesis is only concerned with anomaly detection methods for detecting anomalous trajectories.

A key issue when designing an anomaly detector is how to represent the data in which anomalies can be found. Some anomaly detection methods assume that data, in our case trajectories, is represented as points in a fixed-dimensional *feature space*. This implies that a fixed number of features, e.g., the position at a fixed number of points in time, has to be extracted from each trajectory or trajectory

segment. Other methods only assume that a *similarity* or *dissimilarity measure* is defined for pairs of trajectories or trajectory segments. The choice of features or similarity or dissimilarity measure essentially determines the type of anomalies that can be detected and is therefore very important.

Most of the previously proposed algorithms are essentially designed for *offline anomaly detection* in trajectory databases. Hence, they typically assume that the *complete trajectory* has been observed before it is classified as anomalous or not. This is a limitation in surveillance applications since it delays anomaly alarms and, thus, the ability to react to impending events. In contrast, algorithms for *sequential anomaly detection* allow detection in *incomplete trajectories* (Morris and Trivedi, 2008a), e.g., real-time detection of anomalous trajectories as they evolve. Moreover, learning in previously proposed algorithms is usually *offline,* i.e., fixed model parameters and thresholds are estimated or tuned once based on a batch of historical data. Thus, once the anomaly detector is deployed in the current application, it cannot leverage on new training data that is collected incrementally. The need for *online learning* in the surveillance domain has been identified and discussed by Piciarelli and Foresti (2006) and Rhodes et al. (2007).

Generally, anomaly detection algorithms require the careful setting of multiple application specific parameters in order to achieve good performance; anomalous trajectory detection is no exception to this. According to Markou and Singh (2003a), "an [anomaly] detection method should aim to minimise the number of parameters that are user set". Indeed, Keogh et al. (2007) argue that most data mining algorithms are more or less *parameter-laden*, which increases the risk of overfitting the model to the training data and makes implementing the algorithms more difficult. A central parameter of most anomaly detection algorithms is the *anomaly threshold,* which regulates the trade-off between the *sensitivity* to true and interesting anomalies, and the rate of *false alarms*, i.e., detected anomalies that correspond to normal and uninteresting data. While achieving high sensitivity is important, it has been argued that the false alarm rate is the critical factor for the performance of an anomaly detection system (Axelsson, 2000; Riveiro, 2011). Indeed, Riveiro (2011) argues that a high false alarm rate "may become a nuisance for operators who might turn [the anomaly detector] off". In many cases, an application specific distance or density based threshold is used to decide whether new data is anomalous or not. The thresholds are often not normalised and seem to lack intuition, and the procedures for setting the thresholds seem to be more or less ad-hoc. This increases the risk of high false alarm rates due to poorly calibrated thresholds.

Various algorithms have previously been proposed for anomalous trajectory detection in surveillance applications, such as video surveillance and maritime surveillance. Yet, we argue that these algorithms typically suffer from drawbacks related to one or more of the issues discussed above: they are designed for offline anomaly detection in trajectory databases, they are parameter-laden and they may suffer from high false alarm rates due to ad-hoc and poorly calibrated anomaly thresholds. In this thesis, we aim to bridge this gap in the research.

1.1 Research Question and Objectives

Following the argumentation in the last paragraph above, the overall research question of this thesis is formulated as follows:

Research Question: *What are appropriate algorithms for the detection of anomalous trajectories in surveillance applications?*

In order to answer this research question, the following set of objectives are identified:

$\mathcal{O}1$ Review and analyse previously proposed algorithms for anomaly detection in general and anomalous trajectory detection in particular.

$\mathcal{O}2$ Identify important and desirable properties of algorithms for anomalous trajectory detection in surveillance applications.

$\mathcal{O}3$ Propose and analyse new or updated algorithms that are more appropriate for anomalous trajectory detection in surveillance applications.

$\mathcal{O}4$ Identify appropriate performance measures for evaluating algorithms for anomalous trajectory detection in surveillance applications.

$\mathcal{O}5$ Implement the proposed algorithms and evaluate the performance of new and previous algorithms on synthetic and real trajectory datasets.

1.2 Research Methodology

In order to address the research objectives stated in Section 1.1, we adopt a number of different research methods.

Starting with $\mathcal{O}1$ and $\mathcal{O}2$, we perform two *literature reviews* and *literature analyses* (Berndtsson et al., 2002). The first is focused on algorithms for anomaly detection in general, while the second is focused on algorithms for anomalous trajectory in surveillance applications. An important issue when undertaking a literature analysis is how to systematically search for previously published work that is relevant for the current research (Berndtsson et al., 2002). Therefore, we adopt a number of different search strategies based on:

- Searching the internet in general, and in scientific databases in particular, using different combinations of selected keywords, such as "anomaly detection", "surveillance", "trajectory data" etc.
- Browsing journals and annual proceedings of selected conferences, choosing a subset of papers for further reading based on title, abstract and/or keywords.
- Backwards citation chaining from relevant papers previously found.

In order to enable the evaluation of the proposed algorithms ($\mathcal{O}5$), we develop an *implementation* (Berndtsson et al., 2002) for each of them in MATLAB[6] or Java™.

[6] http://www.mathworks.com

An important issue when developing an implementation of an algorithm is to ensure its *reliability* (Berndtsson et al., 2002), i.e., the robustness and correctness of the implementation. The algorithms implemented in MATLAB are largely based on functions and subroutines from the standard MATLAB library and the official Statistics toolbox[7] by MathWorks, which strengthens the reliability of the implementations.

In order to evaluate the proposed algorithms according to the identified performance measures ($\mathcal{O}5$) , we perform a series of *experiments* (Berndtsson et al., 2002) using the implementations of the corresponding algorithms and different trajectory datasets. Machine learning algorithms are typically evaluated by measuring their classification performance on a *labelled* dataset, which is often publicly available[8]. However, public availability of real world surveillance data is very limited due to confidentiality and proprietary rights. Moreover, a real-world surveillance dataset is typically *unlabelled*, i.e., there is no information regarding which part of the data is actually normal or anomalous, and includes very few, if any, true anomalies; this may threaten the validity and reliability of experimental results and conclusions. An alternative to real world data is to *simulate* data, which has the advantages that 1) the resulting data is labelled and 2) the *reproducibility* of experiments is enhanced, under the assumption that the data can more easily be made publicly available (downloaded) or accurately re-created given the parameters and details of the simulation process. The main drawback of using simulated data is that validity or generalisability may be questioned if, e.g., the simulated anomalies do not reflect the actual anomalies encountered in a real surveillance application. That is, we are actually measuring performance on a different problem (detect simulated anomalies) than the one we aim to measure (detect true anomalies). In this thesis, we evaluate the performance of the proposed algorithms using both real and simulated trajectories. Moreover, we reproduce a number of previously published experiments on labelled datasets.

1.3 Scientific Contributions

This thesis is based on the author's previously published work by the author (Section 1.4), which has been further extended with new empirical results. The main scientific contributions of the thesis, listed below, are organised according to the research areas *anomaly detection* in general, *anomalous trajectory detection* and *conformal prediction*:

1.3.1 Anomaly Detection

- Compilation of key properties of algorithms for anomaly detection and corresponding limitations of previous algorithms (Section 3.1).

[7] http://www.mathworks.com/products/statistics/

[8] see, e.g., the UCI Machine Learning Repository: http://archive.ics.uci.edu/ml/

Various strengths and weaknesses of different approaches to anomaly detection have previously been discussed by others (see, e.g., Chandola et al. (2009)). In this thesis, we complement previous surveys by identifying a number of key properties of any anomaly detector and the corresponding limitations of previously proposed algorithms in the literature. While these key properties have been separately identified by previous authors (see, e.g., Keogh et al., 2007; Axelsson, 2000; Riveiro, 2011), we argue that the compilation and overall discussion of all these properties is a contribution in itself to the field of anomaly detection.

- Proposal, analysis and discussion of a novel algorithmic framework, known as *conformal anomaly detection*, with a unique set of properties that addresses the limitations of previous approaches (Sections 4.3 and 4.4).

  The conformal anomaly detection framework is one of the main contributions of the thesis. The framework addresses the limitations of previous approaches above and supports both unsupervised and semi-supervised, offline and online anomaly detection.

1.3.2 Anomalous Trajectory Detection

- Compilation of key properties of algorithms for anomalous trajectory detection and the corresponding limitations of previous work (Section 3.2).

  In this thesis, we identify and discuss key properties of any algorithm for anomalous trajectory detection. These key properties have also been been identified by previous authors (Lee et al., 2008; Piciarelli and Foresti, 2006); nevertheless, the compilation and overall discussion of these properties is a contribution in itself within the field of anomalous trajectory detection.

- Proposal and analysis of algorithms for the *sequential* detection of anomalous trajectories that address the above limitations (Sections 5.1.3 and 6.1).

  One of the main contributions of the thesis concerns the proposal and analysis of two algorithms for online learning and the sequential detection of anomalous trajectories and sub-trajectories, respectively.

- Results and discussions regarding the classification performance on labelled datasets of real and synthetic trajectories in the domains of video surveillance (Section 5.2) and maritime surveillance (Section 6.3).

  We implement the algorithms proposed for anomalous trajectory detection and evaluate their classification performance on labelled datasets. This is done by reproducing previously published experiments and designing new experiments. For comparative purposes, we also implement the discords algorithm (Keogh et al., 2005) and investigate its performance.

1.3.3 Conformal Prediction

- Extension of the conformal prediction framework for solving the anomaly detection problem (*cf.* Section 1.3.1) (Chapter 4).

  The conformal prediction framework was originally proposed to provide reliable measures of confidence during supervised learning and prediction (Vovk et al., 2005). In this thesis, we have extended conformal prediction to the general problem of semi-supervised and unsupervised anomaly detection. We have a described how any conformal predictor can be adopted for multi-class anomaly detection, including a discussion of its key properties. Moreover, we have proposed the *conformal anomaly detector* (CAD) and the *inductive conformal anomaly detector* (ICAD), which are two general anomaly detection algorithms based on conformal prediction (Vovk et al., 2005) and *inductive* conformal prediction (Vovk et al., 2005), respectively.

- Proposal and analysis of a novel nonconformity measure (NCM) based on the Hausdorff distance (Alt et al., 1995) (Section 5.1.2).

  The NCM is the main design parameter in conformal prediction and conformal anomaly detection. Previously proposed NCMs (Vovk et al., 2005) assume that each example is represented as a vector in a fixed-dimensional feature space. This may be a limitation in applications where examples are naturally represented as sequences or sets of data points of varying length or size, respectively. In the thesis, we propose the *directed Hausdorff k-nearest neighbours nonconformity measure* (DH-kNN-NCM), which is a novel NCM designed for examples that are represented as sets of points. One of the key properties of DH-kNN-NCM is that it allows for sequential calculation of the preliminary nonconformity score for an incomplete example, i.e., when only a subset of the data points or feature values is observed. In particular, we prove that the sequence of preliminary nonconformity scores increases monotonically as the example is updated sequentially with more data points. This property will ensure that the prediction sets of any conformal predictor based on DH-kNN-NCM will remain valid during the sequential update of a new example.

- Proposal and analysis of a novel NCM for trajectory data based on local outlier factor (LOF) (Breunig et al., 2000) (Section 6.1.2).

  As discussed above, previous NCMs are not designed for examples represented as sequences of points that are updated sequentially, such as trajectories. The DH-kNN-NCM above addresses this limitation, but it is not sensitive to local anomalous sub-trajectories. Therefore, we propose the *sequential sub-trajectory local outlier nonconformity measure* (SSTLO-NCM) for the sequential detection of anomalous sub-trajectories.

1.4 Publications

The following list provides a summary of the author's peer-reviewed publications and how they contribute to the thesis. The publications are divided into those of high relevance and those of less relevance for the thesis. An overview of which publications contribute to which objectives can be found in Table 1.1.

Publications of High Relevance for the Thesis

1. **Laxhammar, R.** (2008) Anomaly detection for sea surveillance. In *Proceedings of the 11th International Conference on Information Fusion*, Cologne, Germany, July 2008.

   This paper presents one of the first algorithms and empirical results that have been published in the literature on statistical anomaly detection in vessel traffic. In this approach, the position and velocity vector of vessels is statistically modelled using a cell-based *Gaussian mixture model* (GMM) (Mitchell, 1997), whose parameters are estimated using an *expectation-maximisation* algorithm (Verbeek et al., 2003). Empirical results from an explorative investigation on an unlabelled dataset of recorded vessel trajectories demonstrates the type of anomalous vessel behaviour that can be detected by the algorithm. The paper mainly contributes to the background of the thesis.

2. **Laxhammar, R.**, Falkman, G. and Sviestins, E. (2009) Anomaly Detection in Sea Traffic - a Comparison of the Gaussian Mixture Model and the Kernel Density Estimator. In *Proceedings of the 12th International Conference on Information Fusion*, Seattle, USA, July 2009.

   One of the main objectives of this paper is to empirically investigate whether the *kernel density estimator* (KDE) (Silverman, 1986) is more appropriate than GMM (Publication 1 above) for the statistical modelling of the position-velocity vector during sequential anomaly detection in vessel trajectories. To this end, a new performance measure in the context of anomalous trajectory detection is introduced, known as the *detection delay*, which is defined as the expected number of observed data points prior to the successful detection of an abnormal trajectory. Empirical investigations are performed on a dataset of recorded vessel trajectories, which are assumed to be normal, and a set of simulated trajectories labelled abnormal. Results from preliminary experiments confirm the hypothesis that the KDE more accurately captures finer details of the normal data. Nevertheless, the detection delay for GMM and KDE on the abnormal trajectories is approximately the same and, surprisingly, high in absolute terms. Hence, it is concluded that the cell-based statistical modelling of the position-velocity vector is sub-optimal for sequential anomaly detection in vessel traffic. The paper mainly contributes to $\mathcal{O}4$ and $\mathcal{O}5$.

3. **Laxhammar, R.** and Falkman, G. (2010) Conformal Prediction for Distribution-Independent Anomaly Detection in Streaming Vessel Data, In *Proceedings of the First International Workshop on Novel Data Stream Pattern Mining Techniques*, Washington D.C., USA, July 2010.

   Conformal prediction (CP) is a general method for supervised learning and prediction with valid confidence (Vovk et al., 2005). Given a specified significance level $\epsilon \in (0, 1)$, a conformal predictor outputs a prediction set that includes the true label or value with probability at least $1 - \epsilon$. In this paper, we propose and discuss a novel application of CP for multi-class anomaly detection based on the idea of interpreting erroneous prediction sets as anomalies. One of the key properties of this approach to anomaly detection is that ϵ, which corresponds to a normalised and application independent anomaly threshold, can be interpreted as an upper bound of the probability of a false alarm. The main design parameter in CP is the *nonconformity measure* (NCM). We adapt a previously proposed NCM (Vovk et al., 2005) based on distance to k-nearest neighbours, making it more suitable for multi-class anomaly detection applications. As an application, we consider anomaly detection in vessel trajectories, where vessel class is predicted based on the current position-velocity vector. If the prediction set does not include the actual class (reported by the vessel itself), the vessel is classified as anomalous. Experiments are performed on a set of recorded vessel trajectories, which are assumed to be normal, and a set of simulated anomalous trajectories. Results show that the detection delay on the anomalous trajectories is considerably less for the CP-based anomaly detector compared to the cell-based GMM and KDE (Publication 2). The paper contributes to $\mathcal{O}1$–$\mathcal{O}3$ and $\mathcal{O}5$.

4. **Laxhammar, R.** and Falkman, G. (2011) Sequential Conformal Anomaly Detection in Trajectories based on Hausdorff Distance. In *Proceedings of the 14th International Conference on Information Fusion*, Chicago, USA, July 2011.

   In this paper, we extend our initial work on CP and anomaly detection (Publication 3) and introduce CAD. We discuss theoretical properties of CAD, including its well-calibrated alarm rate; if the training set and new normal data are IID, the rate of normal examples erroneously classified as anomalous is expected to be close to the specified anomaly threshold ϵ. Analogously to a conformal predictor, the NCM is a central parameter in CAD. We propose a novel k-nearest neighbours NCM based on the *directed Hausdorff distance* that in contrast to the previous NCM allows data to be represented as sets or sequences of different sizes or lengths, respectively. We describe and analyse the complexity of a sequential anomaly detection algorithm based on CAD and the proposed NCM. The classification accuracy of the algorithm is empirically investigated by reproducing two previously published experiments on two public datasets of labelled video trajectories. We also carry out some preliminary experiments investigating the detection delay and how

sensitivity to labelled anomalies increases during online learning. The paper contributes to $\mathcal{O}1$–$\mathcal{O}5$.

5. **Laxhammar, R.** (2011) *Anomaly Detection in Trajectory Data for Surveillance Applications*. Licentiate Thesis, Örebro University, 2011.

   This monograph constitutes the author's licentiate thesis and is based on the results from Publications 1–4. It extends the previous work by providing:

   - a more extensive background on the problem of anomalous trajectory detection;
   - additional and previously unpublished empirical results;
   - more extensive analysis and discussions of the results from the previous work;
   - more general conclusions.

   The work contributes to $\mathcal{O}1$–$\mathcal{O}5$ and to the background.

6. **Laxhammar, R.** and Falkman, G. (2012) Online Detection of Anomalous Sub-Trajectories - A Sliding Window Approach Based on Conformal Anomaly Detection and Local Outlier Factor. In *Proceedings of the AIAI 2012 Workshop on Conformal Prediction and its Applications*, volume 382 of *IFIP Advances in Information and Communication Technology*, pages 192-202, Springer Berlin Heidelberg, 2012.

   This paper is concerned with the problem of the sequential detection of local anomalous sub-trajectories. The main result and contribution of the paper is the proposal and discussion of a novel NCM designed to detect local anomalous sub-trajectories in the CAD framework. This NCM is based on Local Outlier Factor (Breunig et al., 2000) and is therefore sensitive to local anomalies. The paper contributes to $\mathcal{O}1$–$\mathcal{O}3$.

7. **Laxhammar, R.** and Falkman, G. (2013) Online Learning and Sequential Anomaly Detection in Trajectories. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 99, PrePrints, September 2013.

   This journal article deepens and expands on the previous results in Publication 4. More specially, we:

   - Present an extensive discussion of limitations of previous algorithms for anomaly detection in general and the detection of anomalous trajectories in particular.
   - Further formalise CAD and the concepts of well-calibrated alarm rate and offline vs. online and unsupervised vs. semi-supervised conformal anomaly detection.

- Formalise DH-kNN-NCM prove a theorem regarding the monotonicity of the p-values during the sequential update of an incomplete trajectory. This theorem implies that the alarm rate of CAD is well-calibrated during sequential anomaly detection in incomplete trajectories.
- Re-introduce our algorithm for the sequential detection of anomalous trajectories (Publication 4) as the *sequential Hausdorff nearest neighbour conformal anomaly detector* (SHNN-CAD). This includes a more detailed description of the algorithm, a discussion of its properties, and the implementation and analysis of its time complexity.
- Extend the empirical investigation of DH-kNN-NCM and SHNN-CAD by including new experiments on new trajectory datasets and comparing it with the *discords* algorithm (Keogh et al., 2005).

The article contributes to $\mathcal{O}1$–$\mathcal{O}5$.

8. **Laxhammar, R.** and Falkman, G. (2013) Inductive Conformal Anomaly Detection for Sequential Detection of Anomalous Sub-Trajectories. In *Annals of Mathematics and Artificial Intelligence: Special Issue on Conformal Prediction and its Applications*, September 2013.

   This article is concerned with the problem of online learning and sequential detection of local anomalous sub-trajectories. The main contributions of the article are two-fold. The first is the proposal and analysis of ICAD, which is a generalisation of CAD based on the concept of inductive conformal predictors (Vovk et al., 2005). The main advantage of ICAD compared to CAD is the improved computational efficiency. The second contribution of this paper concerns the further formalisation, implementation and empirical investigation of the NCM proposed in Publication 6 for the sequential detection of anomalous sub-trajectories. The results from the empirical investigations on an unlabelled set of vessel trajectories illustrate the most anomalous trajectories detected for different parameter values of the NCM, and confirm that the empirical alarm rate for ICAD is indeed well-calibrated. The article contributes to $\mathcal{O}1$–$\mathcal{O}3$ and $\mathcal{O}5$.

9. **Laxhammar, R.** (2014) Anomaly Detection. To appear in *Conformal Prediction for Reliable Machine Learning: Theory, Adaptations, and Applications* (eds. Balasubramanian, V., Ho, S. and Vovk, V.), Elsevier Insight Series, Morgan-Kaufmann Publishers, 2014.

   This invited book chapter, which is mainly based on the results from Publications 7 and 8, provides a self-contained presentation of the framework of conformal anomaly detection. This includes the presentation and discussion of CAD and ICAD, their key properties and how they operate in the unsupervised or semi-supervised mode, and the offline, semi-offline or online mode. The book chapter also presents DH-kNN-NCM and SHNN-CAD and how it can be used for the application of sequential anomaly detection in the

trajectories. Empirical results and discussions of selected experiments from Publication 7 are provided. The book chapter mainly contributes to $\mathcal{O}3$ and $\mathcal{O}5$.

Further Publications Related to the Thesis

10. Brax, C., **Laxhammar, R.** and Niklasson, L. (2008) Approaches for detecting behavioural anomalies in public areas using video surveillance data. *In Proceedings of SPIE Electro-Optical and Infrared Systems: Technology and Applications V*, Cardiff, Wales, September 2008.

    In this paper, an extended version of the cell-based GMM algorithm (Publication 1) and a histogram-based anomaly detection algorithm (Brax et al., 2008) are both evaluated on a labelled set of recorded video trajectories. The author's main contribution is the development, implementation and evaluation of the extended cell-based GMM algorithm. The paper mainly contributes to the background of the thesis.

11. Brax, C., Niklasson, L. and **Laxhammar, R.** (2009) An ensemble approach for increased anomaly detection performance in video surveillance data. In *Proceedings of the 12th International Conference on Information Fusion*, Seattle, USA, July 2009.

    This paper extends the work in Publication 10 above by considering a more crowded scene that involves more complex behaviour. Similar to the previous paper, an updated version of the cell-based GMM (Publication 1) and the histogram-based anomaly detection algorithm (Brax et al., 2008) are evaluated on another dataset of labelled trajectories extracted from recorded video data. In addition to evaluating the classification performance of each individual anomaly detector, the combination of the two detectors is also evaluated. Results show that a simple combination achieves better classification performance than any of the two detectors by themselves. Similar to Publication 10, the author's main contribution is the development, implementation and evaluation of the extended cell-based GMM algorithm. The paper mainly contributes to the background of the thesis.

12. Holst, A., Bjurling, B., Ekman, J., Rudström, Å., Wallenius, K., **Laxhammar, R.**, Björkman, M., Fooladvandi, F. and Trönninger, J. (2012) A Joint Statistical and Symbolic Anomaly Detection System Increasing Performance in Maritime Surveillance. In *Proceedings of the 15th International Conference on Information Fusion*, Singapore, July 2012.

    Algorithms for automated surveillance and detection of abnormal behaviour can generally be categorised as *symbolic* or *statistical*. Symbolic methods typically involve defining rules for abnormal behaviour based on expert knowledge, while statistical methods leverage on historical data for detecting abnormal behaviour that deviates from the normal or expected pattern. This

paper discusses why these two approaches complement each other and how they can be combined in order to maximise their synergies in the maritime domain. The author's main contribution concerns the co-authoring of the background section of the paper and discussions with the first authors regarding some of the ideas in the remaining part of the paper.

1.5 Thesis Outline

In the following, we provide an outline and brief description of each of the remaining chapters of the thesis. An overview of how the chapters relate to the research objectives and thesis publications can be found in Table 1.1.

Chapter 2: Background. This chapter provides a background to the subject of the thesis and introduces the terminology and basic concepts that are needed. The first part is concerned with anomaly detection in general and the second part is focused on anomalous trajectory detection in the video surveillance and maritime surveillance.

Chapter 3: The Research Gap. This chapter discusses limitations of previous algorithms for anomaly detection in general and anomalous trajectory detection in particular. Based on these limitation, a set of key properties of new or updated algorithms for anomalous trajectory detection are identified. These properties constitute the main gap in the research that is addressed in the thesis.

Chapter 4: Conformal Anomaly Detection. Motivated by the discussions in the previous chapter, chapter 4 introduces and discusses the framework of conformal anomaly detection. We start by introducing conformal prediction (Vovk et al., 2005) before describing, at an informal level, how a conformal predictor can be adopted for anomaly detection. In the remainder of the chapter, we propose, analyse and discuss CAD and ICAD, which are general algorithms for anomaly detection based on the conformal anomaly detection framework.

Chapter 5: Sequential Detection of Anomalous Trajectories. This chapter is concerned with the adoption of CAD for the sequential detection of anomalous trajectories. The main design parameter of CAD is the NCM. Hence, key properties of a NCM for detecting anomalous trajectories are first discussed. This is followed by the proposal of DH-kNN-NCM. Based on DH-kNN-NCM, we then propose SHNN-CAD for online learning and the sequential detection of anomalous trajectories. Theoretical properties of SHNN-CAD are analysed and discussed. Finally, we investigate and discuss the classification performance of the proposed algorithms and other previous algorithms on a number of public datasets.

Chapter 6: Sequential Detection of Anomalous Sub-Trajectories. This chapter is concerned with the problem of the sequential detection of local anomalous sub-trajectories. Similar to the previous chapter, we identify key properties of a NCM for this problem. This is followed by an introduction of LOF and the proposal SSTLO-NCM, which based on LOF. An algorithm for SSTLO-NCM is also presented and analysed. Empirical investigations of SSTLO-NCM and the *sequential sub-trajectory local outlier inductive conformal anomaly detector* (SSTLO-ICAD) are carried out in the maritime domain. In particular, we investigate the classification performance for different parameter setups of SSTLO-NCM on a dataset of realistic maritime anomalies.

Chapter 7: Conclusion. The thesis is concluded with a discussion of the main results and how they contribute to the research objectives. Future work and a further generalisation of the thesis results are also discussed.

Appendix A: Performance Results: Realistic Maritime Anomalies. The thesis appendix includes a number of tables that present detailed results for the classification performance of SSTLO-NCM on the maritime dataset with realistic anomalies in Chapter 6.

Table 1.1: Overview of research objectives, publications and thesis chapters.

Objectives	Publications	Thesis Chapters
$\mathcal{O}1$	3–8	2 and 3
$\mathcal{O}2$	3–8	2 and 3
$\mathcal{O}3$	3–8	4–6
$\mathcal{O}4$	2, 4, 5 and 7	5 and 6
$\mathcal{O}5$	2–5 and 7–9	5 and 6

Chapter 2

Background

This chapter presents a background to the subject of the thesis and introduces the terminology and basic concepts required. The first part (Section 2.1) introduces anomaly detection and provides a survey over different aspects of the problem and common algorithms. The second part concerns previous work related to anomalous trajectory detection (Section 2.2).

2.1 Anomaly Detection

Anomaly detection has been identified as an important technique for detecting critical events in a wide range of data rich domains where a majority of the data is considered "normal" and uninteresting (Latecki et al., 2007; Chandola et al., 2009). Yet, it is a rather fuzzy concept and domain experts may have different notions of what constitutes an *anomaly* (Roy, 2008). Common synonyms to anomaly include *outlier, novelty, rarity*, *abnormality*, while other related words to anomaly include *deviating*, *unexpected, suspicious*, *interesting* etc. In the academic world, more or less similar definitions of anomaly detection and the closely related concepts *outlier detection* and *novelty detection* have been proposed by different authors with various backgrounds and application areas. However, the methods and algorithms used in practice are often the same (Chandola et al., 2009).

In the statistical community, the concepts *outlier* and *outlier detection* have been known for quite a long time. Barnett and Lewis (1994) define an outlier in a dataset to be "an observation (or subset of observations) which appears to be inconsistent with the remainder of that set of data". A similar definition of an outlier was given by Hawkins (1980):

> [An outlier is] an observation that deviates so much from other observations as to arouse suspicion that it was generated by a different mechanism.

This mechanism is usually assumed to follow a stationary probability distribution. Hence, outlier detection essentially involves determining whether or not a particular observation has been generated according to the same distribution as the remaining observations. Traditionally, outlier detection in the statistical community has been used to clean datasets before statistical models are fitted; outliers are considered noise and are removed to improve the quality of the statistical models.

In the data mining community, "anomalies are patterns in data that do not conform to a well-defined notion of normal behaviour" (Chandola et al., 2009). Often, the notion of normal behaviour is captured by a *normalcy model*, which is induced from training data. According to Portnoy et al. (2001), "anomaly detection approaches build models of normal data and then attempts to detect deviations from the normal model in observed data". In contrast to traditional statistical applications, data mining applications are often interested in the anomalous observations *per se*, since they may correspond to interesting and important events. Traditional applications of anomaly detection in data mining include *fraud detection* in commercial domains (Chandola et al., 2009), *intrusion detection* in network security (Portnoy et al., 2001), and *fault detection* in industrial domains (Chandola et al., 2009). Common to all these applications is that patterns of interesting behaviour are difficult, if not impossible, to define *a priori*, due to limited knowledge and lack of data.

According to Ekman and Holst (2008), "anomaly detection says nothing about the detection approach and it actually says nothing about what to detect". Indeed, anomaly detection, as it is usually defined, refers to a process that aims to *detect something*; yet it says nothing in particular about *what* to detect. This means that the interpretation and impact of an anomaly is undefined within the scope of the anomaly detector. One could argue that the definition of an anomaly is always relative to a specific model or dataset. Therefore, it is a relative concept rather than an absolute truth; something that appears to be deviating relative to a statistical model, or strange to a human with limited domain knowledge, may be fully understandable and predictable to, e.g., some other model or human domain expert.

2.1.1 General Aspects of Anomaly Detection

In a recent survey by Chandola et al. (2009), a number of general aspects of the anomaly detection problem are discussed. This section reviews these and other aspects related to anomaly detection.

Input Data

Similar to other data mining algorithms, most anomaly detection algorithms assume that the basic input is in the form of a *data point*, also referred to as *example, data instance*, *feature vector, observation, object, pattern,* etc. The data point can be univariate or multivariate, but usually has a fixed number of *feature*s, also referred to as *attributes* or *variable*s. These can be a mix of binary, categorical, or

continuous values. However, some methods do not require explicit data points as input; instead, pairwise distances or similarities between data points are provided in the form of a distance or similarity matrix (Chandola et al., 2009). Another aspect of the input is the relationship between different data points, which in many cases is of a spatial or temporal nature. Trajectory data is an example of time-series, where data points are temporally ordered. However, most anomaly detection techniques explicitly or implicitly assume that there is no relationship between different data points, i.e., that they are independent of each other (Chandola et al., 2009).

Most applications of anomaly detection involve a *feature extraction* process; this corresponds to preprocessing the raw input data and extracting relevant features. In the context of moving object surveillance, such features may be current speed and location of an object, its size and previously visited locations. The choice of an appropriate *feature model* is critical in anomaly detection applications, since it essentially determines the character of detected anomalies. If, for example, we only consider the position feature of an object, it will be hard, if not impossible, to detect anomalies related to the object's speed, whether it is low or high, assuming that speed is more or less independent of position. If inappropriate features are selected, the resulting anomalies may be of little or no interest. Thus, features should be selected carefully and based on available domain knowledge of how interesting anomalies manifest themselves.

Output

There are generally two types of output from an anomaly detector; *scores* and *labels* (Chandola et al., 2009). Scoring techniques assign an anomaly or outlier score to each input data point, where the score value reflects the degree to which the corresponding data point is considered anomalous. Output is usually a list of anomalous data points that are sorted according to their anomaly score. Such a list may include the top-k anomalies, or a variable number of anomalies that have a score above a predefined *anomaly threshold*. Labelling techniques, on the other hand, output a label for each input data point, usually *normal* or *anomalous*. Such techniques may also output the corresponding anomaly score, confidence or probability associated with the label. More details regarding the output of different algorithms are discussed in Sections 2.1.2 and 2.1.3 below.

Supervised, Unsupervised and Semi-Supervised Anomaly Detection

From an application owners point of view, anomaly detection may be considered as a binary classification problem, where each data point either belongs to a *normal class* or an *abnormal class*. The true class of a data point is typically determined by a domain expert, on the basis of his or her experience of what is normal or uninteresting, and what is abnormal and therefore interesting. Assuming that a training set of data points from both the normal and abnormal class is available, where each data point is annotated by a *label* indicating whether it is normal or abnormal,

supervised learning techniques (Mitchell, 1997) are applicable. In practice, however, the collection and annotation of a labelled training set for supervised learning is typically not feasible, due to the rarity and diversity of abnormal data points. Nevertheless, usually there are large amounts of *unlabelled* data available, which is assumed to only include a minority, if any, of abnormal data points. Thus, many anomaly detection algorithms adopt unsupervised learning techniques (Mitchell, 1997), such as density estimation or clustering, to find the most anomalous data points in a set of unlabelled data. These approaches are referred to as *unsupervised anomaly detection* (Chandola et al., 2009). In some cases, normal training data is available, which can be used to learn *one-class anomaly detectors* or *multi-class anomaly detectors*, depending on whether the training data includes *labelled* data points from a single normal class or multiple normal classes, respectively (Chandola et al., 2009). One-class anomaly detectors, e.g., the one-class Support Vector Machine (SVM) (Schölkopf et al., 1999), classify unlabelled data points as either normal or anomalous. Multi-class anomaly detectors can, in addition to this, discriminate data points from different normal classes. Approaches that assume all training data is labelled normal are sometimes referred to as *semi-supervised anomaly detection* (Chandola et al., 2009).

Classification Performance

Assuming that data points belong to either a normal or abnormal class (as discussed above), anomaly detectors are typically evaluated in terms of their *classification performance* on a labelled evaluation set that includes data points from both classes. In particular, the following performance measures from the fields of pattern recognition and machine learning are often used to evaluate anomaly detectors:

- *Accuracy* (Fawcett, 2006), which is equivalent to the overall rate of correctly classified data points.
- *True positives rate* (Fawcett, 2006), also referred to as *sensitivity* or *recall* (Fawcett, 2006), which is equivalent to the rate of data points from the abnormal class that are correctly classified as anomalous.
- *False positives rate* (Fawcett, 2006), also known as *false alarm rate* (Fawcett, 2006), which is equivalent to the rate of data points from the normal class that are incorrectly classified as anomalous.
- *Precision* (Fawcett, 2006), which is equivalent to the fraction of the data points classified as anomalous that belong to the abnormal class. In other words, precision corresponds to the rate of anomaly detections or alarms that are interesting from a domain experts perspective.

Obviously, the higher the accuracy, sensitivity and precision, and the lower the false alarm rate, the better the performance of the anomaly detector. It should be noted that acquiring a labelled evaluation set may be difficult for the same

reasons that acquiring a labelled training set is difficult (see the discussion above regarding supervised anomaly detection).

Types of Anomalies

Chandola et al. (2009) categorise anomalies as either *point anomalies*, *contextual anomalies* or *collective anomalies*. Point anomalies correspond to individual data points that are anomalous relative to *all* other data points; these type of anomalies are captured by, e.g., the definition of an outlier given by Hawkins (1980). Point anomalies are the focus of most research within anomaly detection algorithms (Chandola et al., 2009). Contextual anomalies, also known as conditional anomalies, are data points that are considered anomalous in a particular context. To formalise the notion of context, each data point is defined by *contextual attributes* and *behavioural attributes* (Chandola et al., 2009). If the behavioural attributes of a data point are anomalous relative to the behaviour attributes of the subset of data points having the same or similar contextual attributes, the corresponding data point is considered a contextual anomaly. Examples of contextual attributes could be time of day, season and geographical location. Lastly, collective anomalies consist of a set or sequence of *related* data points that are anomalous relative to the rest of the data points. In this case, the individual data points may not be anomalous by themselves; it is the aggregation of the data points that is anomalous. Examples of collective anomalies can found in sequence data, graph data and spatial data (Chandola et al., 2009). Anomalous trajectories, which we are concerned with further on in the thesis, can be considered as instances of collective anomalies.

Online vs. Offline Learning

Learning in most anomaly detection algorithms is essentially *offline*; static model parameters are learnt from a batch of training data and then used repeatedly when classifying new data. In order to accurately model normalcy, a fairly large training set may be required, which is representative of all possible normal behaviour. However, such a dataset may not be available from the outset. Moreover, "in many domains normal behaviour keeps evolving and a current notion of normal behaviour might not be sufficiently representative in the future" (Chandola et al., 2009). In contrast, *online learning* may account for this by incrementally refining and updating model parameters based on new data points.

Algorithms for Anomaly Detection

A number of surveys attempting to structure different algorithms for anomaly detection have been published during the last few years (see, e.g., Chandola et al., 2009; Patcha and Park, 2007). Chandola et al. (2009) categorise algorithms as belonging to one or more of the following classes: *classification-based techniques*, *parametric or non-parametric statistical techniques*, *nearest neighbour-based techniques*, *clustering-based techniques*, *spectral techniques* and *information theoretic*

techniques. In their survey, various advantages and disadvantages of algorithms from each category are discussed at length.

2.1.2 Statistical Anomaly Detection

Statistical methods for anomaly detection are based on the assumption that "normal data instances occur in high probability regions of a stochastic model, while anomalies occur in the low probability regions of the stochastic model" (Chandola et al., 2009). It is usually assumed that normal data points constitute independent and identically distributed (IID) samples from a stationary probability distribution P, which can be estimated from sample data D. Given a new data point, z, the goal is to determine whether it can be assumed to have been generated by P or not, i.e., if it is anomalous or not relative to the sample data D. Hence, there are two practical problems: how to estimate P based on D, and how to decide whether z is a random sample from P.

Parametric Methods and GMM

Statistical methods for estimating probability distributions can broadly be categorised as either *parametric* or *non-parametric* models (Markou and Singh, 2003a). Starting with the parametric models, they assume that the underlying distribution belongs to a parameterised family of distributions, i.e., $P_\theta : \theta \in \Theta$, where the parameters θ belong to a parameter space Θ and can be estimated from available sample data D. A common and simple parameterised model in anomaly detection applications is the *Gaussian distribution* (Chandola et al., 2009; Markou and Singh, 2003a). Another example is the *Poisson distribution* (Holst et al., 2006). More complex parameterised models in anomaly detection applications include various graphical models, such as *Bayesian networks* (Johansson and Falkman, 2007) and *hidden Markov models* (HMM) (Urban et al., 2010), and *mixture models*, such as the univariate or multivariate *Gaussian mixture model* (GMM) (Laxhammar, 2008; Ekman and Holst, 2008) and mixtures of other parameterised distributions, such as the Poisson and *Gamma distributions* (Ekman and Holst, 2008).

GMM is a common model for approximating continuous multi-modal distributions, when knowledge regarding the structure is limited; it has been used in numerous anomaly detection applications (Chandola et al., 2009). A GMM consists of C multivariate Gaussian densities known as mixture components. Each Gaussian component $c_i, i = 1, \ldots, C$, has its own parameter values $\theta_i = (\mu_i, \Sigma_i)$ and weight w_i, where μ_i is the mean value vector, Σ_i is the covariance matrix and w_i is a non-negative normalised mixing weight where all weights sum to one. The total set of parameters for the GMM is denoted $\theta = \{\theta_1, \ldots, \theta_C, w_1, \ldots, w_C\}$. The probability density function for the multivariate GMM is given by:

$$p(x) = \sum_{i=1}^{C} w_i \frac{1}{(2\pi)^{d/2} \sqrt{|\Sigma_i|}} \exp\left(-\frac{1}{2}(x - \mu_i)^\top \Sigma_i^{-1} (x - \mu_i)\right). \qquad (2.1)$$

A common and relatively simple way to estimate the parameters θ of a distribution P_θ based on a data sample D is to use a *maximum likelihood* (ML) estimator. The *expectation-maximisation* (EM) algorithm (Dempster et al., 1977) is a widely used ML estimator when D is incomplete and data points may have missing values, also known as latent variables. One example of missing values is not knowing which of the components of a mixture model that generated a data point. Typically, the EM algorithm starts by randomising initial values for the parameters and then incrementally estimating the values $\hat{\theta}$ that yield maximum likelihood for the sample data D:

$$\hat{\theta} = \underset{\theta \in \Theta}{argmax} \left(D|\theta \right). \tag{2.2}$$

More specifically, the algorithm consists of two steps, *expectation* and *maximisation,* which are repeated until a certain end condition, usually a convergence condition, is fulfilled. In the case of a GMM with a predefined number of components C, the expectation step involves updating the posterior probabilities $p\left(c_i \,|x_j\right)$ for each data point $x_j \in D, j = 1, \ldots, n$, belonging to each component $c_i, i = 1, \ldots, C$, according to Bayes' rule (Verbeek, 2003):

$$p\left(c_i \,|x_j\right) = \frac{p\left(x_j; c_i\right) w_i}{\sum\limits_{q=1,\ldots,C} p\left(x_j; c_q\right) w_q}, \tag{2.3}$$

where

$$p\left(x_j; c_q\right) = \frac{1}{\left(2\pi\right)^{d/2} \sqrt{|\Sigma_q|}} \exp\left(-\frac{1}{2} \left(x_j - \mu_q\right)^\top \Sigma_q^{-1} \left(x_j - \mu_q\right) \right) \tag{2.4}$$

corresponds to the qth component distribution. This expectation of point to component correspondence is then used in the maximisation step where the parameters of each component are updated based on the ML estimation. The maximisation step involves adjusting the parameters of each component in such a way that the component better fits the data points, taking the updated posterior probabilities into account. More specifically, component parameters are updated according to Equations 2.5 to 2.7 (Verbeek, 2003):

$$w_i = \frac{1}{n} \sum_{j=1}^{n} p\left(c_i \,|x_j\right), \tag{2.5}$$

$$\mu_i = \frac{1}{n w_i} \sum_{j=1}^{n} p\left(c_i \,|x_j\right) x_j, \tag{2.6}$$

$$\Sigma_i = \frac{1}{n w_i} \sum_{j=1}^{n} p\left(c_i \,|x_j\right) \left(x_j - \mu_i\right) \left(x_j - \mu_i\right)^\top. \tag{2.7}$$

The updated model is then used to calculate new posterior probabilities in the expectation step, and so on.

The popularity of the standard EM algorithm is probably due to its relative simplicity and fast convergence. Nevertheless, it has some disadvantages: Firstly, it is not guaranteed to converge to a global optimum; the algorithm is more or less sensitive to the parameter initialisation and may converge to different ML estimates depending on the start values (Verbeek, 2003). The standard procedure to overcome this initialisation dependence is to start the EM algorithm from several random initialisations and to retain the best result obtained (Verbeek, 2003). An extension of the standard EM algorithm calculates the *maximum a posteriori* (MAP) *estimate* based on a prior distribution, $p(\theta)$, on the parameters, thus incorporating prior knowledge and making the algorithm less sensitive to initialisation and noisy data. Another issue is how to determine the optimal number of components, a problem which is not solved by the standard EM algorithm. Verbeek proposed an efficient and greedy version of the EM algorithm that estimates the optimal number of components of a GMM and avoids the need for multiple runs with random parameter initialisation (Verbeek et al., 2003).

ML and MAP estimators do not include any uncertainty in the parameter estimates; they simply calculate the most likely parameter values for a given dataset, regardless of the size of the dataset. Hence there is no information on how confident we can be in the estimates. In the case of a small sample, the estimates are susceptible to random variations in the data; this is a bad property for an anomaly detector, since it will generate a lot of false alarms by focusing on the peculiarities of the data too much (Holst et al., 2006). Moreover, in many applications, including anomaly detection, we are not interested in the model's parameter values *per se*; rather, we are interested in getting an accurate and reliable estimate of the predictive data distribution, $p(x)$, based on the sample data D. In this case, an alternative to ML or MAP is a fully Bayesian parameter estimation, where the posterior distribution for the parameter values, $p(\theta|D)$, is estimated based on the prior distribution, $p(\theta)$, and available sample data, D, according to Bayes' theorem (Gelman et al., 2003):

$$p(\theta|D) = \frac{p(D|\theta)\, p(\theta)}{\int_\theta p(D|\theta)\, p(\theta)\, d\theta}. \tag{2.8}$$

The posterior distribution for the parameters can then be used to estimate the predictive distribution for normal data (Holst et al., 2006):

$$p(x|D) = \int_\theta p(x|\theta)\, p(\theta|D)\, d\theta. \tag{2.9}$$

Non-Parametric Methods and KDE

A general drawback of the parametric techniques is their assumption that a parameterised model exists and that it can be accurately estimated; this is doubtful in many applications where the true structure of the probability distribution is unknown and may be highly complex (Markou and Singh, 2003a). One approach to mitigate this problem is to resort to non-parametric methods, such as *histograms*

or *kernel density estimators* (KDE), which essentially assume nothing about the structural form of the distribution; it is instead determined from the given data (Chandola et al., 2009; Markou and Singh, 2003a).

In histogram-based methods, the feature space is assumed to be discrete and each possible value corresponds to a bin of the histogram. The histogram is constructed by simply counting the number of data points that fall within each of the bins. The unknown data distribution is then approximated by the discrete distribution of observed data points from the different bins, i.e., the relative frequency. A drawback of histogram methods is that continuous input data has to be discretised. It may not be obvious how the discretisation should be done, and subtle, yet significant, information may be lost in the process. Determining the optimal size of the bins is a key challenge for maintaining a low false alarm (false positive) rate and low false negative rate in anomaly detection applications (Chandola et al., 2009). Histogram methods for anomaly detection in multivariate data typically construct a separate univariate histogram for each data feature (Chandola et al., 2009). Thus, they do not capture the potential correlation between different features. However, there is at least one exception to this, in which the correlation between different attributes is accounted for by considering a variant of a *multi-dimensional histogram* (Brax et al., 2008).

In contrast, KDE, also known as *Parzen window estimation,* allows for a non-parametric estimation of a multivariate continuous distribution based on a *kernel function* (Chandola et al., 2009). The unknown distribution is approximated by placing a kernel function K on each and every data point x_i of the data sample:

$$p(x) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{(h_i)^d}K\left(\frac{x - x_i}{h_i}\right), \tag{2.10}$$

where n is the size of the sample data, d is the number of dimensions of the data points and h_i is a kernel width parameter known as *bandwidth* (Latecki et al., 2007), *window width* (Ristic et al., 2008) or *smoothing parameter* (Markou and Singh, 2003a). The kernel function is usually a symmetric probability density function, being non-negative over its domain and should integrate to unity over the defined range (Markou and Singh, 2003a). A common kernel for anomaly detection applications is the multivariate Gaussian function with zero mean and covariance matrix Σ (Ristic et al., 2008; Latecki et al., 2007):

$$K(x) = \frac{1}{(2\pi)^{d/2}\sqrt{|\Sigma|}}\exp\left(-\frac{1}{2}x^{\top}\Sigma^{-1}x\right). \tag{2.11}$$

The covariance matrix Σ can be set to, e.g., the identity matrix (Latecki et al., 2007) or estimated from data as sample covariance (Ristic et al., 2008). The kernel width can either be fixed, i.e., the same for all data points, or adaptive, based on the local (Latecki et al., 2007) or global (Ristic et al., 2008) distribution of the data points. A formula for computing the optimal fixed width for the Gaussian kernel (under the assumption that the underlying PDF is Gaussian) was originally proposed by Silverman (1986). However, a fixed kernel width is inappropriate in

applications where local data density varies (Latecki et al., 2007). In particular, a fixed kernel width is unable to satisfactorily deal with the tails of distributions where local density is low (Ristic et al., 2008). Therefore, kernel widths should be adaptive based on the local data density, i.e., broader windows in low density areas and narrower windows in high density areas. A two-stage approach, known as *adaptive KDE* for computing adaptive window widths from the fixed optimal width, was proposed by Silverman (1986). Another approach where local window widths are based on distance to the kth nearest neighbour was presented by Latecki et al. (2007). There are, however, at least two general drawbacks with KDE methods: they are relatively sensitive to noise in data (Markou and Singh, 2003a) and they require a relatively large sample set for accurate density estimation.

Anomaly Detection in New Data

Assuming that P has been estimated based on a parametric or non-parametric method, as discussed above, various anomaly tests for a new data point z have been proposed (Chandola et al., 2009). Some of these compute an anomaly score, which depends on the probability (discrete data) or probability density value (continuous data), and classify the data point as anomalous if the score is above a particular threshold. Often, the anomaly score is simply the inverse or negative of the data likelihood or its logarithm. Other approaches are based on statistical hypothesis testing where the null hypothesis, H_0, that z constitutes a random sample from the estimated distribution $p(x)$ is tested (Chandola et al., 2009). These tests are based on a specified *test statistic* and *significance level*, which corresponds to the confidence in classifying a data point as anomalous.

For Gaussian models, such as the simple Gaussian distribution or mixtures of Gaussian distributions, a large number of anomaly measures and test statistics for classifying new data points as normal or anomalous have been proposed. A simple technique is to calculate the number of standard deviations σ from the corresponding mean μ and compare this distance to a threshold (Markou and Singh, 2003a). For example, in one dimension, data points located 3σ or more further away more from μ may be classified as anomalous, as the region $\mu \pm 3\sigma$ contains 99.7% of all data points from the corresponding distribution (Chandola et al., 2009). A similar anomaly measure for univariate or multivariate data is the *box plot rule,* which usually defines a region that contains 99.3% of the normal data points (Chandola et al., 2009). A well-known outlier test is *Grubb's test*, where the z-score for a univariate data point is calculated and compared to an adaptive threshold based on data size and the value of a t-distribution taken at a specified significance level (Chandola et al., 2009). Multiple variants of Grubb's test have been proposed for multivariate data (Chandola et al., 2009). A number of other test statistics for anomaly detection have also been proposed for univariate and multivariate data, including *Student's* t-test, *Hotelling's* t^2-test and χ^2-tests. Common to all these test statistics is the assumption that the underlying data distribution is Gaussian.

Ekman and Holst (2008) proposed *principal anomaly* as "the true anomaly

measure". Assuming that X is a random variable from the probability distribution $p(x)$, the principal anomaly of a data point z with respect to $p(x)$ is defined as $Pr\{p(X) \geq p(z)\}$, i.e., the probability that a random sample X has a density equal to or larger than the density of z (Ekman and Holst, 2008). This measure is attractive from a theoretical perspective, since it is normalised and does not assume that $p(x)$ is Gaussian. However, it is typically not easy to estimate the principal anomaly for multivariate distributions and distributions other than the Gaussian (Ekman and Holst, 2008). Therefore, Ekman and Holst (2008) proposed *deviation* as an alternative anomaly measure, which is defined as:

$$\frac{E[\log p(X)] - \log p(z)}{S[\log p(X)]}, \tag{2.12}$$

where E and S correspond to the expectation (mean) and standard deviation, respectively. They argue that this measure is strongly related to principal anomaly and show how it can be easily calculated for multivariate distributions with independent variables (Ekman and Holst, 2008).

In histograms-based methods, there are generally two approaches to anomaly detection. The first is to simply check whether a new data point falls into any one of the histogram bins; if it does, the data point is classified as normal, otherwise, it is anomalous (Chandola et al., 2009). The second approach assigns an anomaly score to each data point, based on the frequency of the corresponding bin (Chandola et al., 2009). If frequency is below a specified threshold, the data point is classified as anomalous.

For other non-Gaussian distributions, such as Bayesian networks, HMM, KDE, etc., anomaly detection is usually done by checking whether the likelihood $p(z)$, or the corresponding logarithmic likelihood, $\log(p(z))$, for the new data point is below an application specific anomaly threshold. A low likelihood indicates an improbable data point and, hence, evidence against the hypothesis that it is normal. More generally, a higher anomaly likelihood threshold increases the probability of detecting true anomalies, but also the probability of false alarms.

2.1.3 Other Anomaly Detection Algorithms

Classification-Based Methods

Classification-based methods essentially assume that a decision boundary in feature space can be learnt from a training dataset, such that the decision boundary separates normal and anomalous data points. Depending on the number of normal classes, classification-based anomaly detectors can be grouped into one-class or multi-class anomaly detectors. Multi-class anomaly detectors assume that training data includes labels from multiple normal classes. Typically, new data points are either classified as belonging to one of the normal classes, or classified as anomalous, if they do not fit any of the normal classes. One-class techniques assume that all data points in the training set have the same label; they typically learn a

discriminative boundary around the training data which is used to check whether new data points are anomalous or not.

Most of the classification-based methods for anomaly detection are based on *neural networks*, *support vector machines* (SVM) or *rule-based methods* (Chandola et al., 2009). Considering neural networks, *multi-layer perceptrons* are the best-known and most widely used (Markou and Singh, 2003b). Other neural networks used for anomaly detection include *auto-associative networks*, *adaptive resonance theory*, *radial basis functions* and *Hopfield networks* (Chandola et al., 2009). Neural networks are powerful classifiers, since they can approximate arbitrary complex decision boundaries in feature space. However, they are designed for *discriminating* between multiple known classes rather than *detecting* new classes (Markou and Singh, 2003b). Since they do not generate closed class boundaries, adapting them to anomaly (novelty) detection and ensuring that the generalisation property of the network does not interfere with its anomaly detection ability are fairly challenging tasks (Markou and Singh, 2003b). In other words, there is typically the risk of overfitting the model to the training data.

A number of different SVM-based methods have been applied to anomaly detection in the one-class setting (Chandola et al., 2009). Similar to neural networks, SVM methods are capable of learning arbitrary complex regions in input feature space. However, they do not have the same problem of overfitting to training data, since they estimate optimal wide-margin hyperplanes in a high-dimensional feature space. A key parameter of any SVM is the *kernel function*, e.g., a polynomial or Gaussian function, which maps input data to a high-dimensional feature space where data is perfectly, or close to, linearly separable. Schölkopf et al. (1999) proposed a one-class SVM for novelty detection based on quantile estimation of arbitrary distributions. The idea is to learn the boundaries in input data feature space for a specified quantile of the underlying distribution, based on available training data. For example, estimating the boundaries for the first percentile results in a region containing approximately 99% of randomly generated data points from the underlying distribution. The specified quantile corresponds to the anomaly threshold; if a new data point falls outside of the region, it is considered anomalous. The region is found by estimating the hyperplane in the high-dimensional feature space that maximises the distance to the origin where only a small fraction of the training data points fall between the hyperplane and the origin (Schölkopf et al., 1999).

Various methods that learn rules from normal data based on, e.g., decision trees, have been proposed for anomaly detection (Chandola et al., 2009). Some of these methods associate a confidence score to each rule, which is proportional to the ratio of the number of training instances correctly classified by the rule and the total number of training data points covered by the rule (Chandola et al., 2009). For new data points, the rule that best captures the data points is sought, and the inverse of the corresponding rule confidence is returned as the anomaly score. If no rule that captures the new data point can be found, it is classified as anomalous.

Nearest Neighbours Methods

Nearest neighbours methods for anomaly detection are based on the assumption that "normal data [points] occur in dense neighbourhoods, while anomalies occur far from their closest neighbours" (Chandola et al., 2009). This assumption suggests that a distance or similarity measure between data points is needed. For continuous features, the Euclidean distance (ED) is a popular distance measure, but other measures have also been proposed (Chandola et al., 2009). For categorical attributes, the simple matching coefficient, or other more complex measures, are often used (Chandola et al., 2009). Generally, distance measures used in nearest neighbours methods are required to be *positive-definite* and *symmetric*; but they are usually not required to satisfy the *triangle inequality* and, hence, not required to be strictly *metric* (Chandola et al., 2009).

A number of different algorithms for calculating anomaly scores based on the nearest neighbours principle have been proposed. A common algorithm is based on the following principle: "The anomaly score of a data [point] is defined as its distance to its kth nearest neighbour in a given dataset" (Chandola et al., 2009). Other variants of this principle calculate the anomaly score as the sum of the distances to the k nearest neighbours (Eskin et al., 2002), or as the inverse of the number of nearest neighbours that are no more than d distance apart from the corresponding data point (Chandola et al., 2009). The latter algorithm has the flavour of kernel density estimation (Section 2.1.2) where the kernel function corresponds to counting the number of neighbouring data points within the hypersphere of radius d. However, analogously to statistical methods, the nearest neighbours techniques discussed above may be suboptimal if the data has regions of varying densities (Chandola et al., 2009). To handle this, techniques for computing the local density of data points, i.e., the density of data points relative to their closest neighbours, have been proposed. One such technique is *local outlier factor* (LOF) (Breunig et al., 2000), in which the anomaly score for a particular data point is equal to the ratio of the average local density of the k nearest neighbours, and the local density of the data point itself. The local densities in LOF are inversely proportional to the radius of the minimum hypersphere that contains the k nearest neighbours. Several variants and extensions of the basic LOF algorithm have been proposed, e.g., to handle different data types and improve computational efficiency.

In most cases, a threshold is applied to the anomaly score during anomaly detection. Alternatively, the n data points with the largest anomaly scores may be returned (Chandola et al., 2009).

Clustering Methods

Clustering is essentially an unsupervised learning technique that groups similar data points into clusters. Chandola et al. (2009) categorise clustering based techniques for anomaly detection into three groups, depending on what assumptions are made regarding normal and anomalous data. The first group assumes that

"normal data [points] belong to a cluster in the data, while anomalies do not belong to any cluster". These techniques are based on an algorithm that clusters the data while not forcing all data points to belong to a cluster; data points that are found not belonging to any cluster are classified as anomalous. According to Chandola et al. (2009), the disadvantage of these methods is that they are designed to find clusters rather than anomalies.

The second group of clustering techniques are more focused on finding anomalies; these are based on the assumption that "normal data [points] lie close to their closest cluster centroid, while anomalies are far away from their closest cluster centroid" (Chandola et al., 2009). Typically, these methods first cluster the data and then assign an anomaly score to each data point, based on the distance to its nearest cluster centroid. Common clustering algorithms include *k-means clustering, self-organising maps* (SOM), and EM (Chandola et al., 2009). In EM, clusters are represented as Gaussian components in a mixture model, i.e., similar to the estimation of a GMM (Section 2.1.2). Techniques from the second group can also operate in a semi-supervised mode, where new test data is compared to a cluster model learnt from previous training data (Chandola et al., 2009).

If anomalies form clusters by themselves, algorithms based on the two principles presented above will obviously not be able to detect those anomalies. To address this issue, algorithms have been proposed that rely on the assumption that "normal data [points] belong to large and dense clusters, while anomalies either belong to small or sparse clusters" (Chandola et al., 2009). These algorithms assign an anomaly score which reflects the size or density of the cluster to which the corresponding data points belong.

2.2 Anomalous Trajectory Detection

Research related to the automated detection of anomalous trajectories has attracted a lot of attention lately, much due to the increasing amounts of historical and real-time trajectory data. There is a clear trend towards more and more advanced sensor systems producing huge amounts of trajectory data from moving objects, such as people, vehicles, vessels and animals. For example, the use of video cameras for public surveillance has increased dramatically since the beginning of the 21th century and a significant amount of research has focused on developing object *tracking* algorithms (Yilmaz et al., 2006) that produce trajectories from raw video data. In the maritime surveillance domain, regulations regarding the mandatory use of AIS transponders for vessels of 300 gross tonnage or greater have resulted in a significant increase of vessel tracking data, which complements existing radar tracking systems (Chang, 2004). In the domain of intelligent transportation, the widespread use of GPS has facilitated the tracking of land-based transports.

Technically, a *trajectory* is generally modelled as a finite sequence of data points:

$$(\chi_1, t_1), \ldots, (\chi_L, t_L) : t_{i-1} < t_i, i = 2, \ldots, L, \tag{2.13}$$

where each $\chi_i \in \mathbb{R}^d$ is a multi-dimensional feature vector at time point t_i and where the total length in time of the trajectory is $t_L - t_1$ (Morris and Trivedi, 2008a). In the simple case, $\chi_i \in \mathbb{R}^2$ represents an object's (estimated) location in the two-dimensional spatial plane at time point t_i. Depending on the application, the feature space may be extended by, e.g., a third spatial dimension or the velocity. Moreover, in some applications it is assumed that the trajectory is uniformly sampled in time:

$$(t_{i+1} - t_i) = (t_i - t_{i-1}) : i = 2, \ldots, L-1, \tag{2.14}$$

in which case (2.13) can be further simplified to:

$$(\chi_1, \ldots, \chi_L)\,. \tag{2.15}$$

It should be noted that in practice, (2.13) is typically the output of a tracking system and constitutes an *estimate* of the true trajectory of the object. Therefore, (2.13) may be inaccurate or incomplete due to various errors and noise introduced by the tracking system. In particular, the trajectory may appear anomalous due to erroneous measurements and other noise.

Most of the previous work related to anomalous trajectory detection has been published in the domains of video surveillance and maritime surveillance. In the following two subsections, we therefore provide a survey of previous algorithms within each of these two domains.

2.2.1 Video Surveillance

A large amount of work related to the automated detection of anomalous trajectories has been published in the domain of video and image processing (Morris and Trivedi, 2008a; Hu et al., 2006; Dee and Velastin, 2008). Most of these methods involve *trajectory clustering*, where cluster models corresponding to normal paths or routes are learnt from historical trajectories; this process is referred to as *trajectory learning* (Morris and Trivedi, 2008a) and is presented in the first part of this subsection. New trajectories are typically assigned an anomaly score based on the distance to the closest cluster model, or likelihood of the most probable cluster model; this is described in the second part of this subsection. Finally, methods that do not involve trajectory clustering are presented in the third and final part of this subsection.

Trajectory Learning

Trajectory learning typically consists of three steps that are sometimes combined: preprocessing of raw trajectories, clustering of preprocessed trajectories and modelling of trajectory clusters (Morris and Trivedi, 2008a).

The purpose of the preprocessing is to produce trajectory representations that are suitable for clustering. A typical problem with raw trajectories is that they are not of equal length. According to Morris and Trivedi (2008a), "steps must

be taken to ensure a meaningful comparison between differing sized inputs" and "trajectory representations should retain the intuitive notion of similarity present in raw trajectories for meaningful clusters". Two main techniques are used for preprocessing trajectories: *normalisation* and *dimensionality reduction* (Morris and Trivedi, 2008a). Normalisation techniques ensure that trajectories are of equal length by, e.g., extending short trajectories by zero padding (Hu et al., 2006) or by re-sampling trajectories. Dimensionality reduction techniques map trajectories into a lower dimensional feature space which is computationally more manageable and enables more robust clustering given less training data (Morris and Trivedi, 2008a). Examples of dimensionality reduction techniques include vector quantisation, various polynomial approximations (Naftel and Khalid, 2006), parametrised spline models (Sillito and Fisher, 2008), wavelets, HMM (Porikli, 2004), principal component analysis (PCA), and spectral methods (Atev et al., 2010).

The aim of the trajectory clustering is to learn the underlying structure and routes in data by grouping similar trajectories. All clustering algorithms require that an appropriate *similarity measure*, also known as *distance measure*, is defined. Assuming that all the points of each trajectory are embedded as a high-dimensional vector, the ED (Euclidean distance) between two such vectors is perhaps the most simple and intuitive similarity measure. However, ED requires that the preprocessed trajectories are of equal length, and it performs poorly if the trajectories are not properly aligned (Morris and Trivedi, 2008a). Other similarity measures and modifications of ED have been proposed for relaxing alignment and length constraints, such as dynamic time warping (DTW) (Morris and Trivedi, 2008a). Longest common sub-sequence (LCSS) is another similarity measure appropriate for trajectories that are of unequal length and/or are not well-aligned (Vlachos et al., 2002). In contrast to ED and DTW, LCSS is more robust to noise in trajectories, since it focuses on parts of the trajectories that are similar rather than dissimilar (Vlachos et al., 2002). Other alternatives to ED include HMM-based trajectory distance (Porikli, 2004) and variants of the Hausdorff distance (HD) (Atev et al., 2010) (HD is described in Section 5.1.1). Different similarity measures, including those presented above, were evaluated for trajectory clustering on a real-world dataset by Zhang et al. (2006). They found that a relatively simple approach based on representing trajectories in a PCA-based coefficient feature space and using ED as distance measure performed the best.

A number of different algorithms have been proposed for creating and updating trajectory clusters based on a specified similarity measure. Morris and Trivedi (2008a) identified five main categories based on fundamental properties of the clustering algorithm: *iterative optimisation* (e.g., k-means), *online adaptive*, *hierarchical methods* (agglomerate and divisive), neural networks (e.g., SOM) and *co-occurrence methods*. The strengths and weaknesses of each category are discussed in their survey (Morris and Trivedi, 2008a). Of particular interest are clustering algorithms that are online adaptive, since they enable the continuous and efficient update and adaption to changes in normal trajectory behaviour. Piciarelli and Foresti (2006) proposed such an algorithm for online clustering of trajectories,

including a novel similarity measure tailored for the clustering algorithm.

Once trajectories have been clustered, appropriate cluster models, sometimes referred to as *path models* (Morris and Trivedi, 2008a) or *motion patterns* (Hu et al., 2006), are defined. The path models correspond to compact representations of the clusters, which enables efficient inference during, e.g., trajectory classification or anomaly detection. Two main types of path models have been adopted. The first considers a complete path, from starting point to endpoint. The second decomposes a path into smaller atomic parts called *subpaths* (Morris and Trivedi, 2008a). Complete path models are typically based on a *centroid* representation of the cluster, corresponding to the "average" trajectory (Morris and Trivedi, 2008a). The centroid is sometimes complemented by an *envelope*, which captures the extension and variance of the trajectories (Morris and Trivedi, 2008a). Hu et al. (2006) represent each path model (referred to as motion pattern) as a chain of Gaussian distributions, where the sequence of mean values and covariance matrices define the centroid and envelope, respectively. Morris and Trivedi (2008b) proposed the modelling of each path by a HMM where each hidden state is modelled by a GMM. Model parameters for each HMM are initially estimated based on the training trajectories belonging to the corresponding cluster. Each HMM can also be updated online as new training data arrives. In the case of subpath models, these are further defined by their connections to other subpaths, which can be probabilistically modelled (Morris and Trivedi, 2008a). An example of a subpath structure was proposed by Piciarelli and Foresti (2006), where each subpath is modelled as a node in a tree-like structure augmented with probabilities for node transitions. A subpath in their approach is represented as a sequence of points, where each point has an associated variance, i.e., similar to Hu et al. (2006).

Anomaly Detection

For complete trajectories, anomaly detection is typically carried out by first determining the path model that best explains the new trajectory, i.e., having the minimal distance to the new trajectory. In the case of probabilistic models, this can be done on the basis of a MAP or ML analysis (Morris and Trivedi, 2008a). The corresponding distance or likelihood is then usually compared to an anomaly threshold, analogously to general anomaly detection algorithms discussed in Section 2.1. Some algorithms, but far from all, support *sequential,* also known as *online* (Morris and Trivedi, 2008a) or *incremental* (Hu et al., 2006) anomaly detection in *incomplete* trajectories. Morris and Trivedi (2008b) proposed an algorithm that monitors the likelihood for the part of the current trajectory that is within a sliding window of a fixed size; if this likelihood drops below a specified threshold, the trajectory is classified as anomalous. Hu et al. (2006) proposed an algorithm for incremental anomaly detection in incomplete trajectories; for each new data point, the algorithm first updates the most probable motion pattern (trajectory cluster model) given the part of the trajectory observed so far. The likelihood for the new point is then calculated relative to the most probable motion pattern and the point is classified as anomalous if its likelihood is below a

specified threshold. If a series of points are classified as anomalous, the trajectory is classified as anomalous. Fu et al. (2005) proposed a sequential anomaly detection algorithm that incrementally determines the MAP trajectory cluster as more data points are observed for an incomplete trajectory. The incomplete trajectory is classified as anomalous if it leaves the envelope of the current MAP cluster, the MAP cluster varies as more data points are observed, or the velocity of the trajectory deviates from the corresponding velocity of the current MAP cluster.

Non-clustering Based Methods

Other approaches to trajectory anomaly detection in video surveillance have been proposed that do not involve the clustering of trajectories as presented above (Piciarelli et al., 2008; Yankov et al., 2008; Brax et al., 2008; Owens and Hunter, 2000). Piciarelli et al. (2008) proposed a trajectory learning and anomaly detection algorithm based on one-class SVM (Section 2.1.3), where each trajectory is represented by a fixed-dimensional feature vector corresponding to evenly sampled points from the raw trajectory. One of the main novelties of the algorithm is its ability to automatically detect and remove anomalies in the training data.

Yankov et al. (2008) proposed the application of *time-series discords* (Keogh et al., 2005) for detecting anomalous trajectories in a database. Assuming preprocessed trajectories or sub-trajectories of equal length, a discord is defined as the trajectory or sub-trajectory, respectively, maximising the ED to its nearest neighbour in the database. This definition can be extended by considering the distance to the kth nearest neighbour. Moreover, if the dataset is assumed to include more than one anomalous trajectory, anomaly detection can be carried out by calculating the top-K discords.

Brax et al. (2008, 2009) proposed an algorithm based on multi-dimensional histograms for detecting anomalous trajectory behaviour in video surveillance. Each data point from a trajectory is represented in a discrete multi-dimensional feature space, referred to as a *composite state space*, which encompasses: current kinematic state (position, speed and course), the object's position relative to the currently closest other object, the object's size, the state of the local environment, and the time of the day. A multi-dimensional histogram is constructed for the composite states of all the data points in the training set. Moreover, a separate histogram for each possible transition from a composite state to another is also constructed. Sequential anomaly detection is based on the relative frequency of each new data point according to the histograms. If the average frequency of the most recent data points within a sliding window is below a predefined threshold, the trajectory is classified as anomalous. One of the main features of this approach is that contextual information, i.e., time of day and state of local environment, is embedded in the state (feature) space. Moreover, the proposed method differs from traditional multi-histograms in that the state boundaries are set on the basis of expert knowledge rather than generated from the data itself (Brax et al., 2009).

Owens and Hunter (2000) proposed an algorithm based on SOM appropriate for learning and sequential (online) anomaly detection in trajectory data. Each

data point from a trajectory is represented by a fixed-length feature vector encompassing the current location, velocity, and acceleration, together with information on the recent position. The SOM is trained using a set of feature vectors as input. During sequential anomaly detection, the feature vector corresponding to each new data point is submitted to the SOM and the ED to the winning neuron is determined. If this distance exceeds a predefined threshold, the corresponding trajectory is classified as anomalous.

Lee et al. (2008) proposed a *partition-and-detect* framework for the detection of anomalous sub-trajectories in a trajectory database. A proposed two-step anomaly (outlier) detection algorithm first partitions each trajectory into a number of line segments. Next, anomalous trajectory partitions, i.e., line segments, are detected according to a combination of distance-based and density-based analysis. In the case of distance, a combination of spatial distance and angular distance between line segments is used.

2.2.2 Maritime Surveillance

Maritime surveillance is another domain for which substantial work related to anomaly detection in trajectory data has been published. In contrast to video surveillance, most of the algorithms proposed for anomaly detection in the maritime domain do not involve the explicit clustering of trajectories as part of the learning phase. The published papers typically provide less information on parameter values and implementation details of the proposed algorithms. Moreover, the experiments are often described in less detail and the datasets are exclusively non-public. Hence, it is generally more difficult to actually implement the proposed algorithms and reproduce the experiments.

A number of methods have been proposed where learning is based on the estimation of a statistical model for the feature values of individual data points from vessel trajectories, e.g., the current location and velocity vector (Brax et al., 2010; Johansson and Falkman, 2007; Ristic et al., 2008; Kraiman et al., 2002). In the work by Ristic et al. (2008), vessel trajectories are first clustered on the basis of their origin, resulting in a number of motion patterns. The PDF for feature values of data points belonging to the same motion pattern is then estimated using KDE. Data points from new trajectories are sequentially classified as normal or anomalous on the basis of their likelihood, which is calculated from the PDF of the corresponding motion pattern. Kraiman et al. (2002) discuss a two-step learning algorithm where feature values of individual data points are first clustered using SOM and then modelled using GMM. Anomaly detection in new data is carried out on the basis of a Bayesian analysis of the probability output of the GMM, which can be accumulated within a sliding time window. A prototypical scenario is presented where properties of the proposed algorithm are discussed. However, no details regarding the learning and anomaly detection algorithm are given.

Other statistical approaches have also been proposed based on Bayesian networks (Johansson and Falkman, 2007) or multi-dimensional histograms (Brax et al., 2010), where point features are discretisized and anomaly scores for new

data points are calculated on the basis of their likelihood or frequency in training data, respectively. Johansson and Falkman (2007) argue that one of the main advantages of the Bayesian network is that it enables the incorporation of human expert knowledge during learning. The multi-histogram method proposed by Brax et al. (2010) has the advantage that it, in contrast to the previous algorithms, captures the relationship between successive data points by modelling the frequency of different state transitions. In order to suppress the effect of noise during anomaly detection, Johansson and Falkman (2007) and Brax et al. (2010) propose a sliding window approach where anomaly scores for individual data points are integrated, i.e., similar to Kraiman et al. (2002) described above.

Urban et al. (2010) propose a two-level approach based on a combination of GMM and HMM for learning and anomaly detection in trajectory data. At the first level, a GMM is estimated for the position feature of individual trajectory points based on EM. At the second level, each component of the GMM is considered a discrete state in a HMM, which is estimated from the trajectory data using the *Baum-Welch algorithm.* Anomaly detection in new trajectories is carried out by thresholding the likelihood according to the HMM. Classification performance of the algorithm is evaluated on a non-public simulated dataset. A similar two-level approach was proposed by Tun et al. (2007), in which clusters, referred to as regions, in the position feature space for vessel trajectories are first discovered using a combination of *density maps* and *Linear scale space.* A HMM is then estimated for the transitions between different regions. Moreover, additional feature values, such as momentary course and speed, are estimated for vessel trajectories within the same region. A short extract from an experimental evaluation with real normal data and simulated anomalous data show that the method can accurately detect the anomalies.

Rhodes et al. (2005, 2007), Garagic et al. (2009), and Bomberger et al. (2006) have proposed algorithms for learning and anomaly detection in vessel traffic, based on different variants of neural networks. The first algorithm is based on a combination of online unsupervised and supervised learning, implemented by a Fuzzy ARTMAP neural network, where clusters in feature space are incrementally detected and optionally labelled by a human operator (Rhodes et al., 2005). The features considered in their application are the momentary location and the speed of vessels. New data points are classified as anomalous, depending on their distance to the nearest cluster. The authors present prototypical scenarios and discuss qualitative properties related to the online performance of the algorithm; however, no details are given regarding parameters and implementation. In a subsequent paper, the authors proposed replacing the Fuzzy ARTMAP algorithm with another mixture-based neural network algorithm with similar properties, including unsupervised and incremental learning (Garagic et al., 2009). The performance of the new algorithm was compared to that of a standard mixture model based on batch learning.

Bomberger et al. (2006) proposed an algorithm based on an associative neural network that learns to predict the future location of vessels based on their cur-

rent location, speed and course. Similar to their previously proposed algorithms, learning is incremental and unsupervised. However, features are also discretisized, where location is specified by the corresponding cell coordinates in a grid over the surveillance area. Anomalous vessel behaviour is detected when the current location (cell) of a vessel is inconsistent with the corresponding prediction. This algorithm is considered to be a complement to the previous algorithms, since it is able to detect anomalous behaviour that develops over time, such as anomalous routes. Results of the learning and prediction performance of the algorithm based on real vessel data were presented. These results indicated that performance is sensitive to local grid resolution. Hence, the authors investigated an extended approach in a subsequent paper, where improvement in prediction performance was achieved by using multiple spatial scales to represent position (Rhodes et al., 2007).

The online clustering algorithm initially proposed by Piciarelli and Foresti (2006) for video surveillance (Section 2.2.1) was evaluated by Dahlbom and Niklasson (2007) in the domain of maritime surveillance. Dahlbom and Niklasson (2007) discuss some practical problems and issues that were discovered during experiments on a simulated vessel trajectories. The paper therefore concludes with a discussion regarding an alternative spline-based clustering approach.

Chapter 3

The Research Gap

This chapter discusses in more detail some general limitations of previous algorithms for anomalous trajectory detection (Sections 2.1.2–2.1.3 and Section 2.2). In the first subsection, we discuss issues that are related to anomaly detection in general. The second subsection is focused on problems that are specific for the trajectory domain. In the final subsection, we conclude with the identification of a set of key properties that need to be addressed by new or updated algorithms for anomalous trajectory detection. These properties motivate the proposal of CAD and ICAD in Chapter 4, and SHNN-CAD and SSLO-ICAD in Chapters 5 and 6, respectively.

3.1 Anomaly Detection

3.1.1 Inaccurate Statistical Models

Parameterised statistical models, such as the Gaussian distribution, are attractive because parameter estimation is rather straightforward and there are statistical tests that provide well-founded confidence for anomaly detection (Chandola et al., 2009). However, the accuracy of estimated models and the robustness of anomaly detection depend on whether the structural assumptions are valid. For example, fitting a Gaussian model to data generated from an approximately uniform distribution will result in overestimation and underestimation of the probability density at the centre (mean) and at the tails of the distribution, respectively. This problem is illustrated by Figure 3.1, where the PDF for the geographical position of vessels was modelled by GMM (Laxhammar et al., 2009). In the case of underestimation, examples of the normal class will be assigned a lower likelihood and, hence, are more likely to be classified as abnormal. In the case of overestimation, the sensitivity to abnormal examples will decrease as there is an increased risk that subtle anomalies will be erroneously classified as normal. In many real-world applications, such as traffic surveillance, where behaviour is governed by social systems

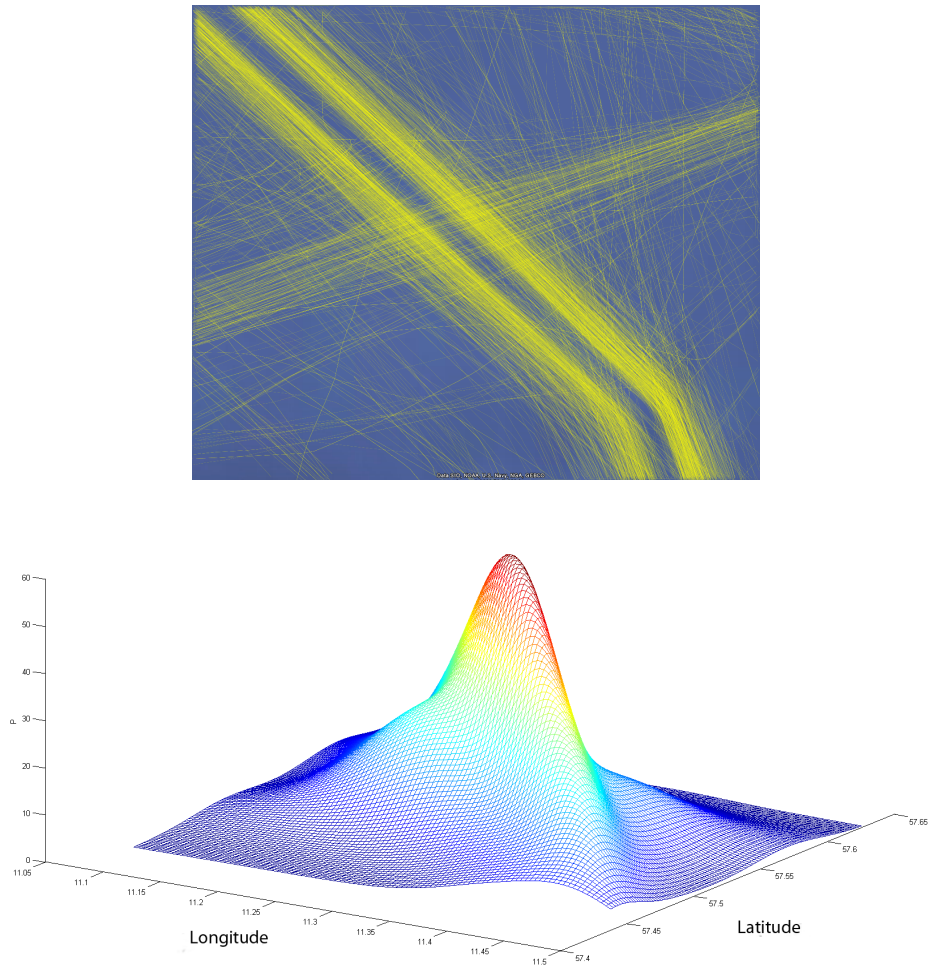


Figure 3.1: Top: plot of a recorded vessel trajectories at open sea. Bottom: PDF for vessel position estimated from recorded trajectories using GMM and EM (Laxhammar et al., 2009).

and structures rather than by physical laws, the Gaussian assumption may indeed be questioned (*cf.* Taleb, 2004).

3.1.2 Parameter-laden Algorithms

Most anomaly detection algorithms require careful preprocessing and the setting of multiple application specific parameters and detection thresholds in order to achieve (near) optimal performance; anomalous trajectory detection is no exception to this. In fact, Keogh et al. (2007) argue that most data mining algorithms are more or less *parameter-laden*, which is undesirable for several reasons. In an extensive empirical study, they show that "in case of anomaly detection, parameter-laden algorithms are particularly vulnerable to overfitting" (Keogh et al., 2007). The risks of overfitting parameter-laden models in real-world applications are also discussed by Hand (2006), who shows empirically that "the marginal gain from complicated models is typically small compared to the predictive power of the simpler models". According to Markou and Singh (2003a), "an [anomaly] detection method should aim to minimise the number of parameters that are user set". Indeed, one may argue that the use of few parameters suppresses bias towards particular types of anomalies and makes the method easier to implement for different applications.

3.1.3 Ad-hoc Anomaly Thresholds and High False Alarm Rates

A key parameter of most anomaly detection algorithms is the anomaly threshold, which regulates the balance between the sensitivity to true and interesting anomalies, and the rate of false alarms (Section 2.1.1). Generally, the higher the sensitivity, the higher the false alarm rate. Obviously, achieving high sensitivity is desirable. Nevertheless, it has been argued that the false alarm rate is the most critical of the two from an application perspective: According to Riveiro (2011) "the primary and most important challenge that needs to be met for using [an anomaly detection] approach [in maritime surveillance] is the development of strategies to reduce the high false alarm rate". Riveiro continues arguing that "the inability of suppressing false alarms is a perennial problem that prevents the widespread adoption of anomaly detection capabilities" and that "this obstacle may become a nuisance for operators that might turn [the anomaly detector] off" (Riveiro, 2011).

The false alarm rate has a major influence on the precision (Section 2.1.1), which is another critical performance measure in anomaly detection applications. If the precision is low, most of the detected anomalies are uninteresting, which decreases the human operators' faith in the system and increases the risk that they will ignore *all* of the alarms, including those corresponding to true and interesting anomalies. It is the imbalanced class distribution in anomaly detection applications, where the prior probability of the abnormal class is very low, which further aggravates the effect of a high false alarm rate. Indeed, the importance

of suppressing the false alarm rate in order to achieve reasonable precision was highlighted by Axelsson (2000) in the context of anomaly-based intrusion detection systems. According to Axelsson, the *Bayesian detection rate*, which is the conditional probability that a detected anomaly corresponds to an intrusion, becomes dominated by the false alarm rate if the prior probability of an intrusion is very low. Therefore, "the factor limiting the performance of an [anomaly based] intrusion detection system is not the ability to correctly identify behaviour as [anomalous], but rather *the ability to suppress false alarms*" Axelsson (2000). Let us further elaborate on this argument. Assume that:

- A is the event that the data *is* a true anomaly, regardless of whether it is detected as anomalous or not.
- D is the event that the data is *detected* as anomalous, regardless of whether it is a true anomaly or not.

Then, $Pr(A|D)$ is the conditional probability of a true anomaly given a detection, i.e., the Bayesian detection rate, which is equal to the expected precision. Based on Bayes' rule, $Pr(A|D)$ can be decomposed as:

$$Pr(A|D) = \frac{Pr(D|A)\,Pr(A)}{Pr(D)} = \\ = \frac{Pr(D|A)\,Pr(A)}{Pr(D|A)\,Pr(A) + Pr(D|\neg A)\,Pr(\neg A)}, \tag{3.1}$$

where:

- $Pr(D|A)$ is the conditional probability that a true anomaly is successfully detected as anomalous, which is equal to the expected sensitivity;
- $Pr(D|\neg A)$ is the conditional probability that a normal data point is erroneously detected as anomalous, which is equal to the expected false alarm rate;
- $Pr(A)$ and $Pr(\neg A)$ are the prior probabilities of a true anomaly and normal data, respectively.

Now, assuming that $Pr(A) \ll Pr(\neg A)$, which is typically the case in anomaly detection applications, it is clear from (3.1) that $Pr(D|\neg A)$ has a major influence on $Pr(A|D)$. To give a numerical example, assume that $Pr(A) = 0.01$, $Pr(D|A) = 1$ and $Pr(D|\neg A) = 0.01$. Then, the expected precision is equal to 0.50, which might be considered acceptable from a human operator's perspective. However, if we increase the probability of a false alarm by four percentage points, $Pr(D|\neg A) = 0.05$, the expected precision becomes less than 0.17, even though the sensitivity is still perfect. The effect of the phenomena, which is referred to as the *base-rate fallacy* by Axelsson (2000), becomes more severe if the prior probability of a true anomaly is even smaller. Thus, in order to achieve reasonably good precision, it is critical to maintain a low or very low false alarm rate.

To summarise the above, suppressing the false alarm rate is critical in order to reduce the workload of the human operators and to decrease the risk that the anomaly detection system is considered a nuisance. In order to maximise the sensitivity, while maintaining a sufficiently low false alarm rate, it is important that the anomaly threshold is *well-calibrated.* Most algorithms define the anomaly threshold in terms of a distance, density, or data likelihood (Chandola et al., 2009). These measures are typically not normalised and the procedures for setting the thresholds seem to be more or less ad-hoc. In particular, the thresholds seem to lack any intuition and it may be difficult to predict the performance in terms of, e.g., the false alarm rate or the overall rate of detected anomalies in future data. Indeed, it has been argued that "while [modern anomaly detection] approaches provide impressive computationally efficient solutions on real data, it is generally difficult to precisely relate tuning parameter choices to desired false alarm probability" (Zhao and Saligrama, 2009). Hence, ad-hoc and poorly calibrated anomaly thresholds increase the risk of a high false alarm rate and low precision.

3.2 Anomalous Trajectory Detection

3.2.1 Offline Learning

With a few exceptions (notably Piciarelli and Foresti (2006)), previous algorithms (e.g., Yankov et al., 2008; Lee et al., 2008) are designed for *offline learning*, in the sense that all training trajectories are assumed to be available from the outset; fixed model parameters and thresholds are estimated or tuned once, based on a batch of training data, and then repeatedly used for anomaly detection. In contrast, algorithms designed for *online learning* incrementally update the model parameters as each new training trajectory is observed. The need for online learning was highlighted by Piciarelli and Foresti (2006), who proposed an efficient online trajectory clustering algorithm. It may be counter-argued that taking the system offline is not strictly necessary in the case of an offline learning process that runs in parallel to the anomaly detector, where updated model parameters are introduced at specific time points. For example, Morris and Trivedi (2008b) discuss regular batch-learning repetitions using an updated training set. However, it is not clear how to determine the appropriate time point for such an update. On the one hand, minimising the learning delay, i.e., the time between successive model updates, may be desirable in order to maintain a more timely and accurate model. On the other hand, the computational complexity of offline learning algorithms, which is typically higher than for online algorithms, may impose restrictions on how short a delay can be achieved in a practical application.

3.2.2 Offline Anomaly Detection

Previously proposed algorithms are mainly designed for anomaly detection in databases and, hence, it is often explicitly or implicitly assumed that the *complete*

trajectory (from start to end points) is available prior to classification. This is true, e.g., for algorithms based on dimensionality reduction and other normalisation techniques (Sillito and Fisher, 2008; Porikli, 2004; Atev et al., 2010; Khalid, 2010). Moreover, most of the proposed trajectory similarity measures, including ED, DTW and LCSS (Section 2.2), are essentially designed for complete trajectories, even though some of them (e.g., LCSS) are more robust to missing data points than others. In surveillance applications where trajectories are sequentially updated in real-time, anomaly detection is delayed until the trajectory has terminated and, thus, the ability to react to impending events is limited.

In contrast, algorithms that are designed for *sequential anomaly detection* in *incomplete* trajectories (also known as trajectory streams (Bu et al., 2009)) enable the timely detection of anomalies as they evolve. Examples of such algorithms include various *point-based anomaly detectors* where a low-dimensional feature model of the current and previous trajectory states is considered (e.g., Laxhammar, 2008; Brax et al., 2008; Owens and Hunter, 2000). However, an obvious limitation of such algorithms is that they do not capture more long-term trajectory behaviour. A compromise between a point-based approach and a traditional model of complete trajectories is to consider a sub-trajectory model. Hu et al. (2006) propose an algorithm where each trajectory is divided into a number of fixed length and non-overlapping subsequences of data points. Each such subsequence is then represented as a high-dimensional feature vector. However, such methods typically involves more preprocessing, such as alignment and interpolation, and more parameters, such as size of the subsequence; it may not be obvious how the size of the subsequence should be chosen. Moreover, there will still be a delay in anomaly detection that is bounded by the length of the subsequence. Bu et al. (2009) developed an algorithm for online learning and sequential anomaly detection in a single continuous trajectory stream. However, this algorithm has at least four free parameters and it is not clear how different values affect the expected anomaly detection rate.

3.2.3 Insensitivity to Local Sub-Trajectory Anomalies

Another limitation of previously proposed algorithms for anomalous trajectory detection is their insensitivity to local anomalous *sub-trajectories*. This is illustrated by Figure 3.2, which highlights two trajectories (Tr_1 and Tr_2) that are anomalous at different scales. The first trajectory (Tr_1) corresponds to a global anomaly that is anomalous in its entirety, since trajectories originating from **A** are expected to finish at **D** rather than **B**. Such anomalies are typically detectable by previously proposed algorithms. The second trajectory (Tr_2) corresponds to a local anomaly; while the overall route of (Tr_2) is consistent with other trajectories, the initial sub-trajectory (red) deviates significantly from the other local sub-trajectories in the area. Yet, most trajectory-based algorithms would probably miss this anomaly, since the distance between the anomalous sub-trajectory and its nearby sub-trajectories is less than the distance between neighbouring sub-trajectories in other less dense areas, such as the central crossing area in Figure 3.2. The under-

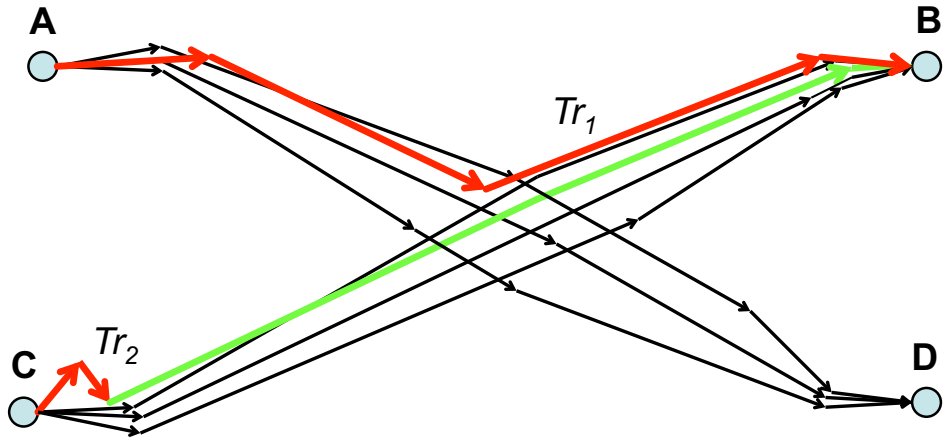


Figure 3.2: Illustration of a global trajectory anomaly (Tr_1), which is all red, vs. a local sub-trajectory anomaly (Tr_2), where red and green correspond to the anomalous and normal part of the trajectory, respectively.

lying models and similarity measures of previous algorithms essentially consider trajectories *as a whole* and do not consider local variations in density. Therefore, local anomalous sub-trajectories of the type in Figure 3.2 are "smoothed out". In fact, this limitation was previously identified by Lee et al. (2008) who argue that "a trajectory may have a long and complicated path" and that "even though some portions of a trajectory show an unusual behaviour, these differences might be averaged out over the whole trajectory". Motivated by this problem, they proposed an algorithm for detecting anomalous sub-trajectories, which takes into account the local data density (Lee et al., 2008). However, similar to most other algorithms (Sections 3.2.1–3.2.2), their algorithm is designed for the offline detection of anomalies in a trajectory database.

3.3 Identification of Key Properties

The limitations discussed above can be translated into a set of requirements that should be fulfilled by an algorithm for anomalous trajectory detection in surveillance applications. More specifically, we identify the following key properties:

Non-parametric The algorithm should not be based on any statistical assumptions regarding the *structure* of the data distribution (*cf.* Section 3.1.1).

Parameter-light The algorithm should have a minimum of parameters (*cf.* Section 3.1.2).

Well-calibrated The algorithm should provide a principled and well-founded approach to the tuning of the anomaly threshold, which can be directly

related to a desired, or acceptable, rate of detected anomalies, or false alarms, in new data (*cf.* Section 3.1.3).

Sequential detection The algorithm should support sequential analysis of each trajectory and detection of anomalies with minimum delay (*cf.* Section 3.2.2).

Unsupervised online learning The algorithm should support automated and incremental learning from unlabelled data, without the need for human feedback (*cf.* Section 3.2.1).

Multi-scale The algorithm should be sensitive to local sub-trajectory anomalies at different scales (*cf.* Section 3.2.3).

Considering previous work, we argue that there is no single algorithm for anomalous trajectory detection that fulfils all of the above requirements, Hence, there is a clear research gap, which we aim to bridge in the following chapters.

Chapter 4

Conformal Anomaly Detection

In the previous section, we identified a number of general limitations of anomaly detection algorithms that need to be addressed. In this section, we introduce and discuss conformal anomaly detection, which is a novel approach to anomaly detection that addresses these issues. Conformal anomaly detection is an extension of the *conformal prediction* (Vovk et al., 2005) framework, and we therefore first provide a brief overview of conformal prediction and its main results.

4.1 Preliminaries: Conformal Prediction

Conformal prediction (CP) is a technique that provides valid measures of confidence for individual predictions made by machine learning algorithms (Gammerman and Vovk, 2007). Assume a training set $z_1, \ldots, z_l$ where each example $z_i = (x_i, y_i) : i = 1, \ldots, l$ consists of features $x_i \in \mathbf{X}$ and label $y_i \in \mathbf{Y}$. Given a new example with observed features x_{l+1} and a predefined *significance level* $\epsilon \in (0, 1)$, the conformal predictor (Algorithm 4.1) outputs a *prediction set* $\Gamma^{\epsilon}_{l+1} \subseteq \mathbf{Y}$ for the corresponding label y_{l+1}. The prediction set is *valid* at the specified significance level in the sense that the probability of an *error*, i.e., the event that the prediction set does *not* include the true label, is guaranteed to be less than or equal to ϵ:

$$Pr\left(y_{l+1} \notin \Gamma^{\epsilon}_{l+1}\right) \leq \epsilon, \tag{4.1}$$

under the assumption that $z_1, \ldots, z_{l+1}$ are *independent and identically distributed* (IID) (Vovk et al., 2005). For example, with the setting $\epsilon = 0.05$, we know that the probability that a prediction set includes the true label is at least 95%, regardless of the structure of the underlying probability distribution, which may be unknown. That is, we have 95% confidence in the prediction set.

The central idea in CP (Algorithm 4.1) is to estimate the *p-value*, $p^Y \in (0, 1)$, for each possible label $Y \in \mathbf{Y}$ of the new example z_{l+1} and include in the prediction set only those labels for which p^Y is at least as large as ϵ (Gammerman and Vovk, 2007):

Algorithm 4.1 Conformal predictor

Input: NCM A, significance level ϵ, training set $(z_1, \ldots, z_l)$, new features x_{l+1}.
Output: Prediction set Γ^{ϵ}_{l+1} for the new label y_{l+1}.

$\Gamma^{\epsilon}_{l+1} \leftarrow \emptyset$
for all $y \in \mathbf{Y}$ **do**
  $z_{l+1} \leftarrow (x_{l+1}, y)$
  for $i \leftarrow 1$ to $l+1$ **do**
    $B_i \leftarrow \{z_1, \ldots, z_{l+1}\} \setminus z_i$
    $\alpha_i \leftarrow A(B_i, z_i)$
  end for
  $p^Y \leftarrow \frac{|\{i=1,\ldots,l+1 : \alpha_i \geq \alpha_{l+1}\}|}{l+1}$
  if $p^Y > \epsilon$ **then**
    $\Gamma^{\epsilon}_{l+1} \leftarrow \Gamma^{\epsilon}_{l+1} \cup y$
  end if
end for

$$\Gamma^{\epsilon}_{l+1} = \left\{ Y : Y \in \mathbf{Y}, p^Y \geq \epsilon \right\}. \tag{4.2}$$

In order to estimate the p-values, the concept of a *nonconformity measure* (NCM) is introduced, which measures how "different" an example is relative to a set of examples. Formally, an NCM is a real-valued function $A\big(B, z\big)$ that assigns a nonconformity score α for the example z relative to the examples in the *bag* (also known as multi-set) B (Shafer and Vovk, 2008). For each $y \in \mathbf{Y}$, the nonconformity scores $\alpha_i : i = 1, \ldots, l+1$ for each example z_i relative to the rest of the examples $B_i = \{z_j : j = 1, \ldots, l+1, j \neq i\}$ are first calculated using A, where $z_{l+1} = (x_{l+1}, y)$. Next, p^Y is estimated as the ratio of the nonconformity scores $\alpha_1, \ldots, \alpha_{l+1}$ that are at least as large as α_{l+1}:

$$\frac{\big|\{i = 1, \ldots, l+1 : \alpha_i \geq \alpha_{l+1}\}\big|}{l+1}. \tag{4.3}$$

Considering an example with observed features x_{l+1}, it is expected that any hypothetical label $Y \neq y_{l+1}$, i.e., any label other than the true label, will result in the corresponding α_{l+1} being relatively large compared to $\alpha_1, \ldots, \alpha_l$. Hence, the p-value is expected to be very low for any label other than the true label.

The validity property of conformal predictors (4.1) can be further strengthened by slightly modifying (4.3) to:

$$\frac{\big|\{i = 1, \ldots, l+1 : \alpha_i > \alpha_{l+1}\}\big| + \tau\big|\{i = 1, \ldots, l+1 : \alpha_i = \alpha_{l+1}\}\big|}{l+1}, \tag{4.4}$$

where $\tau \in [0, 1]$ is an independent and random sample from the uniform distribution (Vovk et al., 2005). The result of estimating *smoothed p-values* according to (4.4) is that the probability of error is now *exactly* equal to ϵ (Vovk et al., 2005):

$$Pr\left(y_{l+1} \notin \Gamma^{\epsilon}_{l+1}\right) = \epsilon. \tag{4.5}$$

One of the most important properties of conformal predictors based on (4.4) is that the successive prediction errors are *independent* during online learning and prediction (Gammerman and Vovk, 2007). In online learning and prediction, the training set is incrementally updated with the features *and* the label of the current test example before the prediction of the label of the next example: Assume a sequence of new examples $z_{l+1}, \ldots, z_N$. For each $n = l+1, \ldots, N$, the online conformal predictor first outputs a prediction set for y_n based on the observed features x_n and the training set of examples $z_1, \ldots, z_{n-1}$ that have been observed so far. The true value of y_n is then provided and $z_n = (x_n, y_n)$ appended to the training set before the label of the next example z_{n+1} is predicted, and so forth. Let $\mathrm{err}^{\epsilon}_n \in \{0, 1\}$ be an indicator variable for the event that $y_n \notin \Gamma^{\epsilon}_n$. Then, the sequence of random variables $\mathrm{err}^{\epsilon}_{l+1}, \ldots, \mathrm{err}^{\epsilon}_N$ are independent and will each take value 1 with probability ϵ, under the assumption that $z_1, \ldots, z_N$ are IID (Gammerman and Vovk, 2007). If $N-1$ is large, it follows from the law of large numbers that the empirical error rate will be equal to ϵ up to statistical fluctuations. That is, the online conformal predictor is *well-calibrated* (Gammerman and Vovk, 2007).

A conformal predictor will produce valid prediction sets that satisfy (4.1) or (4.5) using *any* real-valued function $A\big(B, z\big)$ as the NCM. However, the prediction sets will only be small and informative if an appropriate NCM is chosen. Generally, the performance of conformal predictors is measured in terms of the size of the prediction sets, which is known as the *efficiency* (Gammerman and Vovk, 2007). Several NCMs based on different machine learning algorithms have previously been proposed and investigated for CP, e.g., the k-nearest neighbours algorithm (Vovk et al., 2005), (kernel) ridge regression (Vovk et al., 2005), SVM (Vovk et al., 2005), neural networks (Papadopoulos et al., 2007), random forests (Devetyarov and Nouretdinov, 2010) and genetic algorithms (Lambrou et al., 2009).

A drawback of the CP framework presented above is that it may be computationally inefficient. Re-calculating the nonconformity score for each possible label and example in the training set may become computationally unfeasible, particularly if the underlying NCM has high computational complexity. This is typically the case for NCMs based on SVM, neural networks, etc. The computational inefficiency of conformal predictors is addressed by a modification of CP known as *inductive conformal prediction* (ICP) (Papadopoulos, 2008; Vovk et al., 2005), which has been investigated in combination with a neural network-based NCM (Papadopoulos et al., 2007). In general, the computational efficiency is considerably improved at the expense of some potential loss in prediction efficiency (Gammerman and Vovk, 2007), i.e., the prediction sets may, on average, be slightly larger for inductive conformal predictors. Nevertheless, inductive conformal predictors retain the validity properties of regular conformal predictors (Vovk et al., 2005).

4.2 Conformal Prediction for Multi-Class Anomaly Detection

Consider the general supervised classification setting (Section 4.1) where each example z_i consists of the object x_i and the label y_i and the task is to predict the unknown label y_{l+1} based on the observed object x_{l+1} and the training set $(z_1, \ldots, z_l)$. A conformal predictor outputs a prediction set $\Gamma^\epsilon(z_1, \ldots, z_l, x_{l+1}) \subseteq \mathbf{Y}$ for the label y_{n+1} at the specified significance level ϵ. Now, assuming that the significance level is low, say 1% or less, an interesting situation arises if the prediction set is empty; this happens if all p-values (one for each possible class) are below ϵ and it indicates that the new object x_{l+1} is a novelty that does not match any of the previous objects $x_i : i = 1, \ldots, l$. An empty prediction set at significance level ϵ is equivalent to the case of a single label prediction that has credibility less than ϵ (Gammerman and Vovk, 2007). As discussed by Gammerman and Vovk (2007), "low credibility means that either the training set is non-random or the test object is not representative of the training set". As an example, the authors discuss an Optical Character Recognition (OCR) application, where an empty prediction set may arise if the new image corresponds to a letter while the training set consists of images of digits. Another interesting situation arises if a non-empty prediction set does not include the true label. This means that the object was recognised but the (later observed) class label was unexpected. An example of such a situation would be a badly written "7" that resembles a "1" (or vice versa). In both situations, the prediction set is incorrect, but we may suspect that the corresponding example is an outlier according to Hawkins definition. That is, it is not a random sample from the same distribution as the training set. In general, if the new example and the training set are IID or exchangeable, we know from (4.1) that the probability of error is bounded by ϵ. Thus, if we classify examples as anomalous when the corresponding prediction set is incorrect, we know that the specified significance level ϵ corresponds to an upper bound of the probability of an anomaly detection.

4.2.1 A Nonconformity Measure for Multi-class Anomaly Detection

Gammerman and Vovk (2007) proposed the following k-nearest neighbours NCM for classification applications:

$$\alpha_i := \frac{\sum_{j=1}^{k} d_{ij}^{+}}{\sum_{j=1}^{k} d_{ij}^{-}}, \tag{4.6}$$

where d_{ij}^{+} is the jth shortest distance in the object space from z_i to other examples with the same label and d_{ij}^{-} is the jth shortest distance from z_i to examples labelled differently. The intuition is rather simple: It is assumed that examples of the same (normal) class are close to each other in feature space, while examples of different

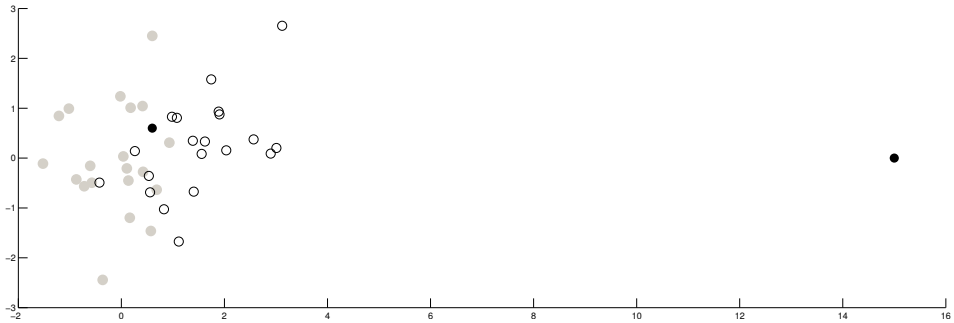


Figure 4.1: Illustration of the problem with the NCM based on distances to nearest neighbours of the same class and nearest neighbours of other classes discussed in Section 4.2.1

(normal) classes are further away from each other. Nevertheless, we argue that this NCM is suboptimal in multi-class anomaly detection applications if there are reasons to believe that normal classes overlap in the object space. The intuition of this argument is demonstrated with the following example, illustrated by Figure 4.1:

Assume a binary label space where the two class distributions (circles and solid light grey points) overlap in a high-density region in the object space. Considering an example that is located in the overlapping high-density region (left solid black point), we expect that the sum of the distances to its nearest neighbours will be approximately the same for the two classes. Thus, the nonconformity score according to (4.6) would be close to 1, regardless of which class the example actually belongs to. Moving away from both class distributions and considering another example that is located far to the right (right solid black point), the ratio of the sum of the nearest neighbours distances for the two classes will approach 1. Hence, the p-value of a normal point located in the overlapping high density area will be close to the p-value of an obviously anomalous point located far away from both class distributions. In response to this problem, we propose a modified NCM for the case of partially overlapping class distributions:

$$\alpha_i := \sum_{j=1}^{k} d_{ij}^{+}. \tag{4.7}$$

This NCM, which in contrast to (4.6) does not consider the distances to other classes, is expected to be more sensitive to anomalous points located far away from all known classes.

Remark. The problem with using (4.6) for detecting outliers was in fact first identified by Barbará and Domeniconi (2006) and later independently rediscovered by Laxhammar and Falkman (2010). In the work by Barbará and Domeniconi (2006), a cluster-based outlier detection algorithm was proposed based on ideas

from *transductive confidence machines* (Proedrou et al., 2002), which is a precursor to conformal predictors. Given a cluster model and a new example, they proposed the use of (4.7) to estimate the p-values for the new example belonging to each of the corresponding clusters. If all p-values are below a specified threshold, the example is considered to be an outlier.

4.3 Conformal Anomaly Detection

In many applications, one is only interested in detecting anomalies and not in determining which, if any, of the normal classes the example belongs to. In this case, it makes more sense and is computationally more efficient to only estimate the p-value for the *observed* label, rather than to calculate p-values for all possible labels $Y \in \mathbf{Y}$. Moreover, the structured prediction setting discussed in Sections 4.1–4.2 may be considered as a special case of the more general and "unsupervised" problem of predicting the example z_{l+1} based on the training set $(z_1, \ldots, z_l)$ (Laxhammar, 2014). Hence, in the remainder of the thesis, we consider the problem of classifying an unlabelled example z_{l+1} as normal or abnormal based on a training set $(z_1, \ldots, z_l)$.

4.3.1 The Conformal Anomaly Detector

The conformal anomaly detector (CAD) (Algorithm 4.2) is a general algorithm for anomaly detection that is based on the framework of CP (Section 4.1). Given a test example z_{l+1}, CAD estimates the p-value p_{l+1} for z_{l+1} relative to the training set $(z_1, \ldots, z_l)$ based on a specified NCM A. This is done by first calculating the nonconformity score α_i for each example $z_i : i = 1, \ldots, l+1$ relative to the rest $\{z_1, \ldots, z_{l+1}\} \setminus z_i$ using A. Next, p_{l+1} is estimated according to (4.3). If p_{l+1} is below the predefined anomaly threshold $\epsilon \in (0, 1)$, z_{l+1} is classified as a *conformal anomaly*:

$$\text{Anom}_{l+1}^{\epsilon} = 1. \tag{4.8}$$

Otherwise, z_{l+1} is considered to be *normal*:

$$\text{Anom}_{l+1}^{\epsilon} = 0. \tag{4.9}$$

The definition of a conformal anomaly is consistent with the statistical definition of an outlier according to Hawkins (1980). That is, a conformal anomaly corresponds to an example z_{l+1} that deviates so much from $(z_1, \ldots, z_l)$ as to arouse suspicion that it was generated by a different mechanism. Conformal anomaly detection can be viewed from the perspective of *statistical hypothesis testing*, where the null hypothesis that the new example z_{l+1} and the training set $(z_1, \ldots, z_l)$ are IID is tested at significance level ϵ (*cf.* Shafer and Vovk (2008)). If the null hypothesis is true, i.e., z_{l+1} and $(z_1, \ldots, z_l)$ are in fact IID, then ϵ is an upper

Algorithm 4.2 The Conformal Anomaly Detector (CAD)

Input: NCM A, anomaly threshold ϵ, training set $(z_1, \ldots, z_l)$ and test example z_{l+1}.
Output: Indicator variable $\text{Anom}^{\epsilon}_{l+1} \in \{0, 1\}$.

1: **for** $i \leftarrow 1$ to $l + 1$ **do**
2: $\quad \alpha_i \leftarrow A\left(\{z_1, \ldots, z_{l+1}\} \setminus z_i, z_i\right)$
3: **end for**
4: $p_{l+1} \leftarrow \frac{|\{i=1,\ldots,l+1 : \alpha_i \geq \alpha_{l+1}\}|}{l+1}$
5: **if** $p_{l+1} < \epsilon$ **then**
6: $\quad \text{Anom}^{\epsilon}_{l+1} \leftarrow 1$
7: **else**
8: $\quad \text{Anom}^{\epsilon}_{l+1} \leftarrow 0$
9: **end if**

bound of the probability of the event that the null hypothesis is rejected, i.e., that z_{l+1} is classified as a conformal anomaly. Hence, for any choice of NCM A, the specified anomaly threshold ϵ is an upper bound of the probability of the event that z_{l+1} is classified as a conformal anomaly:

$$Pr\left(\text{Anom}^{\epsilon}_{l+1} = 1\right) \leq \epsilon, \tag{4.10}$$

under the assumption that $z_1, \ldots, z_{l+1}$ are IID. The larger the value of ϵ, the higher the expected rate of detected conformal anomalies.

Analogously to CP (Section 4.1), the deterministic p-value (4.3) (line 4 of Algorithm 4.2) may be replaced by the corresponding smoothed p-value (4.4), in which case the probability of a conformal anomaly becomes exactly equal to ϵ:

$$Pr\left(\text{Anom}^{\epsilon}_{l+1} = 1\right) = \epsilon. \tag{4.11}$$

From an application perspective, there are at least three different explanations for a conformal anomaly. Firstly, it may correspond to a relatively rare or previously unseen example generated from the same probability distribution as $(z_1, \ldots, z_l)$. Secondly and thirdly, it may be a "true" anomaly in the sense that it was not generated according to the same probability distribution as $(z_1, \ldots, z_l)$; either z_{l+1} is a true novelty, or $(z_1, \ldots, z_l)$ itself is in fact not IID. A non-IID training set may be explained by incomplete or biased data collection. In a public video surveillance application, e.g., observed behaviour during early morning or late afternoon may appear anomalous if the training set is based on data recorded during a limited time of the day, e.g., 10 a.m. to 2 p.m. Another possible reason for a non-IID training set is that the underlying probability distribution has actually changed. In maritime surveillance, e.g., new vessel trajectories may arise as a result of new traffic regulations. If the rate of detected conformal anomalies suddenly starts to deteriorate from ϵ in an online application, there may be reasons to suspect that the underlying probability distribution has changed recently.

4.3.2 Offline and Online Conformal Anomaly Detection

Assume an initial training set $(z_1, \ldots, z_l)$ and a sequence of test examples $z_{l+1}, \ldots, z_N$. The *offline* CAD classifies each $z_n : n = l+1, \ldots, N$ based on the same training set $(z_1, \ldots, z_l)$. In contrast, the *online* CAD classifies each z_n based on the *updated* training set $(z_1, \ldots, z_{n-1})$. That is, z_{n-1} is appended to the training set before the classification of z_n. If $z_1, \ldots, z_n$ are IID, we know that the *unconditional* probability of the event that z_n is classified as a conformal anomaly is bounded by ϵ (Section 4.3.1). Nevertheless, the *empirical rate* of detected anomalies for the offline CAD may be significantly higher than ϵ, even if $N - l$ is large, since the sequence of random variables:

$$\text{Anom}^{\epsilon}_{l+1}, \ldots, \text{Anom}^{\epsilon}_{N} \tag{4.12}$$

are *not* independent.

For the online CAD, however, the empirical rate of detected anomalies is *guaranteed* to be less or equal to ϵ, up to statistical fluctuations. To see this, let us temporarily assume that the deterministic p-value (line 5 of Algorithm 4.2) for each test example $z_n : n = l+1, \ldots, N$ is replaced by the corresponding smoothed p-value estimated according to (4.4): It has previously been shown that the sequence of smoothed p-values $p_{l+1}, \ldots, p_N$ are *independent* in the online framework under the assumption that $z_{l+1}, \ldots, z_N$ are IID (Vovk et al., 2005, Theorem 8.2). This implies that (4.12) are also independent and that each will take value 1 with probability ϵ. The rate of detected anomalies will therefore be equal to ϵ, up to statistical fluctuations. In particular, it follows from the law of large numbers that:

$$\lim_{N \to \infty} \frac{|\text{Anom}^{\epsilon}_{n} = 1 : n = l+1, \ldots, N|}{N - l} = \epsilon. \tag{4.13}$$

Now, considering the case of deterministic p-values, it is intuitively clear that the rate of detected conformal anomalies must be less or equal to the corresponding rate for the case of smoothed p-values above, since the probability that each $z_n : n = l+1, \ldots, N$ is classified as a conformal anomaly is always less or equal to ϵ. The rate of detected anomalies for the online CAD is therefore guaranteed to be less or equal to ϵ, up to statistical fluctuations. Asymptotically, the rate of detected anomalies will never exceed ϵ:

$$\limsup_{N \to \infty} \frac{|\text{Anom}^{\epsilon}_{n} = 1 : n = l+1, \ldots, N|}{N - l} \leq \epsilon. \tag{4.14}$$

We refer to this property as *well-calibrated alarm rate*.

4.3.3 Unsupervised and Semi-Supervised Conformal Anomaly Detection

The conformal anomaly detection framework is essentially based on an unsupervised learning paradigm where there is no explicit notion of normal and abnormal

classes. However, from an application owner's or domain expert's point of view, each example is often considered to belong to either a normal or an abnormal class (Section 2.1.1). Hence, when classifying an unlabelled example using CAD, there are generally two types of possible errors. The first error is when a normal example is classified as a conformal anomaly, i.e., a false alarm. The second error is when an abnormal example is *not* classified as a conformal anomaly.

Analogously to Eskin (2000), let us assume that examples of the normal and abnormal classes are generated according to the unknown probability distributions P_{Normal} and $P_{Abnormal}$ and that the unknown prior probability of the abnormal class is equal to λ. The generative distribution for *unlabelled* examples can then be modelled as a *mixture model*:

$$P_{Examples} = (1 - \lambda) P_{Normal} + \lambda P_{Abnormal}. \tag{4.15}$$

Depending on whether the training set is assumed to be unlabelled or labelled normal, CAD can be considered to operate in an unsupervised or semi-supervised anomaly detection mode, respectively (Section 2.1). The unsupervised online CAD is perhaps the most interesting from both a theoretical and practical point of view; it does not require any label feedback and the overall rate of detected conformal anomalies is well-calibrated, regardless of P_{Normal}, $P_{Abnormal}$ and λ. The only assumption is that $(z_1, \ldots, z_N)$ constitutes an IID sample from $P_{Examples}$, which does not seem to be an impractical or otherwise unrealistic assumption in an anomaly detection application.

The semi-supervised online CAD has a similar notion of well-calibrated alarm rate: Assume that $\left(z'_{l+1}, \ldots, z'_Q\right) : Q \leq N$ corresponds to the subsequence of normal test examples of the full sequence $(z_{l+1}, \ldots, z_N)$, and that each $z'_n : n = l + 2, \ldots, Q$ is classified based on the updated training set $\left(z_1, \ldots, z_l, z'_{l+1}, \ldots, z'_{n-1}\right)$. Further, assume that $Q - l$ is large and that $\left(z_1, \ldots, z_l, z'_{l+1}, \ldots, z'_Q\right)$ constitutes an IID sample from P_{Normal}. Then, the rate of the normal test examples $\left(z'_{l+1}, \ldots, z'_Q\right)$ that are erroneously classified as anomalous, i.e., the false alarm rate (Section 2.1.1), will be well-calibrated. The semi-supervised approach is, however, less practical in the online mode, since it requires label feedback for each $z_n : n = l + 1, \ldots, N$ in order to know whether z_n is part of $\left(z'_{l+1}, \ldots, z'_Q\right)$ and, hence, should be added to the training set before the classification of z_{n+1}. The semi-supervised offline CAD mitigates this practical problem at the cost of losing the theoretical guarantee of a well-calibrated false alarm rate.

4.3.4 Importance of the Nonconformity Measure

Regardless of whether CAD operates in the unsupervised or semi-supervised mode, classification performance is obviously dependent on the chosen NCM and how well it discriminates between examples from $P_{Abnormal}$ and P_{Normal}. In the unsupervised mode, classification performance is further dependent on the character of $P_{Abnormal}$; intuitively, if the abnormal examples vary greatly, i.e., $P_{Abnormal}$ has high entropy, then performance in the unsupervised mode should not be worse

than in the semi-supervised mode. However, if abnormal examples vary relatively little from each other, i.e., $P_{Abnormal}$ has very small entropy, then performance in the unsupervised mode may suffer as new abnormal examples might not differ significantly from the abnormal examples in the training set.

4.3.5 Tuning of the Anomaly Threshold

The parameter ϵ regulates the sensitivity to abnormal examples, i.e., the proportion of abnormal examples that are detected as conformal anomalies. It also affects the precision, i.e., the proportion of abnormal examples among the detected conformal anomalies. A higher value of ϵ may increase the sensitivity, but at the same time reduce the precision due to an increased false alarm rate. As discussed in Section 3.1.3, the precision is of critical importance and is heavily influenced by the false alarm rate in an anomaly detection application.

Considering the unsupervised CAD, it may be argued that ϵ should be set close to λ from (4.15) in order to achieve a good balance between the sensitivity and precision. Indeed, assuming an ideal NCM such that $\alpha_i > \alpha_j$ for any examples z_i and z_j belonging to the abnormal and normal classes, respectively, it is intuitively clear that setting $\epsilon = \lambda$ will result in the sensitivity and precision both being close to 1. Nevertheless, it may be difficult to estimate the true value of λ. An alternative strategy is to set ϵ on the basis of the rate of false alarms that is acceptable: Recall that the overall alarm rate of the unsupervised online CAD is guaranteed to be less than or close to ϵ under the IID assumption (Sections 4.3.2 and 4.3.3). Assuming that the sensitivity is at least as large as the false alarm rate, the false alarm rate is also guaranteed to be less than or close to ϵ. Hence, ϵ may be set to the maximum rate of false alarms that is acceptable to the human operator. The latter strategy also applies to the case of the semi-supervised CAD, where ϵ corresponds to the expected false alarm rate (Section 4.3.3).

Nevertheless, setting $\epsilon < \frac{1}{l+1}$ should always be avoided for both the unsupervised and semi-supervised CAD and regardless of the NCM, since the sensitivity to abnormal examples will then be zero. To see this, assume we observe an abnormal example z_{l+1} such that $\alpha_{l+1} \gg \alpha_i : i = 1, \ldots, l$. From (4.3), it is clear that $p_{l+1} = \frac{1}{l+1}$. Hence, if $\epsilon < \frac{1}{l+1}$, then z_{l+1} will not be classified as anomalous, even though it appears to be very extreme.

4.4 Inductive Conformal Anomaly Detection

A drawback of CAD is that it is computationally inefficient; in addition to calculating the nonconformity score α_{l+1} for the test example z_{l+1}, CAD has to recalculate the nonconformity score for each example $z_1, \ldots, z_l$ in the training set. When l is large, CAD may therefore become computationally unfeasible, especially when the computational complexity of the underlying NCM is relatively high. In response to this problem, we propose the *inductive conformal anomaly detector* (ICAD) (Algorithm 4.3), which is an extension of CAD based on the concept of ICP (Sec-

tion 4.1). Similar to ICP, the idea is to use the precomputed nonconformity scores for previous examples when estimating the p-value for a new test example. More specifically, the training set is split into the *proper training set* $(z_1, \ldots, z_m)$ of size $m < l$ and the *calibration set* $(z_{m+1}, \ldots, z_l)$ of size $l-m$. Further, the nonconformity score α_i is precomputed for each example $z_i : i = m+1, \ldots, l$ of the calibration set relative to the proper training set. Given a test example z_{l+1}, ICAD calculates the nonconformity score α_{l+1} for z_{l+1} relative to the proper training set. Next, p_{l+1} is estimated by comparing α_{l+1} with the precomputed nonconformity scores $(\alpha_{m+1}, \ldots, \alpha_l)$:

$$\frac{|\{i = m+1, \ldots, l+1 : \alpha_i \geq \alpha_{l+1}\}|}{l-m}. \tag{4.16}$$

Thus, the computational complexity of ICAD is reduced by a factor l as compared to CAD. Analogously to CAD (Section 4.3.1), the probability of an inductive conformal anomaly is bounded by ϵ, i.e., (4.10) is still true for ICAD.

Algorithm 4.3 The Inductive Conformal Anomaly Detector (ICAD)

Input: NCM A, anomaly threshold $\epsilon \in (0, 1)$, proper training set $(z_1, \ldots, z_m)$, pre-computed nonconformity scores $(\alpha_{m+1}, \ldots, \alpha_l)$ and test example z_{l+1}.
Output: Indicator variable $\text{Anom}^{\epsilon}_{l+1} \in \{0, 1\}$.

1: $\alpha_{l+1} \leftarrow A\left(\{z_1, \ldots, z_m\}, z_{l+1}\right)$
2: $p_{l+1} \leftarrow \frac{|\{i=m+1,\ldots,l+1:\alpha_i \geq \alpha_{l+1}\}|}{l-m}$
3: **if** $p_{l+1} < \epsilon$ **then**
4: $\quad \text{Anom}^{\epsilon}_{l+1} \leftarrow 1$
5: **else**
6: $\quad \text{Anom}^{\epsilon}_{l+1} \leftarrow 0$
7: **end if**

4.4.1 Offline and Semi-Offline Inductive Conformal Anomaly Detection

Analogously to ICP (Vovk et al., 2005), ICAD can run in an offline, semi-offline or online mode, depending on whether the set of precomputed nonconformity scores and the proper training set are updated or not. For each test example $z_n : n = l+1, \ldots, N$ the *offline* ICAD reuses the same proper training set $(z_1, \ldots, z_m)$ and sequence of precomputed nonconformity scores $(\alpha_{m+1}, \ldots, \alpha_l)$. Similar to the offline CAD, the unconditional probability that z_n is classified as a conformal anomaly is bounded by ϵ under the assumption that $(z_{m+1}, \ldots, z_l)$ and z_n are IID. Yet, the empirical rate of detected anomalies for the offline ICAD may still be significantly higher than ϵ, since (4.12) are not independent.

In contrast, the *semi-offline* ICAD uses the *updated* sequence of nonconformity scores $(\alpha_{m+1}, \ldots, \alpha_l, \ldots, \alpha_n)$ when classifying each test example $z_n : n = l+1, \ldots, N$. That is, the nonconformity score α_{n-1} for the previous example z_{n-1} is

added to the sequence of precomputed nonconformity scores before classifying z_n. Analogously to the online CAD (Section 4.3.2), the online update of the calibration set implies that the alarm rate is well-calibrated. That is, the rate of detected anomalies is guaranteed to be less or equal to ϵ, up to statistical fluctuations, as long as the calibration set is IID. To see this, let us temporarily assume that line 2 of Algorithm 4.3 is replaced by the corresponding smoothed p-value estimated according to:

$$\frac{\left|\{i = m+1, \ldots, l+1 : \alpha_i \geq \alpha_{l+1}\}\right| + \tau\left|\{i = m+1, \ldots, l+1 : \alpha_i = \alpha_{l+1}\}\right|}{l-m}. \tag{4.17}$$

From Vovk et al. (2005, Theorem 8.2) it then follows that $p_{l+1}, \ldots, p_N$ are independent under the assumption that $z_{l+1}, \ldots, z_N$ are IID. Hence, (4.12) are also independent and will each take value 1 with probability ϵ. Following the same argument for the alarm rate of the online CAD (Section 4.3.2), it is clear that 4.14 is also true for the semi-offline ICAD.

4.4.2 Online Inductive Conformal Anomaly Detection

The *online* ICAD is a generalisation of the semi-offline ICAD; in addition to updating the sequence of precomputed nonconformity scores, the online ICAD involves updating the training set at the predefined *update trials*:

$$m_1, m_2, \ldots : l < m_1 < m_2 < \cdots, \tag{4.18}$$

i.e., similar to the online ICP (Vovk et al., 2005, Section 4.1). Assume the initial proper training set $(z_1, \ldots, z_{m_0})$, the calibration set $(z_{m_0+1}, \ldots, z_l)$ and the corresponding precomputed nonconformity scores $(\alpha_{m_0+1}, \ldots \alpha_l)$, where $m_0 < l$. For each test example $z_n : n = l+1, \ldots, m_1$, the fixed proper training set $(z_1, \ldots, z_{m_0})$ and the updated sequence of nonconformity scores $(\alpha_{m_0+1}, \ldots, \alpha_{n-1})$ are used for the classification, i.e., similar to the semi-offline ICAD above. However, prior to the classification of the next test example z_{m_1+1}, which corresponds to the first update trial, the proper training set is updated to $(z_1, \ldots, z_{m_1})$ and the sequence of nonconformity scores is reset. For each subsequent example $z_n : n = m_1+2, \ldots, m_2$, the fixed proper training set $(z_1, \ldots, z_{m_1})$ and updated sequence of nonconformity score $(\alpha_{m_1+1}, \ldots, \alpha_{n-1})$ are used. Generally, for each test example $z_{m_j+1} : j = 1, 2, \ldots$, the proper training set is updated to $\left(z_1, \ldots, z_{m_j}\right)$ and the sequence of precomputed nonconformity scores is reset.

A drawback of the online approach above is that the initial sequence of p-values after each update trial is not very useful. The reason for this is that the size of the calibration set is still too small to provide any significant p-values for new examples; note that even if $\alpha_n \gg \alpha_i : i = m_j + 1, \ldots, n-1$, the p-value for z_n can never be less than $(n - m_j)^{-1}$, where m_j corresponds to the most recent update trial. If $\epsilon < (n - m_j)^{-1}$, which is typically the case after an update trial, sensitivity to abnormal examples will suffer severely, since no conformal anomalies will be

detected. Of course, this problem can be mitigated by temporarily increasing the value of ϵ, but this will disrupt the well-calibrated alarm rate and likely result in more false alarms.

A More Practical Online Approach

In order to address the practical problem discussed above, we propose the following alternative update strategy for the online ICAD: For each test example $z_{m_j+1} : j = 1, 2, \ldots$, only the subsequence $\left(z_{m_{j-1}+1}, \ldots, z_{m_j - \lceil \epsilon^{-1} \rceil}\right)$ of the calibration set is removed and added to the training set. For each $z_i : i = m_j - \left\lceil \epsilon^{-1} \right\rceil + 1, \ldots, m_j$ that remains in the calibration set, we re-calculate α_i relative to the updated proper training set $\left(z_1, \ldots, z_{m_j - \lceil \epsilon^{-1} \rceil}\right)$. The sequence of examples $z_n : n = m_j + 1, \ldots, m_{j+1}$ until the next update trial m_{j+1} is classified using the fixed proper training set:

$$\left(z_1, \ldots, z_{m_j - \lceil \epsilon^{-1} \rceil}\right) \tag{4.19}$$

and the updated sequence of nonconformity scores:

$$\left(\alpha_{m_j - \lceil \epsilon^{-1} \rceil + 1}, \ldots, \alpha_{n-1}\right). \tag{4.20}$$

This strategy ensures that sensitivity to abnormal examples will not suffer from a too small calibration set; the number of calibration nonconformity scores is never less than $\left\lceil \epsilon^{-1} \right\rceil$. Nevertheless, the property of well-calibrated alarm rate across different update trials is now lost; the corresponding smoothed p-values from different update trails are no longer independent, since some of the examples are used for calibration across two or more update trails. Hence, in contrast to the standard online inductive approach, the overall alarm rate is not guaranteed to be well-calibrated when considering a sequence of test examples over multiple update trails. In practice, however, this seems to have limited impact on the empirical alarm rate, which we investigate in Section 6.3.2.

Tuning of the Update Frequency

For simplicity, the sequence of update trails can be chosen so that:

$$m_j = m_{j-1} + \eta : j = 1, 2, \ldots, \tag{4.21}$$

where the parameter $\eta \in \mathbb{N}^+$ regulates the frequency of the update trails. Setting $\eta = \infty$ is equivalent to the semi-offline ICAD and is obviously computationally efficient since the training set is never updated. Nevertheless, this setting may result in reduced sensitivity to abnormal examples, especially if the initial training set is small. Conversely, setting $\eta = 1$ implies that every iteration is an update trail and that maximum possible sensitivity is achieved. The drawback of setting $\eta = 1$ is, of course, increased computational complexity and that the guarantee of a well-calibrated alarm rate is completely lost.

4.5 Summary

In this chapter, we have presented the framework of conformal anomaly detection, which is a novel approach to anomaly detection based on conformal prediction (Section 4.1). Conformal prediction was originally developed for the problem of supervised learning and prediction with valid confidence. However, some of the main theoretical results of conformal prediction are also highly relevant to the problem of anomaly detection. The key observations are two-fold: The first is that the p-value for an observed example is a general and useful anomaly measure and that the significance level ϵ, when used as an anomaly threshold, corresponds to an upper bound of the unconditional probability of detecting an anomaly. The second key observation is that the smoothed p-values for a sequence of IID examples are independent in the online framework, which implies that the rate of detected anomalies can be regulated by the setting of ϵ.

In Section 4.2, the application of a supervised conformal predictor for multi-class anomaly detection is discussed. This includes the proposal and discussion of an updated k-nearest neighbours NCM that is more appropriate for multi-class anomaly detection applications than previous k-nearest neighbours based NCMs. In Section 4.3.1, CAD is formalised, which is a general algorithm for one-class anomaly detection. The main idea of CAD is to estimate the p-value for a test example relative to a training set based on a specified NCM. If the p-value is below the predefined anomaly threshold ϵ, the example is classified as a conformal anomaly. Offline, online, unsupervised and semi-supervised conformal anomaly detection are formalised and properties related to the alarm rate and the classification performance in the different modes discussed. One of the key properties of the online CAD is that the overall alarm rate and the false alarm rate are well-calibrated in the unsupervised and the semi-supervised mode, respectively. Moreover, the effect of the NCM and ϵ on the classification performance of CAD is discussed. In the unsupervised mode, ϵ should generally be close to the estimated prior probability of an abnormal example. Alternatively, ϵ may be set to the maximum rate of false alarms that is acceptable. The latter strategy also applies in the semi-supervised mode.

One of the main drawbacks of CAD is that it is computationally inefficient. In response to this, ICAD is proposed (Section 4.4), which is based on ICP (Section 4.1) and, hence, is computationally more efficient than CAD. Analogously to ICP, ICAD can operate in an offline, semi-offline or online mode. The semi-offline and online ICAD are well-calibrated in the same sense as the online CAD. However, the sensitivity of the online ICAD after an update trial may suffer severely if the calibration set is too small. In order to address this, an alternative online protocol for ICAD is proposed, where the size of the calibration set is bounded. This ensures that the sensitivity does not deteriorate after an update trial, but at the loss of the theoretical guarantee of a well-calibrated alarm rate.

Chapter 5

Sequential Detection of Anomalous Trajectories

In the previous chapter, we are concerned with the general problem of anomaly detection, for which we propose the conformal anomaly detection framework, including CAD and ICAD. In this chapter, we adopt CAD for the problem of the sequential detection of anomalous trajectories. This includes the proposal of the *directed Hausdorff k-nearest neighbours nonconformity measure* (DH-kNN-NCM), which is a novel NCM designed for examples that are represented as sets of points and sequentially updated, such as trajectories. Based on DH-kNN-NCM, we propose and investigate the *sequential Hausdorff nearest neighbours conformal anomaly detector* (SHNN-CAD) for online learning and the sequential detection of anomalous trajectories. This includes empirical investigations on a set of public datasets of synthetic and real video trajectories. The algorithms and empirical results of this chapter have previously been published in Laxhammar and Falkman (2013a).

5.1 Sequential Conformal Anomaly Detection

The only design parameter of CAD is the NCM. In principle, any real-valued function $A(B, z)$ that adequately discriminates between examples from the normal and abnormal classes would be appropriate. Assuming that each example represents a complete trajectory, most, if not all, of the previously proposed trajectory similarity measures (Section 2.2) would be applicable as NCMs for offline conformal anomaly detection in a trajectory database. However, the application of CAD for sequential anomaly detection in trajectories imposes some further requirements on the NCM. That is, it should enable the computation of a *preliminary* nonconformity score for a partially observed trajectory; otherwise, anomaly detection is delayed until the complete trajectory has been observed. Further, the p-value

for an incomplete trajectory should *monotonically decrease* as the trajectory is sequentially updated; if this is not true, the online CAD is no longer guaranteed to have a well-calibrated alarm rate (Section 4.3.2). These requirements are not fulfilled by previous NCMs (e.g., Vovk et al., 2005) or other trajectory similarity measures proposed for anomaly detection.

In order to address these requirements, we propose DH-kNN-NCM for sequential conformal anomaly detection in trajectories, which is based on a k-nearest neighbours NCM in combination with the directed Hausdorff distance (DHD) (Section 5.1.1). One of the key properties of DHD is that examples, in our case trajectories, are represented as *point sets* of arbitrary size, which may be incrementally updated. That is, trajectories are not required to be complete or normalised to fixed-dimensional feature vectors. Moreover, DHD has some properties that can be used to prove the monotonicity of the p-values during the sequential update of an incomplete trajectory using DH-kNN-NCM (Theorem 1).

5.1.1 Preliminaries: Hausdorff Distance

Generally speaking, the Hausdorff distance is a dissimilarity measure for two sets of points in a metric space. It is a well-known distance measure in the field of computational geometry and image processing, where it has been applied for shape matching and shape recognition (Alt, 2009). Given two sets of points $A, B \subseteq \mathbb{R}^d$, the Directed Hausdorff Distance (DHD), $\overrightarrow{\delta}_H$, from A to B is defined as:

$$\overrightarrow{\delta}_H(A, B) = \max_{a \in A} \left\{ \min_{b \in B} \{dist(a, b)\} \right\}, \tag{5.1}$$

where distance between points is measured by some metric $dist(a, b)$, typically ED. That is, $\overrightarrow{\delta_H}(A, B)$ corresponds to the maximum distance from a point in A to the closest point in B. Assuming that the point sets represent different shapes, the DHD captures the degree to which shape A resembles some *part* of shape B. Hence, it is a natural distance measure when A is incomplete, since it does not require that every part of B is matched with some part of A. The DHD between two polygonal curves is illustrated in Figure 5.1. It should be noted that the DHD is not a metric since it is not symmetric; the reverse $\overrightarrow{\delta_H}(B, A)$ is in general different from $\overrightarrow{\delta_H}(A, B)$.

5.1.2 A Nonconformity Measure for Examples Represented as Sets of Points

We propose the following extension of (4.7) based on DHD: Assume a bag of unlabelled examples $\{z_1, \ldots, z_n\}$ where each $z_i \subseteq \mathbb{R}^d : i = 1, \ldots, n$ is represented by a non-empty set of points in a d-dimensional space. DH-kNN-NCM for z_i

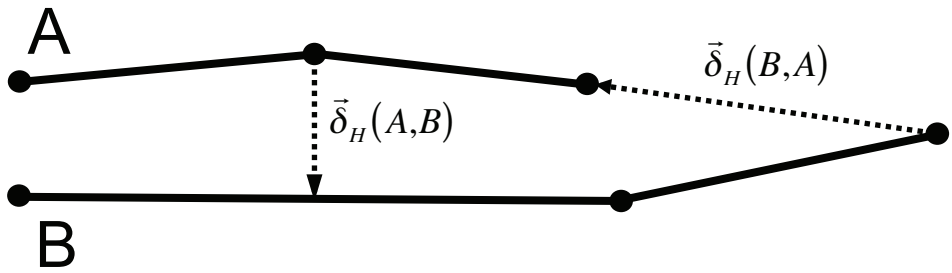


Figure 5.1: Illustration of the directed Hausdorff distances between two polygonal curves A and B.

relative to $\{z_1, \ldots, z_n\} \setminus z_i$ is defined as:

$$\alpha_i = \sum_{j=1}^{k} \overrightarrow{\delta_H}\left(z_i, NN\left(z_i, \{z_1, \ldots, z_n\} \setminus z_i, j\right)\right), \tag{5.2}$$

where $NN\left(z_i, \{z_1, \ldots, z_n\} \setminus z_i, j\right) \in \{z_1, \ldots, z_n\} \setminus z_i$ corresponds to the jth nearest neighbour to z_i according to (5.1). One of the key properties of (5.2) is that the preliminary p-value $\hat{p}_{l+1}$ for an incomplete test example $z^*_{l+1} \subset z_{l+1}$ monotonically decreases as more points are included from z_{l+1}; this property is formalised by Theorem 1, which is proven below:

Theorem 1. *Assume that z_i^* and z_i^{**} are non-empty subsets of z_i such that $z_i^* \subset z_i^{**} \subset z_i$. Then, the corresponding preliminary p-values $\hat{p}_i$ and $\hat{p}'_i$ and the final p-value p_i for the ith example relative to $\{z_1, \ldots, z_n\} \setminus z_i$, estimated according to (5.2), must satisfy $\hat{p}_i \geq \hat{p}'_i \geq p_i$. That is, the p-value for a subset of z_i monotonically decreases as more points are included in the subset.*

Proof. From (5.1) it is clear that $\overrightarrow{\delta_H}(z_i^*, z) \leq \overrightarrow{\delta_H}(z_i^{**}, z) \leq \overrightarrow{\delta_H}(z_i, z)$ for any $z \in \{z_1, \ldots, z_n\}$; the maximum distance from any point $a \in A$ to the closest point in B monotonically increases as more points are added to A. Thus, the sum in (5.2) also monotonically increases as more points are considered from z_i. The preliminary nonconformity scores $\hat{\alpha}_i$ and $\hat{\alpha}'_i$ and the final nonconformity score α_i for the ith example relative to $\{z_1, \ldots, z_n\} \setminus z_i$, calculated according to (5.2), must therefore satisfy:

$$\hat{\alpha}_i \leq \hat{\alpha}'_i \leq \alpha_i. \tag{5.3}$$

Further, considering (5.1) we see that the distance from each point a from a fixed set A to the closest point in B monotonically decreases as more points are added to B. Hence, it follows that $\overrightarrow{\delta_H}(z_j, z_i^*) \geq \overrightarrow{\delta_H}(z_j, z_i^{**}) \geq \overrightarrow{\delta_H}(z_j, z_i)$ for any $z_j \in \{z_1, \ldots, z_n\}$. Consequently, the sum in (5.2) also monotonically decreases as more points are considered from z_i. This implies that preliminary nonconformity scores $\hat{\alpha}_j$ and $\hat{\alpha}'_j$ and the final nonconformity score α_j for any other example

$z_j \in \{z_1, \ldots, z_n\} \setminus z_i$ relative to $\{z_1, \ldots, z_n\} \setminus z_j$, calculated according to (5.2), must satisfy:

$$\hat{\alpha}_j \geq \hat{\alpha}'_j \geq \alpha_j. \tag{5.4}$$

From (5.3) and (5.4), it follows that:

$$|\{\hat{\alpha}_j \geq \hat{\alpha}_i\}| \geq \left|\{\hat{\alpha}'_j \geq \hat{\alpha}'_i\}\right| \geq |\{\alpha_j \geq \alpha_i\}| : j = 1, \ldots, n.$$

Hence, the corresponding p-values must satisfy $\hat{p}_i \geq \hat{p}'_i \geq p_i$. □

A consequence of Theorem 1 is that a well-calibrated alarm rate is maintained for any conformal anomaly detector based on (5.2), when the new example is sequentially updated with new points. Note that this would not be the case if we used another NCM based on, e.g., the *average* (instead of maximum) distance to the closest point of the other set.

Nearest neighbours methods are relatively easy to implement, require little in the way of tuning and often perform quite well (Russel and Norvig, 2003). In particular, nearest neighbours methods are appropriate for online learning, since they do not require an extensive update of model parameters when new training data is added (compared to, e.g., neural networks and SVMs). Moreover, nearest neighbours algorithms have some strong consistency results; the asymptotic error rate is less than twice the Bayes error rate, which corresponds to the minimum achievable error rate given the distribution of the data (Duda and Hart, 1973).

5.1.3 The Sequential Hausdorff Nearest Neighbours Conformal Anomaly Detector

Based on (5.2), we propose SHNN-CAD (Algorithm 5.1) for sequential anomaly detection applications. Given:

- the anomaly threshold ϵ,
- the number of nearest neighbours k to consider,
- the training set $(z_1, \ldots, z_l)$, where $z_i \subseteq \mathbb{R}^d : i = 1, \ldots, l$,
- the distance matrix H, where each element $H_{i,j} : i = 1, \ldots, l, j = 1, \ldots, k$ corresponds to the DHD from z_i to its jth-nearest neighbour among:

$$(z_1, \ldots, z_{i-1}, z_{i+1}, \ldots, z_l),$$

- the empty priority queue Q (Goodrich and Tamassia, 1998),
- the test example $z_{l+1} = \{x_1 \cup x_2 \cup \cdots \cup x_L\}$ observed as a sequence of disjunct subsets $x_1, \ldots, x_L$ such that $x_i \cap x_j = \emptyset : i, j = 1, \ldots, L \wedge j \neq i$,

SHNN-CAD sequentially updates the classification of z_{l+1} and outputs:

- the sequence of indicator variables $\text{Anom}^{\epsilon}_{l+1,1}, \ldots, \text{Anom}^{\epsilon}_{l+1,L}$, where each $\text{Anom}^{\epsilon}_{l+1,i} : i = 1, \ldots, L-1$ corresponds to the preliminary classification of z_{l+1} based on the subset $\{x_1 \cup \cdots \cup x_i\} \subset z_{l+1}$, and $\text{Anom}^{\epsilon}_{l+1,L}$ corresponds to the final classification of z_{l+1},

- the vector $(h_1, \ldots, h_l)$ where $h_i : i = 1, \ldots, l$ corresponds to the DHD from z_{l+1} to z_i,

- the vector $(h'_1, \ldots, h'_l)$ where $h'_i : i = 1, \ldots, l$ corresponds to the DHD from z_i to z_{l+1}.

The algorithm works as follows. For each $z_i : i = 1, \ldots, l$, the DHD from z_{l+1} to z_i, h_i, is initialised to zero and the sum of the distances to the $(k-1)$-nearest neighbours of z_i is precomputed (lines 1–4 of Algorithm 5.1). The following nested loops (lines 5–31) are then repeated for each $x_j : j = 1, \ldots, L$ and $z_i : i = 1, \ldots, l$: First, h_i is updated as the DHD from $\{x_1 \cup \cdots \cup x_j\}$ to z_i, which is calculated in an incremental manner based on the distance from $\{x_1 \cup \cdots \cup x_{j-1}\}$ to z_i calculated in the previous iteration (line 7). If Q, which stores the current k-nearest neighbour distances to $\{x_1 \cup \cdots \cup x_j\}$, contains less than k distance values, h_i is simply inserted into Q (line 9). If Q contains k distance values and h_i is less than the current kth nearest neighbour distance, the maximum distance value in Q is removed and h_i is inserted into Q (lines 12–13). The DHD from z_i to z_{l+1}, h'_i, is updated as the distance from z_i to $\{x_1 \cup \cdots \cup x_j\}$. The preliminary nonconformity score $\hat{\alpha}_i$ for z_i is then updated depending on whether h'_i is smaller than the DHD from z_i to its kth-nearest neighbour in the training set (lines 16–21). Next, the k distance values are extracted from Q and $\hat{\alpha}_{l+1}$ is updated as the sum of these distances (lines 23–24). Finally, the preliminary p-value and the classification of the test example are updated (lines 25–30).

In the case of the unsupervised online SHNN-CAD, z_{l+1} is added to the training set when $x_1, \ldots, x_L$ have been observed. This is also true for the semi-supervised online SHNN-CAD, if z_{l+1} is assumed to belong to the normal class. If z_{l+1} is added to the training set, then:

1. the sorted distances to the k-nearest neighbours of each $z_1, \ldots, z_l$, i.e., rows 1 to l of H, are updated based on h' (output of Algorithm 5.1),

2. the sorted distances to the k-nearest neighbours of z_{l+1} are extracted from h (output of Algorithm 5.1) and appended as the last row of H.

The updated training set $(z_1, \ldots, z_{l+1})$ and the updated distance matrix H are then provided as input to SHNN-CAD when classifying the next test example z_{l+2}.

Implementation and Complexity Analysis

A natural approach to implementing SHNN-CAD for sequential anomaly detection in trajectories is to adopt a polyline representation; each trajectory (2.13) is

Algorithm 5.1 The Sequential Hausdorff Nearest Neighbours Conformal Anomaly Detector (SHNN-CAD)

Input: Anomaly threshold ϵ, number of nearest neighbours k, training set $(z_1, \ldots, z_l)$, distance matrix H, empty priority queue Q, sequence of updates $x_1, \ldots, x_L$ of test example z_{l+1}

Output: Sequence of indicator variables $\text{Anom}^{\epsilon}_{l+1,1}, \ldots, \text{Anom}^{\epsilon}_{l+1,L}$, distance vectors $(h_1, \ldots, h_l)$ and $(h'_1, \ldots, h'_l)$.

for $i \leftarrow 1$ to l **do**
  $v_i \leftarrow \text{sum}\{H_{i,1}, \ldots, H_{i,k-1}\}$
  $h_i \leftarrow 0$
end for
for $j \leftarrow 1$ to L **do**
  for $i \leftarrow 1$ to l **do**
    $h_i \leftarrow \max\left\{\overrightarrow{\delta_H}(x_j, z_i), h_i\right\}$
    if $i \leq k$ **then**
      Q.insertElement(h_i)
    else
      if Q.maxElement() $> h_i$ **then**
        Q.removeMaxElement()
        Q.insertElement(h_i)
      end if
    end if
    $h'_i \leftarrow \overrightarrow{\delta_H}(z_i, \{x_1 \cup \cdots \cup x_j\})$
    if $h'_i < H_{i,k}$ **then**
      $\hat{\alpha}_i \leftarrow v_i + h'_i$
    else
      $\hat{\alpha}_i \leftarrow v_i + H_{i,k}$
    end if
  end for
  $(h^*_1, \ldots, h^*_k) \leftarrow Q$.removeAllElements()
  $\hat{\alpha}_{l+1} \leftarrow \text{sum}\{h^*_1, \ldots, h^*_k\}$
  $\hat{p}_{l+1} \leftarrow \frac{|\{i=1,\ldots,l+1 : \hat{\alpha}_i \geq \hat{\alpha}_{l+1}\}|}{l+1}$
  if $\hat{p}_{l+1} < \epsilon$ **then**
    $\text{Anom}^{\epsilon}_{l+1,j} \leftarrow 1$
  else
    $\text{Anom}^{\epsilon}_{l+1,j} \leftarrow 0$
  end if
end for

represented by an example:

$$z_i = \{x_1 \cup x_2 \cup \cdots \cup x_{L_i}\}, \tag{5.5}$$

where $x_1 = \chi_1$ and

$$x_j = \{\chi_{j-1} + s \cdot (\chi_j - \chi_{j-1}) : s \in (0, 1]\} \tag{5.6}$$

for $j = 2, \ldots, L_i$. That is, the first subset x_1 represents the first trajectory point and each remaining subset x_j represents the set of all points along the line segment that connects χ_{j-1} and χ_j. This approach allows for the efficient compression of trajectories and the exact calculation of (5.1) with time complexity $O((L_i + L_j)\, log\,(L_i + L_j))$ based on algorithms from computational geometry (Alt, 2009), where L_i and L_j are the number of line segments of the corresponding trajectories. Moreover, assuming that Q is implemented as a heap based on a binary tree, the time complexity of insertElement (), removeMaxElement (), maxElement () and removeAllElements () in Algorithm 5.1 is $O(\log(k))$, $O(\log(k))$, $O(1)$, and $O(k)$, respectively, where k is the number of elements of the queue (Goodrich and Tamassia, 1998).

Based on the assumptions above, the time complexity of Algorithm 5.1 can be analysed as follows: The sections corresponding to lines 7, 8–15, 16 and 17–21 have complexity $O(L_i \log(L_i))$, $O(\log(k))$, $O((L_i + j)\log(L_i + j))$ and $O(1)$, respectively. Thus, the overall complexity of the inner loop (lines 6–22), which corresponds to a trajectory update, is:

$$\begin{aligned} &O(l \log(k) + (L_1 + j)\log(L_1 + j) + (L_l + j)\log(L_l + j) = \\ &\quad = O(l \log(k) + l \cdot L_{max} \log(L_{max})), \end{aligned} \tag{5.7}$$

where L_{max} is the maximum number of line segments among all of the trajectories. The sections corresponding to lines 23, 24 and 25 have complexity $O(k)$, $O(k)$ and $O(l)$, respectively. Since $l > k$, the complexity of each iteration of the outer loop (lines 5–31), i.e., the update of the classification of z_n based on x_j, is equivalent to (5.7). Further, the pre-computations (lines 1–4) have complexity $O(l \cdot k)$. Thus, the overall time complexity of Algorithm 5.1 is:

$$O\left(l \cdot L_{max} \log(k) + l \cdot L_{max}^2 \log(L_{max}) + l \cdot k\right). \tag{5.8}$$

Assuming that $L_{max} > k$, which is typically the case, (5.8) is reduced to:

$$O\left(l \cdot L_{max}^2 \log(L_{max})\right). \tag{5.9}$$

5.1.4 Discussion

An alternative approach to the continuous polyline representation (5.6) is to only consider the end points of the line segments:

$$x_j = \{\chi_j\} : j = 1, \ldots, L. \tag{5.10}$$

This representation corresponds to a less detailed model of the actual trajectory, but allows for a more simple implementation of the DHD.

In contrast to previously proposed algorithms (Section 2.2), we argue that SHNN-CAD fulfils all of the key properties identified in Section 3.3, except for the multi-scale property: Firstly, SHNN-CAD is non-parametric, since it is based on the conformal anomaly detection framework and a nearest neighbour-based NCM. Secondly, SHNN-CAD offers a well-founded approach to the tuning of the anomaly threshold ϵ (Section 4.3.5), and it has a well-calibrated alarm rate in the online mode (Section 4.3.2 and Theorem 1). Apart from the anomaly threshold, there is only one free parameter k; hence, SHNN-CAD is parameter-light. Finally, the algorithm is designed for sequential anomaly detection in incomplete trajectories and it supports unsupervised online learning.

It should also be noted that SHNN-CAD requires no particular preprocessing or normalisation of the trajectories. Further, due to intrinsic properties of the DHD, it may be argued that SHNN-CAD is not only robust to future data points of the trajectory that have not yet been observed; it is also robust to previous data points that are missing. Assuming that the trajectories are output of a tracking system (Section 2.2), some data points may be missing due to delayed track initialisation, re-initialisation of a previous track that was lost, etc.

The DHD is by definition insensitive to the ordering of the data points. Thus, if only position is included in the point feature model, this may result in contra-intuitive matching. An example is two objects that follow the same path but travel in opposite directions. This could be addressed by simply extending the point feature model and incorporating the current velocity vector, which would also capture the speed of the object. Nevertheless, this would require that the different features are further normalised. In the empirical investigations that follow in this chapter, we have only considered the two-dimensional feature space comprising the current position.

5.2 Empirical Investigations

In this section, we first implement and investigate the accuracy of DH-kNN-NCM (Section 5.1.2) on three public datasets of labelled trajectories (Section 5.2.1). Next, we implement the unsupervised online SHNN-CAD (Sections 4.3.2–4.3.3 and Section 5.1.3) and investigate its classification performance on a larger public dataset of labelled trajectories (Section 5.2.2). For simplicity, we implement an algorithm for calculating (5.1) that only considers the finite set of points of the trajectories. That is, we have not implemented an algorithm for calculating the exact DHD between two trajectories represented as polylines, where all the intermediate points along the line segments are considered (Section 5.1.4).

For comparative purposes, we also implement and investigate the performance of the time-series discords algorithm (Keogh et al., 2005), which was originally proposed by Keogh et al. for detecting anomalies in time-series data. The discords algorithm is parameter-light and has been demonstrated to have competitive

classification performance on a wide range of time-series databases (e.g., Keogh et al., 2005), including trajectory datasets (Yankov et al., 2008; Piciarelli et al., 2008). Hence, we use the discords algorithm as a benchmark for evaluating the relative performance of DH-kNN-NCM and SHNN-CAD in all of the following experiments.

All algorithms are implemented in Java™ and executed on a Macbook Pro 2.66 GHz Intel Core 2 Duo processor with 8 GB of RAM.

5.2.1 Accuracy of Trajectory Outlier Measures

In this subsection, we investigate the accuracy of DH-kNN-NCM and the discords algorithm by reproducing three previously published experiments on three different datasets of labelled trajectories. The main objective is to investigate how accurately DH-kNN-NCM discriminates complete abnormal trajectories from complete normal trajectories, compared to previously proposed trajectory outlier measures.

Synthetic Trajectories

The first experiment was originally published by Piciarelli et al. (2008) who investigated the accuracy of an outlier measure based on one-class SVM on a synthetic dataset. The dataset[1] was created by the authors and consists of 1000 randomly generated trajectory subsets. Each subset contains 260 two-dimensional trajectories of length 16 without any time information. Of the 260 trajectories, 250 belong to five different clusters and are labelled normal. The remaining 10 are stray trajectories that do not belong to any cluster and are labelled abnormal (see Figure 5.2 for a plot of one of the subsets). For each of the 1000 subsets, the authors calculated the outlier score for each of the 260 trajectories relative to the remaining 259. The *error rate* was calculated by averaging over the number of normal trajectories among the top-10 trajectories with the highest outlier scores. We repeat this procedure for DH-kNN-NCM, where the outlier score for each trajectory is calculated according to (5.2). Further, we calculate the top-ten discords, where each trajectory corresponds to a subsequence. The average accuracy, i.e., 1 minus the corresponding average error rate, for all three algorithms are summarised in Table 5.1.

First Set of Recorded Video Trajectories

In the second experiment, we investigate accuracy on a public dataset of 239 recorded video trajectories[2] that were extracted from IR surveillance videos using a motion detection and tracking algorithm (Pokrajac et al., 2007). The trajectories are three-dimensional, where the third dimension corresponds to the time stamp, and each trajectory has length five. Of the 239 trajectories, 237 are labelled normal and two are labelled abnormal (see Figure 5.3). We calculate the nonconformity

[1] http://avires.dimi.uniud.it/papers/trclust/
[2] www.cs.umn.edu/~aleks/inclof

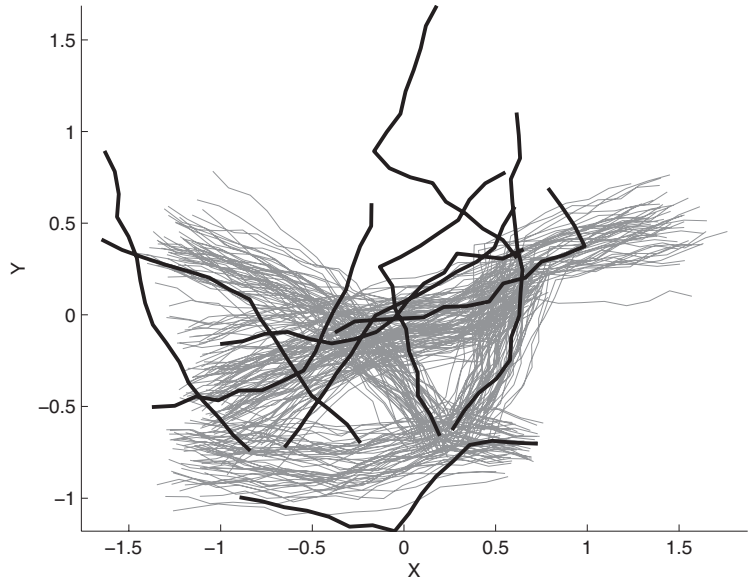


Figure 5.2: Plot of trajectories from one of the 1000 subsets of synthetic trajectories used to evaluate the accuracy of different trajectory outlier measures (Section 5.2.1). Grey trajectories (250) are labelled normal and black trajectories (10) are labelled abnormal.

Table 5.1: Average accuracy on the synthetic trajectories (Section 5.2.1).

	# of most similar neighbours considered				
Outlier Measure	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
DH-kNN-NCM	96.51%	97.09%	97.05%	96.95%	96.77%
SVM (Piciarelli et al., 2008)	96.30%*				
Discords (Keogh et al., 2005)	97.06%				

* Accuracy results for SVM are 1 minus the corresponding error rate reported by Piciarelli et al. (2008).

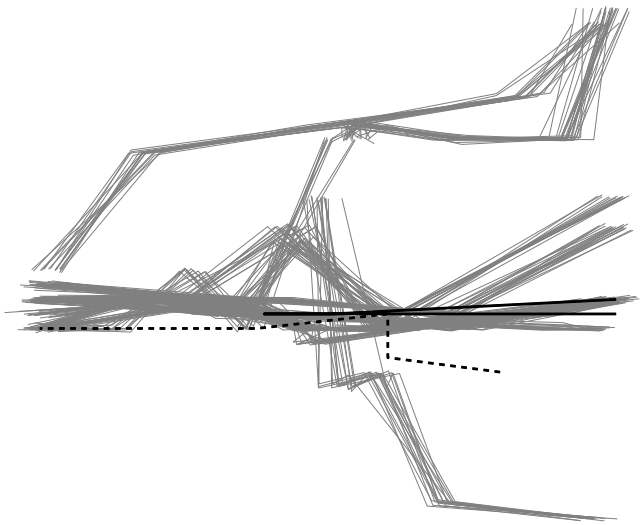


Figure 5.3: Plot of the 239 trajectories from the first set of recorded video trajectories (Section 5.2.1). Grey corresponds to normal trajectories while solid and dashed black corresponds to the two abnormal trajectories.

score for each of the 239 trajectories relative to the rest according to (5.2) for $k = 1, \ldots, 5$. Since it is not obvious how the time dimension should be normalised when using DH-kNN-NCM, we only consider the two-dimensional spatial location of each trajectory point. Sorting the resulting nonconformity scores, the two trajectories labelled abnormal have the top-two largest nonconformity scores, regardless of k. Furthermore, the top-two discords (for the two-dimensional representation) also correspond to the abnormal trajectories. Hence, similar to previous algorithms (Pokrajac et al., 2007), DH-kNN-NCM and discords achieve perfect accuracy on this dataset.

Second Set of Recorded Video Trajectories

The second set, which is known as the LAB-dataset (Khalid, 2010), consists of 152 trajectories labelled normal and 8 trajectories labelled abnormal. The trajectories contain no time stamps and are of varying length, where the maximum and average length is 425 and 192 points, respectively (Figure 5.4). The trajectories were extracted from a video recording where people were tracked in a controlled laboratory environment. Movements were planned so that normal trajectories can be grouped into four clusters and the abnormal trajectories deliberately varied from the normal trajectories. Analogously to the experimental setup described by Khalid (2010), we evaluate the accuracy of DH-kNN-NCM and the discords algorithm as follows: We first construct a training set by randomly sampling without

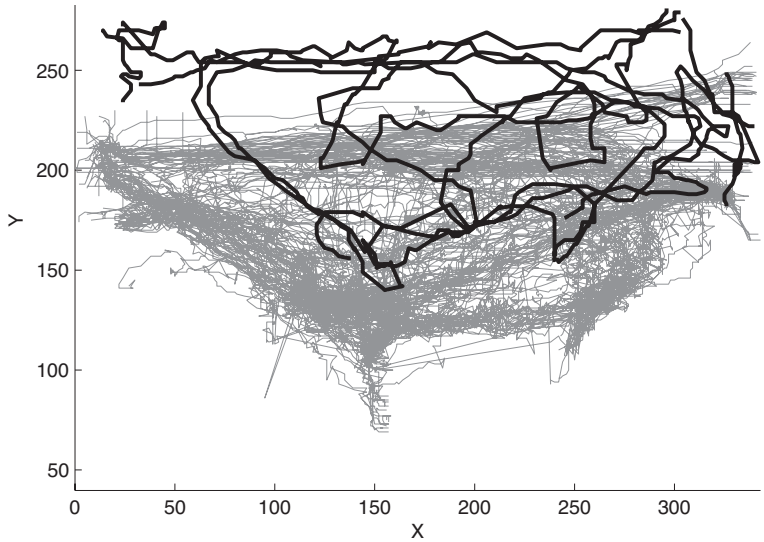


Figure 5.4: Plot of the second set of recorded video trajectories, also known as the LAB-dataset (Khalid, 2010) (Section 5.2.1). Grey trajectories (152) are labelled normal and black trajectories (8) are labelled abnormal.

replacement half of the 152 normal trajectories. The remaining 76 normal trajectories and the eight abnormal trajectories are used as the test set. For each test trajectory, the nonconformity score relative to the training set is estimated according to (5.2). Moreover, the top-eight discords among the test trajectories are calculated relative to the training set, where each trajectory in the training and test sets is normalised to length 50, based on linear interpolation; note that the discords algorithm requires that the trajectories have the same length. Analogously to the previous experiments in this subsection, accuracy is calculated as the fraction of the top-eight nonconforming test trajectories and the top-eight discords that are labelled abnormal. This procedure is repeated 100 times. The average accuracy for DH-kNN-NCM with $k = 1, \ldots, 5$ and the discords algorithm is summarised in Table 5.2. Note that the results reported by Khalid (2010) are based on a single run, i.e., not the average of 100 experiments.

5.2.2 Unsupervised Online Learning and Sequential Anomaly Detection Performance

In this subsection, we investigate the sequential classification performance of the unsupervised online SHNN-CAD and the discords algorithm on a relatively large synthetic dataset. The objectives of the experiment are to:

1. Investigate how the sensitivity and precision depends on the size of the training set and the value of ϵ.

Table 5.2: Average accuracy on the 100 test sets that were randomly sampled from the second set of labelled video trajectories (Section 5.2.1).

	# of nearest neighbours considered				
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
DH-kNN-NCM	87.9%	88.3%	87.3%	85.9%	84.1%
Discords	98.6%				
m-Mediods (Khalid, 2010)	100%*				

* The results for m-Mediods were reported by Khalid (2010) and are, in contrast to the results of the other two algorithms, based on only one test set.

2. Investigate how the *detection delay*, i.e., the number of observed trajectory points prior to the detection of an abnormal trajectory, depends on the size of the training set and the value of ϵ. Recall that a low detection delay is advantageous, since it enables an earlier response in a real-time surveillance application (Section 3.2.2).

3. Investigate the relative performance of SHNN-CAD vs. discords.

4. Validate that the rate of detected anomalies (both normal and abnormal trajectories) is indeed well-calibrated for SHNN-CAD.

Note that the order of objectives above does not reflect any particular priority or similar.

The Dataset

For this experiment, we generated a public dataset of synthetic trajectories[3] using the publicly available trajectory generator software[4] written by C. Piciarelli. The dataset contains 100 random trajectory sequences, where each sequence consists of 2000 two-dimensional trajectories of length 16 that are labelled normal or abnormal. Each sequence, which is independent of the other 99 sequences, was created as follows. First, a set of 2000 normal trajectories from 10 different trajectory cluster seeds and another set of 1000 abnormal trajectories from 1000 different cluster seeds were created using the trajectory generator with the randomness parameter set to the default value 0.7. Next, a sequence of 2000 trajectories was created by randomly permuting the 2000 normal trajectories. Finally, each of the normal trajectories was independently and with probability 1% replaced by an abnormal trajectory, which was randomly sampled without replacement from the set of abnormal trajectories. Hence, the trajectory generation process is consistent with (4.15) where $\lambda = 0.01$. A subset of the normal trajectories and all abnormal trajectories from the first sequence are shown in Figure 5.5.

[3] https://www.researchgate.net/profile/Rikard_Laxhammar/
[4] http://avires.dimi.uniud.it/papers/trclust/create_ts2.m

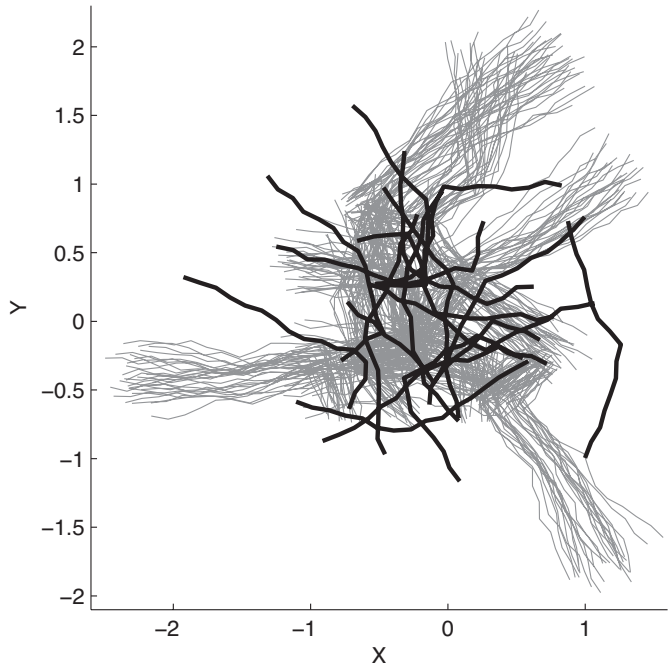


Figure 5.5: Plot of trajectories from the first sequence of synthetic trajectories used in the online learning and sequential anomaly detection experiments (Section 5.2.2). Grey trajectories (300) are labelled normal and black trajectories (17) are labelled abnormal. Note that for clarity, only 300 of 1983 normal trajectories are plotted.

Design

For SHNN-CAD, we fix $k = 2$ since it was shown to give the best accuracy results for DH-kNN-NCM on all of the previous datasets (Table 5.1–5.2). For each of the 100 sequences of 2000 trajectories, we allocate the first three trajectories as an initial training set and calculate the corresponding distance matrix M of size 3×3 (Section 5.1.3); note that three is the minimum size of the training set for which (5.2) is defined when $k = 2$. The remaining 1997 trajectories are sequentially classified by the unsupervised online SHNN-CAD (Algorithm 5.1). The whole process is repeated for three different anomaly thresholds: $\epsilon = 0.005, 0.01$ and 0.02. The choice of $\epsilon = 0.01$ is motivated by the discussion in Section 4.3.5, i.e., that the anomaly threshold should be set close to λ. Nevertheless, the exact value of λ may be unknown and, hence, we also investigate the performance for $\epsilon = 0.005$ and 0.02. Moreover, in order to avoid the problem of zero sensitivity when the size of the training set is relatively small (Section 4.3.5), the anomaly threshold is dynamically re-tuned to $(l+1)^{-1}$ as long as $l < \epsilon^{-1}$.

The experiment is also reproduced for a brute-force implementation of an online version of the top-k discords algorithm. According to the general definition (Keogh et al., 2005), the top-k discords of a time-series correspond to the top-k non-overlapping subsequences of predefined length m that have the largest distances to their nearest non-overlapping match. In previous work (Piciarelli et al., 2008; Yankov et al., 2008) and in the previous experiments of this paper, each subsequence simply corresponds to a complete and normalised trajectory of fixed length m. However, since we are now concerned with sequential anomaly detection in incomplete trajectories, we need to consider a subsequence length that is less than the full length of the trajectories. Setting m to a low value may decrease the detection delay but may also decrease the sensitivity to abnormal trajectories when the length of the abnormal part is greater than m. In these experiments, we consider $m = 4, 8$ and 12, which correspond to a quarter, a half and three quarters of the full trajectory, respectively. For each update $j = m, \ldots, 16$ of trajectory $i = 4, \ldots, 2000$, the subsequence $\chi^i_{j-m+1}, \ldots, \chi^i_j$ is extracted, added to the training set of previous subsequences and classified as abnormal or normal depending on whether it is among the top-k discords of the updated training set. Intuitively, k should be chosen depending on the expected number of discords in the training set. Assuming that approximately 1% of all trajectories are abnormal and that each abnormal trajectory corresponds to one discord, k is dynamically tuned based on the number of trajectories observed so far, i.e., $k = \lceil 0.01i \rceil$. Nevertheless, the prior probability of an abnormal trajectory may be unknown and abnormal trajectories may include more than one discord, especially when m is relatively small. Hence, we also investigate performance when $k = \lceil 0.005i \rceil$ and $\lceil 0.02i \rceil$.

Results

In order to evaluate the relative performance of SHNN-CAD vs. discords (objective 3), we calculate the average F_1-score and the average detection delay for each

Table 5.3: Average F_1-score and mean detection delay for the online learning and sequential anomaly detection experiments (Section 5.2.2).

			F_1	Detection delay*
SHNN-CAD	$\epsilon = 0.005$		0.53	12.7
	$\epsilon = 0.01$		**0.75**	10.3
	$\epsilon = 0.02$		0.62	**8.0**
Discords	$k = \lceil 0.005i \rceil$	$m = 4$	0.46	10.2
		$m = 8$	0.58	10.6
		$m = 12$	0.62	12.8
	$k = \lceil 0.01i \rceil$	$m = 4$	0.62	8.8
		$m = 8$	0.74	9.7
		$m = 12$	**0.76**	12.3
	$k = \lceil 0.02i \rceil$	$m = 4$	0.63	**7.3**
		$m = 8$	0.65	8.9
		$m = 12$	0.60	12.1

* The maximum possible detection delay is 16, since this is the length of each trajectory.

combination of the detectors' parameters (Table 5.3). The F_1-score corresponds to an evenly weighted combination of sensitivity and precision:

$$F_1 = \frac{2 \cdot sensitivity \cdot precision}{sensitivity + precision}, \tag{5.11}$$

and it is therefore useful for comparing the *overall* classification performance of multiple anomaly detectors. In order to address objective 1, we estimate logistic regression models for the sensitivity and precision, respectively, dependent on the size of the training set (Figure 5.6). The coefficients of the logistic regression models for the sensitivity are estimated on the basis of the final classification results for the subset of trajectories that are labelled abnormal. In the case of the precision, the coefficients are estimated on the basis of the true label for those trajectories that are detected as anomalous. Further, in order to address objective 2, we estimate a linear regression model for the detection delay dependent on the current size of the training set (Figure 5.7). Finally, a histogram of the overall alarm rate for SHNN-CAD $\epsilon = 0.01$ is shown in Figure 5.8 (objective 4). The average processing time for each sequence of 2000 trajectories was 19.1 seconds for SHNN-CAD with $\epsilon = 0.01$ and 45.0 seconds for discords with $m = 8$ and $k = \lceil 0.01i \rceil$.

5.2.3 Discussion

For the first two datasets (Section 5.2.1), it is clear that DH-kNN-NCM is an accurate outlier measure, regardless of k (Table 5.1). On the third dataset, however, results for DH-kNN-NCM were slightly worse compared to discords (Table 5.2),

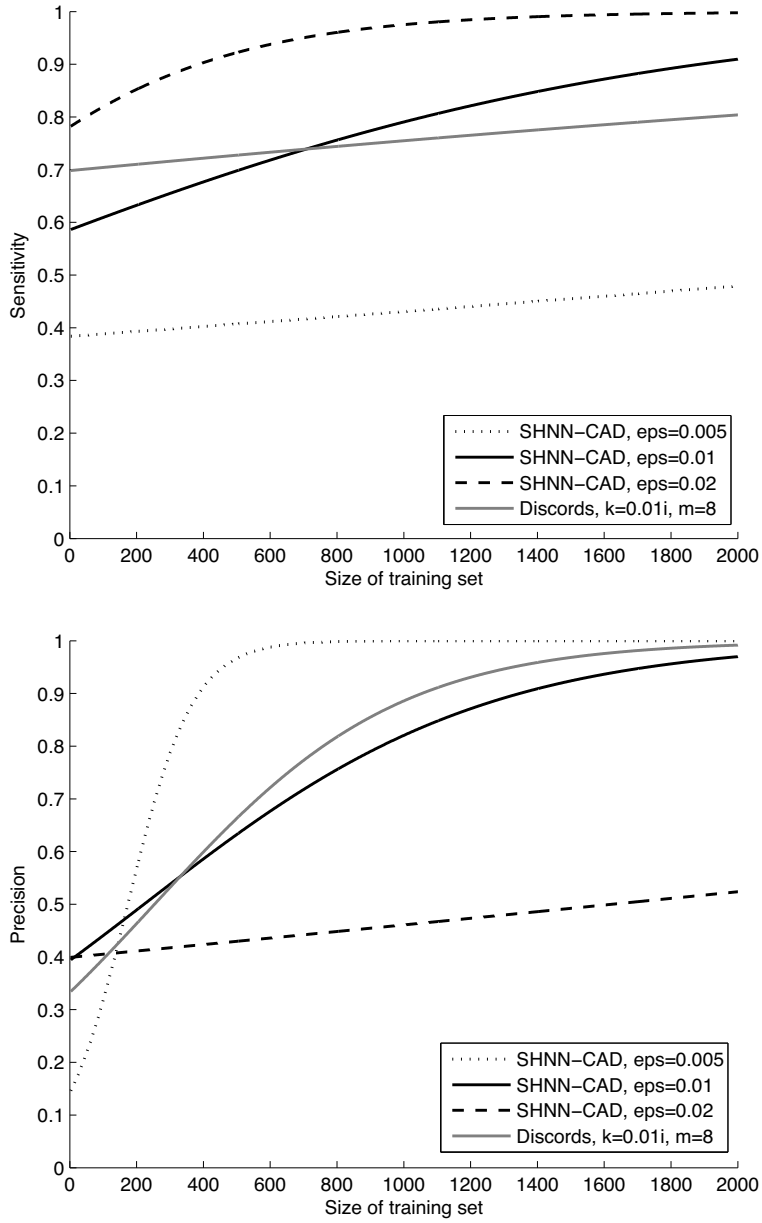


Figure 5.6: Plots of the sensitivity (upper plot) and precision (lower plot) dependent on the size of the training set according to the logistic regression models, which are estimated on the basis of the online learning and sequential anomaly detection results (Section 5.2.2).

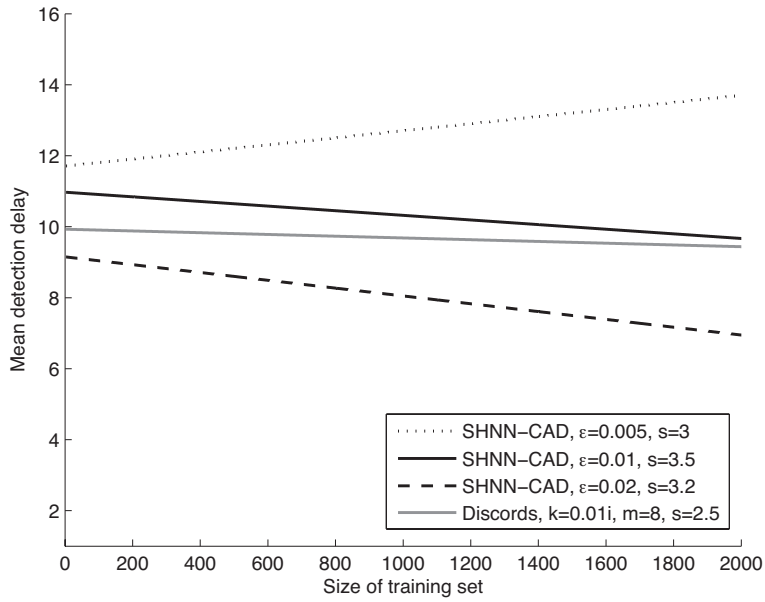


Figure 5.7: Plot of the mean detection delay dependent on the size of the training set according to the linear regression models, which are estimated on the basis of the online learning sequential anomaly detection results (Section 5.2.2). The variable s in the legend is an unbiased estimate of the standard deviation of the error term of the linear regression model.

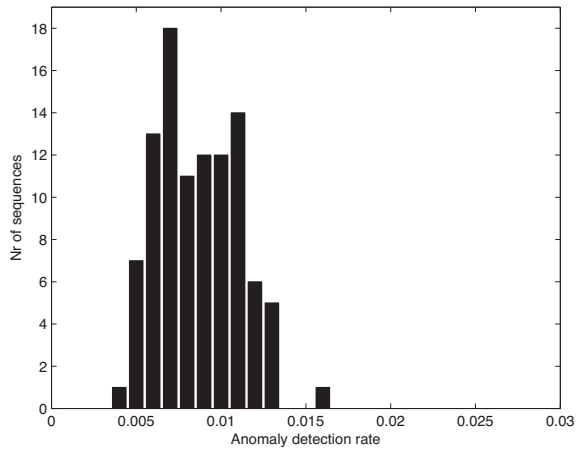


Figure 5.8: Histogram of the empirical alarm rate (both normal and abnormal trajectories) for the unsupervised online SHNN-CAD with $\epsilon = 0.01$. Each data point corresponds to the average anomaly detection rate for one of the 100 random sequences of trajectories (Section 5.2.2). The corresponding histograms for $\epsilon = 0.005$ and $\epsilon = 0.02$ have a similar structure with the mean value close to 0.005 and 0.02, respectively.

which indicates that the latter has some edge over the former in the case of complete trajectories. Nevertheless, the advantage of DH-kNN-NCM becomes clear when we consider the results for SHNN-CAD during online learning and sequential anomaly detection:

Considering Table 5.3, it is clear that the best classification performance for SHNN-CAD ($F_1 = 0.75$) is achieved when $\epsilon = 0.01$. This confirms our hypothesis that ϵ should be close to λ (Section 4.3.5). The best classification performance for SHNN-CAD is approximately the same as the best results for discords: $F_1 = 0.74$ and $F_1 = 0.76$ when $k = \lceil 0.01i \rceil$ and $m = 8, 12$, respectively. The detection delay for SHNN-CAD with $\epsilon = 0.01$ is also similar to that of discords with $m = 8$ (10.3 and 9.7 points, respectively). However, the detection delay for discords with $m = 12$ is considerably higher (12.3 points). Hence, we select $k = \lceil 0.01i \rceil$ and $m = 8$ when we further investigate how sensitivity, precision and detection delay depend on the size of the training set[5] (Figure 5.6–5.7).

Examining Figure 5.6, it is clear that the sensitivity and precision improve as more unlabelled training data is accumulated. In particular, there is generally a large increase in precision. Considering the end of the curves, i.e., at the point where the size of the training set approaches 2000, we see that the sensitivity and precision are still, in most cases, increasing, albeit at a slower rate. This indicates that the anomaly detectors may still benefit from more training data but

[5]For clarity and brevity, we omit more detailed results for the remaining parameter values of discords in Figures 5.6–5.7.

are approaching the point where they may be considered fully trained. Comparing the sensitivity for SHNN-CAD $\epsilon = 0.01$ and discords, we see that the latter initially has a slight advantage until the size of the training set is 700. From that point, the sensitivity of SHNN-CAD becomes superior and increases more rapidly than for discords. When the size of the training set has reached 2000, the sensitivity of SHNN-CAD and discords is approximately 0.9 and 0.8, respectively. In the case of precision, the difference is more subtle; SHNN-CAD has an initial advantage, but it is passed by discords after the size of the training set has reached 400. When the size of the training set reaches 2000, the precision of both SHNN-CAD and discords is close to 1, with a slight advantage to discords.

The expected detection delay generally decreases as more training is accumulated, with the exception of SHNN-CAD with $\epsilon = 0.005$ (Figure 5.7). The explanation for this exception is not obvious. Nevertheless, the same pattern was observed for discords with $k = \lceil 0.005i \rceil$. This suggests that for low anomaly thresholds, the detection delay generally *increases* as the training set grows.

It should be noted that all the parameter values may have a significant impact on performance for both SHNN-CAD and discords. However, the discords algorithm involves the tuning of one more parameter than SHNN-CAD, i.e., the length of the subsequence m. Moreover, the choice of k for discords may not be as straightforward as the choice of ϵ for SHNN-CAD, since the optimal value of the former may depend on m. For example, the best classification performance for $m = 8, 12$ is achieved with $k = \lceil 0.01i \rceil$. But for $m = 4$, the classification performance and the detection delay are both superior with $k = \lceil 0.02i \rceil$.

Finally, considering Figure 5.8, we see that the alarm rate for each of the 100 sequences is close to the specified anomaly threshold. Hence, the alarm rate of the online unsupervised SHNN-CAD is indeed well-calibrated in practice.

5.3 Summary

In this chapter, we have adopted the framework of conformal anomaly detection (Chapter 4) for the sequential detection of anomalous trajectories. The sequential anomaly detection setting imposes some further requirements that are not fulfilled by previously proposed NCM. Therefore, DH-kNN-NCM is proposed, which is a novel NCM based on the directed Hausdorff distance for examples that are represented as sets of points. A key property of DH-kNN-NCM is that it enables the calculation of a preliminary nonconformity score for an incomplete example and that this score monotonically increases as the example is updated with more points. Further, for each of the remaining examples, the corresponding nonconformity score monotonically decreases as the incomplete example is updated. This implies that a well-calibrated alarm rate is maintained for any conformal anomaly detector based on DH-kNN-NCM when the new example, e.g., a trajectory, is sequentially updated. Based on DH-kNN-NCM and CAD, a general and parameter-light algorithm for sequential anomaly detection applications is presented, called SHNN-CAD. The implementation of SHNN-CAD for the sequential

detection of anomalous trajectories is discussed and a corresponding complexity analysis presented. Compared to previous algorithms, the main advantages of SHNN-CAD are that it is distribution independent and parameter-light, requires no preprocessing of trajectories, supports online learning and sequential anomaly detection, is well-calibrated and offers a well-founded approach to the tuning of the anomaly threshold.

In the empirical investigations of this chapter, we have first implemented and evaluated the accuracy of DH-kNN-NCM on three public and labelled datasets of synthetic and recorded video trajectories. For comparative purposes, we have also implemented and evaluated the discords algorithms, which is another parameter-light algorithm previously proposed for detecting anomalous trajectories. On the first two datasets, the results show that DH-kNN-NCM is a competitive outlier measure, regardless of the parameter value k. However, the results show that DH-kNN-NCM performs worse than discords on the third dataset. This indicates that the discords algorithm has some edge over DH-kNN-NCM in the case of complete trajectories. The unsupervised online learning and sequential anomaly detection performance of SHNN-CAD and discords on a relatively large dataset of synthetic trajectories is also investigated. The best overall classification performance for SHNN-CAD is achieved when $\epsilon = 0.01$, which confirms our hypothesis that ϵ should be set close to the expected prior probability of an anomaly. It is clear from the results that both sensitivity and precision improve as more unlabelled training data is accumulated. Considering the results for the discords algorithm, the best overall performance is approximately the same as for SHNN-CAD. However, when the training set grow relatively large, the sensitivity of SHNN-CAD is superior to discords, while precision is approximately the same for both algorithms. Generally, the anomaly threshold has a significant impact on the performance for both SHNN-CAD and discords. However, the tuning of the anomaly threshold for discords is not as straightforward as for SHNN-CAD. Moreover, the discords algorithm requires the tuning of an additional parameter, i.e., the length of the sub-trajectory and the optimal choice of anomaly threshold may be dependent on this parameter.

Chapter 6

Sequential Detection of Anomalous Sub-Trajectories

As discussed in Section 3.2.3, previous algorithms are generally not sensitive to local anomalous sub-trajectories. This is also true for SHNN-CAD, which we have investigated in the previous chapter. In this chapter, we therefore propose the *sequential sub-trajectory local outlier inductive conformal anomaly detector* (SSTLO-ICAD) for the sequential detection of local anomalous sub-trajectories. SSTLO-ICAD is the combination of ICAD and the *sequential sub-trajectory local outlier nonconformity measure* (SSTLO-NCM), which is a novel NCM that is based on *local outlier factor* (LOF) (Breunig et al., 2000) and, hence, is sensitive to local anomalous sub-trajectories. The performance of SSTLO-ICAD and SSTLO-NCM is empirically investigated on a number of different datasets from the maritime domain. In particular, the classification performance of SSTLO-NCM is evaluated on a labelled dataset of real vessel trajectories and realistically simulated anomalies. The chapter is based on previously published results in Laxhammar and Falkman (2013b) that have been updated and extended with new theoretical and empirical results in the thesis.

6.1 A Nonconformity Measure for the Sequential Detection of Local Anomalous Sub-Trajectories

Adopting the framework of inductive conformal anomaly detection (Section 4.4), we assume that each example z represents a uniformly sampled trajectory (2.15). We are interested in an NCM for calculating the sequence of preliminary nonconformity scores $\hat{\alpha}_1, \ldots, \hat{\alpha}_{L-1}$ and the final nonconformity score α for the sequentially updated trajectory:

$$(\chi_1^n, \ldots, \chi_s^n) : s = 1, \ldots, L_n \tag{6.1}$$

relative to the bag of m trajectories:

$$B = \left\{ \left(\chi_1^1, \ldots, \chi_{L_1}^1\right), \ldots, \left(\chi_1^m, \ldots, \chi_{L_m}^m\right) \right\} \tag{6.2}$$

where $n > m$. In addition to accurately discriminating between abnormal and normal trajectories, we argue that this NCM should have the following properties:

1. It should be sensitive to local features of the trajectory at different scales. That is, it should be able to discriminate anomalous sub-trajectories that may be smoothed out if the trajectory is considered as a whole (*cf.* Lee et al., 2008).

2. It should consider the local density and variance of the neighbourhood of sub-trajectories. For example, it should account for the fact that the distance between vessel sub-trajectories in some areas, e.g., harbours, is relatively small compared to other less trafficked areas.

3. The sequence $\hat{\alpha}_1, \ldots, \hat{\alpha}_{L-1}, \alpha$ should monotonically increase:

$$\hat{\alpha}_1 \leq \cdots \leq \hat{\alpha}_{L-1} \leq \alpha. \tag{6.3}$$

Since the precomputed nonconformity scores of the calibration set (Section 4.4) are independent of the current test trajectory, (6.3) is a sufficient condition for ensuring that the online and semi-offline ICAD (Section 4.4.1–4.4.2) are well-calibrated.

6.1.1 Preliminaries: Local Outlier Factor

LOF (Breunig et al., 2000) is a powerful density-based outlier measure that has shown good results in a wide range of anomaly detection applications (Chandola et al., 2009). The main advantage of LOF is that it does not suffer from the local density problem (Janssens and Postma, 2009), as illustrated in Figure 6.1: Assume that C_1 and C_2 are two clusters with low and high densities, respectively, and that x_1 and x_2 are two anomalous data points located in low and high density areas of the feature space, respectively. Clearly, x_1 is easily detected by, e.g., a standard nearest neighbour method, since it has the largest nearest neighbour distance among *all* points from C_1 and C_2. However, this is not true for x_2, since its nearest neighbour distance is approximately equal to or less than the corresponding nearest neighbour distances of the normal points belonging to C_1. LOF addresses this problem by considering the neighbourhood density of x_2 in relation to the neighbourhood densities of the k-nearest neighbours to x_2. Analogously to Janssens and Postma (2009), we describe the three steps involved when calculating LOF for each data point $x_i \in \mathcal{D}$:

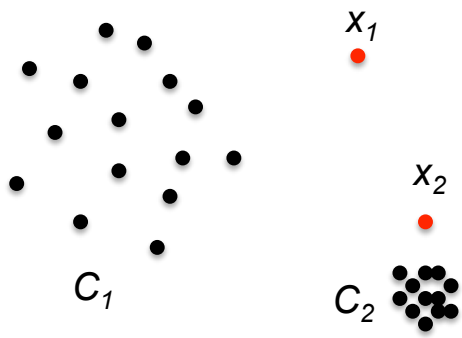


Figure 6.1: Illustration of the local density problem in 2D, where C_1 and C_2 correspond to two clusters with different densities, and x_1 and x_2 correspond to anomalous data points in low and high density areas, respectively.

1. The first step involves determining the *neighbourhood* of each data point: Let the *neighbourhood border distance* of x_i be defined as:

$$dist_{border}\left(x_i, k\right) = dist\left(x_i, \mathrm{NN}\left(x_i, k\right)\right), \tag{6.4}$$

where $dist$ is a distance function and $\mathrm{NN}\left(x_i, k\right)$ corresponds to the kth nearest neighbour to x_i in $\mathcal{D}$. Then, the neighbourhood of x_i is given by:

$$\mathcal{N}\left(x_i, k\right) = \left\{x_j \in \mathcal{D} \setminus x_i : dist\left(x_i, x_j\right) \leq dist_{border}\left(x_i, k\right)\right\}. \tag{6.5}$$

Note that in the absence of ties, $\mathcal{N}\left(x_i, k\right)$ is simply the set of k nearest neighbours to x_i.

2. The second step involves estimating the *neighbourhood density* of each data point: Let the *reachability distance* from x_i to x_j be defined as:

$$dist_{reach}\left(x_i, x_j, k\right) = \max\left\{dist_{border}\left(x_j, k\right), dist\left(x_i, x_j\right)\right\}. \tag{6.6}$$

Then, the neighbourhood density of x_i is defined as:

$$\rho\left(x_i, k\right) = \frac{\left|\mathcal{N}\left(x_i, k\right)\right|}{\sum\limits_{x_j \in \mathcal{N}\left(x_i, k\right)} dist_{reach}\left(x_i, x_j, k\right)}. \tag{6.7}$$

The use of $dist_{reach}\left(x_i, x_j, k\right)$ instead of $dist\left(x_i, x_j\right)$ in (6.7) has a smoothening effect on the neighbourhood density, which is illustrated in Figure 6.2. The larger the value of k, the larger is the smoothening effect.

3. In the third and final step, the local outlier factor for each data point is calculated by comparing its neighbourhood density with the neighbourhood densities of its neighbouring data points:

$$\mathrm{LOF}\left(x_i, k\right) = \frac{\sum\limits_{x_j \in \mathcal{N}(x_i,k)} \rho\left(x_j, k\right)}{\rho\left(x_i, k\right) \left|\mathcal{N}\left(x_i, k\right)\right|}. \tag{6.8}$$

Generally, a data point that lies deep inside a cluster will have a local outlier factor close to 1, while a data point that lies relatively far from the nearest cluster will have a high local outlier factor (Breunig et al., 2000).

6.1.2 The Sequential Sub-Trajectory Local Outlier Nonconformity Measure

The key idea of SSTLO-NCM is to consider the set of all sub-trajectories of fixed length $w \in \mathbb{N}^+$ as high-dimensional data points (feature vectors) and adopt the framework of LOF (Section 6.1.1). That is, each uniformly sampled sub-trajectory:

$$\left(\chi_j^i, \ldots, \chi_{j+w-1}^i\right) : j = 1, \ldots, L_i - w + 1 \tag{6.9}$$

from each trajectory $i = 1, 2, \ldots, m$ in (6.2) and $i = n$ is embedded as a high-dimensional feature vector $x_j^i \in \mathbb{R}^{w \cdot d}$. The preliminary nonconformity score $\hat{\alpha}_s$ or the final nonconformity score α for (6.1) relative to (6.2) is recursively defined as:

$$\begin{aligned}
&\text{SSTLO-NCM}\Big(\left(\chi_1^n, \ldots, \chi_s^n\right), B, w, k\Big) = \\
&\qquad = \begin{cases}
\max\Big\{\mathrm{LOF}\left(x_{s-w+1}^n, k\right), \\
\text{SSTLO-NCM}\Big(\left(\chi_1^n, \ldots, \chi_{s-1}^n\right), B, w, k\Big)\Big\} & s > w \\
\mathrm{LOF}\left(x_1^n, k\right) & s = w \\
0 & s < w
\end{cases},
\end{aligned} \tag{6.10}$$

where B is defined according to (6.2), x_{s-w+1}^n is the embedded feature vector corresponding to the sub-trajectory $\left(\chi_{s-w+1}^n, \ldots, \chi_s^n\right)$, $\mathrm{LOF}\left(x_{s-w+1}^n, k\right)$ and $\mathrm{LOF}\left(x_1^n, k\right)$ are defined according to (6.8), and $\rho\left(x_j^i, k\right)$ is defined according to (6.7). In other words, the nonconformity score of a trajectory is equivalent to the maximum LOF score of its sub-trajectories observed so far. However, in order to avoid "self-matching" in the sense that the neighbourhood of an embedded sub-trajectory x_j^i includes another sub-trajectory $x_{j'}^i$ from the same trajectory i, or "multiple-matching" in the sense that the neighbourhood includes two or more sub-trajectories $x_{j'}^{i'}, x_{j''}^{i'}, \ldots$ from the same trajectory i', we re-define the neighbourhood (6.5) for an embedded sub-trajectory x_j^i as:

$$\mathcal{N}_j^i = \left\{x \in \mathcal{D}_j^i : dist\left(x_j^i, x\right) \leq dist_{border}\left(x_j^i, k\right)\right\}, \tag{6.11}$$

where:

$$\mathcal{D}_j^i = \bigcup_{i'=1,\ldots m: i' \neq i} \left\{ x_{j'}^{i'} : j' \in \underset{j'=1,\ldots,L_{i'}+1-w}{\arg\min} dist\left(x_j^i, x_{j'}^{i'}\right) \right\} \tag{6.12}$$

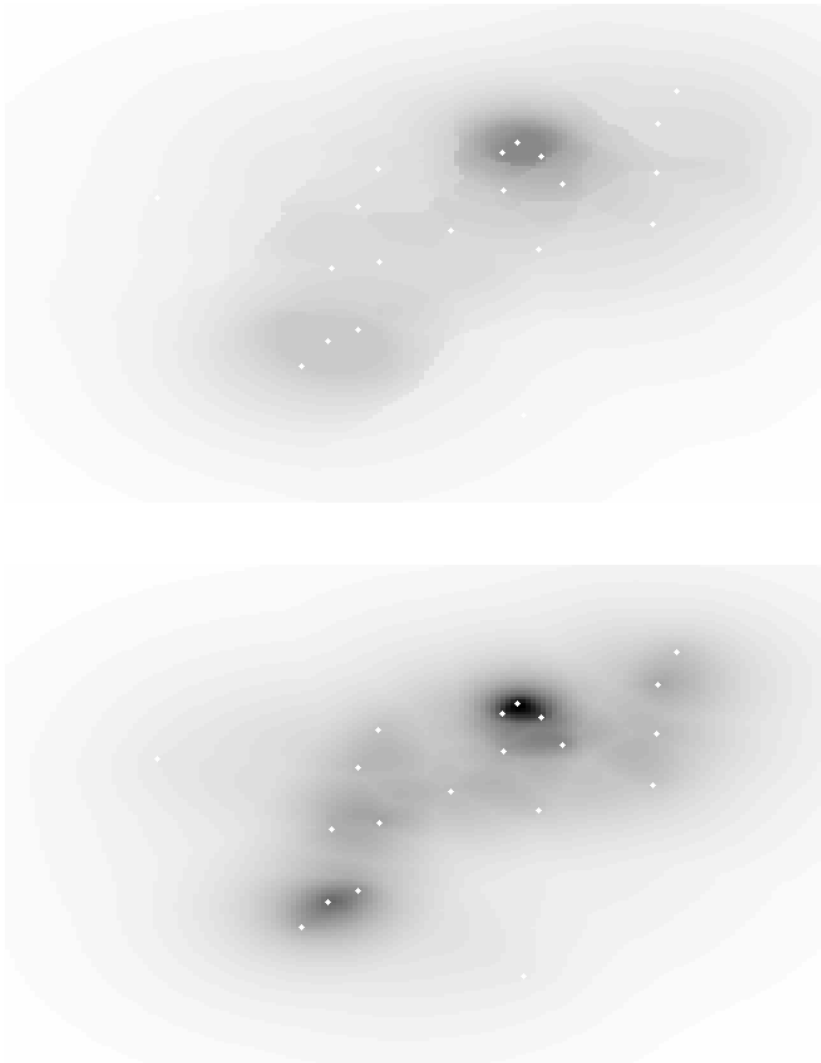


Figure 6.2: The use of the reachability distance (6.6) has a smoothening effect on the neighbourhood density. The upper subfigure shows a plot of the neighbourhood density according (6.7), where the data points are indicated by the white squares, and $k = 3$. The lower subfigure shows a plot of the density based on the same data points and $k = 3$, but with the reachability distance replaced by the standard distance. The shades of grey reflect the density; the darker the shade, the higher the density. Note that for comparative purposes, the grey scale is normalised in the two subfigures.

corresponds to the *set of aligned sub-trajectories* to x_j^i and includes exactly one sub-trajectory for each other trajectory $i' \neq i$. Assuming that there are no ties, (6.11) is equivalent to the k nearest sub-trajectories to x_j^i from k different trajectories.

Implementation

Assuming an exhaustive nearest neighbours search strategy, the calculation of the neighbourhood is the computationally most intensive step of LOF (Goldstein, 2012). One approach to reduce the complexity of the nearest neighbours search is to index the feature vectors using, e.g., a k-d tree (Friedman et al., 1977). In this work, we therefore propose and investigate a k-d tree-based implementation of SSTLO-NCM (Algorithm 6.1).

Given:

- the neighbourhood size k,
- the sub-trajectory length w,
- the k-d tree T, which stores the training set:

$$\left\{x_1^1, \ldots, x_{L_1+1-w}^1, \ldots, x_1^m, \ldots, x_{L_m+1-w}^m\right\} \tag{6.13}$$

  of embedded sub-trajectory feature vectors that have been extracted from (6.2) based on w, and
- the maximum number of sub-trajectories λ_{max} of any trajectory in (6.2),

Algorithm 6.1 outputs the sequence of preliminary nonconformity scores $\hat{\alpha}_1, \ldots, \hat{\alpha}_{L-1}$ and the final nonconformity score α for the sequence of trajectory points $\chi_1^n, \ldots, \chi_L^n$. For the first $s = 1, \ldots, w-1$ points, the nonconformity score is set to zero (line 3 of Algorithm 6.1). For each of the remaining $s = w, \ldots, L$ points, the nonconformity score is calculated according to the following steps:

1. First, the embedded sub-trajectory x_{s-w+1}^n is extracted (line 5) and the corresponding neighbourhood $\mathcal{N}_{s-w+1}^n$ calculated (line 6) based on the *sub-trajectory neighbourhood algorithm* (STNA) (Algorithm 6.2).
2. This is followed by the calculation of the neighbour density ρ_{s-w+1}^n for x_{s-w+1}^n (line 12) based on the sum of the reachability distance to its k-nearest neighbours (lines 7–11).
3. Next, the neighbour density of each of the k-nearest neighbours to x_{s-w+1}^n is calculated (lines 14–19).
4. Finally, the LOF score for x_{s-w+1}^n is calculated (line 23), based on its own neighbourhood density (line 12) and the sum of the neighbourhood densities of its k nearest neighbours (line 21). Generally, the current nonconformity score is set to the maximum of the current LOF score and the previous

Algorithm 6.1 k-d tree based SSTLO-NCM (sequential sub-trajectory local outlier nonconformity measure)

Input: Sequence of trajectory points $\chi_1^n, \ldots, \chi_L^n$, sub-trajectory length w, neighbourhood size k, set of embedded sub-trajectory feature vectors stored as k-d tree T, upper bound λ_{max} of the number of sub-trajectories per trajectory.

Output: Sequence of preliminary nonconformity scores $\hat{\alpha}_1, \ldots, \hat{\alpha}_{L-1}$ and final nonconformity score α.

for $s \leftarrow 1$ to L **do**
  if $s < w$ **then**
    $\hat{\alpha}_s \leftarrow 0$
  else
    $x_{s-w+1}^n \leftarrow \left(\chi_{s-w+1}^n, \ldots, \chi_s^n\right)$
    $\mathcal{N}_{s-w+1}^n \leftarrow$STNA$(x_{s-w+1}, k, T, \lambda_{max})$
    $distSum \leftarrow 0$
    for each $x_j^i \in \mathcal{N}_{s-w+1}^n$ **do**
      $\mathcal{N}_j^i \leftarrow$STNA$\left(x_j^i, k, T, \lambda_{max}\right)$
      $distSum \leftarrow distSum + \max\left\{\max_{x \in \mathcal{N}_j^i}\left\{dist\left(x, x_j^i\right)\right\}, dist\left(x_{s-w+1}^n, x_j^i\right)\right\}$
    end for
    $\rho_{s-w+1}^n \leftarrow k/distSum$
    $densitySum \leftarrow 0$
    for each $x_j^i \in \mathcal{N}_{s-w+1}^n$ **do**
      $distSum \leftarrow 0$
      for each $x_{j'}^{i'} \in \mathcal{N}_j^i$ **do**
        $\mathcal{N}_{j'}^{i'} \leftarrow$STNA$\left(x_{j'}^{i'}, k, T, \lambda_{max}\right)$
        $distSum \leftarrow distSum + \max\left\{\max_{x \in \mathcal{N}_{j'}^{i'}}\left\{dist\left(x, x_{j'}^{i'}\right)\right\}, dist\left(x_j^i, x_{j'}^{i'}\right)\right\}$
      end for
      $\rho_j^i \leftarrow k/distSum$
      $densitySum \leftarrow densitySum + \rho_j^i$
    end for
    $LOF \leftarrow densitySum/\left(k \cdot \rho_{s-w+1}^n\right)$
    if $s == w$ **then**
      $\hat{\alpha}_s \leftarrow LOF$
    else
      $\hat{\alpha}_s \leftarrow \max\left(\hat{\alpha}_{s-1}, LOF\right)$
    end if
  end if
end for
$\alpha \leftarrow \hat{\alpha}_L$

Algorithm 6.2 Sub-trajectory neighbourhood algorithm (STNA)

Input: Embedded sub-trajectory x_j^i, neighbourhood size k, set of embedded sub-trajectories stored as k-d tree T, upper bound λ_{max} for number of sub-trajectories per trajectory.
Output: Sub-trajectory neighbourhood $\mathcal{N}_j^i$.

```
1: N_j^i ← ∅
2: I ← i
3: v ← 1
4: V ← T.nearestNeighbourSearch(x_j^i, k · λ_max + 1)
5: while |N_j^i| < k do
6:    x_j^i' ← V.getElementAt(v)
7:    if i' ∉ I then
8:       N_j^i ← N_j^i ∪ x_j^i'
9:       I ← I ∪ i'
10:   end if
11:   v ← v + 1
12: end while
```

nonconformity score (line 27). However, in the special case that $w = 1$ and $s = 1$, the first nonconformity score is simply set to the current LOF score.

The search strategy of STNA (Algorithm 6.2) is to first extract a list of embedded sub-trajectories V from the k-d tree T, such that V is sorted in ascending order according to the distance to x_j^i and V is *guaranteed* to include the k-nearest sub-trajectories to x_j^i from k different trajectories (line 4 of Algorithm 6.2). The neighbourhood $\mathcal{N}_j^i$ of x_j^i is then incrementally calculated by iterating over V and adding each embedded sub-trajectory $x_{j'}^{i'}$ from V if no other sub-trajectory from the same trajectory i' has previously been added to $\mathcal{N}_j^i$ (lines 5–12). The iterative procedure is repeated until the size of $\mathcal{N}_j^i$ has reached k. In order to guarantee that V contains nearby sub-trajectories from *at least* k distinct trajectories other than i, the range for the nearest neighbours search in T is set to $k \cdot \lambda_{max} + 1$ neighbours, where λ_{max} is an input parameter corresponding to an upper bound of the number of sub-trajectories of any trajectory $i = 1, \ldots, m$. That is, the length of V is $k \cdot \lambda_{max} + 1$.

Complexity Analysis

In theory, the worst-case computational complexity of the nearest neighbours search in a k-d tree is linear in the number of elements of the tree (Friedman et al., 1977), i.e., the same as for the exhaustive k-nearest neighbours search. In practice, however, the search in a k-d tree is usually significantly faster than the corresponding exhaustive search; the expected complexity of the search is logarithmic in the number of elements of the tree (Friedman et al., 1977). Hence, the

expected complexity of the nearest neighbours search in T (line 4 of Algorithm 6.2) is $O(\log(m \cdot \lambda_{avg}))$, where m is the number of trajectories in (6.2) and λ_{avg} is the average number of sub-trajectories per trajectory. The iterative calculation of the neighbourhood based on V (lines 5–12 of Algorithm 6.2) has worst-case complexity $O(k \cdot \lambda_{max})$. Hence, the expected complexity of STNA is no worse than $O(\log(m \cdot \lambda_{avg}) + k \cdot \lambda_{max})$.

Considering Algorithm 6.1, line 6, the first inner loop (lines 8–11) and the second inner loop (lines 14–22) have complexity:

- $O(\log(m \cdot \lambda_{avg}) + k \cdot \lambda_{max})$,
- $O(k(\log(m \cdot \lambda_{avg}) + k \cdot \lambda_{max}))$, and
- $O\left(k^2(\log(m \cdot \lambda_{avg}) + k \cdot \lambda_{max})\right)$,

respectively. The remainder of each iteration of the outer loop (lines 1–30) has constant complexity. Thus, the expected complexity of calculating each nonconformity score $\hat{\alpha}_s : s = w, \ldots, L$ is:

$$O\left(k^2 \log(m \cdot \lambda_{avg}) + k^3 \cdot \lambda_{max}\right) \tag{6.14}$$

and the overall complexity of Algorithm 6.1 is:

$$O\left(L \cdot k^2 \log(m \cdot \lambda_{avg}) + L \cdot k^3 \cdot \lambda_{max}\right) \tag{6.15}$$

The computational complexity of constructing T from (6.13) is:

$$O(w \cdot d \cdot m \cdot \lambda_{avg} \cdot \log(m \cdot \lambda_{avg})) \tag{6.16}$$

(Friedman et al., 1977), where $m \cdot \lambda_{avg}$ corresponds to the number of elements of the tree and $w \cdot d$ corresponds to the dimension of each element. However, the computational time required for constructing T is less important than the time it takes to search T, since T is constructed only once in the case of the offline and semi-offline ICAD (Section 4.4.1), or re-constructed during each update trial in the online mode (Section 4.4.2).

Discussion and Analysis

SSTLO-NCM considers all sub-trajectories of length w and will assign a relatively high nonconformity score to a trajectory whenever any of its sub-trajectories is anomalous, regardless of whether the remaining part of the trajectory is perfectly normal. The parameter w corresponds to the scale of the anomalous sub-trajectory and, depending on the application, it might be appropriate to run multiple anomaly detectors in parallel using different values of w. Hence, SSTLO-NCM fulfils property 1 (see introduction of Section 6.1). A sub-trajectory will be assigned a large LOF score if the local density is relatively small compared to the local density

of the nearest sub-trajectories from other trajectories. Therefore, SSTLO-NCM fulfils property 2.

From (6.11) and (6.12), it is clear that none of the embedded sub-trajectories $x_1^n, \ldots, x_{s-w+1}^n$ are included in any neighbourhood:

$$\mathcal{N}_j^i : i = 1, \ldots, m, j = 1, \ldots, L_i.$$

This implies that each and every $\rho\left(x_j^i, k\right)$ is independent of $x_1^n, \ldots, x_{s-w+1}^n$ and, hence, $\mathrm{LOF}\left(x_1^n, k\right), \ldots, \mathrm{LOF}\left(x_{s-w}^n, k\right)$ are independent of each other. Since $\hat{\alpha}_s$ is equal to the maximum of $\mathrm{LOF}\left(x_1, k\right), \ldots, \mathrm{LOF}\left(x_{s-w+1}, k\right)$, it follows that (6.3) is true and SSTLO-NCM fulfils property 3.

6.2 The Sequential Sub-Trajectory Local Outlier Inductive Conformal Anomaly Detector

Following the results and discussion in Section 6.1, we propose SSTLO-ICAD for the sequential detection of local anomalous sub-trajectories, where SSTLO-ICAD is equivalent to Algorithm 4.3 such that A is implemented by Algorithm 6.1. In contrast to previous algorithms, including SHNN-CAD, we argue that SSTLO-ICAD addresses *all* of the key properties in Section 3.3. Since SSTLO-NCM fulfils property 3 in Section 6.1, SSTLO-ICAD is guaranteed to be well-calibrated in the semi-offline and online modes (Section 4.4.1–4.4.2)[1]. As a result of the fact that SSTLO-NCM does not involve any assumptions regarding the probability distribution of the trajectories, SSTLO-ICAD is purely non-parametric. Further, SSTLO-NCM is obviously designed for the sequential analysis of trajectories, and the property of unsupervised online learning is naturally inherited from ICAD. Finally, SSTLO-NCM fulfils properties 1 and 2 in Section 6.1. Hence, in contrast to SHNN-CAD, SSTLO-ICAD is sensitive to local anomalous trajectories at different scales. This additional property comes at the expense of an additional scale parameter w, which is the sub-trajectory length.

6.3 Empirical Investigations in the Maritime Domain

In this section, we empirically investigate SSTLO-NCM and SSTLO-ICAD on a number of different trajectory datasets from the maritime domain. We start by investigating the most anomalous trajectories detected in an unlabelled set of vessel trajectories for different parameter values of SSTLO-NCM (Section 6.3.1). This is followed by the investigation of the empirical alarm rate of SSTLO-ICAD

[1]Note that the online CAD (Algorithm 4.2) based on SSTLO-NCM is *not* guaranteed to be well-calibrated; there is no guarantee that the nonconformity score for a trajectory of the training set does not increase as (6.1) is updated with more points and, hence, the p-value of (6.1) is not guaranteed to monotonically decrease.

on the same dataset (Section 6.3.2). In Section 6.3.3, we reproduce a previously published experiment where the detection delay of SSTLO-NCM is evaluated on a set of simulated anomalous vessel trajectories. In the final subsection, we evaluate the classification performance of SSTLO-NCM on a labelled trajectory dataset that includes realistic maritime anomalies (Section 6.3.4).

We implement Algorithms 6.1 and 6.2 in Java™ based on the publicly available k-d tree implementation by Rednaxela[2]. Similar to the empirical investigations in the previous chapter (Section 5.2), we also implement the discords algorithm (Keogh et al., 2005) in Java™ and investigate its classification performance for comparative purposes. However, we extend the previous implementation of the discords algorithm by incorporating the same k-d tree implementation as above for indexing the sub-trajectories of the training set and speeding up the nearest neighbours searches of the discords algorithm. All algorithms are executed on a Macbook Pro 2.66 GHz Intel Core 2 Duo processor with 8 GB of RAM.

6.3.1 Preliminary Investigations

In this subsection, we investigate the anomalous trajectories detected by SSTLO-NCM on a dataset of unlabelled vessel trajectories. The objective of this experiment is to investigate the character of the most anomalous trajectories detected for different values of k and w.

Dataset and Preprocessing

The dataset consists of 7211 uniformly sampled vessel trajectories:

$$\left\{\left(\chi_1^i, \ldots, \chi_{L_i}^i\right) : i = 1, \ldots, 7211\right\}, \tag{6.17}$$

where each trajectory point $\chi_j^i \in \mathbb{R}^2 : j = 1, \ldots, L_i$ represents the vessel's position in the two-dimensional plane at some time point. The trajectories have been extracted from a database of recorded AIS[3] data, which was originally provided by Saab AB in cooperation with the Swedish Maritime Administration. The database corresponds to approximately three weeks of recorded vessel traffic along the western coast of Sweden. The data is *unlabelled* in the sense that there is no information whether a particular trajectory belongs to the normal or abnormal class. Nevertheless, the AIS data includes information regarding the vessel type, i.e., whether the corresponding vessel is a tanker, cargo vessel, passenger ship, pilot, etc.

Each row in the database corresponds to a vessel report that contains, amongst other things, the MMSI (Maritime Mobile Service Identity) of the corresponding vessel, the current time and the current geographical position in latitudinal and

[2]https://bitbucket.org/rednaxela/knn-benchmark/src/tip/ags/utils/dataStructures/trees/thirdGenKD/

[3]Automatic Identification System (AIS). For more information, see, e.g., http://www.imo.org/ourwork/safety/navigation/pages/ais.aspx.

longitudinal coordinates. We transform each geographical position to the two-dimensional plane and extract only those vessel reports that are within a specified rectangle of size 40×35 km, located outside the port area of Gothenburg, Sweden. Further, we extract trajectories from each vessel by tracking it based on the MMSI, which is assumed to uniquely identify each vessel. Since stationary vessels are not assumed to be of interest, we apply an *exponential moving average filter* with parameter value $\alpha = 0.12$ and only initiate a trajectory if the filtered speed value exceeds 0.6 knots. If a vessel leaves the specified rectangle, or if the filtered speed value drops below 0.3 knots, the current trajectory is terminated. During tracking, the trajectory is uniformly re-sampled where the distance in time between successive data points is 60 seconds. The experimental dataset is defined as the subset of 7211 trajectories, such that the length in time of each trajectory is at least 5 minutes and the distance between the start and end points is at least 1 km. The choice of the preprocessing parameter values above was found to generate trajectories that correspond to natural vessel routes that are not too short and typically terminate in harbour areas.

Design

For each combination of the parameter values $k = 2$ or $k = 5$, and $w = 6$ or $w = 61$, we investigate the trajectories having the top-three highest nonconformity scores according to SSTLO-NCM. The length of the sub-sequences correspond to 5 minutes and 1 hour, where the former is considered to be very short and captures the momentary behaviour of the vessel, while the latter reflects more of the route of the vessel. Hence, they capture different aspects of the vessel behaviour and may be considered to complement each other. For each trajectory:

$$\left(\chi_1^i, \ldots, \chi_{L_i}^i\right) : i = 1, \ldots, 7211 \tag{6.18}$$

we calculate the final nonconformity score α_i relative to the remaining set of trajectories:

$$\left\{ \left(\chi_1^j, \ldots, \chi_{L_j}^j\right) : j = 1, \ldots, 7211, j \neq i \right\}, \tag{6.19}$$

according to (6.10). The sequence of nonconformity scores $\alpha_1, \ldots, \alpha_{7211}$ are then sorted in descending order. For each combination of parameter values, the three trajectories having the top-three nonconformity scores are shown in Figures 6.3–6.6.

Analysis of Results

Let us start by considering the results in Figure 6.3, where $k = 5$ and $w = 6$ (5 minutes). It is clear from the overview that the three sub-trajectories with the highest nonconformity scores are located in areas where traffic is relatively dense and follows well-defined sea lanes. The first and the second sub-trajectory, which are almost identical, seem to follow an infrequent path. The third sub-trajectory

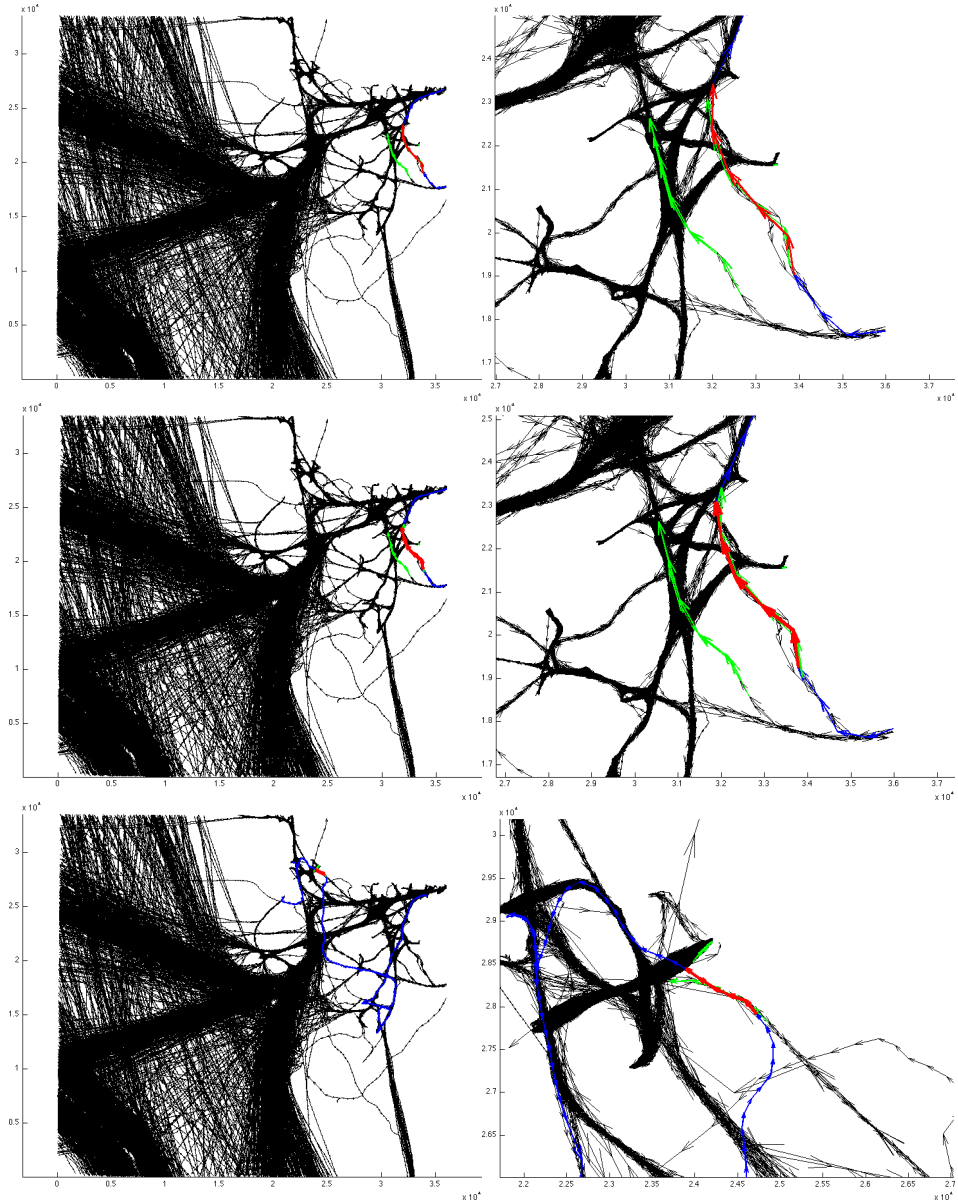


Figure 6.3: Illustration of the three trajectories (blue) having the top-three nonconformity scores relative to the remaining trajectories (black) according to SSTLO-NCM, with $w = 6$ (5 minutes) and $k = 5$. The left and the right columns of plots show an overview and a zoomed-in view, respectively, of each of the top-three nonconforming trajectories. Each red sub-trajectory corresponds to the sub-trajectory that has the largest LOF score. The green sub-trajectories are the k-nearest neighbours to the corresponding red sub-trajectory.

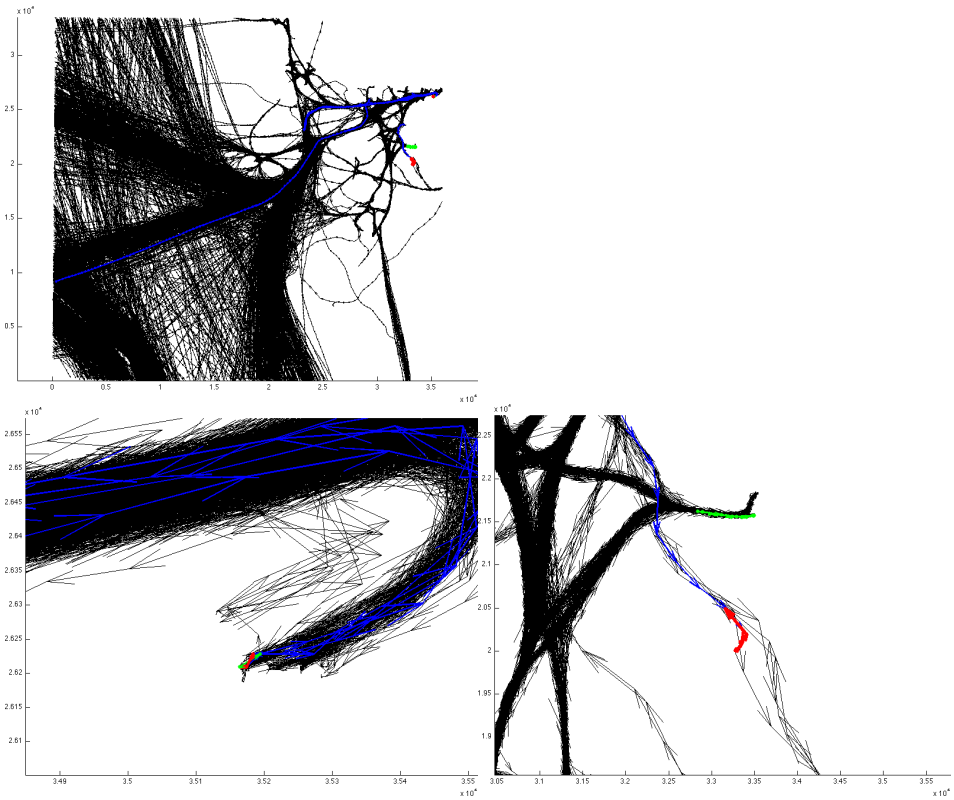


Figure 6.4: Illustration of the three trajectories (blue) having the top-three nonconformity scores relative to the remaining trajectories (black) according to SSTLO-NCM, with $w = 6$ (5 minutes) and $k = 2$. The top plot shows an overview and the two bottom plots show a zoomed-in view of the top-three nonconforming trajectories. Each red sub-trajectory corresponds to the sub-sequence that has the largest LOF score. The green sub-trajectories are the k-nearest neighbours to the corresponding red sub-trajectory.

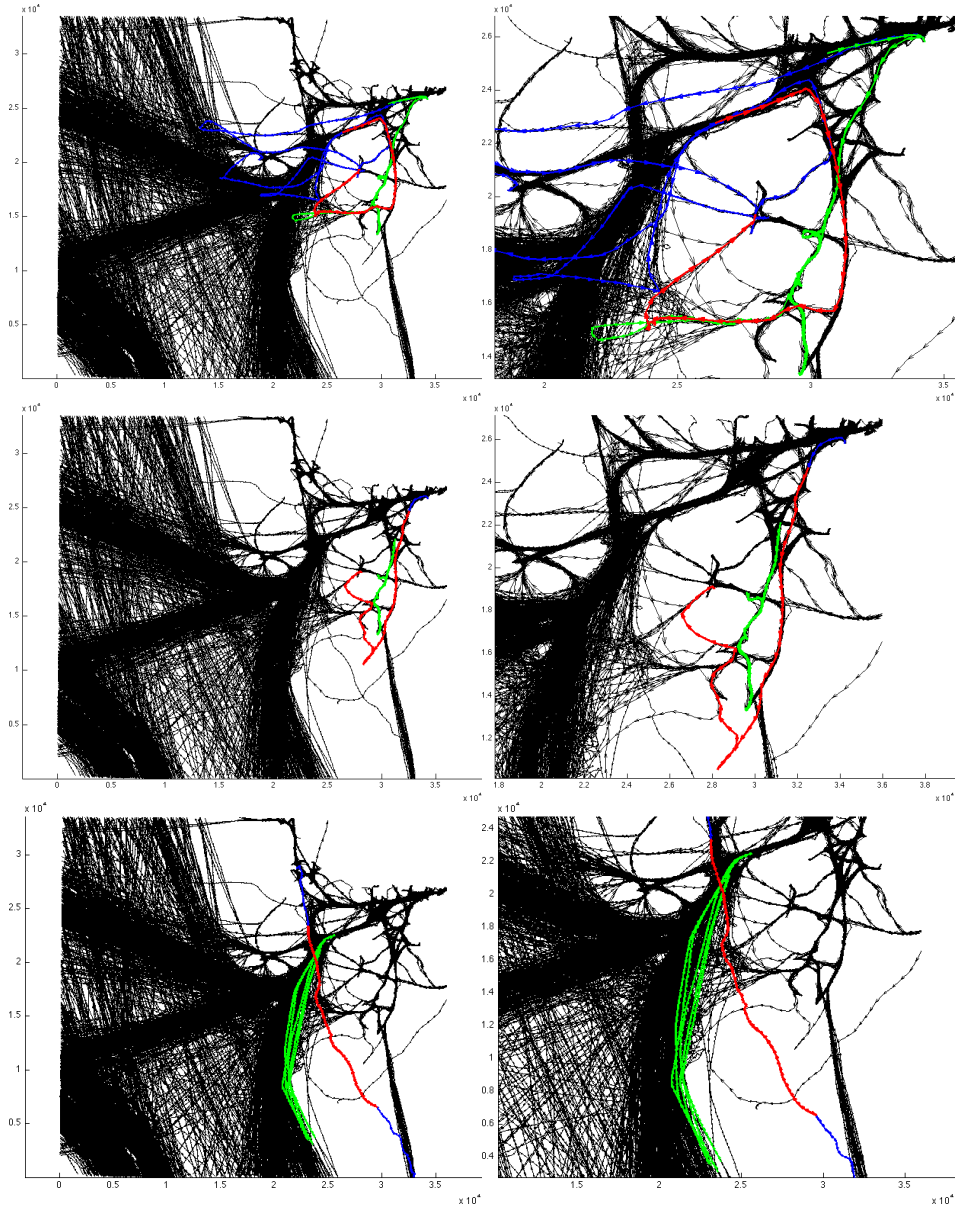


Figure 6.5: Illustration of the three trajectories (blue) having the top-three nonconformity scores relative to the remaining trajectories (black) according to SSTLO-NCM, with $w = 61$ (1 hour) and $k = 5$. The left and the right columns of plots show an overview and a zoomed-in view, respectively, of each of the top-three nonconforming trajectories. Each red sub-trajectory corresponds to the sub-sequence that has the largest LOF score. The green sub-trajectories are the k-nearest neighbours to the corresponding red sub-trajectory.

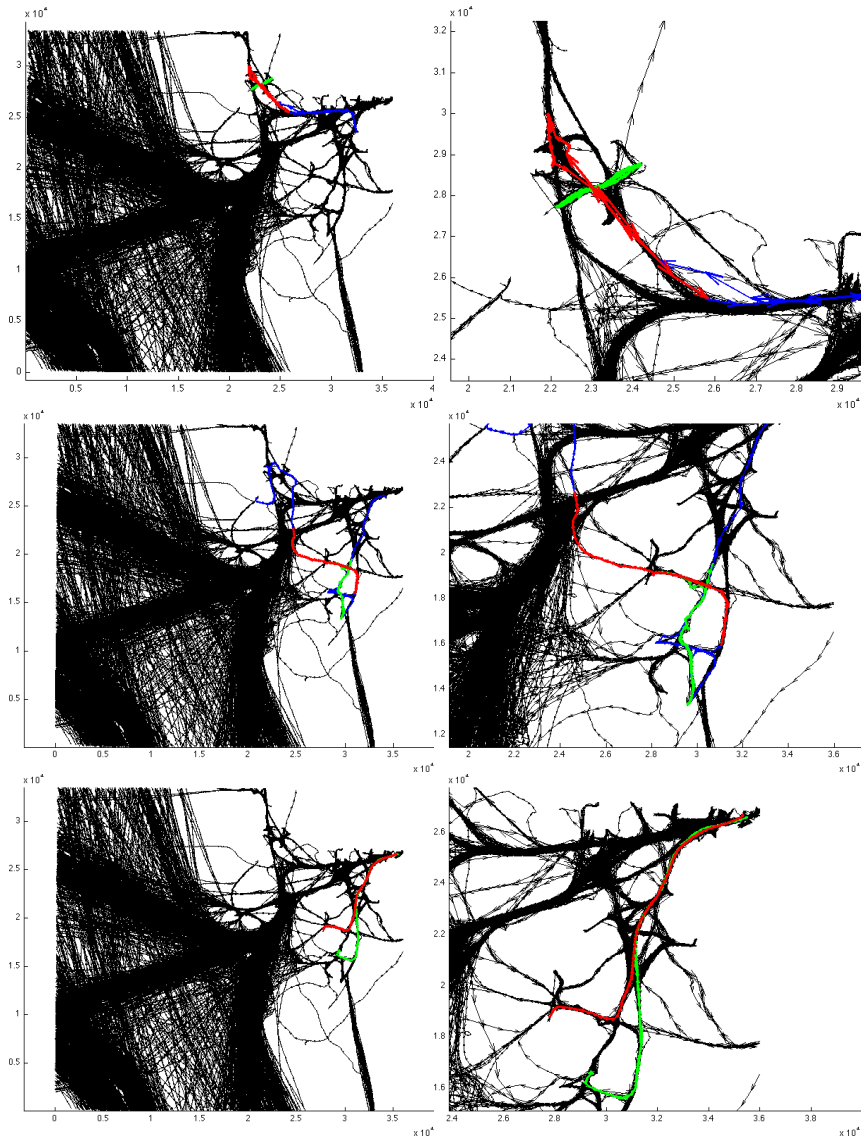


Figure 6.6: Illustration of the three trajectories (blue) having the top-three nonconformity scores relative to the remaining trajectories (black) according to SSTLO-NCM, with $w = 61$ (1 hour) and $k = 2$. The left and the right columns of plots show an overview and a zoomed-in view, respectively, of each of the top-three nonconforming trajectories. Each red sub-trajectory corresponds to the sub-sequence that has the largest LOF score. The green sub-trajectories are the k-nearest neighbours to the corresponding red sub-trajectory.

also follows a relatively infrequent path; four of its five nearest neighbours (green) correspond to slow moving sub-trajectories located north in a very dense area. Interestingly, all three nonconforming sub-trajectories belong to three different rescue vessels; the behaviour of this vessel type, which constitutes a small minority of all vessels in this area, is likely to be different from the tankers, cargo and passenger vessels, which constitute the majority of all trajectories.

Considering the smaller neighbour size, i.e., $k = 2$ (Figure 6.4), we see that one of the top-three nonconforming sub-trajectories is located close to the nonconforming sub-trajectories in Figure 6.3. The other two sub-trajectories, however, are quite different; they correspond to two slow moving vessels that are about to moor in or leave a harbour area. At first, these results may seem strange, since the location of the vessels seems perfectly normal. However, the reason for the high LOF scores of these sub-trajectories is that the neighbourhood density of their neighbouring sub-trajectories is extremely large in this very dense area. This type of problem is generally mitigated by increasing the value of k which has a smoothing effect.

When $w = 61$ (Figures 6.5–6.6), i.e., when sub-trajectories correspond to one hour, we see that the most nonconforming sub-trajectories correspond to unique routes that are clearly different from all other trajectories. In particular, some of the sub-trajectories pass isolated areas far from the closest sea lane. This might be explained by the fact that the corresponding vessels are of the minority vessel type, pilot, rescue, and fishing vessel, respectively, the behaviour of which differs from the majority vessel types, cargo, tankers, and passenger vessels. However, in contrast to the case when $w = 6$, the choice of k seems to have less impact on the character of the detected anomalies.

To summarise, the sub-trajectories with the highest nonconformity scores correspond to unique or infrequent routes, sometimes in areas with no previous vessel traffic, and appear anomalous with respect to the local neighbourhood. They belong to odd vessel types, such as pilots, rescue vessels and fishing boats, which may explain their anomalous nature. Regarding the choice of k, it seems that small values, such as 2, should be avoided when w is relatively small.

6.3.2 Empirical Alarm Rate

In this section, we investigate the empirical alarm rate of the unsupervised SSTLO-ICAD in the semi-offline mode (Section 4.4.1) and in the alternative online mode (Section 4.4.2) on the set of unlabelled vessel trajectories used in the previous experiment (Section 6.3.1). The main objective of the experiment is to determine whether the alarm rate is well-calibrated in practice, i.e., whether it is less than or close to the specified anomaly threshold ϵ.

Design

We generate 100 random permutations of the set of 7211 unlabelled vessel trajectories (6.17) (Section 6.3.1). For each such sequence, the first 1111 trajectories

are allocated to the initial proper training set and the nonconformity scores are precomputed for the following 100 trajectories. The remaining sequence of 6000 trajectories is classified by the unsupervised SSTLO-ICAD. The anomaly threshold ϵ is set to 1%, which is assumed to correspond to a reasonable alarm rate in an anomaly detection application. The neighbourhood parameter k is set to 2 or 5 and the sub-sequence length w is set to 6 (5 minutes) or 61 (1 hour), i.e., four different parameter setups are evaluated similar to the previous experiment (Section 6.3.1). The procedure is repeated on the same 100 random permutations for the semi-offline and the alternative online version of ICAD with $\eta = 100$.

Analysis of Results

The mean of the distribution of the alarm rates for the semi-offline SSTLO-ICAD (Figure 6.7) is between 0.0095 and 0.0097 for each of the different parameter values. This is expected since the alarm rate of the semi-offline SSTLO-ICAD is guaranteed to be less or equal to ϵ, up to statistical fluctuations, under the assumption that the examples are IID (Section 4.4.1). More interesting, however, are the results for the alternative online SSTLO-ICAD in Figure 6.8; the mean of the distribution of the alarm rates is equal to 0.0069 for all values of k and w. Further, the alarm rate never exceeds 0.01 for any of the 100 sequences. Hence, the alarm rate of the alternative online SSTLO-ICAD is well-calibrated in practice, although it is not guaranteed to be well-calibrated in theory (Section 4.4.2). Further, examining the detailed results in Figure 6.9 for the first of the 100 random sequences, we see that the total number of alarms (blue) never exceeds the upper bound of the expected number of alarms for a well-calibrated anomaly detector (solid black line) and that the alarm rate is close to constant (dashed black).

6.3.3 Detection Delay: Simulated Anomalies

In this subsection, we reproduce a previously published experiment (Laxhammar et al., 2009) in which the detection delay on a set of simulated anomalous trajectories was investigated. The main objectives of the experiment are to:

- Investigate whether SSTLO-NCM is more sensitive to the anomalous vessel trajectories, in terms of less detection delay, compared to the discords algorithm and other previously proposed algorithms (Laxhammar, 2011).
- Investigate how different values of k and w affect the detection delay of SSTLO-NCM.

Dataset

The original dataset used in this experiment consists of a training set of 2310 unlabelled vessel trajectories and a test set of 1000 vessel trajectories labelled abnormal (Laxhammar, 2011). Each trajectory (2.15) is four-dimensional where $\chi_i \in \mathbb{R}^4$ represents the longitude, latitude, speed (knots) and course (degrees)

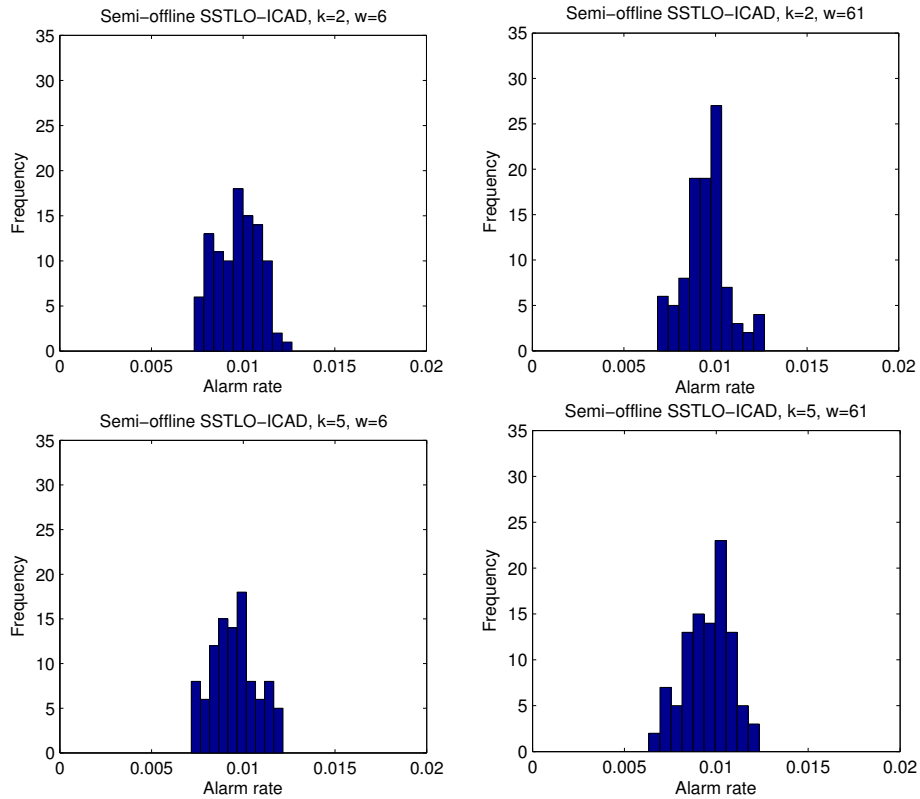


Figure 6.7: Histograms of the empirical alarm rates of the semi-offline SSTLO-ICAD with $\epsilon = 0.01$ on each of the 100 sequences of unlabelled trajectories (Section 6.3.2). First and second rows correspond to the cases when $k = 2$ and $k = 5$, respectively. First and second columns correspond to the cases when $w = 6$ and $w = 61$, respectively. The mean of the alarm rates for the top-left, top-right, bottom-left and bottom-right histograms is 0.0097, 0.0095, 0.0096 and 0.0095, respectively.

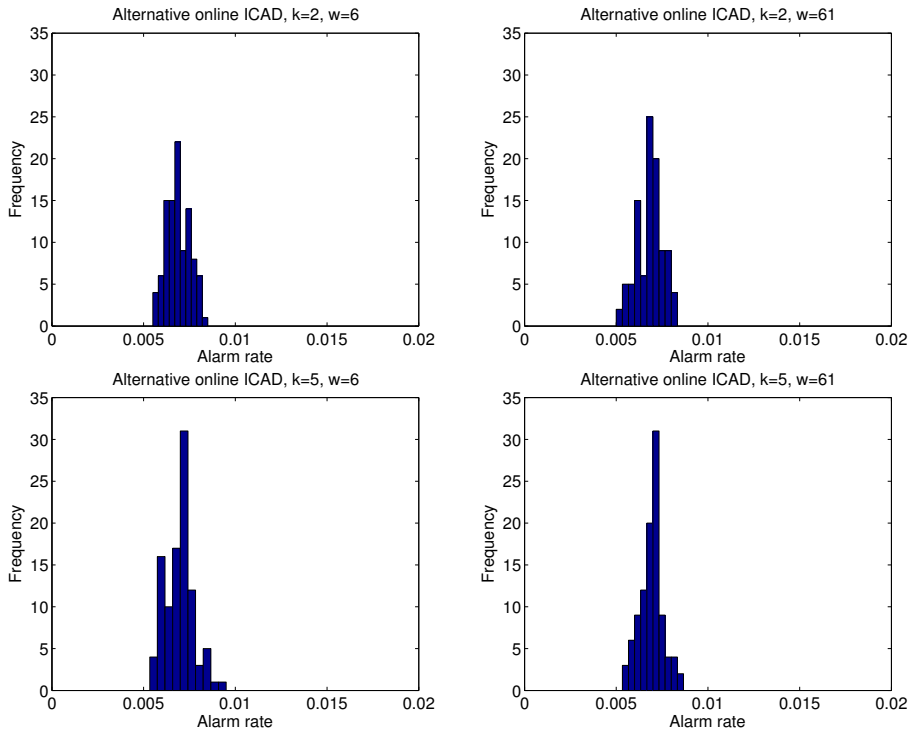


Figure 6.8: Histograms of the empirical alarm rates of the alternative online SSTLO-ICAD with $\epsilon = 0.01$ and $\eta = 100$ on each of the 100 sequences of unlabelled trajectories (Section 6.3.2). First and second rows correspond to the cases when $k = 2$ and $k = 5$, respectively. First and second columns correspond to the cases when $w = 6$ and $w = 61$, respectively. The mean of the alarm rates is equal to 0.0069 for all of the four cases above.

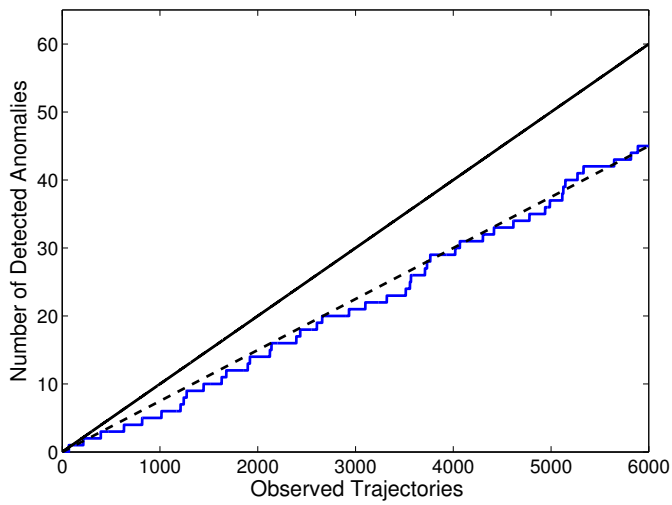


Figure 6.9: Plot of the accumulated number of anomalous trajectories detected (y-axis) as a function of the number of observed trajectories. The black solid line corresponds to the expected upper bound of the accumulated number of anomalies for a well-calibrated anomaly detector with $\epsilon = 0.01$. The blue graph corresponds to the actual result for the unsupervised alternative online SSTLO-ICAD on the first random sequence of 6000 unlabelled vessel trajectories (Section 6.3.2), where $k = 5$, $w = 61$, $\epsilon = 0.01$ and $\eta = 100$. The dashed line corresponds to the average rate of detected anomalies, i.e., the straight line from $(0, 0)$ to $(45, 6000)$.



Figure 6.10: Screen image captured from Google Earth where the 2888 unlabelled trajectories extracted from the AIS database have been superimposed on a photo (copyright 2009 Lantmäteriet/Metria) of the port area of Gothenburg. The size of the area is approximately 12×8 km.

of the vessel at time point i. The training set was randomly sampled without replacement from a set of 2888 unlabelled vessel trajectories, which were extracted from the same AIS database as in Section 6.3.1 but from a smaller area of size 12×8 km outside the port of Gothenburg (Figure 6.10). The trajectories were geographically sampled every 200 meters and terminated or initiated whenever the corresponding vessel left or entered the area, respectively, or when the vessel remained stationary for more than 5 minutes or started to move again, respectively (Laxhammar et al., 2009).

Each trajectory from the test set consists of an initial unlabelled segment:

$$\chi_1, \ldots, \chi_b, \tag{6.20}$$

which is followed by an abnormal segment:

$$\chi_b, \ldots, \chi_L, \tag{6.21}$$

where $L = b + 100$. The unlabelled segment was constructed by first randomly sampling a trajectory with replacement among the remaining 578 unlabelled trajectories that were not part of the training set. A breakpoint $\chi_b : 1 \leq b < L'$ was then randomly selected and (6.20) extracted as the unlabelled segment, where L' is the length of sampled trajectory. The abnormal segment (6.21) is a random walk which was generated as follows: Starting at χ_b, new values for the speed and course are randomly sampled from the uniform distributions over $[0, 30]$ and

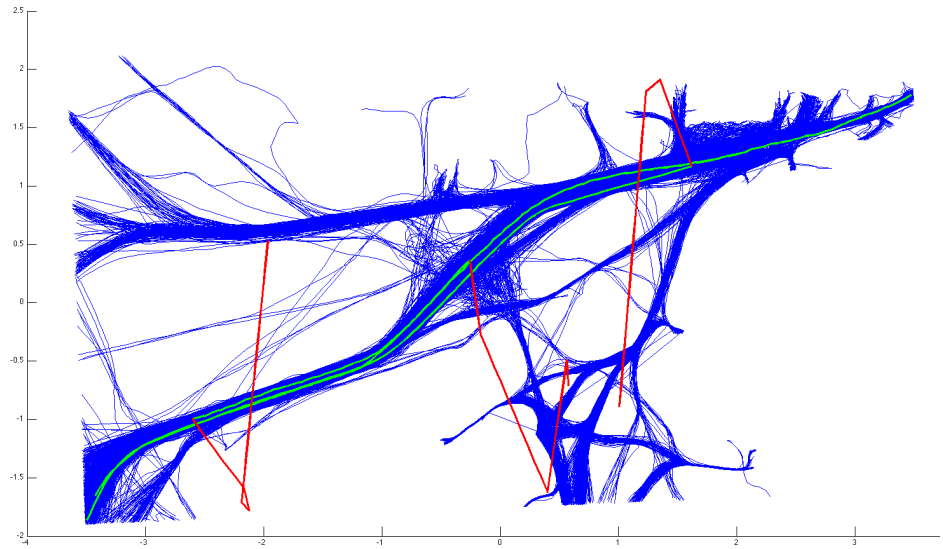


Figure 6.11: Plot of three randomly selected trajectories from the test set of 1000 abnormal trajectories. The unlabelled segments of the test trajectories are green and the simulated abnormal segments are red (Section 6.3.3). The blue trajectories (2310) belong to the training set. Note that only the first 20 points from the abnormal segments have been plotted for clarity.

$[0, 360)$, respectively. Next, the latitudinal and longitudinal coordinates for each $\chi_i : i = b+1, \ldots, L$ are calculated by projecting the next position 200 m ahead of χ_{i-1} based on the current course. Thus, the sampling distance of (6.21) is equal to that of (6.20). For each $\chi_i : i = b+1, \ldots, L$, there is also a 10% probability that new values for the speed and course are sampled from the uniform distributions above, independently of each other. If the trajectory is about to leave the sampling area (Figure 6.10), a new course is sampled, which ensures that the trajectory does not leave the area. A sample of the test trajectories are illustrated in Figure 6.11.

Preprocessing

In the reproduced experiments of this work, each trajectory of the original training and test sets above has been further preprocessed as follows: First, we project the geographical position (latitude and longitude) of each $\chi_i \in \mathbb{R}^4 : i = 1, \ldots, L$ to the two-dimensional spatial plane. Next, we set the timestamp t_1 of the first projected trajectory point $\chi'_1 \in \mathbb{R}^2$ to zero and estimate the remaining timestamps[4]

[4]Note that the original timestamp of each vessel report of the AIS database was discarded during the previous extraction of the trajectories from the AIS database (Laxhammar, 2011);

$t_i : i = 2, \ldots, L$ for each projected trajectory point $\chi'_i \in \mathbb{R}^2$ according to:

$$t_i = t_{i-1} + \frac{\left\|\chi'_i - \chi'_{i-1}\right\|}{\left(v_i + v_{i-1}\right)/2}, \tag{6.22}$$

where v_i is the momentary speed of the vessel extracted from χ_i. Finally, we resample the points $(\chi'_1, t_1), \ldots, (\chi'_L, t_L)$ based on linear interpolation in time with constant sampling rate dt.

Design

Analogously to the original experiment (Laxhammar, 2011), we investigate the detection delay for SSTLO-NCM on each of the 1000 abnormal test trajectories as follows: Starting with (6.20), we incrementally calculate the sequence of preliminary nonconformity scores $\hat{\alpha}_i : i = b+1, b+2 \ldots$ for each update χ'_i of the incomplete test trajectory $(\chi'_1, \ldots, \chi'_i)$ relative to the training set of 2310 trajectories based on Algorithm 6.1. The detection delay for the test trajectory is defined as $j - b$, where $j > b$ is the index of the first point χ'_j for which the corresponding preliminary nonconformity score $\hat{\alpha}_j \geq \alpha_{Threshold}$. That is, the detection delay corresponds to the number of observed points from (6.21) prior to anomaly detection. The anomaly threshold $\alpha_{Threshold}$ is calibrated by calculating the nonconformity scores for each of the 1000 unlabelled test segments (6.20) relative to the training set, sorting the scores and setting $\alpha_{Threshold}$ equal to the 10th largest nonconformity score. Hence, similar to the original experiment, the non-normalised anomaly threshold $\alpha_{Threshold}$ is calibrated so that 1% of the unlabelled test segments are classified as anomalous.

The procedure above is repeated for each combination of $k = 1, \ldots, 9$ and:

$$(w, dt) = (4, 10), (7, 10), (7, 20), (10, 20), (10, 30), (11, 30), \tag{6.23}$$

which correspond to the sub-trajectory lengths 0.5, 1, 2, 3, 4 and 5 minutes, respectively. The choices of w and dt are done such that the dimension of each sub-trajectory feature vector (Section 6.1.2) is at most 22, which ensures that the curse of dimensionality and the computational complexity of the k-nearest neighbour searches are suppressed. The corresponding mean and median detection delay are reported in Tables 6.1 and 6.2.

We also repeat the above procedure for the discords algorithm where the length of each subsequence corresponds to w. Analogously to SSTLO-NCM above, we calibrate the anomaly threshold by first calculating the nearest neighbour distance of the sub-trajectory discord for each of the 1000 unlabelled test segments relative to the training set. The 1000 discord distances are then sorted and the anomaly threshold set to the 10th largest distance. The discord distance for the incomplete test trajectory $(\chi'_1, \ldots, \chi'_i) : i = b + 1, b + 2, \ldots$ relative to the training set is

there was no need for this information in the original experiments (Laxhammar, 2011) since the cell-based GMM and KDE algorithms do not consider the timestamp of each point (in contrast to SSTLO-NCM and the discords algorithm).

incrementally updated and the detection delay defined as the index $j-b$ of the first point χ'_j for which the discord distance equals or exceeds the anomaly threshold. Results for the mean and median detection delay of the discords algorithm are summarised in Tables 6.1 and 6.2.

Discussion

To start with, it is clear that the performance of both SSTLO-NCM and discords improves as the length of the sub-trajectories decreases; the lowest detection delay is achieved when $(w, dt) = (4, 10)$, which corresponds to half a minute and is the smallest length of the sub-trajectories in these experiments. This could to some extent be explained by the fact that when $t_b < (w-1) \cdot dt$, i.e., when the time point t_b for the corresponding breakpoint χ'_b is less than the time length of the sub-trajectory, there is a lower bound on the detection delay that is equal to $(w-1) \cdot dt - t_b$.

It is not very surprising that the performance of the discords algorithm is approximately the same as the performance of the cell-based GMM and KDE (right-side columns of Tables 6.1 and 6.2) when $(w-1) \cdot dt$ is small, since the latter are point-based anomaly detectors (Section 3.2.2) and the former can also be considered point-based when $w \to 1$ and $dt \to 0$. Nevertheless, SSTLO-NCM outperforms both discords and the cell-based GMM and KDE when the sub-trajectory length is two minutes or less and $k \leq 4$. This is most likely explained by the fact that SSTLO-NCM, in contrast to the other algorithms, takes into account the local density of the sub-trajectories. Examining Figure 6.10, we see that the local traffic density (blue) varies greatly in different areas. As argued in our previous work (Laxhammar, 2011, Section 5.4.5), the relatively infrequent, yet normal, vessel routes are marginalised by the GMM and KDE, which consider the global density. Considering the longer sub-trajectories of a length of two or more minutes, we see that SSTLO-NCM consistently outperforms discords, regardless of k.

It interesting to note that the value of k seems to have significant impact on the detection delay and that the optimal choice of k is dependent on the length of the sub-trajectory. In particular, it seems that the optimal value for k is close to 3 when the sub-trajectory length is two minutes or less, but increases when the length is three minutes or more.

6.3.4 Classification Performance: Realistic Anomalies

In this final subsection, we evaluate the classification performance of SSTLO-NCM and the discords algorithm on a labelled dataset that includes realistic anomalies from a number of different maritime anomaly classes. The main objectives of the experiment are to investigate:

- whether SSTLO-NCM is capable of detecting realistic anomalies in the maritime domain with reasonable classification performance,

Table 6.1: *Mean* detection delay (number of trajectory points) for SSTLO-NCM, discords and cell-based GMM/KDE (Laxhammar et al., 2009) on the simulated anomalous vessel trajectories (Section 6.3.3). The numbers in bold indicate the best result for each sub-trajectory length of SSTLO-NCM.

	SSTLO-NCM									Discords	Cell-based GMM	Cell-based KDE
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	$k=8$	$k=9$			
$w=4$, $dt=10$	9.0	8.0	**7.2**	7.2	10.1	10.6	15.6	15.8	14.8	10.1	11.3	10.1
$w=7$, $dt=10$	13.8	**7.3**	7.4	7.9	9.2	10.4	12.0	13.1	13.9	11.6		
$w=7$, $dt=20$	8.5	**7.5**	8.4	8.3	8.3	9.2	9.5	10.0	9.6	10.2		
$w=10$, $dt=20$	11.4	10.7	10.6	10.1	**9.3**	9.5	10.2	9.8	10.2	13.5		
$w=9$, $dt=30$	13.2	11.4	10.5	9.6	10.3	10.0	8.9	**8.6**	9.5	17.7		
$w=11$, $dt=30$	13.3	15.2	13.0	12.7	12.7	12.1	11.3	**10.2**	11.5	18.6		

Table 6.2: *Median* detection delay (number of trajectory points) for SSTLO-NCM, discords and cell-based GMM/KDE (Laxhammar et al., 2009) on the simulated anomalous vessel trajectories (Section 6.3.3). The numbers in bold indicate the best result for each sub-trajectory length of SSTLO-NCM.

	SSTLO-NCM									Discords	Cell-based GMM	Cell-based KDE
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	$k=8$	$k=9$			
$w=4$, $dt=10$	6	5	**4**	4	6	6	9	9	9	7	7.5	7
$w=7$, $dt=10$	9	**4**	4	5	6	6	7	8	8.5	8		
$w=7$, $dt=20$	5	**4**	5	5	5	6	6	6	6	8		
$w=10$, $dt=20$	7	**6**	7	6	6	6	7	7	7	11		
$w=9$, $dt=30$	9	7	7	**6**	7	7	6	6	7	14		
$w=11$, $dt=30$	9	10	9	9	9	9	8	**7**	9	14.5		

- how different parameter values of SSTLO-NCM affect the classification performance,

- the relative performance of SSTLO-NCM, discords and SBAD (Brax, 2011), which is a previously proposed anomaly detection algorithm that was evaluated on the same dataset by Brax (2011).

Dataset

The dataset used in the experiments of this subsection was originally created by Brax (2011), who used it to investigate the classification performance of different anomaly detectors in the maritime domain. In the following, we will describe the dataset and the original preprocessing according to Brax (2011).

A set of 8867 unlabelled trajectories was extracted from an AIS database corresponding to 50 days of vessel traffic continuously recorded from the 2nd February 2010 to the the 6th April 2010, in an area outside the port of Gothenburg, Sweden, as indicated by Figure 6.12. Each vessel report that was extracted from the AIS database includes MMSI, timestamp, position in latitude and longitude coordinates, speed (knots), course over ground, vessel type and vessel length (meters). Similar to Section 6.3.1, the trajectories were formed by grouping vessel reports according to MMSI. Duplicate reports, or reports from locations outside the sampling area (Figure 6.12), were excluded. In order to reduce the size of the dataset even further, vessel reports were resampled every 100 meters in case the speed was greater than 0.01 knots, or every 60 seconds in case the speed was less than 0.01 knots. Moreover, only vessel reports corresponding to *cargo*, *tanker*, and *passenger* vessels were extracted, which was motivated by the fact that these are the most interesting vessels to monitor for surveillance operators (Brax, 2011). Finally, trajectories were split into two whenever the time difference between two successive reports was greater than 30 minutes.

The above set of 8867 unlabelled trajectories was randomly split into a *training set* of size 7980 and an *evaluation set* of size 887. The training set was used to train anomaly detectors and tune their parameters. The evaluation set was used to create a *normal test set* of 604 vessel trajectories labelled normal and an *anomalous test set* of 604 vessel trajectories labelled abnormal. The trajectories of the normal test set were randomly sampled without replacement from the evaluation set. Figure 6.13 shows an overview of the trajectories from the training set and the normal test set. The anomalous test set was developed by Brax (2011) in collaboration with maritime subject matter experts from the Swedish Maritime Administration. A list of potentially interesting anomaly classes was first compiled on the basis of previously published work resulting from two maritime security workshops (van Laere and Nilsson, 2009; Roy, 2008). These anomaly classes were then discussed together with the subject matter experts. Based on their feedback, the list of anomaly classes was further refined. Finally, a number of instances for each of the anomaly classes was created by Brax (2011). In this work we consider seven

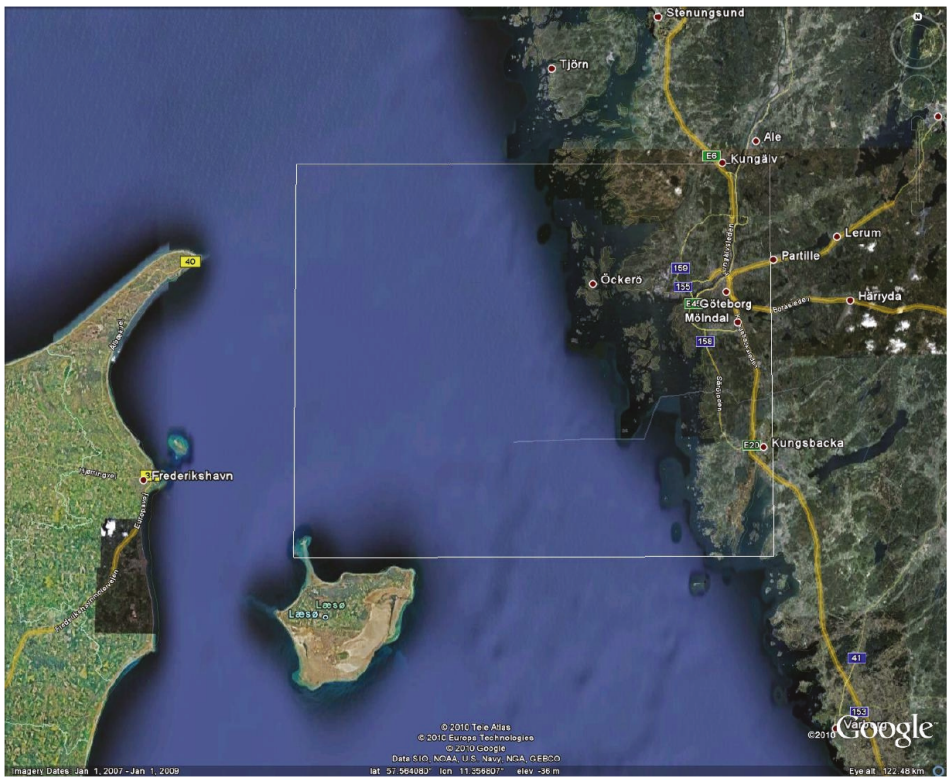


Figure 6.12: Geographical coverage of the dataset of recorded vessel trajectories and realistic anomalies (Section 6.3.4). The image was originally exported from Google Earth (Brax, 2011) (©2010 Tele Atlas, ©2010 Europa Technologies, ©2010 Google, Data SIO, NOAA, U.S. Navy, NGA, GEBCO). Reproduced from Brax (2011) with permission.

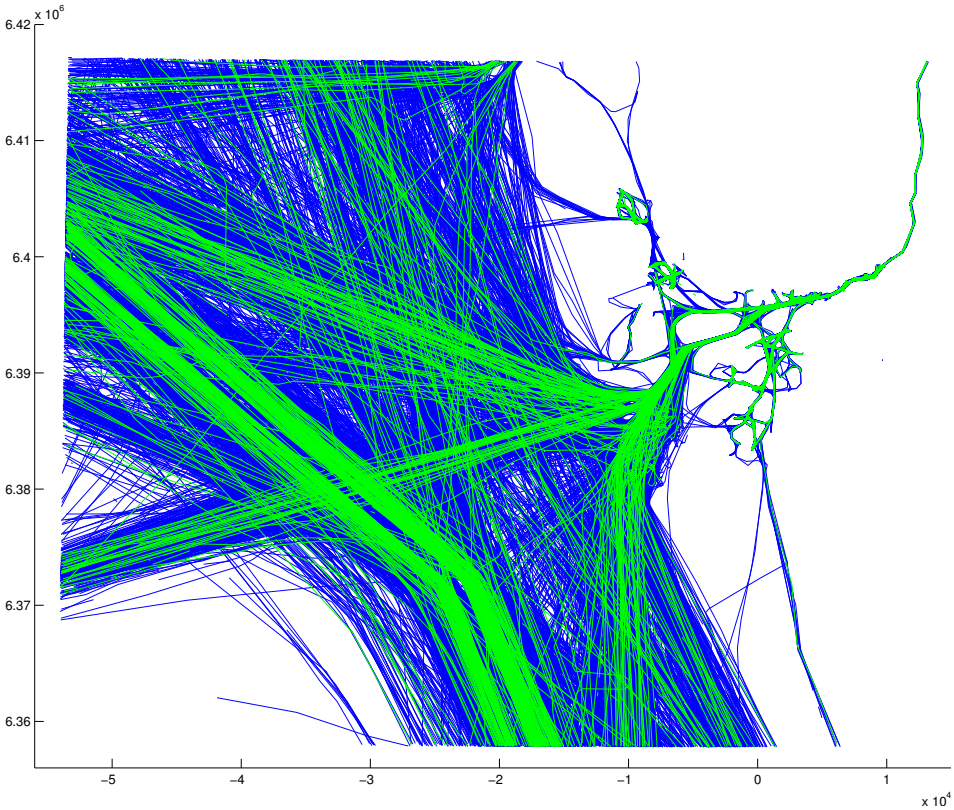


Figure 6.13: Plot of the 7980 vessel trajectories of the training set (blue) and the 604 vessel trajectories from the normal test set (green).

anomaly classes[5], which are summarised in Table 6.3. In the following, we briefly describe the seven anomaly classes and how the corresponding trajectory instances were created (Brax, 2011).

1. The *circle and land* class aims to simulate a vessel whose rudder is malfunctioning or whose officer on the bridge has fallen asleep. It is based on a real incident during which the vessel went in circles before running ashore. The trajectory instances of this class were handcrafted, i.e., the position, speed, course, and timestamps were manually calculated. Since this is a time-consuming process, only two instances were created (Figure 6.14).

2. The *missed turn* anomaly class simulates the situation when a vessel follows the shipping lane but misses a turn and travels towards shallows. Similar to above, only two such instances were manually calculated (Figure 6.15), both of which correspond to two high-risk areas where, according to the subject matter experts, vessels may miss a turn.

3. The *unexpected stop* class reflects the situation when a vessel suddenly makes a suspicious stop for approximately 50 minutes before resuming its regular route. Each of the 100 instances of this class (Figure 6.16) was automatically generated as follows. First, a trajectory was randomly sampled with replacement from the evaluation set. A trajectory point outside of the normal anchoring areas was then randomly selected as the stopping point. Finally, an additional 50 points were inserted at the stopping position, with 60 seconds between each one, and the timestamp of the remaining trajectory points was updated accordingly.

4. The *unusual speed* class corresponds to the case of a vessel that for a limited time travels relatively slowly or fast, with respect to other vessels in the same area or of the same type. Each of the 100 instances of this class was automatically generated as follows: A trajectory was randomly sampled with replacement from the evaluation set. Next, a sub-sequence of length 50 was randomly selected from the corresponding trajectory. Finally, the speed of every point of the sub-sequence was either increased or decreased by 90% with equal probability. Two of the instances are illustrated in Figure 6.17. The length of the abnormal segment is between 2–10 minutes for 74 of the trajectories. For the remaining 26 trajectories, the average length of the abnormal segment is 66 minutes; these correspond to unusually slow speed segments.

5. The *strange manoeuvres* class includes 100 instances of strange and erratic vessel behaviour, where a vessel repeatedly changes its course during a limited time frame. Each instance is created by randomly sampling a trajectory with replacement from the evaluation set. The trajectory is modified

[5]The eighth and last anomaly class in Brax (2011) is concerned with normal vessel movements at unusual time points of the day and is considered out of the scope of the thesis.

Table 6.3: Overview of the different maritime anomaly classes in the anomalous test set of vessel trajectories used in Section 6.3.4.

Anomaly Class	Nr. of test trajectories	Description	Illustration
Circle and land	2	Vessel that suddenly starts to travel in circles and eventually runs aground.	Figure 6.14
Missed turn	2	Vessel that follows the shipping lane towards a port but misses the normal turn and travels towards shallows.	Figure 6.15
Unexpected stop	100	Vessel that unexpectedly stops for 50 minutes.	Figure 6.16
Unusual speed	100	Vessel that temporarily travels slower/faster with respect to the normal speed of other vessels in the same area and/or of the same type.	Figure 6.17
Strange manoeuvres	100	Erratic vessel behaviour; repeated change of course with high frequency.	Figure 6.18
Large vessel in unusual location	100	Large vessel located in an area usually not trafficked by large vessels.	Figure 6.19
Unexpected vessel type/behaviour	100	Strange vessel behaviour for given vessel type; behaviour more similar to behaviour of another vessel type.	Figures 6.20–6.21

by skewing a random number of consecutive points from a randomly selected start point. More specifically, each point is skewed by translating it $\triangle d$ meters perpendicular to the current course, where $\triangle d$ is independently sampled from the uniform distribution on $[-500, 500]$. An example of the strange manoeuvre instances is shown in Figure 6.18. The distribution of the time length of the 100 abnormal segments is unimodal with the mean value close to 11 minutes and the standard deviation equal to 3.7.

6. The *large vessel in unusual location* class simulates the situation when a large vessel is located in an area that is usually not trafficked by large vessels. Each of the 100 instances of this class (Figures 6.19) was generated by first randomly sampling a trajectory from the evaluation set such that the size of the corresponding vessel is small, and then changing the size to large.

7. The *unexpected vessel type/behaviour* class reflects the case when observed vessel behaviour is inconsistent with the reported vessel type, or it is more similar to the behaviour of another type. The 100 instances of this class were generated in similar fashion as above; a trajectory was randomly sampled from the evaluation set and its corresponding type randomly changed to another vessel type. For example, if the sampled trajectory is of type cargo vessel, the type would be changed to tanker or passenger vessel or some other type not present in the training set, such as pilot, fishing vessel, etc. The number of test trajectories where the corresponding false vessel type is cargo, tanker or passenger vessel is 13, 11 and 16, respectively. The type of each of the remaining 60 test trajectories is neither cargo, tanker or passenger vessel. Figures 6.20–6.21 illustrate the subsets of the 100 test trajectories where the unexpected type is passenger vessel and tanker, respectively.

For more details regarding the datasets and the preprocessing above, the reader is referred to Brax (2011).

Preprocessing

In this work, we have further preprocessed the trajectories from the training set and the two test sets above in a similar way as in Section 6.3.3: First, the geographical position of each trajectory point has been projected to the two-dimensional spatial plane. Second, the timestamps have been re-calculated. The original datasets above include timestamps for each trajectory point, which have been directly extracted from the AIS database. However, a closer examination of the dataset reveals that these timestamps are not always consistent and reliable; the time difference between some of the trajectory points is relatively large, even though the corresponding geographical distance is small. This could be explained by, amongst other things, the unsynchronised internal clocks of the different AIS base stations that register the same vessel report. Therefore, we have initialised the timestamp of the first point of each trajectory to zero and estimated the timestamp of the

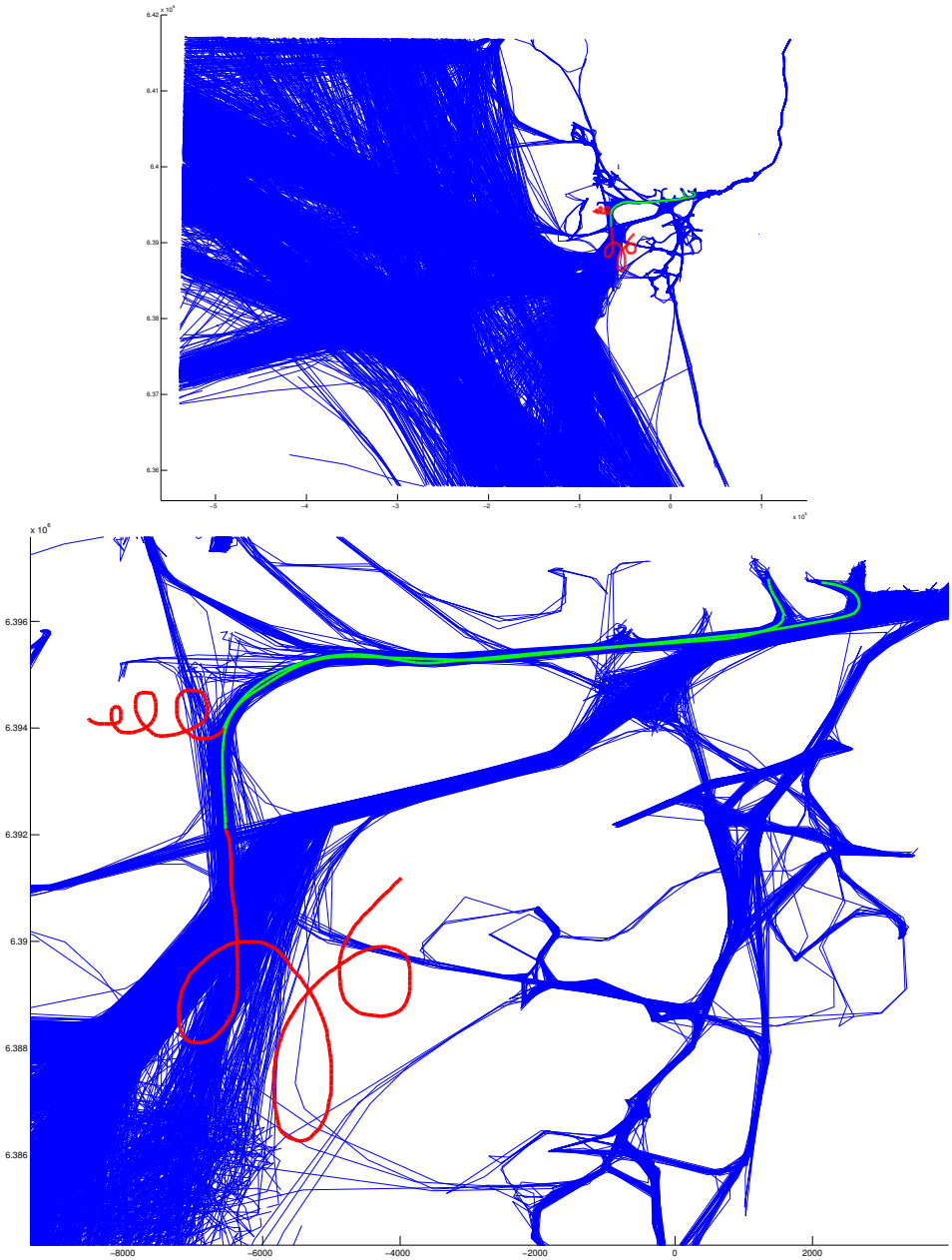


Figure 6.14: Overview (top) and zoomed-in view (bottom) of the two circle and land anomalies (Section 6.3.4), where red and green correspond to the abnormal and normal segments, respectively, and the blue trajectories correspond to the training set. The lengths of the top and bottom abnormal segments are 14 and 38 minutes, respectively.

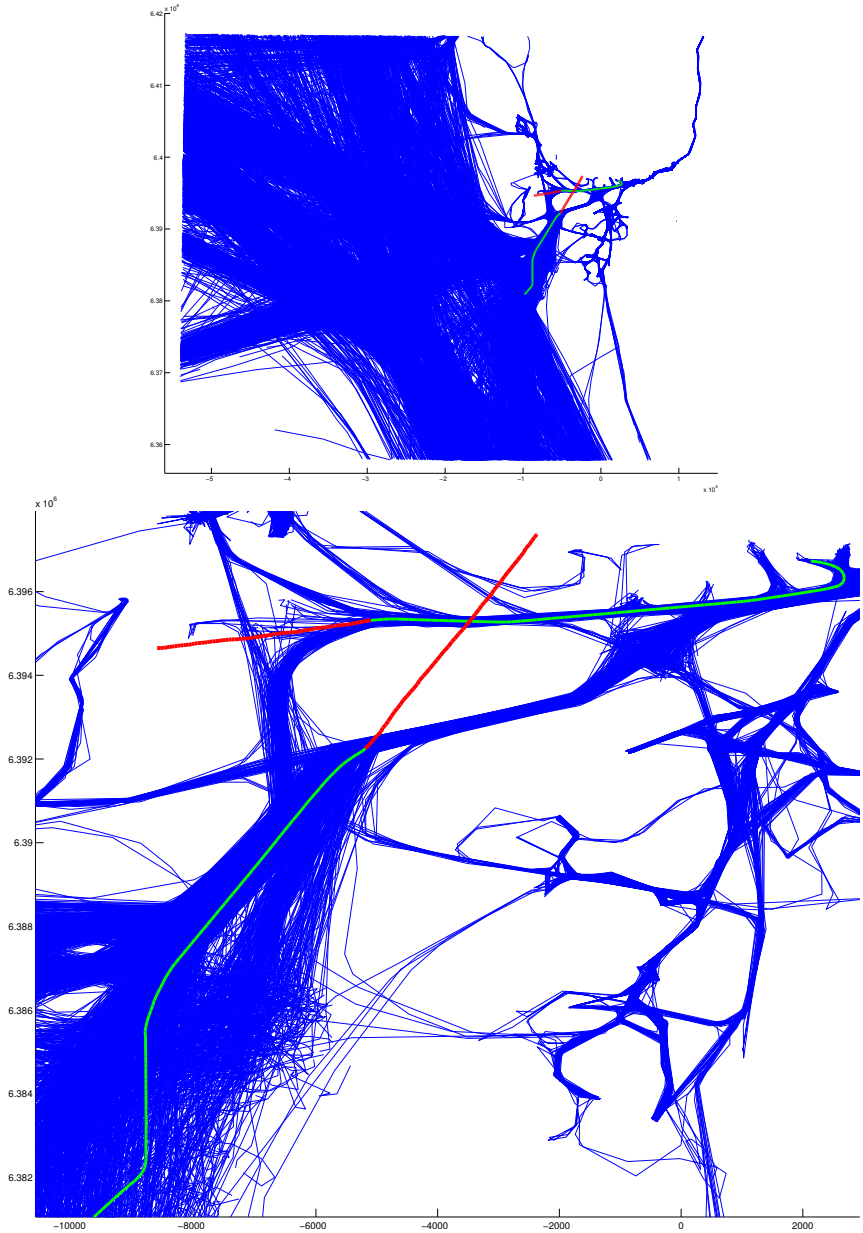


Figure 6.15: Overview (top) and zoomed-in view (bottom) of the two missed turn anomalies (Section 6.3.4), where red and green correspond to the abnormal and normal segments, respectively, and the blue trajectories correspond to the training set. The lengths of the left and right abnormal segments are 7 and 15 minutes, respectively.

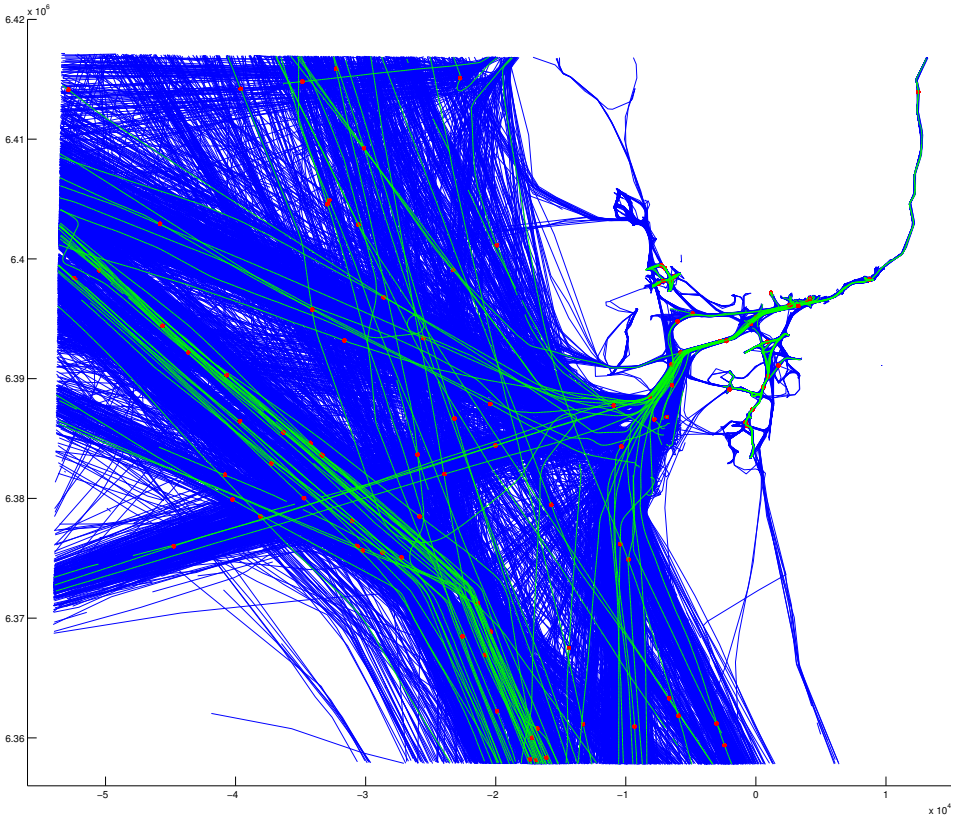


Figure 6.16: Plot of the 100 anomalous trajectories involved in unexpected stops (Section 6.3.4). Red and green correspond to the unexpected stops and the normal segments, respectively, and the blue trajectories correspond to the full training set.

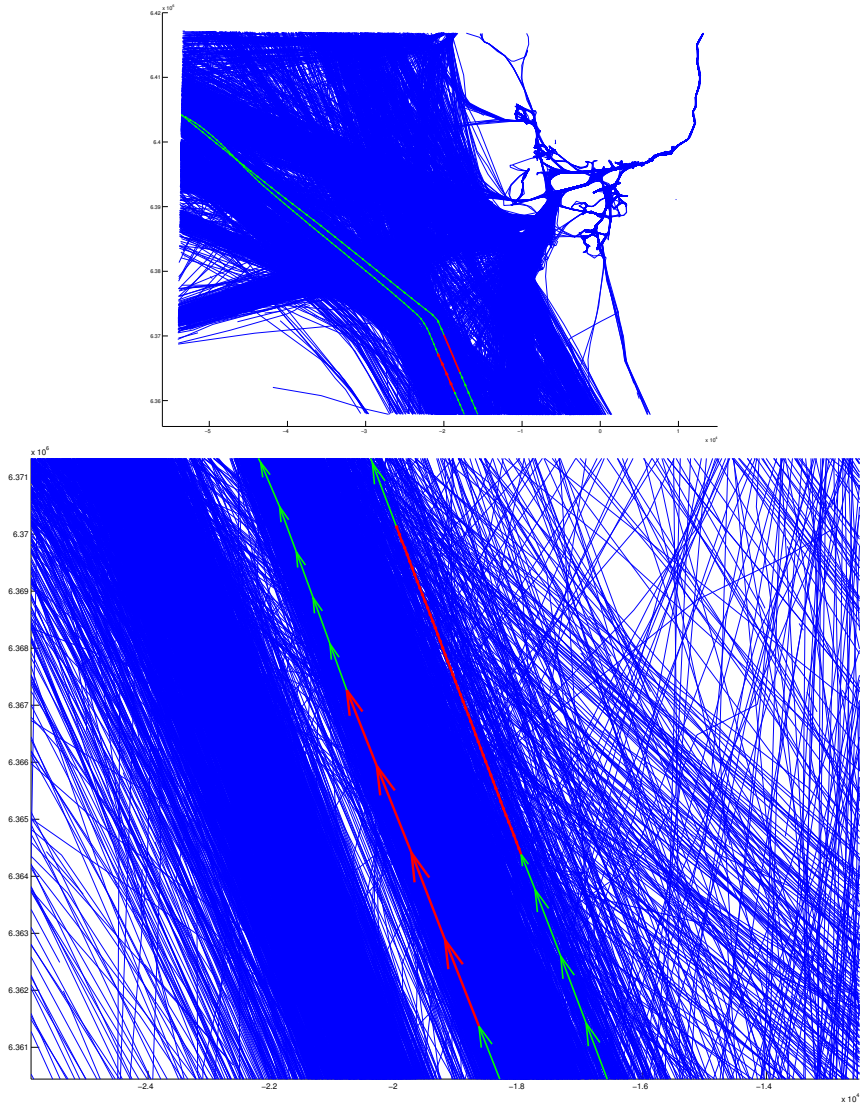


Figure 6.17: Overview (top) and zoomed-in view (bottom) of two arbitrarily selected trajectories from the set of 100 unusual speed anomalies (Section 6.3.4). Red and green correspond to the abnormal and normal segments, respectively, and the blue trajectories correspond to the full training set. In order to visualise the relative speed of the normal and abnormal segments, the two trajectories have been uniformly sampled in time and the velocity vectors between successive points plotted. Note that the left and the right abnormal segments correspond to high-speed and low-speed segments, respectively, which is indicated by the length of the velocity vectors.

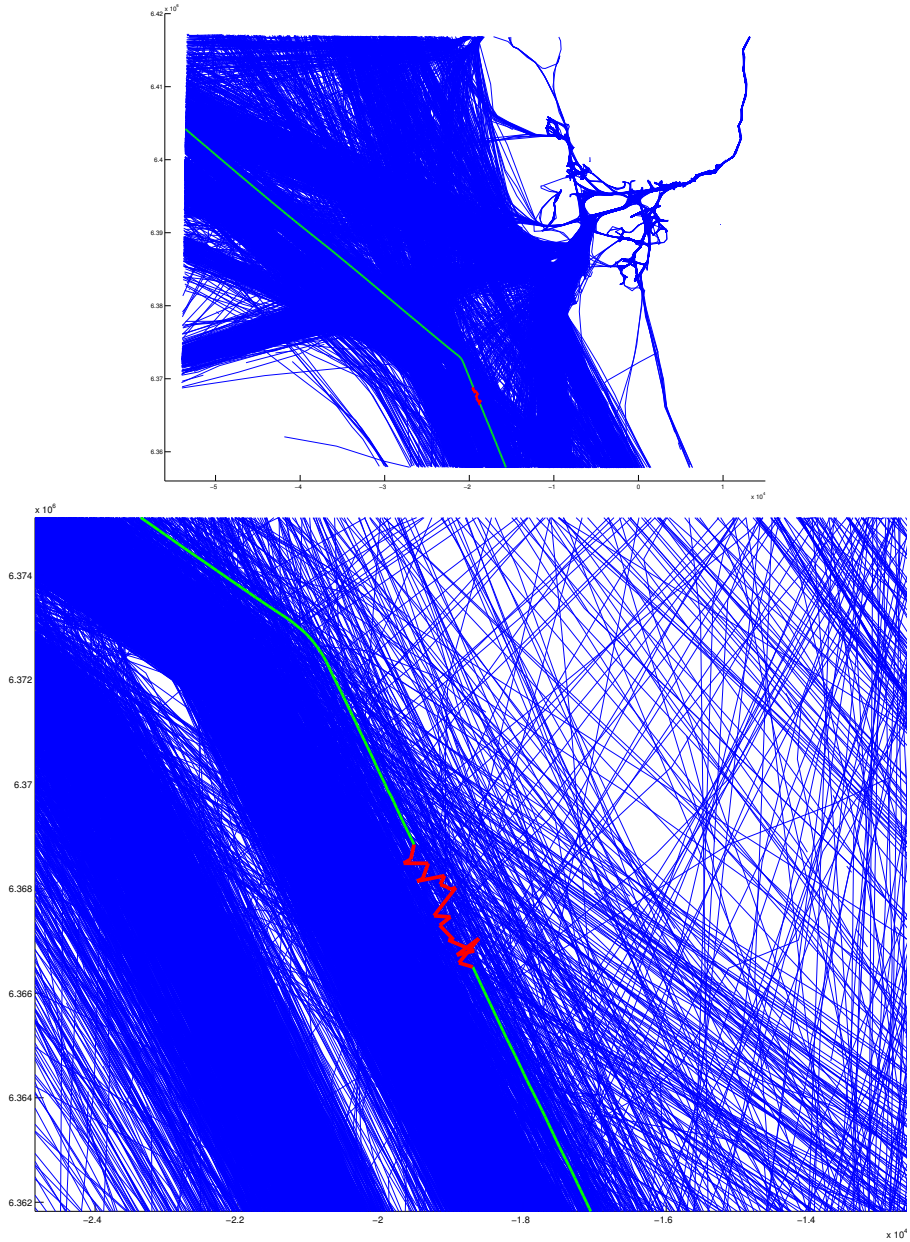


Figure 6.18: Overview (top) and zoomed-in view (bottom) of an arbitrarily selected trajectory from the set of 100 strange manoeuvre anomalies (Section 6.3.4). Red and green correspond to the abnormal and normal segments, respectively, and the blue trajectories correspond to the full training set.

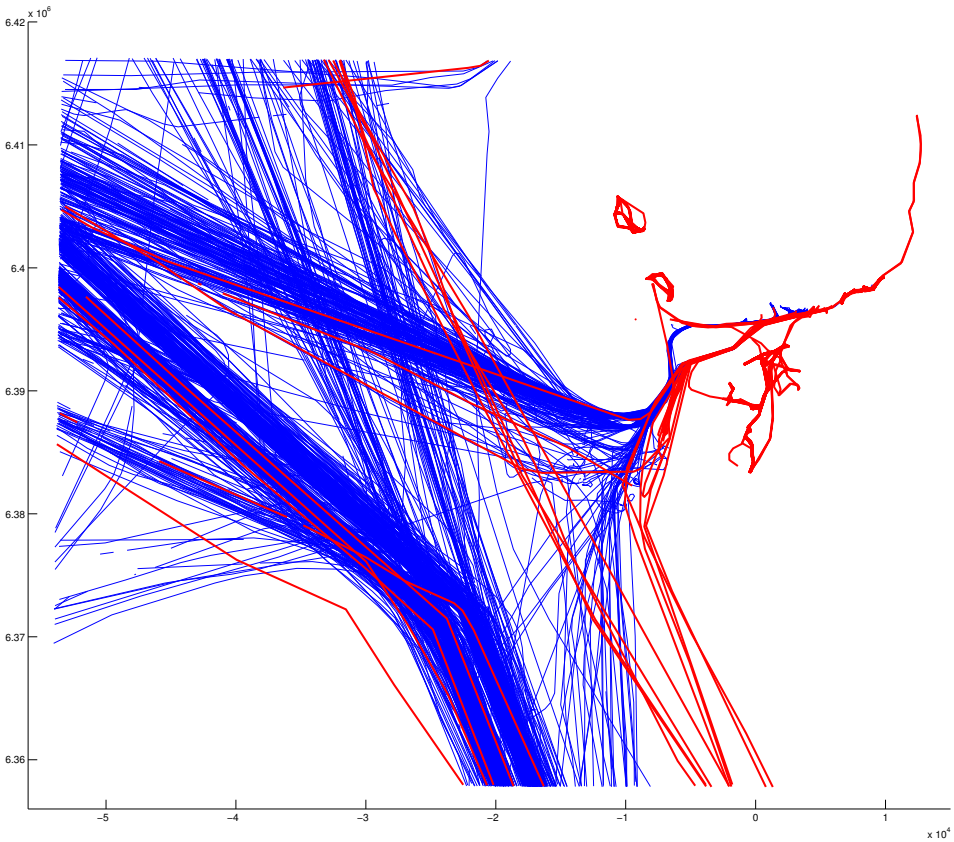


Figure 6.19: Plot of the 100 anomalous trajectories corresponding to large vessels in unusual locations (red) (Section 6.3.4). The blue trajectories are the subset of the training set for which the size of the corresponding vessel is large.

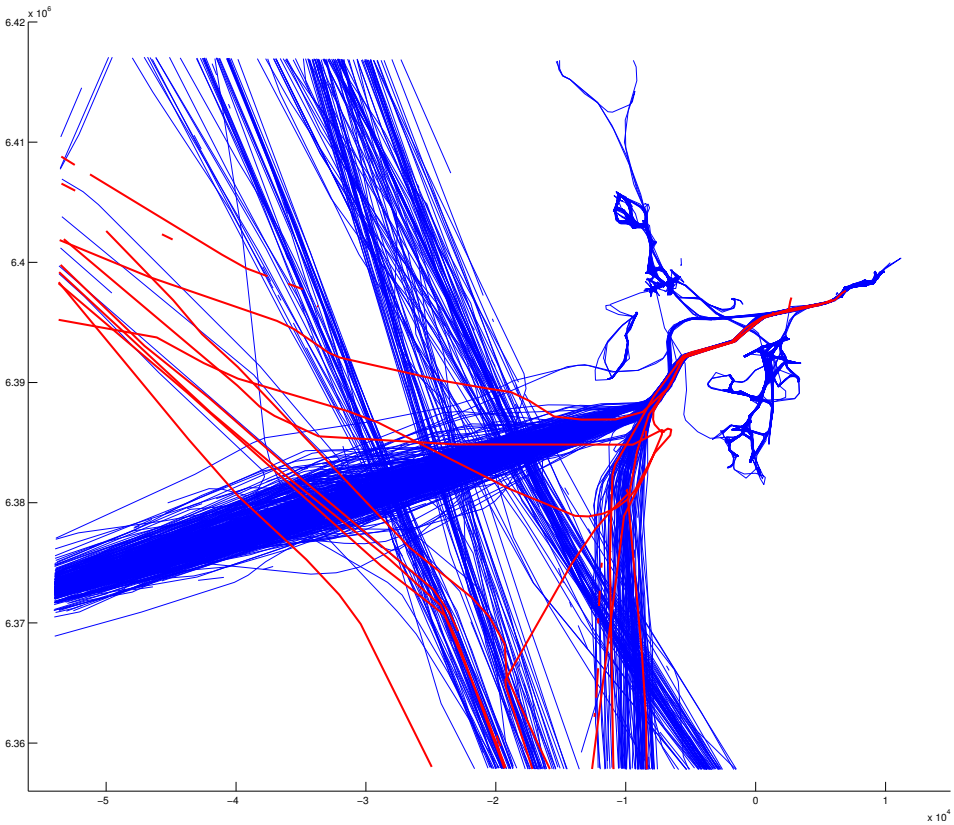


Figure 6.20: Plot of the subset of passenger vessel trajectories (red) from the set of unexpected vessel type/behaviour anomalies (Section 6.3.4). The blue trajectories are the subset of the training set for which the (true) type of the corresponding vessel is passenger vessel.

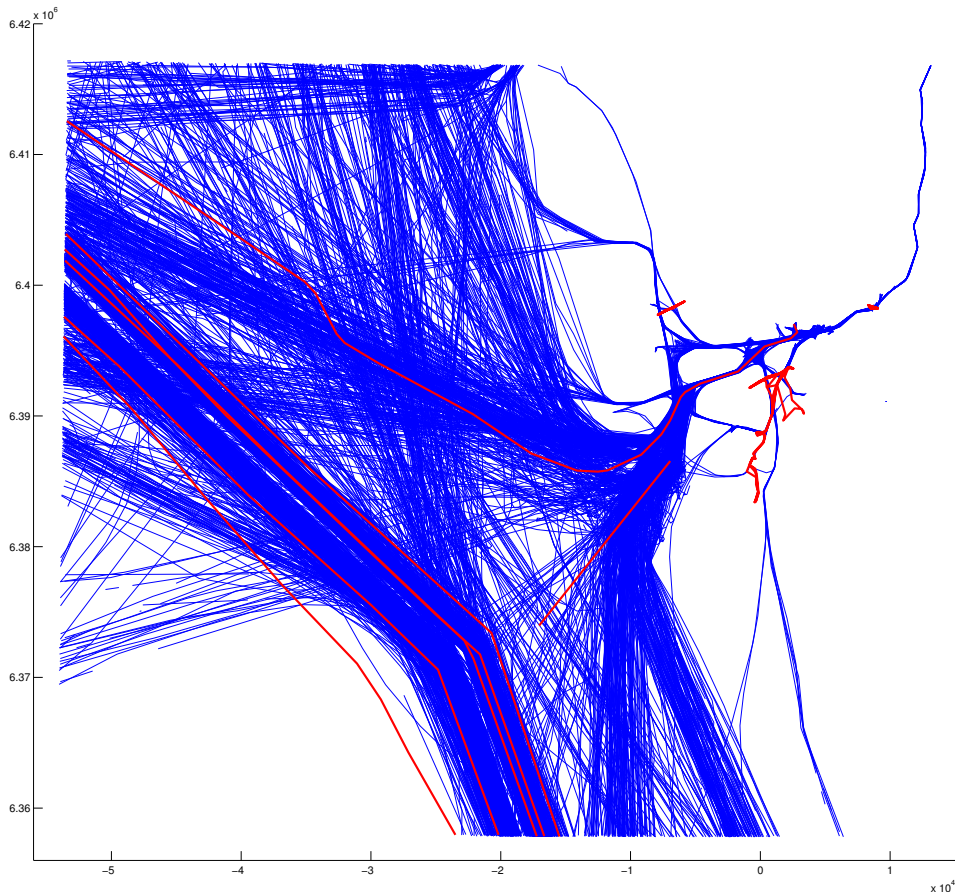


Figure 6.21: Plot of the subset of tanker trajectories (red) from the set of unexpected vessel type/behaviour anomalies (Section 6.3.4). The blue trajectories are the subset of the training set for which the (true) type of the corresponding vessel is tanker vessel.

remaining points according to (6.22). Finally, each trajectory has been uniformly resampled in time with sampling rate dt.

In addition to the previous feature model, where each $\chi_i = (x_i, y_i) : i = 1, \ldots, L_i$ represents the location of the vessel in the two-dimensional plane at t_i, we also investigate a velocity-based feature model such that $\chi_i = (vx_i, vy_i)$ represents the *velocity vector* of the vessel in the two-dimensional plane at t_i. Thus, we estimate the velocity vector (vx_i, vy_i) for each point (x_i, y_i) of the uniformly sampled trajectories according to:

$$vx_i = \begin{cases} \frac{|x_i - x_{i-1}|}{dt} & i = 2, \ldots, L \\ \frac{|x_2 - x_1|}{dt} & i = 1 \end{cases} \tag{6.24}$$

and

$$vy_i = \begin{cases} \frac{|y_i - y_{i-1}|}{dt} & i = 2, \ldots, L \\ \frac{|y_2 - y_1|}{dt} & i = 1 \end{cases}. \tag{6.25}$$

Each preprocessed trajectory is annotated by $type \in \{\text{cargo}, \text{tanker}, \text{passenger}\}$ and $size \in \{\text{small}, \text{medium}, \text{large}\}$. The vessel size is discretisized based on the vessel length, similar to Brax (2011), i.e., small, medium and large correspond to vessels of length less than 120 meters, between 120 and 200 meters and more than 200 meters, respectively. Further, the abnormal segment of each test trajectory from anomaly classes 3–5 is also annotated.

In addition to the main training set D_{Train} of size 7980, the following six subsets of the training set are extracted:

- D_{Train}^{Cargo}, which corresponds to the subset of 3420 trajectories such that $type = \text{cargo}$.
- $D_{Train}^{\text{Tanker}}$, which corresponds to the subset of 2558 trajectories such that $type = \text{tanker}$.
- $D_{Train}^{\text{Passenger}}$, which corresponds to the subset of 2002 trajectories such that $type = \text{passenger}$.
- D_{Train}^{Large}, which corresponds to the subset of 828 trajectories such that $size = \text{large}$.
- $D_{Train}^{\text{Medium}}$, which corresponds to the subset of 2746 trajectories such that $size = \text{medium}$.
- D_{Train}^{Small}, which corresponds to the subset of 4388 trajectories such that $size = \text{small}$.

From the anomalous test set of size 604, the seven subsets $D_{Anom}^{i} : i = 1, \ldots, 7$, corresponding to the abnormal trajectories from anomaly classes 1–7 (Table 6.3) have been extracted. Note that the total number of trajectories from the seven anomalous subsets is 504; the remaining 100 anomalous test trajectories belong to the eighth and last anomaly class that is not used in this work (Footnote 5).

Performance Measures

In the original experiments by Brax (2011), each trajectory from the normal and anomalous test sets was classified on the basis of the training set using different variants of the *state-based anomaly detector* (SBAD) (Brax, 2011). Classification performance was measured in terms of the sensitivity on the anomalous test set for the predefined false alarm rates 0.05 and 0.01, i.e., the threshold of the corresponding anomaly detector was calibrated so that approximately 5% or 1% of the normal test trajectories were classified as anomalous. In this work, we consider the *receiver operating characteristics* (ROC) *curve* (Fawcett, 2006) for each anomaly detector, which describes the general relationship between the sensitivity and the false alarm rate. Further, we consider the *area under the ROC curve* (AUC) (Fawcett, 2006), which summarises the classification performance over all possible thresholds and, hence, is a more general performance measure than the sensitivity and false alarm rate.

Estimating ROC graphs is a general approach to tuning and evaluating binary classifiers in machine learning and data mining (Fawcett, 2006). A ROC curve for a binary scoring-based classifier, in our case a scoring-based anomaly detector (Section 2.1.1), is a two-dimensional curve in which the sensitivity, also known as the *true positives rate* (tpr)(Fawcett, 2006), is plotted on the Y axis and the false alarm rate, also known as *false positives rate* (fpr) (Fawcett, 2006), is plotted on the X axis. Thus, a ROC curve visualises the relative tradeoff between $tpr \in [0, 1]$ and $fpr \in [0, 1]$ for each possible anomaly threshold. Given a labelled test set, the ROC curve for an anomaly detector can be estimated on the basis of the ranking of the test examples according to their anomaly scores (Fawcett, 2006). In the case of SSTLO-NCM, the anomaly scores are equivalent to the nonconformity scores for each test example relative to the training set.

AUC is a standard measure for the classification performance of binary scoring-based classifiers (Fawcett, 2006). The larger the AUC, the better the overall classification performance over all possible thresholds. AUC corresponds to a portion of the area of the unit square and, thus, its value will always be within $[0, 1]$. Intuitively, the AUC of an anomaly detector is equivalent to the probability that the anomaly detector will rank a random example from the abnormal class higher than a random example from the normal class (Fawcett, 2006). Nevertheless, the *partial AUC* (pAUC) (Narasimhan and Agarwal, 2013) has been proposed as an alternative to AUC in binary classification applications when only a *sub-range* of the ROC curve is of practical interest, e.g., $fpr \in [0, 0.01]$. We argue that anomaly detection is an example of such an application. As discussed in Sections 3.1.3 and 4.3.5, it is important that precision is not too low, for the corresponding anomaly detection system to be of any practical use. Assuming that the rate of true anomalies is very low, say less than 1%, any false alarm rate greater than 1% implies that precision will be less than 50%, regardless of the sensitivity. That is, the precision becomes dominated by the false alarm rate (Section 4.3.5). Hence, we consider the pAUC for $fpr \in [0, 0.01]$, which is also normalised to be within $[0, 1]$. It is important to note that pAUC may be larger

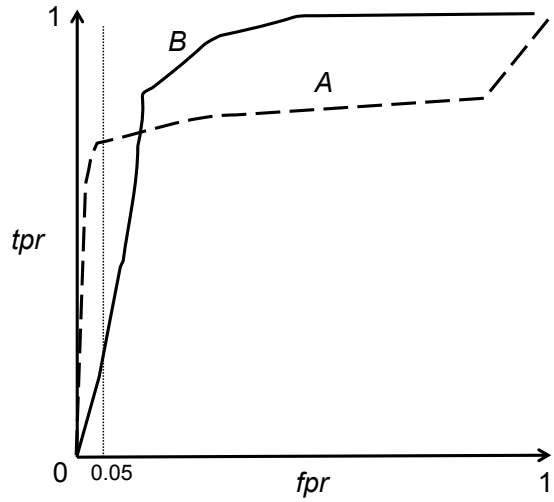


Figure 6.22: Example of ROC curves for two anomaly detectors A and B, such that pAUC $fpr \in [0, 0.05]$ is greater for A, even though B has greater AUC.

for an anomaly detector A compared to an anomaly detector B, even though AUC for B is greater than for A. This is illustrated by Figure 6.22.

Design

In these experiments, we estimate ROC curves according to the algorithm proposed by Fawcett (2006) and report the corresponding AUC and pAUC for $fpr \in [0, 0.01]$ on each of the anomaly classes 1–7 and for different parameter setups of SSTLO-NCM and discords. The general procedure is to first calculate the anomaly score for each of the 604 normal test trajectories relative to the training set. For each anomaly class, we then calculate the anomaly scores for the corresponding subset of abnormal test trajectories relative to the training set. The ROC curve for a particular anomaly class is estimated on the basis of the ranking of the subset of abnormal test trajectories and the 604 normal test trajectories according to the corresponding anomaly scores. In the case of SSTLO-NCM, the anomaly score for a test trajectory is equivalent to the corresponding nonconformity score relative to the training set. For the discords algorithm, the anomaly score for a trajectory is equivalent to the nearest neighbour distance of its top sub-trajectory discord relative to the training set.

For anomaly classes 1–5, D_{Train} is used when calculating the anomaly score for each test trajectory. Nevertheless, for anomaly classes 6 and 7, we obviously need to take into account the actual size and type of the vessels, respectively, in order to achieve good classification performance. Therefore, in the case of anomaly class 6, we calculate the anomaly score for each normal or abnormal

test trajectory of type $t \in \{\text{cargo,tanker, passenger}\}$ relative to D^t_{Train}, i.e., the subset of the training set that only includes vessels of type t. In other words, we consider three separate and independent anomaly detectors, one for each type of vessel, which are based on different subsets of the training set. Analogously, for anomaly class 7, we calculate the anomaly score for each test trajectory of size $s \in \{\text{small, medium, large}\}$ relative to D^s_{Train}.

In addition to AUC and pAUC, we also investigate the sensitivity and the detection delay for $fpr = 0.01$, which may be considered a typical operating point for an anomaly detector. The sensitivity for $fpr = 0.01$ was also investigated by Brax (2011), for different variants of SBAD. In order to measure the detection delay, we have to define an anomaly threshold that corresponds to the operating point $fpr = 0.01$. This is done by simply sorting the corresponding 604 anomaly scores of the normal test trajectories and setting the threshold equal to the 7th largest score.

Two different trajectory representations are investigated. The first is referred to as the *spatial trajectory representation* and corresponds to the standard model where each $\chi_i = (x_i, y_i) : i = 1, \ldots, L_i$ represents the location of the vessel at time t_i. The second is referred to as the *velocity-based trajectory representation* where each $\chi_i = (vx_i, vy_i)$ represents the velocity vector of the vessel at time t_i. The procedure for calculating the ROC curves and detection delay for each anomaly class is repeated for each combination of the spatial and velocity-based trajectory representations and the following pairs of sub-trajectory lengths and sampling distances:

$$(w, dt) = (3, 30), (11, 30), (11, 60), (11, 120), (11, 180), \tag{6.26}$$

which correspond to 1, 5, 10, 20 and 30 minutes, respectively. These sub-trajectory lengths have been arbitrarily chosen. Nevertheless, they are assumed to reflect a range of different anomaly scales that may be relevant in maritime surveillance applications. Among the anomalous test sets, 3.2% of the trajectories have a total length less than 30 minutes and are therefore not detectable when the sub-trajectory length is 30 minutes. Hence, considering a larger sub-trajectory length would imply that more than 3.2% of the anomalous test trajectories are not detectable.

Results and Discussion

The complete results for SSTLO-NCM and discords on each anomaly class, including all combinations of parameters and trajectory representations, can be found in Appendix A. The sections of the appendix are organised according to the seven anomaly classes 1–7, where the last section presents the average results over all anomaly classes. For each section (anomaly class), there are two subsections corresponding to the spatial and the velocity vector-based representations, respectively. Further, for each subsection, there are four tables that presents AUC, pAUC, sensitivity, and mean detection delay, respectively, for each combination of parameters

Table 6.4: Performance results of SSTLO-NCM for each value of k, averaged over all the sub-trajectory lengths (6.26) and all anomaly classes 1–7.

		k (size of neighbourhood)				
		1	2	3	4	5
Spatial trajectory representation	AUC	0.86	0.88	0.87	**0.89**	**0.89**
	pAUC	0.39	**0.41**	0.40	0.37	0.38
	Detection delay (s)	1371	1083	1096	**1044**	1336
Velocity trajectory representation	AUC	0.88	0.88	**0.89**	**0.89**	**0.89**
	pAUC	0.44	0.44	0.44	0.44	**0.45**
	Detection delay (s)	1046	1002	**930**	964	**930**

Table 6.5: Performance results of SSTLO-NCM for each sub-trajectory length, averaged over $k = 1, \ldots, 5$ and all anomaly classes 1–7.

		Sub-trajectory length (minutes)				
		1	5	10	20	30
Spatial trajectory representation	AUC	0.82	0.91	**0.92**	0.91	0.82
	pAUC	**0.49**	0.42	0.45	0.34	0.26
	Detection delay (s)	894	**806**	921	1359	1949
Velocity trajectory representation	AUC	0.87	**0.92**	**0.92**	0.89	0.83
	pAUC	0.44	**0.51**	**0.51**	0.36	0.40
	Detection delay (s)	**298**	435	854	1450	1835

of SSTLO-NCM and discords. In the remainder of this subsection, we present and discuss selected results that have been extracted from the tables in Appendix A.

Let us first examine Table 6.4, which presents the average classification performance of SSTLO-NCM for the different choices of k. For the spatial representation, there seems to be a slight advantage for $k = 2$ in terms of the average pAUC. However, for the velocity vector representation, the choice of k does not seem to have any significant impact on the overall classification performance. Further, consider Table 6.5, which presents the average classification performance of SSTLO-NCM for the different choices of the sub-trajectory length. In contrast to k, it is clear that the choice of the sub-trajectory length has a significant impact on the overall classification performance of SSTLO-NCM, especially pAUC and detection delay. In the following, we fix $k = 2$ and investigate, in more detail, how the performance on each particular anomaly class depends on the choice of the sub-trajectory length and the trajectory representation (Tables 6.6–6.7).

In the case of the spatial trajectory representation (Table 6.6), pAUC for anomaly classes 1–2 and 6–7 generally decreases as the sub-trajectory length increases; the best performance for SSTLO-NCM and discords is achieved for the smallest sub-trajectory length, i.e., one minute. For anomaly classes 3 and 5 the reverse is true, i.e., the performance generally increases as the sub-trajectory

Table 6.6: pAUC for SSTLO-NCM ($k = 2$) and discords based on the *spatial* trajectory representation.

Anomaly class	Detector	Sub-trajectory length (minutes)				
		1	5	10	20	30
1. Circle and land	SSTLO-NCM	**1.00**	1.00	1.00	0.36	0.43
	Discords	**1.00**	0.86	0.43	0.21	0.29
2. Missed turn	SSTLO-NCM	**0.93**	0.50	0.50	0.43	0.00
	Discords	**0.92**	0.50	0.50	0.00	0.00
3. Unexpected stop	SSTLO-NCM	0.00	0.03	0.20	0.21	**0.31**
	Discords	0.00	0.10	0.26	0.16	**0.33**
4. Unusual speed	SSTLO-NCM	0.01	0.08	**0.15**	0.10	0.08
	Discords	0.01	0.06	**0.06**	0.03	0.03
5. Strange manoeuvres	SSTLO-NCM	0.00	0.02	0.05	0.04	**0.05**
	Discords	0.00	0.00	0.01	0.01	**0.01**
6. Large vessel in unusual location	SSTLO-NCM	**0.81**	0.73	0.68	0.66	0.56
	Discords	**0.62**	0.62	0.61	0.50	0.48
7. Unexpected vessel type/behaviour	SSTLO-NCM	**0.76**	0.74	0.74	0.69	0.66
	Discords	**0.80**	0.75	0.73	0.74	0.75

Table 6.7: pAUC for SSTLO-NCM ($k = 2$) and discords based on the *velocity vector* trajectory representation.

Anomaly class	Detector	Sub-trajectory length (minutes)				
		1	5	10	20	30
1. Circle and land	SSTLO-NCM	0.74	0.74	**1.00**	0.70	0.94
	Discords	0.71	0.86	**1.00**	1.00	1.00
2. Missed turn	SSTLO-NCM	**0.29**	0.29	0.00	0.00	0.00
	Discords	**0.29**	0.21	0.00	0.00	0.00
3. Unexpected stop	SSTLO-NCM	0.15	**0.20**	0.18	0.11	0.08
	Discords	**0.18**	0.11	0.10	0.14	0.18
4. Unusual speed	SSTLO-NCM	0.33	**0.52**	0.49	0.33	0.36
	Discords	**0.39**	0.33	0.32	0.20	0.18
5. Strange manoeuvres	SSTLO-NCM	0.52	0.72	**0.77**	0.31	0.20
	Discords	0.55	0.69	**0.75**	0.15	0.04
6. Large vessel in unusual location	SSTLO-NCM	0.36	**0.50**	0.48	0.42	0.40
	Discords	0.34	0.43	0.55	**0.56**	0.52
7. Unexpected vessel type/behaviour	SSTLO-NCM	0.66	0.65	0.65	0.67	**0.70**
	Discords	0.66	0.65	0.66	0.69	**0.71**

length increases; the best performance for both anomaly detectors is achieved for sub-trajectories of length 30 minutes. For anomaly class 4, the best performance for both anomaly detectors is achieved for medium length sub-trajectories, i.e., 10 minutes.

The pattern for the velocity vector representation (Table 6.7) is quite different from that of the spatial representation above, with the exception of anomaly class 2, for which 1 minute is still the best choice of sub-trajectory length. For anomaly classes 1 and 5, the best performance is achieved for medium length sub-trajectories, i.e., 10 minutes. Considering anomaly classes 3–4, pAUC is largest for the relatively short sub-trajectories, i.e., 1–5 minutes, while the reverse is true for anomaly class 7. Finally, the optimal sub-trajectory length for anomaly class 6 is close to 5 minutes for SSTLO-NCM and close to 20 minutes for discords; this is the only anomaly class for which the optimal sub-trajectory length is considerably different for the two anomaly detectors based on the velocity representation.

The main conclusions from the results in Tables 6.6–6.7 are two-fold. Firstly, the classification performance of SSTLO-NCM and discords is highly dependent on the choice of sub-trajectory length and the choice of trajectory representation (spatial or velocity). Secondly, the optimal choice of trajectory representation and sub-trajectory length is *not* the same for all the anomaly classes. This suggests that multiple instances of SSTLO-NCM (or discords), based on different trajectory representations and sub-trajectory lengths, should be combined in order to achieve the best possible performance over all anomaly classes. In particular, considering the results for SSTLO-NCM in Tables 6.6–6.7, it would be interesting to investigate a combination of the spatial representation with sub-trajectory length 1 minute and the velocity representation with sub-trajectory length 5–10 minutes.

Examining Tables A.59 and A.63 in the appendix, we observe that the best overall sensitivity of SSTLO-NCM, $tpr = 0.70$, is achieved for the velocity representation with $k = 2$ and the sub-trajectory length of 5 minutes. Analogously, the best overall sensitivity for discords, $tpr = 0.67$, is achieved for the velocity representation and the sub-trajectory length of 1 minute. In Table 6.8, we present the detailed sensitivity and mean detection delay results on each of the anomaly classes for these two setups of SSTLO-NCM and discords, together with the corresponding best results for SBAD reported by Brax (2011). The results demonstrate that SSTLO-NCM and discords, when properly tuned, have more or less similar performance on most of the anomaly classes and are very competitive compared to SBAD. In particular, SSTLO-NCM and discords clearly outperform SBAD on anomaly classes 1–2 and 4. Assuming that the anomaly classes are equally frequent and that the overall frequency of abnormal trajectories is 0.01, the operating point $tpr = 0.70$ and $fpr = 0.01$ implies that precision is equal to 0.414. In a practical surveillance application, this implies that 1.7% of all vessel trajectories generate an alarm and that close to every second alarm corresponds to an interesting anomaly.

Considering SSTLO-NCM vs. discords, the results do not indicate whether there is any significant difference in the classification performance on each of the anomaly classes. Therefore, future work should investigate whether there are any

Table 6.8: Sensitivity and mean detection delay in seconds (within parentheses) for $fpr = 0.01$ on each anomaly class.

Anomaly class	SSTLO-NCM[a]	Discords[b]	SBAD[c]
1. Circle and land	1.00 (282)	1.00 (282)	0
2. Missed turn	0.50 (633)	0.50 (603)	0
3. Unexpected stop	0.35 (341)	0.33 (384)	0.29
4. Unusual speed	0.76 (253)	0.67 (158)	0.04
5. Strange manoeuvres	0.98 (107)	0.84 (45)	0.97
6. Large vessel in unusual location	0.62 (776)	0.63 (226)	0.78
7. Unexpected vessel type/behaviour	0.69 (503)	0.72 (158)	0.78

[a] Velocity vector representation with $k = 2$ and sub-trajectory length 5 minutes.
[b] Velocity vector representation with sub-trajectory length 1 minute.
[c] Results for anomaly classes 1–5 correspond to the best results for the joint position, speed and course state model reported in Brax (2011, Table A.13), while results for anomaly classes 6–7 correspond to the best results for the joint position, speed, course, vessel size and type state model reported in Brax (2011, Table A.16). No results for the detection delay of SBAD were reported by Brax (2011).

statistically significant differences between the performance of SSTLO-NCM and discords on each of the anomaly classes. It may be pointed out that adopting the discords algorithm as a NCM for trajectories in the framework of ICAD is straightforward; the nonconformity score of a test trajectory is simply the nearest neighbour distance of its sub-trajectory discord relative to the sub-trajectories of the training set.

6.4 Summary

In this chapter, we have proposed and investigated SSTLO-NCM for the sequential detection of local anomalous sub-trajectories. SSTLO-NCM is a novel NCM that is based on LOF and is therefore sensitive to local anomalous sub-trajectories. The key intuition of SSTLO-NCM is to consider the set of all sub-trajectories of predefined length w as high-dimensional feature vectors in the LOF framework. The nonconformity score for a test trajectory is equal to the largest LOF score among its sub-trajectories relative to the training set. Apart from the neighbourhood parameter k, the only design parameter of SSTLO-NCM is w, which corresponds to the scale of the anomalous sub-trajectories. An important property of SSTLO-NCM is that the sequence of nonconformity scores for a test trajectory monotonically increases; this implies that the alarm rate of the semi-offline and the online SSTLO-ICAD is guaranteed to be well-calibrated. In general, LOF involves a large number of nearest neighbours searches, which may be computationally very demanding. Therefore, a k-d tree-based algorithm for SSTLO-NCM is presented, including an analysis of its computational complexity.

In the empirical section, an explorative investigation of the performance of

SSTLO-NCM, for different values of k and w, on a recorded dataset of unlabelled vessel trajectories is first presented. The results show that the most anomalous sub-trajectories correspond to unique or infrequent vessel routes, sometimes in areas with no previous vessel traffic. Further, these trajectories typically belong to vessel types other than cargo ships and passenger liners, such as pilots, rescue vessels and fishing boats, which may explain the anomalous nature. In the following experiment, the empirical alarm rate of the unsupervised SSTLO-ICAD, in the semi-offline mode and the alternative online mode, is investigated on 100 randomly permuted sequences of the trajectories used in the previous experiment. As expected, the empirical alarm rate on each of the 100 sequences is either less than or close to the specified anomaly threshold ϵ in the semi-offline mode, i.e., the unsupervised semi-offline SSTLO-ICAD is indeed well-calibrated in practice. More interestingly, however, are the results for the alternative online mode; none of the 100 empirical alarm rates exceed ϵ, even though the alarm rate is not guaranteed to be well-calibrated in the alternative online mode.

A previously published experiment has also been reproduced, where the detection delay is investigated on a dataset of simulated anomalous vessel trajectories. The anomalous trajectories each consist of a normal segment, which was sampled from a real vessel trajectory, and an abnormal segment, which was generated according to a random walk. In addition to SSTLO-NCM, we have also implemented and evaluated the discords algorithm. The results show that the lowest detection delay for SSTLO-NCM and discords is achieved for the smallest sub-trajectory length, which is 30 seconds. Further, for sub-trajectory lengths up to two minutes and $k \leq 4$, SSTLO-NCM outperforms both discords and two other previous anomaly detectors. This is explained by the fact that SSTLO-NCM, in contrast to the other algorithms, takes into account the local density of the sub-trajectories. Considering sub-trajectories of lengths of two or more minutes, SSTLO-NCM consistently outperforms discords, regardless of k. Nevertheless, the results indicate that the value of k has a significant impact on the detection delay and that the optimal choice of k is dependent on the length of the sub-trajectory.

In the final experiments, the classification performance of SSTLO-NCM and the discords algorithm are evaluated on a labelled dataset that includes realistic anomalies from a number of different maritime anomaly classes. In addition to the traditional spatial trajectory representation, the performance of a velocity vector-based trajectory representation is investigated. The ROC curve is estimated and AUC, pAUC (for $fpr \in [0, 0.01]$), sensitivity, and mean detection delay (for $fpr = 0.01$) calculated on each anomaly class and for each combination of $k = 1, \ldots, 5$ and sub-trajectory lengths of 1, 5, 10, 20 and 30 minutes. The main conclusions from the results are three-fold: Firstly, the choice of k has limited or no impact on the overall classification performance on the anomaly classes. Secondly, the classification performance of SSTLO-NCM and discords is highly dependent on the choice of sub-trajectory length and the choice of trajectory representation (spatial or velocity), especially in the critical range of $fpr \in [0, 0.01]$. Thirdly, the optimal choice of trajectory representation and sub-trajectory length

is not the same for all anomaly classes. This suggests that multiple instances of SSTLO-NCM (or discords), based on different trajectory representations and sub-trajectory lengths, should be combined in order to achieve the best possible performance over all anomaly classes (see Section 7.2). Nevertheless, the best overall sensitivity for SSTLO-NCM when $fpr = 0.01$ is 0.70, which is achieved with the velocity representation, $k = 2$ and sub-trajectory length 5 minutes. The results demonstrate that SSTLO-NCM and discords, when properly setup, have more or less similar performance on most of the anomaly classes and are very competitive compared to previous results for SBAD (Brax, 2011). In addition to investigating the combination of different trajectory representations and sub-trajectory lengths, future work should investigate whether there are any statistically significant differences between the performance of SSTLO-NCM and discords on each of the anomaly classes.

Chapter 7

Conclusion

In this final chapter, we discuss the main conclusions and contributions of the thesis (Section 7.1). This is followed by a discussion of future work (Section 7.2) and how the results of the thesis generalise to other domains (Section 7.3). Final remarks are given in Section 7.4.

7.1 Contributions

The discussion of the conclusions and contributions is organised according to the research objectives and the research question that were defined in Sections 1.1. We first discuss how each of the research objectives $\mathcal{O}1$–$\mathcal{O}5$ has been fulfilled. Based on these results, we elaborate on an answer to the research question (Section 7.1.1).

$\mathcal{O}1$: Review and analyse previously proposed algorithms for anomaly detection in general and anomalous trajectory detection in particular.

In this thesis, we have undertaken a literature study in which previously proposed models and algorithms for anomaly detection in general (Section 2.1) and anomalous trajectory detection in particular (Section 2.2) have been reviewed. A substantial portion of the review is based on two recently published surveys by Chandola et al. (2009) on anomaly detection in general and Morris and Trivedi (2008a) on anomalous trajectory detection in video surveillance. The surveys have been complemented by studying other literature related to anomaly detection in general and anomalous trajectory detection, including algorithms for anomaly detection in the maritime domain (e.g., Bomberger et al., 2006; Brax et al., 2010; Dahlbom and Niklasson, 2007; Johansson and Falkman, 2007; Kraiman et al., 2002; Ristic et al., 2008; Tun et al., 2007).

In addition to constituting an updated overview of the research area, the literature study serves as a basis for the analysis of previous work, which includes the identification and discussion of key limitations of previous algorithms for anomaly

detection in general and anomalous trajectory detection in particular (Chapter 3). To summarise, previous algorithms typically suffer from one or more of the following limitations:

- They are based on unrealistic or inaccurate statistical models, e.g., the Gaussian distribution, which decreases the sensitivity to true anomalies and increases the false alarm rate.

- They are more or less parameter-laden, which makes implementing and tuning the algorithms more difficult and increases the risk of overfitting the models.

- They are based on ad-hoc and application-specific anomaly thresholds that are difficult to tune, which may result in too high false alarm rates and low precision.

- They are designed for offline learning and anomaly detection in trajectory databases, which delays the detection of potential anomalies and makes updating the training set in a real-time surveillance application less straightforward.

- They are insensitive to local sub-trajectory anomalies.

$\mathcal{O}2$: Identify important and desirable properties of algorithms for anomalous trajectory detection in surveillance applications.

In conjunction with the analysis of previous algorithms and the identification of their principal limitations in $\mathcal{O}1$, a number of key properties of new or updated algorithms for anomalous trajectory detection in surveillance applications are identified. These properties, which constitute the research gap that is addressed in the thesis, are presented in Section 3.3 and summarised in Table 7.1. It should be noted that each property has been explicitly or implicitly identified and discussed by previous authors in related research fields (Keogh et al., 2007; Lee et al., 2008; Piciarelli and Foresti, 2006; Rhodes et al., 2005; Zhao and Saligrama, 2009). Nevertheless, the compilation and further discussion of all these properties in the context of anomaly detection in general and anomalous trajectory detection in particular is a contribution by itself. Furthermore, it should be stressed that we do not claim that the list of properties in Section 3.3 is exhaustive; there are certainly other important properties of algorithms for the detection of anomalous trajectories in surveillance applications. In particular, high sensitivity to the true anomalies that we are interested in detecting, and low computational complexity, are obviously important properties. The former is discussed in $\mathcal{O}4$ and empirically evaluated in $\mathcal{O}5$, while the latter is analysed for the proposed algorithm in $\mathcal{O}3$.

$\mathcal{O}$3: Propose and analyse new or updated algorithms that are more appropriate for anomalous trajectory detection in surveillance applications.

In this thesis, two novel algorithms for online learning and the sequential detection of anomalous trajectories are proposed and analysed. The first algorithm, SHNN-CAD, fulfills all of the key properties identified in $\mathcal{O}2$ (Section 3.3), except the multi-scale property. The second algorithm, SSTLO-ICAD, fulfills all of the key properties in $\mathcal{O}2$. Both algorithms are based on the framework of conformal anomaly detection (Chapter 4), which is an extension of conformal prediction (Vovk et al., 2005) that has been developed in the thesis.

SHNN-CAD is an instance of CAD, which is a general algorithm for one-class anomaly detection introduced in Section 4.3.1. The only parameters of CAD are the anomaly threshold ϵ and, possibly, the parameters of the chosen NCM. In Sections 4.3.2–4.3.3, offline, online, unsupervised and semi-supervised conformal anomaly detection are formalised and properties related to the alarm rate in the different modes discussed. One of the key properties of the online CAD is that the overall alarm rate and the false alarm rate are guaranteed to be well-calibrated, i.e., less than or close to the predefined anomaly threshold ϵ, in the unsupervised and the semi-supervised mode, respectively; the only assumption is that the examples are IID. The effect of the NCM and ϵ on the classification performance of CAD is also discussed (Section 4.3.5). In the unsupervised mode, ϵ should generally be close to the prior probability of an abnormal example. In the semi-supervised mode, ϵ should be as small as possible while still maintaining reasonable sensitivity.

One of the main drawbacks of CAD is that it is computationally inefficient. In response to this, ICAD is proposed (Section 4.4), which is based on ICP (Section 4.1) and, hence, is computationally more efficient than CAD. Analogously to ICP, ICAD can operate in an offline, semi-offline or online mode. The semi-offline and online ICAD are well-calibrated in the same sense as the online CAD. However, the sensitivity of the online ICAD after an update trial may suffer severely as the size of the calibration set is small. In order to address this, we have proposed an alternative online protocol for ICAD where the size of the calibration set is bounded. This ensures that the sensitivity does not deteriorate after an update trial, but at the loss of the theoretical guarantee of a well-calibrated alarm rate.

In Chapter 5, CAD is adopted for the sequential detection of anomalous trajectories. The sequential anomaly detection setting imposes requirements that are not fulfilled by the previously proposed NCMs. Therefore, DH-kNN-NCM is proposed, which is a novel NCM based on the directed Hausdorff distance for examples that are represented as sets of points, such as trajectories. A key property of DH-kNN-NCM is that it enables the calculation of a preliminary nonconformity score for an incomplete example and that this score monotonically increases as the example is updated with more points. This implies that a well-calibrated alarm rate is maintained for any conformal anomaly detector based on DH-kNN-NCM when the new example, e.g., a trajectory, is sequentially updated. Based on DH-kNN-NCM and CAD, a general and parameter-light algorithm for sequential anomaly

detection applications is presented, called SHNN-CAD. The implementation of SHNN-CAD for the sequential detection of anomalous trajectories is discussed and a corresponding complexity analysis provided. Compared to previous algorithms, the main advantages of SHNN-CAD are that it is distribution independent and parameter-light, requires no preprocessing of trajectories, supports online learning and sequential anomaly detection, is well-calibrated, and offers a well-founded approach to the tuning of the anomaly threshold. However, similar to previous algorithms, SHNN-CAD is not sensitive to local anomalous sub-trajectories.

In Chapter 6, SSTLO-ICAD is proposed for the sequential detection of local anomalous sub-trajectories. This algorithm is based on SSTLO-NCM, which is a novel NCM that is sensitive to local anomalous sub-trajectories. Apart from the neighbourhood parameter k, the only design parameter of SSTLO-NCM is w, which corresponds to the scale of the anomalous sub-trajectories. Similar to DH-kNN-NCM, the sequence of nonconformity scores for a test trajectory monotonically increases. This implies that the alarm rates of the semi-offline and the online SSTLO-ICAD are guaranteed to be well-calibrated. In general, LOF involves a large number of nearest neighbours searches, which may be computationally very demanding. Therefore, a k-d tree-based algorithm for SSTLO-NCM is presented and analysed.

$\mathcal{O}4$: Identify appropriate performance measures for evaluating algorithms for anomalous trajectory detection in surveillance applications.

In this work, we consider anomaly detection as a binary classification problem (Section 2.1.1). Hence, we have used standard performance measures from the fields of pattern recognition and machine learning (Fawcett, 2006), such as accuracy, sensitivity, precision, false alarm rate (fpr) and AUC, for evaluating the classification performance of different anomaly detectors on labelled datasets. Due to the low frequency of true anomalies in surveillance applications, we argue that suppressing the false alarm rate is critical, in order to achieve acceptable precision (Section 3.1.3). Therefore, the sensitivity for $fpr = 0.01$ and the pAUC (Narasimhan and Agarwal, 2013) for $fpr \in [0, 0.01]$ (Section 6.3.4) are proposed and used as performance measures; as far as we know, pAUC has never previously been used to evaluate the classification performance of anomaly detectors.

In the case of sequential anomaly detection, it is also important to minimise the time, or the number of data points, required to accurately detect anomalies in incomplete trajectories. Detection delay, which is a well-known performance measure in the domain of change-detection (Ho and Wechsler, 2010), is therefore proposed as a complement to the classification performance measures above, when evaluating sequential anomaly detectors. As far as we know, detection delay has never previously been used as a performance measure in the domain of trajectory anomaly detection.

$\mathcal{O}$5: Implement the proposed algorithms and evaluate the performance of new and previous algorithms on synthetic and real trajectory datasets.

Based on the identified performance measures from $\mathcal{O}4$, we have implemented and evaluated the proposed algorithms from $\mathcal{O}3$ on a number of labelled datasets with real and synthetic trajectories (Sections 5.2 and 6.3).

In Section 5.2.1, the accuracy of DH-kNN-NCM is investigated on three public and labelled datasets of synthetic and recorded video trajectories. For comparative purposes, we have also implemented and evaluated the discords algorithm, which is another parameter-light algorithm previously proposed for detecting anomalous trajectories. On two of the datasets, the results show that DH-kNN-NCM is a competitive outlier measure, regardless of the parameter value k. On the third dataset, however, results for DH-kNN-NCM are slightly worse compared to discords, which indicates that the latter has some edge over the former in the case of complete trajectories. The unsupervised online learning and sequential anomaly detection performance of SHNN-CAD and discords is also investigated on a relatively large dataset of synthetic trajectories (Section 5.2.2), where the frequency of true anomalies is approximately 0.01. The best overall classification performance for SHNN-CAD is achieved when $\epsilon = 0.01$, which confirms our hypothesis that ϵ should be set close to the expected prior probability of an anomaly. The results show that both sensitivity and precision improve as more unlabelled training data is accumulated. Comparing the results of SHNN-CAD and discords, the best overall performance is approximately the same for both algorithms. However, when the training set is relatively large, the sensitivity of SHNN-CAD is superior to that of discords, while precision is approximately the same for both algorithms. Generally, the anomaly thresholds have a significant impact on the performance for both SHNN-CAD and discords. However, the tuning of the anomaly threshold for discords is not as straightforward as for SHNN-CAD. Moreover, the discords algorithm requires tuning of an additional parameter, i.e., the length of the sub-trajectory. As expected, the empirical alarm rate of the unsupervised online SHNN-CAD is either less than or close to the specified anomaly threshold ϵ, i.e., SHNN-CAD is indeed well-calibrated in practice.

In Section 6.3.1, the character of the most anomalous trajectories detected by SSTLO-NCM, for different values of k and w, is investigated on a recorded dataset of unlabelled vessel trajectories. Further, the empirical alarm rate of the unsupervised SSTLO-ICAD, in the semi-offline mode and in the alternative online mode, is investigated on 100 randomly permuted sequences of the trajectories used in the previous experiment. As expected, the empirical alarm rate on each of the 100 sequences is either less than or close to the specified anomaly threshold ϵ in the semi-offline mode, i.e., the unsupervised semi-offline SSTLO-ICAD is indeed well-calibrated in practice. More interestingly, however, are the results for the alternative online mode; none of the 100 empirical alarm rates exceed ϵ, even though the alarm rate is not guaranteed to be well-calibrated in the alternative online mode.

We have also reproduced a previously published experiment where the detection delay is investigated on a dataset of simulated anomalous vessel trajectories. The anomalous trajectories each consist of a normal segment, which was sampled from a real vessel trajectory, and an abnormal segment, which was generated according to a random walk. In addition to SSTLO-NCM, we also implemented and evaluated the discords algorithm. The results show that the lowest detection delay for SSTLO-NCM and discords is achieved for the sub-trajectory length of 30 seconds, which was the smallest length evaluated. In addition, for sub-trajectory lengths up to two minutes and $k \leq 4$, SSTLO-NCM outperforms both discords and two other previous anomaly detectors. This is explained by the fact that SSTLO-NCM, in contrast to the other algorithms, takes into account the local density of the sub-trajectories. Considering sub-trajectories of lengths of two or more minutes, SSTLO-NCM consistently outperforms discords, regardless of k. Nevertheless, the results indicate that the value of k has a significant impact on the detection delay and that the optimal choice of k is dependent on the length of the sub-trajectory.

In the final experiments, the classification performance of SSTLO-NCM and the discords algorithm is evaluated on a labelled dataset that includes realistically simulated anomalies from a number of different maritime anomaly classes. In addition to the traditional spatial trajectory representation, the performance of a velocity vector-based trajectory representation is investigated. AUC, pAUC, sensitivity, and mean detection delay are reported for each anomaly class and for each combination of $k = 1, \ldots, 5$ and sub-trajectory lengths of 1, 5, 10, 20, and 30 minutes. The main conclusions from the results are three-fold. Firstly, the choice of k has limited or no impact on the overall classification performance on all the anomaly classes. Secondly, the classification performance of SSTLO-NCM and discords is highly dependent on the choice of sub-trajectory length and the choice of trajectory representation (spatial or velocity), especially in the critical range $fpr \in [0, 0.01]$. Thirdly, the optimal choice of trajectory representation and sub-trajectory length is not the same for all anomaly classes. This suggests that multiple instances of SSTLO-NCM (or discords), based on different trajectory representations and sub-trajectory lengths, should be combined in order to achieve the best possible performance over all anomaly classes. Nevertheless, the best overall sensitivity for SSTLO-NCM and $fpr = 0.01$ is 0.70, which is achieved with the velocity representation, $k = 2$ and sub-trajectory length of 5 minutes. The results demonstrate that SSTLO-NCM and discords, when properly setup, perform similarly on most of the anomaly classes and are very competitive compared to previous results for SBAD (Brax, 2011).

7.1.1 Research Question

The aim of the thesis is to address the following research question:

> *What are appropriate algorithms for the detection of anomalous trajectories in surveillance applications?*

Table 7.1: Overview of the key properties identified in $\mathcal{O}2$ (Section 3.3), the algorithms proposed in $\mathcal{O}3$ and a selection of previous algorithms evaluated in $\mathcal{O}5$.

	SHNN-CAD	SSTLO-ICAD	Top-K discords (Keogh et al., 2005)	Cell-based GMM (Laxhammar, 2008)	SBAD (Brax, 2011)
1: Non-parametric	Yes	Yes	Yes	No	Yes
2: Parameter-light	Yes	Yes	Yes	No	No
3: Well-calibrated	Yes	Yes	No	No	No
4: Sequential detection	Yes	Yes	Yes	Yes	Yes
5: Unsupervised online learning	Yes	Yes	Yes[*]	No	Yes
6: Multi-scale	No	Yes	Yes	No[**]	No[**]

[*] The top-K discords algorithm, as described in Keogh et al. (2005); Yankov et al. (2008), is designed for offline detection of anomalous trajectories in databases. Nevertheless, as discussed in the thesis, it is relatively straightforward to apply to the online learning setting.

[**] The cell-based GMM and SBAD are essentially point-based anomaly detectors, i.e., they do not model trajectories or sub-trajectories. Hence, they are, by definition, not multi-scale.

Let us therefore elaborate on an answer to this question based on the results presented in the thesis.

To start with, we have identified a set of key properties of new or updated algorithms for detecting anomalous trajectories in surveillance applications. These properties, together with a selection of the algorithms that have been investigated in the thesis, are summarised in Table 7.1. In contrast to many of the algorithms that have previously been proposed for detecting anomalous trajectories, the algorithms in Table 7.1 all support sequential anomaly detection in incomplete trajectories (property 4). Further, all algorithms in Table 7.1, with the exception of the cell-based GMM, are non-parametric (property 1) and support unsupervised online learning (property 5). A drawback of SBAD (Brax, 2011) and other previously proposed algorithms is that they require the setting of many application specific parameters, compared to SHNN-CAD, SSTLO-ICAD and the top-K discords algorithm, which are parameter-light (property 2). The top-K discords algorithm, which was previously proposed by Yankov et al. (2008) for detecting anomalous trajectories, has all but one of the key properties. Nevertheless, the discords algorithm lacks a well-founded approach to the tuning of the anomaly threshold K during online learning and sequential anomaly detection, and there are no guarantees regarding the alarm rate. In fact, none of the previous algorithms provide any valid guarantees regarding the alarm rate during online learning and sequential anomaly detection. Hence, the property of a well-calibrated alarm rate is unique for SHNN-CAD and SSTLO-ICAD. The main limitation of SHNN-CAD is that it is not multi-scale (property 6). Thus, the only algorithm that has all of the key properties in Table 7.1 is SSTLO-ICAD.

The results from the experiments on the video trajectory datasets indicate that the discords algorithm is more accurate than SHNN-CAD in the case of offline anomaly detection in complete trajectories. Nevertheless, SHNN-CAD achieves competitive classification performance with minimum parameter tuning during unsupervised online learning and sequential anomaly detection. Furthermore, the alarm rate of SHNN-CAD is also well-calibrated in practice. Hence, we conclude that *SHNN-CAD is an appropriate algorithm for online learning and the sequential detection of anomalous trajectories in video surveillance applications, assuming that anomalies do not occur at different scales.*

The experimental results on the dataset with realistic maritime anomalies demonstrate that when the sub-trajectory length is well-tuned, SSTLO-NCM is competitive to the discords algorithm. Further, SSTLO-NCM is competitive to, or outperforms, the cell-based GMM and SBAD. Assuming that the rate of abnormal trajectories is approximately equal to 1%, SSTLO-NCM can achieve an overall sensitivity of 70%, while maintaining a precision of 41%; this level of performance may be argued to be of practical use in a real surveillance system. Finally, it has been demonstrated that the empirical alarm rate of SSTLO-ICAD is well-calibrated in practice. Thus, *we conclude that SSTLO-ICAD is an appropriate algorithm for the detection of anomalous sub-trajectories in maritime surveillance.*

7.2 Future Work

In this section, we list a number of directions for future work, based on the results and conclusions of the thesis:

- Investigation of hierarchical NCMs based on multiple instances of SSTLO-NCM, which are each based on different trajectory representations and sub-trajectory lengths.

  From the results of the experiments on the maritime dataset with realistic anomalies (Section 6.3.4), it is clear that the optimal choice of trajectory representation (spatial or velocity vector-based) and sub-trajectory length is not the same for all anomaly classes. This suggests that a hierarchical NCM based on different combinations of trajectory representations and sub-trajectory lengths should be adopted, in order to achieve improved overall classification performance. One such approach is to define the overall nonconformity score of a trajectory as the product of the nonconformity scores for a number of different sub-trajectory lengths and trajectory representations. In addition to the two-dimensional spatial representation and the two-dimensional velocity vector representation, a joint four-dimensional position and velocity vector representation should be investigated.

- Statistical analysis of differences in classification performance for SSTLO-NCM based on different values of k.

  The results from the experiment on the simulated anomalous vessel trajectories in Section 6.3.3 indicate that the choice of k has a significant impact on the detection delay for SSTLO-NCM. However, considering the results in Section 6.3.4, the choice of k seems to have limited or no impact on the overall classification performance of SSTLO-NCM on all the maritime anomaly classes. Hence, further investigating to what extent k affects the classification performance for the optimal sub-trajectory length on each of the anomaly classes would be interesting.

- Investigate whether the categorisation of trajectories based on, e.g., vessel type and/or vessel size has a significant impact on classification performance in general.

  When evaluating the classification performance on anomaly classes 6–7 in Section 6.3.4, we considered six different subsets of the training set that corresponds to vessels of type cargo, tanker or passenger ship, or vessels of size small, medium or large, respectively. Each test trajectory was classified on the basis of the corresponding subset of the training set for which the vessel type or size was the same. Obviously, this is necessary in order to be able to detect anomalous trajectories from anomaly classes 6–7. The remaining anomaly classes 1–5 are not explicitly conditioned on the type or size of the corresponding vessel. Hence, the complete training set was used when classifying the test trajectories from anomaly classes 1–5 in this work.

Nevertheless, categorising the training set as above might increase the overall classification performance on all anomaly classes if each of the subsets are sufficiently large.

- Statistical analysis of differences in classification performance of SSTLO-NCM and discords.

  The results from the experiments on the maritime dataset with realistic anomalies (Section 6.3.4) show that the classification performance of SSTLO-NCM and discords is competitive and, for some anomaly classes, superior to previous algorithms. However, for most anomaly classes, the performance of SSTLO-NCM and discords is quite similar. Therefore, future work should investigate whether there are any statistically significant differences between the performance of SSTLO-NCM and discords on each of the anomaly classes.

- Investigations of long-term learning strategies for addressing *population-drift* (Hand, 2006) and complexity issues related to the increasing size of the training set.

  An important aspect that is not addressed in this work is long-term learning strategies. If no data pruning strategy is used, online learning and anomaly detection may eventually become intractable due to the size of the training set. Moreover, parts of the training set may become obsolete or invalid due to changes of the underlying data distribution, known as population drift (Hand, 2006). For example, the extension of the sea lanes and the speed of vessels that follow the sea lanes may change as an effect of new or updated traffic regulations. A possible approach to detecting population drift in the framework of conformal anomaly detection is to adopt the martingale framework proposed by Ho and Wechsler (2010). More specifically, a martingale is used for sequentially testing the hypothesis that the sequence of examples observed so far is IID, where the martingale is sequentially updated based on the p-values of the new examples. If the value of the martingale exceeds a predefined threshold λ, the hypothesis that the examples are IID is rejected and the training set may be re-initialised with the most recent examples. A key property of the martingale approach is that $1/\lambda$ is equivalent to the probability of a false alarm (Ho and Wechsler, 2010), i.e., the probability of detecting population-drift when, in fact, there is none.

7.3 Generalisation

Assuming that an appropriate NCM is defined, CAD and ICAD are applicable in virtually any unsupervised or semi-supervised, online or offline, anomaly detection problem. In fact, any existing anomaly or outlier detection algorithm, which has been proved to have good classification performance in the current application domain, may potentially be adopted as a NCM in the conformal anomaly detection

framework. The main benefit of such a wrapper strategy is that it mitigates the problems with ad-hoc anomaly thresholds and offers theoretical guarantees regarding the calibration of the false alarm rate or the overall alarm rate. Hence, an interesting path for future work is to investigate NCMs that are appropriate for other data types than trajectories.

Considering DH-kNN-NCM and SSTLO-NCM, we argue that these NCMs are applicable in other anomaly detection domains where examples are naturally represented as sets or sequences of varying length, such as multi-dimensional time-series data. Further, from the perspective of conformal prediction, DH-kNN-NCM and SSTLO-NCM may be useful for supervised classification applications. For example, SSTLO-NCM may potentially be used to predict the vessel type (cargo, passenger, fishing, etc.) based on the observed trajectory.

7.4 Final Remarks

Detecting abnormal trajectories is important in many applications, such as video surveillance and maritime surveillance, since these trajectories may correspond to early indications of dangerous, illegal or otherwise interesting situations. Various algorithms have previously been proposed to assist analysts in detecting anomalous trajectories. However, as discussed in the thesis, these algorithms typically suffer from one or more limitations. One of these limitations is that they are often parameter-laden, which means that they require the careful setting of multiple application specific parameters. This makes it more difficult to implement the algorithms and achieve good performance in real applications. In particular, it may be difficult to calibrate ad-hoc anomaly thresholds in order to strike a good balance between the sensitivity to true anomalies and the rate of false or otherwise uninteresting alarms. A poorly calibrated threshold may result in a high false alarm rate and low precision, which increases the risk that the operator simply ignores the system. Conversely, if the threshold is unbalanced in the opposite direction, sensitivity may become unnecessarily low, which increases the risk of missing true and interesting anomalies.

Another limitation of previous algorithms is that they are essentially designed for offline anomaly detection in trajectory databases, i.e., they typically assume that all trajectories are available from the outset and that each trajectory is complete. This is obviously a limitation in a real-time surveillance application where trajectories are observed in sequence and each trajectory is sequentially updated. As a result, the detection of anomalous trajectories is delayed until the trajectory is complete and, hence, the ability to respond proactively to impending situations is reduced. Further, the anomaly detector's ability to efficiently leverage on new training data is limited.

In this thesis, we have developed a novel and non-parametric framework for anomaly detection called conformal anomaly detection. In particular, we have proposed two general anomaly detection algorithms: CAD and the computationally more efficient ICAD. These two algorithms have only two parameters each: the

anomaly threshold ϵ and the NCM, which measures how different an example is, e.g., a trajectory, relative to a set of examples. CAD and ICAD can be used for unsupervised or semi-supervised, offline or online anomaly detection. However, the unsupervised online CAD (or ICAD) is perhaps of most practical interest, since it may benefit from online learning and does not require the true class label (normal or abnormal) for each example (trajectory) in the training set. Furthermore, the unsupervised online CAD is well-calibrated in the sense that the overall empirical alarm rate is guaranteed to be close to or less than the predefined anomaly threshold ϵ. As a rule of thumb, ϵ should be set close to the estimated frequency of true anomalies in data.

The main challenge when applying CAD or ICAD is the choice of an appropriate NCM. It is critical that the NCM is able to accurately discriminate normal examples (trajectories) from the examples of the abnormal classes that we are interested in detecting. In principal, any scoring-based anomaly detector, which has been shown to have good classification performance in the current application domain, may be adopted as NCM. In this thesis, we have proposed and empirically investigated the performance of two novel and, more or less, parameter-light NCMs for complete or incomplete trajectories. The first, which is called DH-kNN-NCM, is appropriate in applications where the trajectories are of roughly the same length and are confined to a limited geographical area, such as video surveillance of public areas, etc. It has only one free parameter, which does not seem to be critical for the classification performance. The second, which is called SSTLO-NCM, is more sensitive to local anomalous sub-trajectories. It is therefore appropriate in wide-area surveillance applications, such as maritime surveillance, where the length and extension of trajectories vary greatly and where anomalies may occur at multiple scales. However, SSTLO-NCM involves the tuning of an additional parameter, which corresponds to the length of the sub-trajectories; the optimal value of this parameter has been found to not be the same for all the maritime anomaly classes investigated in the thesis. Further, the choice of trajectory representation, i.e., the features of each trajectory point, such as the current position and velocity, may have a significant impact on the classification performance for different anomaly classes. Therefore, in order to achieve improved overall performance on all anomaly classes, a hierarchal version of SSTLO-NCM, based on multiple sub-trajectory lengths and trajectory representation, should be further investigated. Nevertheless, it may be pointed out that an alternative to DH-kNN-NCM and SSTLO-NCM is to adopt the discords algorithm (Keogh et al., 2005) as NCM, especially if there is no particular reason to believe that the local density of sub-trajectories differs significantly. Indeed, the discords algorithm is also parameter-light and has been shown to have, more or less, similar classification performance as DH-kNN-NCM and SSTLO-NCM, in most of the experiments of this thesis.

Appendix A

Classification Performance Results: Realistic Maritime Anomalies

In this appendix, we present the complete results from the experiments in Section 6.3.4. The first seven sections of the appendix correspond to the detailed results for the seven anomaly classes 1–7. The last section presents the average results over all anomaly classes. For each section, there are two subsections corresponding to the results for the spatial trajectory representation and the velocity vector-based trajectory representation, respectively. In each subsection, there are four tables that present AUC, pAUC (for $tpr \in [0.01]$), sensitivity, and mean detection delay (for $tpr = 0.01$, respectively, for each combination of parameters of SSTLO-NCM and discords.

A.1 Circle and Land

Spatial Trajectory Representation

Table A.1: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=30$	0.999	1.000	1.000	0.998	0.998	0.998
$w=11$, $dt=60$	0.999	1.000	1.000	1.000	1.000	0.990
$w=11$, $dt=120$	0.984	0.993	0.993	0.991	0.993	0.985
$w=11$, $dt=180$	0.988	0.993	0.993	0.992	0.993	0.972

Table A.2: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=30$	0.929	1.000	1.000	0.786	0.786	0.857
$w=11$, $dt=60$	0.929	1.000	1.000	1.000	1.000	0.429
$w=11$, $dt=120$	0.429	0.357	0.357	0.214	0.357	0.214
$w=11$, $dt=180$	0.143	0.429	0.429	0.286	0.357	0.286

Table A.3: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=30$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=60$	1.000	1.000	1.000	1.000	1.000	0.500
$w=11$, $dt=120$	0.500	1.000	1.000	0.500	1.000	0.500
$w=11$, $dt=180$	0.500	0.500	1.000	0.500	0.500	0.500

Table A.4: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	777	702	672	672	672	717
$w=11$, $dt=30$	672	372	417	417	417	702
$w=11$, $dt=60$	537	417	417	537	567	1031
$w=11$, $dt=120$	1691	687	1347	523	1347	1811
$w=11$, $dt=180$	1031	1031	897	1031	1031	1031

Velocity Trajectory Representation

Table A.5: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.998	0.997	0.997	0.997	0.997	0.997
$w=11$, $dt=30$	0.997	0.998	0.997	0.997	0.997	0.998
$w=11$, $dt=60$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=120$	0.998	0.996	0.996	0.996	0.998	1.000
$w=11$, $dt=180$	0.999	0.999	0.999	0.999	1.000	1.000

Table A.6: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.857	0.714	0.714	0.714	0.714	0.714
$w=11$, $dt=30$	0.714	0.857	0.714	0.714	0.714	0.857
$w=11$, $dt=60$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=120$	0.786	0.643	0.643	0.643	0.786	1.000
$w=11$, $dt=180$	0.929	0.929	0.929	0.929	1.000	1.000

Table A.7: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=30$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=60$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=120$	1.000	1.000	1.000	1.000	1.000	1.000
$w=11$, $dt=180$	1.000	1.000	1.000	1.000	1.000	1.000

Table A.8: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	297	282	282	282	282	282
$w=11$, $dt=30$	282	282	282	282	282	297
$w=11$, $dt=60$	417	417	417	417	417	327
$w=11$, $dt=120$	1047	447	387	387	387	387
$w=11$, $dt=180$	717	717	627	627	537	447

A.2 Missed Turn

Spatial Trajectory Representation

Table A.9: AUC

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	0.999	0.999	0.999	0.999	0.999	0.999
$w = 11$, $dt = 30$	0.951	0.979	0.986	0.985	0.983	0.993
$w = 11$, $dt = 60$	0.987	0.994	0.990	0.992	0.992	0.978
$w = 11$, $dt = 120$	0.969	0.980	0.982	0.975	0.977	0.905
$w = 11$, $dt = 180$	0.437	0.509	0.557	0.570	0.603	0.700

Table A.10: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	0.929	0.929	0.929	0.929	0.929	0.929
$w = 11$, $dt = 30$	0.500	0.500	0.500	0.500	0.500	0.500
$w = 11$, $dt = 60$	0.500	0.500	0.500	0.500	0.500	0.500
$w = 11$, $dt = 120$	0.429	0.429	0.429	0.429	0.429	0.000
$w = 11$, $dt = 180$	0.000	0.000	0.000	0.000	0.000	0.000

Table A.11: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	1.000	1.000	1.000	1.000	1.000	1.000
$w = 11$, $dt = 30$	0.500	0.500	0.500	0.500	0.500	0.500
$w = 11$, $dt = 60$	0.500	0.500	0.500	0.500	0.500	0.500
$w = 11$, $dt = 120$	0.500	0.500	0.500	0.500	0.500	0.000
$w = 11$, $dt = 180$	0.000	0.000	0.000	0.000	0.000	0.000

Table A.12: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	241	241	241	241	241	241
$w = 11$, $dt = 30$	243	213	213	213	213	303
$w = 11$, $dt = 60$	333	273	273	273	273	453
$w = 11$, $dt = 120$	393	393	393	393	393	NaN
$w = 11$, $dt = 180$	NaN	NaN	NaN	NaN	NaN	NaN

Velocity Trajectory Representation

Table A.13: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.743	0.737	0.733	0.731	0.728	0.796
$w=11$, $dt=30$	0.807	0.769	0.800	0.818	0.833	0.796
$w=11$, $dt=60$	0.877	0.857	0.827	0.809	0.739	0.781
$w=11$, $dt=120$	0.656	0.579	0.690	0.752	0.782	0.727
$w=11$, $dt=180$	0.446	0.466	0.455	0.461	0.454	0.788

Table A.14: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.286	0.286	0.286	0.286	0.286	0.286
$w=11$, $dt=30$	0.357	0.286	0.357	0.286	0.214	0.214
$w=11$, $dt=60$	0.000	0.000	0.000	0.000	0.000	0.000
$w=11$, $dt=120$	0.000	0.000	0.000	0.000	0.000	0.000
$w=11$, $dt=180$	0.000	0.000	0.000	0.000	0.000	0.000

Table A.15: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.500	0.500	0.500	0.500	0.500	0.500
$w=11$, $dt=30$	0.500	0.500	0.500	0.500	0.500	0.500
$w=11$, $dt=60$	0.000	0.000	0.000	0.000	0.000	0.000
$w=11$, $dt=120$	0.000	0.000	0.000	0.000	0.000	0.000
$w=11$, $dt=180$	0.000	0.000	0.000	0.000	0.000	0.000

Table A.16: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	603	603	603	603	603	603
$w=11$, $dt=30$	633	633	693	693	693	723
$w=11$, $dt=60$	NaN	NaN	NaN	NaN	NaN	NaN
$w=11$, $dt=120$	NaN	NaN	NaN	NaN	NaN	NaN
$w=11$, $dt=180$	NaN	NaN	NaN	NaN	NaN	NaN

A.3 Unexpected Stop

Spatial Trajectory Representation

Table A.17: AUC

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	0.492	0.505	0.508	0.513	0.510	0.620
$w = 11$, $dt = 30$	0.802	0.837	0.849	0.856	0.856	0.806
$w = 11$, $dt = 60$	0.852	0.890	0.900	0.899	0.900	0.854
$w = 11$, $dt = 120$	0.899	0.925	0.924	0.926	0.926	0.888
$w = 11$, $dt = 180$	0.909	0.913	0.914	0.919	0.923	0.899

Table A.18: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	0.000	0.000	0.000	0.000	0.000	0.000
$w = 11$, $dt = 30$	0.013	0.026	0.064	0.060	0.054	0.010
$w = 11$, $dt = 60$	0.063	0.199	0.186	0.131	0.107	0.026
$w = 11$, $dt = 120$	0.080	0.207	0.196	0.160	0.199	0.163
$w = 11$, $dt = 180$	0.231	0.314	0.233	0.159	0.156	0.327

Table A.19: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	0.000	0.000	0.000	0.000	0.000	0.000
$w = 11$, $dt = 30$	0.020	0.130	0.150	0.140	0.140	0.020
$w = 11$, $dt = 60$	0.120	0.270	0.270	0.250	0.200	0.050
$w = 11$, $dt = 120$	0.120	0.360	0.310	0.290	0.350	0.280
$w = 11$, $dt = 180$	0.380	0.420	0.360	0.330	0.240	0.530

Table A.20: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	
$w = 3$, $dt = 30$	NaN	NaN	NaN	NaN	NaN	NaN
$w = 11$, $dt = 30$	3642	768	1597	1231	1435	5669
$w = 11$, $dt = 60$	666	1037	1435	1028	866	1722
$w = 11$, $dt = 120$	1702	1907	1837	2148	1841	1411
$w = 11$, $dt = 180$	1836	1629	1465	1551	1579	1205

Velocity Trajectory Representation

Table A.21: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.706	0.753	0.783	0.787	0.794	0.889
$w=11$, $dt=30$	0.920	0.921	0.917	0.913	0.907	0.921
$w=11$, $dt=60$	0.914	0.905	0.895	0.889	0.893	0.909
$w=11$, $dt=120$	0.868	0.893	0.902	0.895	0.895	0.908
$w=11$, $dt=180$	0.853	0.879	0.885	0.888	0.889	0.903

Table A.22: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.174	0.147	0.150	0.136	0.140	0.183
$w=11$, $dt=30$	0.196	0.197	0.199	0.153	0.146	0.113
$w=11$, $dt=60$	0.187	0.184	0.141	0.136	0.126	0.104
$w=11$, $dt=120$	0.054	0.106	0.101	0.097	0.099	0.144
$w=11$, $dt=180$	0.101	0.076	0.113	0.133	0.156	0.179

Table A.23: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.310	0.310	0.310	0.290	0.300	0.330
$w=11$, $dt=30$	0.300	0.350	0.320	0.270	0.250	0.240
$w=11$, $dt=60$	0.280	0.340	0.200	0.210	0.180	0.190
$w=11$, $dt=120$	0.110	0.200	0.180	0.230	0.250	0.200
$w=11$, $dt=180$	0.130	0.110	0.170	0.180	0.220	0.210

Table A.24: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	598	503	503	576	322	384
$w=11$, $dt=30$	801	341	367	319	96	189
$w=11$, $dt=60$	1565	1160	807	1103	891	660
$w=11$, $dt=120$	2156	2231	2033	1993	2250	1410
$w=11$, $dt=180$	1644	2628	2057	2533	2330	1847

A.4 Unusual Speed

Spatial Trajectory Representation

Table A.25: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.572	0.573	0.576	0.571	0.564	0.694
$w=11$, $dt=30$	0.814	0.836	0.844	0.848	0.848	0.785
$w=11$, $dt=60$	0.834	0.867	0.870	0.866	0.865	0.775
$w=11$, $dt=120$	0.794	0.828	0.823	0.822	0.817	0.737
$w=11$, $dt=180$	0.755	0.773	0.762	0.758	0.761	0.704

Table A.26: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.017	0.014	0.014	0.016	0.020	0.013
$w=11$, $dt=30$	0.059	0.080	0.076	0.073	0.064	0.059
$w=11$, $dt=60$	0.064	0.146	0.154	0.120	0.107	0.063
$w=11$, $dt=120$	0.050	0.102	0.106	0.111	0.129	0.033
$w=11$, $dt=180$	0.069	0.077	0.071	0.043	0.034	0.033

Table A.27: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.040	0.020	0.020	0.020	0.030	0.020
$w=11$, $dt=30$	0.090	0.130	0.140	0.140	0.110	0.070
$w=11$, $dt=60$	0.100	0.190	0.220	0.210	0.170	0.080
$w=11$, $dt=120$	0.060	0.160	0.160	0.180	0.190	0.050
$w=11$, $dt=180$	0.090	0.120	0.110	0.090	0.070	0.070

Table A.28: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	972	36	36	36	2736	36
$w=11$, $dt=30$	492	342	970	976	361	140
$w=11$, $dt=60$	1180	770	900	1523	1639	1605
$w=11$, $dt=120$	1971	1314	1595	1608	1360	792
$w=11$, $dt=180$	4533	2679	2730	3197	2728	2003

Velocity Trajectory Representation

Table A.29: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.818	0.909	0.913	0.917	0.917	0.953
$w=11$, $dt=30$	0.953	0.955	0.954	0.954	0.954	0.949
$w=11$, $dt=60$	0.934	0.945	0.944	0.944	0.944	0.937
$w=11$, $dt=120$	0.896	0.902	0.903	0.897	0.897	0.889
$w=11$, $dt=180$	0.863	0.867	0.865	0.861	0.859	0.848

Table A.30: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.347	0.329	0.360	0.346	0.369	0.387
$w=11$, $dt=30$	0.460	0.520	0.526	0.510	0.494	0.333
$w=11$, $dt=60$	0.511	0.487	0.479	0.487	0.466	0.319
$w=11$, $dt=120$	0.239	0.331	0.357	0.370	0.404	0.197
$w=11$, $dt=180$	0.311	0.364	0.431	0.461	0.473	0.176

Table A.31: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.580	0.620	0.660	0.660	0.670	0.670
$w=11$, $dt=30$	0.660	0.760	0.740	0.740	0.710	0.550
$w=11$, $dt=60$	0.640	0.690	0.650	0.640	0.600	0.510
$w=11$, $dt=120$	0.360	0.440	0.480	0.540	0.580	0.290
$w=11$, $dt=180$	0.420	0.430	0.490	0.550	0.550	0.240

Table A.32: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	189	119	100	96	99	158
$w=11$, $dt=30$	210	253	195	194	192	98
$w=11$, $dt=60$	773	741	747	712	877	234
$w=11$, $dt=120$	874	774	632	535	552	529
$w=11$, $dt=180$	1087	818	696	663	587	930

A.5 Strange Manoeuvres

Spatial Trajectory Representation

Table A.33: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.687	0.724	0.753	0.750	0.758	0.845
$w=11$, $dt=30$	0.817	0.865	0.878	0.883	0.889	0.774
$w=11$, $dt=60$	0.812	0.872	0.872	0.874	0.878	0.712
$w=11$, $dt=120$	0.820	0.861	0.854	0.860	0.856	0.692
$w=11$, $dt=180$	0.799	0.807	0.813	0.820	0.821	0.691

Table A.34: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.004	0.001	0.000	0.000	0.006	0.001
$w=11$, $dt=30$	0.010	0.016	0.019	0.021	0.013	0.004
$w=11$, $dt=60$	0.010	0.051	0.047	0.037	0.029	0.010
$w=11$, $dt=120$	0.024	0.041	0.030	0.027	0.049	0.010
$w=11$, $dt=180$	0.039	0.054	0.046	0.031	0.024	0.013

Table A.35: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.020	0.010	0.000	0.000	0.010	0.010
$w=11$, $dt=30$	0.030	0.050	0.060	0.050	0.030	0.010
$w=11$, $dt=60$	0.020	0.070	0.080	0.060	0.040	0.010
$w=11$, $dt=120$	0.030	0.070	0.050	0.070	0.110	0.010
$w=11$, $dt=180$	0.050	0.090	0.080	0.070	0.050	0.020

Table A.36: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	1624	2967	NaN	NaN	7067	3087
$w=11$, $dt=30$	1734	936	827	735	501	1580
$w=11$, $dt=60$	1063	709	1471	1748	2008	1610
$w=11$, $dt=120$	2877	1980	442	737	782	1730
$w=11$, $dt=180$	2236	1832	1193	1157	1217	1483

Velocity Trajectory Representation

Table A.37: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.950	0.968	0.983	0.982	0.982	0.989
$w=11$, $dt=30$	0.993	0.994	0.994	0.994	0.994	0.994
$w=11$, $dt=60$	0.993	0.996	0.996	0.996	0.995	0.992
$w=11$, $dt=120$	0.954	0.975	0.980	0.982	0.980	0.961
$w=11$, $dt=180$	0.918	0.936	0.931	0.934	0.934	0.909

Table A.38: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.597	0.524	0.510	0.500	0.493	0.553
$w=11$, $dt=30$	0.687	0.716	0.759	0.724	0.679	0.691
$w=11$, $dt=60$	0.823	0.774	0.735	0.726	0.691	0.750
$w=11$, $dt=120$	0.201	0.310	0.337	0.343	0.340	0.146
$w=11$, $dt=180$	0.191	0.204	0.263	0.283	0.311	0.043

Table A.39: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.850	0.820	0.830	0.830	0.810	0.840
$w=11$, $dt=30$	0.950	0.980	0.980	0.970	0.930	0.960
$w=11$, $dt=60$	0.950	0.980	0.950	0.910	0.850	0.940
$w=11$, $dt=120$	0.340	0.470	0.530	0.660	0.620	0.250
$w=11$, $dt=180$	0.260	0.280	0.360	0.380	0.440	0.080

Table A.40: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	79	33	37	57	39	45
$w=11$, $dt=30$	249	107	148	196	173	108
$w=11$, $dt=60$	396	380	446	437	435	392
$w=11$, $dt=120$	951	1006	996	993	1014	1079
$w=11$, $dt=180$	1623	1633	1287	1282	1197	1526

A.6 Large Vessel in Unusual Location

Multi-class anomaly detection based on vessel size.

Spatial Trajectory Representation

Table A.41: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.966	0.977	0.939	0.979	0.979	0.974
$w=11$, $dt=30$	0.959	0.968	0.889	0.973	0.979	0.965
$w=11$, $dt=60$	0.967	0.946	0.873	0.963	0.965	0.959
$w=11$, $dt=120$	0.944	0.871	0.847	0.930	0.943	0.941
$w=11$, $dt=180$	0.873	0.795	0.779	0.878	0.893	0.890

Table A.42: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.690	0.806	0.757	0.739	0.729	0.621
$w=11$, $dt=30$	0.727	0.730	0.716	0.520	0.669	0.620
$w=11$, $dt=60$	0.709	0.683	0.681	0.526	0.624	0.609
$w=11$, $dt=120$	0.754	0.660	0.593	0.510	0.506	0.501
$w=11$, $dt=180$	0.570	0.563	0.417	0.420	0.551	0.477

Table A.43: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.770	0.880	0.800	0.900	0.910	0.740
$w=11$, $dt=30$	0.760	0.780	0.790	0.600	0.790	0.730
$w=11$, $dt=60$	0.740	0.700	0.720	0.550	0.680	0.720
$w=11$, $dt=120$	0.800	0.670	0.670	0.520	0.520	0.680
$w=11$, $dt=180$	0.700	0.620	0.480	0.450	0.560	0.620

Table A.44: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	693	996	996	670	653	628
$w=11$, $dt=30$	969	993	1206	902	941	865
$w=11$, $dt=60$	1188	996	1364	1035	1056	1168
$w=11$, $dt=120$	1947	1496	1977	1604	1567	1654
$w=11$, $dt=180$	2494	2183	2760	1896	1961	2183

Velocity Trajectory Representation

Table A.45: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.825	0.833	0.836	0.834	0.853	0.911
$w=11$, $dt=30$	0.905	0.905	0.907	0.900	0.896	0.898
$w=11$, $dt=60$	0.896	0.895	0.899	0.895	0.888	0.894
$w=11$, $dt=120$	0.873	0.878	0.879	0.877	0.875	0.894
$w=11$, $dt=180$	0.834	0.838	0.829	0.826	0.824	0.858

Table A.46: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.356	0.361	0.363	0.364	0.353	0.341
$w=11$, $dt=30$	0.529	0.494	0.506	0.503	0.501	0.427
$w=11$, $dt=60$	0.490	0.479	0.527	0.499	0.529	0.547
$w=11$, $dt=120$	0.441	0.423	0.344	0.353	0.369	0.557
$w=11$, $dt=180$	0.421	0.397	0.397	0.371	0.414	0.517

Table A.47: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.650	0.640	0.640	0.610	0.610	0.630
$w=11$, $dt=30$	0.610	0.620	0.610	0.620	0.600	0.610
$w=11$, $dt=60$	0.590	0.580	0.630	0.600	0.570	0.570
$w=11$, $dt=120$	0.450	0.550	0.510	0.580	0.560	0.570
$w=11$, $dt=180$	0.480	0.470	0.460	0.480	0.490	0.540

Table A.48: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	240	262	244	411	411	296
$w=11$, $dt=30$	733	776	870	787	1066	829
$w=11$, $dt=60$	1595	1443	1310	1240	1308	1115
$w=11$, $dt=120$	3176	3065	2915	2654	2376	1400
$w=11$, $dt=180$	3818	3780	3510	3784	3696	1910

A.7 Unexpected Vessel Type/Behaviour

Multi-class anomaly detection based on vessel type.

Spatial Trajectory Representation

Table A.49: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.927	0.940	0.944	0.946	0.947	0.948
$w=11$, $dt=30$	0.913	0.928	0.928	0.928	0.927	0.929
$w=11$, $dt=60$	0.903	0.918	0.920	0.921	0.919	0.917
$w=11$, $dt=120$	0.853	0.897	0.911	0.905	0.903	0.921
$w=11$, $dt=180$	0.835	0.875	0.887	0.891	0.890	0.908

Table A.50: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.757	0.759	0.769	0.757	0.751	0.801
$w=11$, $dt=30$	0.750	0.741	0.737	0.720	0.707	0.751
$w=11$, $dt=60$	0.724	0.741	0.717	0.716	0.703	0.727
$w=11$, $dt=120$	0.690	0.686	0.699	0.693	0.687	0.737
$w=11$, $dt=180$	0.657	0.651	0.661	0.647	0.646	0.746

Table A.51: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.760	0.770	0.780	0.800	0.790	0.810
$w=11$, $dt=30$	0.770	0.760	0.770	0.750	0.750	0.780
$w=11$, $dt=60$	0.730	0.750	0.750	0.750	0.720	0.770
$w=11$, $dt=120$	0.690	0.700	0.730	0.720	0.730	0.770
$w=11$, $dt=180$	0.670	0.660	0.690	0.690	0.690	0.780

Table A.52: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	251	362	411	404	349	397
$w=11$, $dt=30$	566	589	728	701	670	664
$w=11$, $dt=60$	1027	926	970	899	832	845
$w=11$, $dt=120$	1400	1459	1514	1515	1440	1582
$w=11$, $dt=180$	1916	1893	2251	2264	2270	2038

Velocity Trajectory Representation

Table A.53: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.859	0.865	0.875	0.883	0.881	0.873
$w=11$, $dt=30$	0.868	0.876	0.878	0.881	0.886	0.865
$w=11$, $dt=60$	0.874	0.859	0.864	0.874	0.881	0.871
$w=11$, $dt=120$	0.871	0.848	0.865	0.871	0.881	0.879
$w=11$, $dt=180$	0.853	0.844	0.865	0.872	0.878	0.868

Table A.54: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.661	0.657	0.657	0.659	0.656	0.664
$w=11$, $dt=30$	0.656	0.653	0.654	0.654	0.650	0.650
$w=11$, $dt=60$	0.689	0.653	0.647	0.657	0.650	0.657
$w=11$, $dt=120$	0.669	0.674	0.670	0.661	0.671	0.694
$w=11$, $dt=180$	0.696	0.699	0.687	0.699	0.716	0.711

Table A.55: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.710	0.700	0.700	0.710	0.710	0.720
$w=11$, $dt=30$	0.690	0.690	0.690	0.690	0.690	0.690
$w=11$, $dt=60$	0.710	0.680	0.670	0.670	0.660	0.700
$w=11$, $dt=120$	0.680	0.680	0.690	0.670	0.690	0.720
$w=11$, $dt=180$	0.720	0.710	0.710	0.730	0.770	0.730

Table A.56: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	137	235	228	232	232	158
$w=11$, $dt=30$	477	503	469	463	481	375
$w=11$, $dt=60$	1052	928	964	1365	861	918
$w=11$, $dt=120$	1505	1498	1583	1481	1590	1500
$w=11$, $dt=180$	2232	2173	2170	2266	2333	2061

A.8 Overall Performance

Average performance over all the anomalous trajectories in Section A.1–A.7.

Spatial Trajectory Representation

Table A.57: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.806	0.817	0.817	0.823	0.823	0.869
$w=11$, $dt=30$	0.894	0.916	0.910	0.924	0.926	0.893
$w=11$, $dt=60$	0.908	0.927	0.918	0.931	0.931	0.884
$w=11$, $dt=120$	0.895	0.908	0.905	0.916	0.916	0.867
$w=11$, $dt=180$	0.800	0.809	0.815	0.833	0.841	0.823

Table A.58: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.485	0.501	0.496	0.491	0.491	0.481
$w=11$, $dt=30$	0.427	0.442	0.444	0.383	0.399	0.400
$w=11$, $dt=60$	0.428	0.474	0.469	0.433	0.439	0.338
$w=11$, $dt=120$	0.351	0.355	0.344	0.306	0.336	0.237
$w=11$, $dt=180$	0.244	0.298	0.265	0.227	0.253	0.269

Table A.59: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.513	0.526	0.514	0.531	0.534	0.511
$w=11$, $dt=30$	0.453	0.479	0.487	0.454	0.474	0.444
$w=11$, $dt=60$	0.459	0.497	0.506	0.474	0.473	0.376
$w=11$, $dt=120$	0.386	0.494	0.489	0.397	0.486	0.327
$w=11$, $dt=180$	0.341	0.344	0.389	0.304	0.301	0.360

Table A.60: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	760	884	471	405	1953	851
$w=11$, $dt=30$	1188	602	851	739	649	1418
$w=11$, $dt=60$	856	733	976	1006	1034	1205
$w=11$, $dt=120$	1712	1319	1301	1218	1247	1497
$w=11$, $dt=180$	2341	1875	1883	1849	1798	1657

Velocity Trajectory Representation

Table A.61: AUC

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.843	0.866	0.874	0.876	0.879	0.915
$w=11$, $dt=30$	0.920	0.917	0.921	0.922	0.924	0.917
$w=11$, $dt=60$	0.927	0.922	0.918	0.915	0.906	0.912
$w=11$, $dt=120$	0.873	0.867	0.888	0.896	0.901	0.894
$w=11$, $dt=180$	0.824	0.833	0.833	0.835	0.834	0.882

Table A.62: Partial AUC for $fpr \in (0, 0.01)$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.468	0.431	0.434	0.429	0.430	0.447
$w=11$, $dt=30$	0.514	0.532	0.531	0.506	0.486	0.469
$w=11$, $dt=60$	0.529	0.511	0.504	0.501	0.494	0.482
$w=11$, $dt=120$	0.341	0.355	0.350	0.352	0.381	0.391
$w=11$, $dt=180$	0.379	0.381	0.403	0.411	0.439	0.375

Table A.63: Sensitivity (tpr) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	0.657	0.656	0.663	0.657	0.657	0.670
$w=11$, $dt=30$	0.673	0.700	0.691	0.684	0.669	0.650
$w=11$, $dt=60$	0.596	0.610	0.586	0.576	0.551	0.559
$w=11$, $dt=120$	0.420	0.477	0.484	0.526	0.529	0.433
$w=11$, $dt=180$	0.430	0.429	0.456	0.474	0.496	0.400

Table A.64: Mean detection delay (seconds) for $fpr = 0.01$

	SSTLO-NCM					Discords
	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	
$w=3$, $dt=30$	306	291	285	322	284	275
$w=11$, $dt=30$	484	413	432	419	426	374
$w=11$, $dt=60$	966	845	782	879	798	608
$w=11$, $dt=120$	1618	1504	1424	1341	1361	1051
$w=11$, $dt=180$	1854	1958	1725	1859	1780	1454

References

H. Alt. The Computational Geometry of Comparing Shapes. In H. Albers, S.and Alt and S. Näher, editors, *Efficient Algorithms*, volume 5760 of *Lecture Notes in Computer Science*, pages 235–248. Springer Berlin / Heidelberg, 2009.

H. Alt, B. Behrends, and J. Blömer. Approximate matching of polygonal shapes. *Annals of Mathematics and Artificial Intelligence*, 13(3-4):251–265, 1995.

S. Atev, G. Miller, and N. Papanikolopoulos. Clustering of Vehicle Trajectories. *IEEE Transactions on Intelligent Transportation Systems*, 11(3):647–657, September 2010.

S. Axelsson. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security (TISSEC)*, 3(3):185–205, August 2000.

D. Barbará and C. Domeniconi. Detecting outliers using transduction and statistical testing. In *Proceedings of the 12th Annual SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2006.

V. Barnett and T. Lewis. *Outliers in Statistical Data*. John Wiley and Sons, Inc., third edition edition, 1994.

M. Berndtsson, J. Hansson, B. Olsson, and B. Lundell. *Planning and Implementing your Final Year Project - with Success!: A Guide for Students in Computer Science and Information Systems*. Springer, 2002.

N. Bomberger, B. J. Rhodes, M. Seibert, and A. Waxman. Associative learning of vessel motion patterns for maritime situation awareness. In *Proceedings of the 9th International Conference on Information Fusion*, 2006.

C. Brax. *Anomaly Detection in the Surveillance Domain*. PhD thesis, Örebro University, 2011.

C. Brax, L. Niklasson, and M. Smedberg. Finding behavioral anomalies in public areas using video surveillance data. In *Proceedings of the 11th International Conference on Information Fusion*, Cologne, Germany, July 2008.

C. Brax, L. Niklasson, and R. Laxhammar. An ensemble approach for increased anomaly detection performance in video surveillance data. In *Proceedings of the 12th International Conference on Information Fusion*, pages 694–701, Seattle, July 2009.

C. Brax, A. Karlsson, S. Andler, R. Johansson, and L. Niklasson. Evaluating precise and imprecise state-based anomaly detectors for maritime surveillance. In *Proceedings of the 13th International Conference on Information Fusion*, 2010.

M. M. Breunig, H. P. Kriegel, R. T. Ng, and J. Sander. LOF: Identifying Density-Based Local Outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000.

Y. Bu, L. Chen, and D. Wai-Chee Fu, A. Liu. Efficient anomaly monitoring over moving object trajectory streams. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2009.

V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

S. Chang. Development and analysis of AIS applications as an efficient tool for vessel traffic service. In *Proceedings of Ocean'04. MTS/IEEE Techno-Ocean: Bridges across the Oceans*, volume 4, pages 2249–2253, 2004.

A. Dahlbom and L. Niklasson. Trajectory clustering for coastal surveillance. In *Proceedings of the 10th International Conference on Information Fusion*, Quebec city, Canada, July 2007.

A. Dahlbom, L. Niklasson, and G. Falkman. Situation recognition and hypothesis management using petri nets. In *Proceedings of the 6th International Conference on Modeling Decisions for Artificial Intelligence*, 2009.

Danish Maritime Authority. Casualty report - collision between Chinese bulk carrier FU SHAN HAI and Cypriot container vessel GDYNIA, 2003.

H. Dee and S. Velastin. How close are we to solving the problem of automated visual surveillance?: A review of real-world surveillance, scientific progress and evaluative mechanisms. *Machine Vision and Applications*, 19(5-6):329–343, September 2008.

A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society, series B*, 39:1–38, 1977.

D. Devetyarov and I. Nouretdinov. Prediction with Confidence Based on a Random Forest Classifierlearning for medical diagnosis. In *Proceedings of 6th IFIP WG 12.5 International Conference: Artificial Intelligence Applications and Innovations (AIAI)*, volume 339 of *IFIP Advances in Information and Communication Technology*, pages 37–44. Springer, 2010.

R. Duda and P. Hart. *Pattern Classification and Scene Analysis.* John Wiley and Sons, Inc., 1973.

J. Edlund, M. Grönkvist, A. Lingvall, and E. Sviestins. Rule-based situation assessment for sea surveillance. In B. V. Dasarathy, editor, *Proceedings of the SPIE Multisensor, Multisource Information Fusion: Architectures, Algorithms, and Applications*, volume 6242, April 2006.

J. Ekman and A. Holst. Incremental stream clustering and anomaly detection. Technical Report T2008:1, Swedish Institute of Computer Science (SICS), 2008.

E. Eskin. Anomaly detection over noisy data using learned probability distributions. In *Proceedings of the 7th International Conference on Machine Learning*, pages 255–262, 2000.

E. Eskin, A. Arnold, M. Prerau, L. Portnoy, and S. Stolfo. A geometric framework for unsupervised anomaly detection. In *Proceedings of the Conference on Applications of Data Mining in Computer Security*, 2002.

T. Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters - Special issue: ROC analysis in pattern recognition*, 27(8):861–874, June 2006.

F. Fooladvandi, C. Brax, P. Gustavsson, and M. Fredin. Signature-based activity detection based on bayesian networks acquired from expert knowledge. In *Proceedings of the 12th International Conference on Information Fusion*, 2009.

J. Friedman, J. Bentley, and R. Finkel. An Algorithm for Finding Best Matches in Logarithmic Expected Time. *ACM Transactions on Mathematical Software*, 3(3):209–226, September 1977.

Z. Fu, W. Hu, and T. Tan. Similarity based vehicle trajectory clustering and anomaly detection. In *Proceedings of IEEE International Conference on Image Processing*, 2005.

A. Gammerman and V. Vovk. Hedging predictions in machine learning. *The Computer Journal*, 50(2):151–163, 2007. ISSN 0010-4620.

D. Garagic, B. J. Rhodes, N. Bomberger, and M. Zandipour. Adaptive mixture-based neural network approach for higher-level fusion and automated behavior monitoring. In *Proceedings of the IEEE International conference on Communications*, 2009.

A. Gelman, J. B. Carlin, H. S. Stern, and D. B. Rubin. *Bayesian Data Analysis.* Chapman and Hall/CRC, 2003.

M. Goldstein. FastLOF: An Expectation-Maximization based Local Outlier detection algorithm. In *Proceedings of the 21st International Conference on Pattern Recognition*, pages 2282–2285, 2012.

M. Goodrich and R. Tamassia. *Data Structures and Algorithms in JAVA*. John Wiley and Sons, Inc., 1998.

D. Hall and S. McMullen. *Mathematical Techniques in Multisensor Data Fusion*. Artech House Inc., Boston, 2004.

D. Hand. Classifier technology and the illusion of progress. *Statistical Science*, 21 (1):1–14, 2006.

D. Hawkins. *Identification of Outliers*. Chapman and Hall, London, 1980.

S. Ho and H. Wechsler. A martingale framework for detecting changes in data streams by testing exchangeability. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(12), December 2010.

A. Holst, J. Ekman, and S. Larsen. Abnormality detection in event data and condition counters on regina trains. In *Proceedings of the IET International Conference on Railway Condition Monitoring 2006 (RCM 2006), 29-30 November 2006, Birmingham, UK*, 2006.

A. Holst, B. Bjurling, J. Ekman, Å. Rudström, K. Wallenius, R. Laxhammar, M. Björkman, F. Fooladvandi, and J. Trönninger. A Joint Statistical and Symbolic Anomaly Detection System Increasing Performance in Maritime Surveillance. In *Proceedings of the 15th International Conference on Information Fusion*, pages 1919–1926, 2012.

W. Hu, X. Xiao, Z. Fu, D. Xie, T. Tan, and S. Maybank. A system for learning statistical motion patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(9):1450–1464, September 2006.

J. Janssens and E. Postma. One-class classification with LOF and LOCI: An empirical comparison. In *Proceedings of the 18th Belgian-Dutch Conference on Machine Learning*, 2009.

F. Johansson and G. Falkman. Detection of vessel anomalies - a bayesian network approach. In *Proceedings of 3rd International Conference on Intelligent Sensors, Sensor Networks and Information Processing*, 2007.

E. Keogh, J. Lin, and A. Fu. Hot sax: Efficiently finding the most unusual time series subsequence. In *Proceedings of the 5th IEEE International Conference on Data Mining*, 2005.

S. Keogh, E.and Lonardi, C. Ratanamahatana, L. Wei, S.-H. Lee, and J. Handley. Compression-based data mining of sequential data. *Data Mining and Knowledge Discovery*, 14(1):99–129, 2007.

S. Khalid. Motion-based behaviour learning, profiling and classification in the presence of anomalies. *Pattern Recognition*, 43(1):173–186, 2010.

J. Kraiman, S. Arouh, and M. Webb. Automated anomaly detection processor. In A. Sisti and D. Trevisani, editors, *Proceedings of SPIE: Enabling Technologies for Simulation Science VI*, volume 4716, pages 128–137, July 2002.

A. Lambrou, H. Papadopoulos, and A. Gammerman. Evolutionary Conformal Prediction for Breast Cancer Diagnosis. In *Proceedings of 9th International Conference on Information Technology and Applications in Biomedicine*, pages 1–4, 2009.

L. J. Latecki, A. Lazarevic, and D. Pokrajac. Outlier Detection with Kernel Density Function. In *Proceedings of the 5th international conference on Machine Learning and Data Mining in Pattern Recognition*, pages 61–75, 2007.

R. Laxhammar. Anomaly detection for sea surveillance. In *Proceedings of the 11th International Conference on Information Fusion*, pages 1–8, Cologne, Germany, July 2008.

R. Laxhammar. *Anomaly Detection in Trajectory Data for Surveillance Applications*. Studies from the school of science and technology at Örebro university; 19. Örebro university, 2011.

R. Laxhammar. Anomaly Detection. In V. Balasubramanian, S. Ho, and V. Vovk, editors, *Conformal Prediction for Reliable Machine Learning: Theory, Adaptations, and Applications*, Elsevier Insight Series. Morgan-Kaufmann Publishers, 2014.

R. Laxhammar and G. Falkman. Conformal prediction for distribution-independent anomaly detection in streaming vessel data. In *Proceedings of the First International Workshop on Novel Data Stream Pattern Mining Techniques*, pages 47–55. Association for Computing Machinery (ACM), 2010.

R. Laxhammar and G. Falkman. Online Learning and Sequential Anomaly Detection in Trajectories. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 99(PrePrints), 2013a. doi: http://doi.ieeecomputersociety.org/10.1109/TPAMI.2013.172.

R. Laxhammar and G. Falkman. Inductive Conformal Anomaly Detection for Sequential Detection of Anomalous Sub-Trajectories. *Annals of Mathematics and Artificial Intelligence: Special Issue on Conformal Prediction and its Applications*, September 2013b.

R. Laxhammar, G. Falkman, and E. Sviestins. Anomaly detection in sea traffic - a comparison of the gaussian mixture model and the kernel density estimator. In *Proceedings of the 12th International Conference on Information Fusion*, pages 756–763, Seattle, USA, July 2009.

J. Lee, J. Han, and X. Li. Trajectory outlier detection: A partition-and-detect framework. In *Proceedings of the 24th IEEE International Conference on Data Engineering (ICDE)*, pages 140–149, 2008.

M. Markou and S. Singh. Novelty detection: a review - part 1: statistical approaches. *Signal Processing*, 83(12):2481–2497, December 2003a.

M. Markou and S. Singh. Novelty detection: a review - part 2: neural network based approaches. *Signal Processing*, 83(12):2499–2521, December 2003b.

T. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.

B. Morris and M. Trivedi. A survey of vision-based trajectory learning and analysis for surveillance. *IEEE Transactions on Circuits and Systems for Video Technology*, 18(8):1114–1127, August 2008a.

B. Morris and M. Trivedi. Learning and classification of trajectories in dynamic scenes: A general framework for live video analysis. In *Proceedings of IEEE Fifth International Conference on Advanced Video and Signal Based Surveillance*, 2008b.

A. Naftel and S. Khalid. Classifying spatiotemporal object trajectories using unsupervised learning in the coefficient feature space. *Multimedia Systems*, 12(3): 227–238, 2006.

H. Narasimhan and S. Agarwal. A Structural SVM Based Approach for Optimizing Partial AUC. In *Proceedings of the 30th International Conference on Machine Learning*, pages 516–524, 2013.

J. Owens and A. Hunter. Application of the self-organising map to trajectory classification. In *Proceedings of the 3rd IEEE International Workshop on Visual Surveillance*, 2000.

H. Papadopoulos. Inductive Conformal Prediction: Theory and Application to Neural Networks. In P. Fritzsche, editor, *Tools in Artificial Intelligence*. InTech, 2008.

H. Papadopoulos, V. Vovk, and A. Gammerman. Conformal prediction with neural networks. In *Proceedings of the 19th IEEE International Conference on Tools with Artificial Intelligence*, volume 2, pages 388–395, 2007.

A. Patcha and J. Park. An overview of anomaly detection techniques: Existing solutions and latest technological trends. *Computer Networks: The International Journal of Computer and Telecommunications Networking*, 2007.

C. Piciarelli and G. Foresti. On-line trajectory clustering for anomalous events detection. *Pattern Recognition Letters - Special issue on vision for crime detection and prevention*, 27, 2006.

C. Piciarelli, C. Micheloni, and G. Foresti. Trajectory-based anomalous event detection. *IEEE Transactions on Circuits and Systems for Video Technology*, 18(11):1544–1554, November 2008.

D. Pokrajac, A. Lazarevic, and L. Latecki. Incremental local outlier detection for data streams. In *IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*, 2007.

F. Porikli. Trajectory Distance Metric Using Hidden Markov Model Based Representation. In *Proceedings of 6th IEEE International Workshop on Performance Evaluation of Tracking and Surveillance (PETS)*, 2004.

L. Portnoy, E. Eskin, and S. Stolfo. Intrusion detection with unlabeled data using clustering. In *Proceedings of ACM CSS Workshop on Data Mining Applied to Security (DMSA-2001)*, 2001.

K. Proedrou, I. Nouretdinov, V. Vovk, and A. Gammerman. Transductive Confidence Machines for Pattern Recognition. In *Proceedings of the 13th European Conference on Machine Learning*, pages 381–390, 2002.

B. Rhodes. Anomaly detection and behavior prediction: Higher-level fusion based on computational neuroscientific principles. In *Sensor and Data Fusion*. InTech, 2009.

B. Rhodes, N. Bomberger, M. Seibert, and A. Waxman. Maritime situation monitoring and situation awareness using learning mechanisms. In *Proceedings of the Military Communications Conference 2005*, Atlantic City, NJ, USA, October 2005.

B. Rhodes, N. Bomberger, and M. Zandipour. Probabilistic associative learning of vessel motion patterns at multiple scales for maritime situation awareness. In *Proceedings of the 10th International Conference on Information Fusion*, Quebec, Canada, July 2007.

B. Ristic, B. La Scala, M. Morelande, and N. Gordon. Statistical analysis of motion patterns in ais data: Anomaly detection and motion prediction. In *Proceedings of the 11th International Conference on Information Fusion*, Cologne, Germany, July 2008.

M. Riveiro. *Visual Analytics for Maritime Anomaly Detection*. PhD thesis, Örebo University, 2011.

J. Roy. Anomaly detection in the maritime domain. In C. S. Halvorson, D. Lehrfeld, and T. T. Saito, editors, *Proceedings of the SPIE Optics and Photonics in Global Homeland Security IV*, volume 6945, Orlanda, USA, March 2008.

S. Russel and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson Education, second edition edition, 2003.

B. Schölkopf, R. Williamson, A. Smola, J. Shawe-Taylor, and J. Platt. Support Vector Method for Novelty Detection. In *Advances In Neural Information Processing Systems 12*, pages 582–588. 1999.

G. Shafer and V. Vovk. A Tutorial on Conformal Prediction. *Journal of Machine Learning Research*, 9:371–421, March 2008.

R. R. Sillito and R. B. Fisher. Semi-supervised learning for anomalous trajectory detection. In *Proceedings of the British Machine Vision Conference (BMVC)*, pages 1034–1044, 2008.

B. W. Silverman. *Density Estimation for Statistics and Data Analysis*. Chapman and hall, 1986.

Swedish Maritime Safety Inspectorate. Dry Cargo Vessel OOSTERBRUG - Grounding in Malmö, 2004.

N. Taleb. *Fooled by Randomness: The Hidden Role of Chance in the Markets and in Life*. Penguin Books, 2004.

P. Tan, M. Steinbach, and V. Kumar. *Introduction to Data Mining*. Addision-Wesley, 2006.

M. H. Tun, G. S. Chambers, T. Tan, and T. Ly. Maritime port intelligence using ais data. In *Recent advances in security technology*, 2007.

S. Urban, M. Jakob, and M. Pechoucek. Probabilistic modeling of mobile agents' trajectories. In *Proceedings of the International Workshop on Agents and Data Mining Interaction (ADMI)*, 2010.

J. van Laere and M. Nilsson. Evaluation of a workshop to capture knowledge from subject matter experts in maritime surveillance. In *Proceedings of the 12th International Conference on Information Fusion*, pages 171–178, 2009.

J. J. Verbeek. *Mixture Models for Clustering and Dimension Reduction*. PhD thesis, University of Amsterdam, Department of Computer Science, 2003.

J. J. Verbeek, N. Vlassis, and B. Kröse. Efficient greedy learning of gaussian mixture models. *Neural Comput.*, 15(2):469–485, 2003. ISSN 0899-7667.

M. Vlachos, G. Kollios, and D. Gunopoulos. Discovering similar multidimensional trajectories. In *Proceedings of the 18th IEEE International Conference on Data Engineering*, 2002.

V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2005. ISBN 0387001522.

D. Yankov, E. Keogh, and U. Rebbapragada. Disk aware discord discovery: finding unusual time series in terabyte sized datasets. *Knowl. Inf. Syst.*, 17(2):241–262, November 2008.

A. Yilmaz, O. Javed, and M. Shah. Object tracking: A survey. *ACM Computing Surveys*, 38(4), 2006.

Z. Zhang, K. Huang, and T. T. Comparison of similarity measures for trajectory clustering in outdoor surveillance scenes. In *Proceedings of 18th International Conference on Pattern Recognition (ICPR)*, 2006.

M. Zhao and V. Saligrama. Anomaly Detection with Score functions based on Nearest Neighbor Graphs. In *Advances in Neural Information Processing Systems 22*, pages 2250–2258. 2009.

PUBLICATIONS IN THE DISSERTATION SERIES

PUBLICATIONS IN THE DISSERTATION SERIES

1. Berg Marklund, Björn (2013) Games in formal educational settings: Obstacles for the development and use of learning games, Informatics.
   Licentiate Dissertation, ISBN 978-91-981474-0-7
2. Aslam, Tehseen (2013) Analysis of manufacturing supply chains using system dynamics and multi-objective optimization, Informatics.
   Doctoral Dissertation, ISBN 978-91981474-1-4
3. Laxhammar, Rikard (2014) Conformal anomaly detection: Detecting abnormal trajectories in surveillance applications, Informatics.
   Doctoral Dissertation, ISBN 978-91-981474-2-1



RIKARD LAXHAMMAR

Rikard Laxhammar obtained his MSc in Computer Science and Engineering in 2007 from the Royal Institute of Technology, Sweden. In cooperation with Saab AB, Sweden, he was an industrial PhD student at University of Skövde, Sweden, from 2008 to 2013. Rikard has been employed by Saab since 2007 and worked part-time as a software engineer during his PhD studies. His main research interests and expertise are machine learning and anomaly detection.

In his PhD thesis, Rikard investigates algorithms for the automated detection of abnormal motion patterns in surveillance applications. In the maritime domain, for example, abnormal vessel trajectories may indicate potentially dangerous situations related to sea safety and security. One of the main scientific contributions of the thesis is the development of a new method for anomaly detection, called *conformal anomaly detection*, which is based on the machine learning theory of *conformal prediction* (Vovk et al.).

ISBN 978-91-981474-2-1
Dissertation Series, No. 3 (2014)